

昇腾 310B 实战

从入门到精通边缘计算与人工智能

周贤中

2026 年 3 月 15 日

前言

书名中的“实战”，核心是“在编程实践中学习”。本书作为昇腾 310B 芯片的入门指南，将跳出单纯的理论讲解，通过真实的 AI 推理部署案例，带读者直观理解昇腾 310B 的硬件架构特性、Atlas 工具链使用逻辑与端侧 AI 项目开发流程——从模型适配、量化优化到推理服务部署，每一个知识点都配套可落地的代码示例，让读者在动手编码的过程中，真正掌握昇腾 310B 的实战应用能力，实现从“了解芯片”到“能用芯片落地项目”的跨越。

本书定位与目标读者

本书面向以下三类读者：

- **高校学生 / 科研新人**：希望通过一套系统化路径快速理解边缘 AI 硬件与部署流程。
- **嵌入式 / IoT 工程师**：已有一定 Linux / C / Python 基础，希望把 AI 模型真正跑在边缘端并做性能调优。
- **AI 应用开发者 / 创客**：已经能使用主流深度学习框架，希望将训练好的模型迁移到昇腾 310B 进行高效推理与产品化落地。

阅读预期：

- 零基础读者可依照“快速起步路径”完成第 1~3 章 + 精选案例；
- 进阶读者可继续深入算子优化、系统整合与复杂多模型协同部署；
- 有项目诉求的团队可参考“方法论 + 附录模板”直接搭建属于自己的边缘 AI 应用。

学习路径速览（建议路线）

1. 环境 + 工具链：硬件认知 □ 开发环境装配 □ CANN 工具初试
2. 基础模型部署：图像分类 □ 目标检测 □ 语义分割 / OCR / NLP
3. 性能优化：模型结构裁剪 □ 精度-性能权衡（FP16 / INT8） □ 并行与 pipeline
4. 低级能力：自定义算子 □ Profiling □ ACL / GE 原理 □ 内存与数据通路调优

- 5. 系统构建：多进程/多模型协同 □ 任务调度 □ 监控与日志体系
- 6. 实战案例：从需求分析 □ 方案设计 □ 模型适配 □ 部署脚本 □ 交付验收

全书结构（初版规划）

本书分为 **Part I：理论教程**与 **Part II：实验教程**两大部分。

0.1. Part I：理论教程

章节	标题	内容聚焦	读者收益
Chapter 1	昇腾 310B 边缘计算基础	边缘计算概念、竞品对比、310B 规格	建立边缘计算认知与选型能力
Chapter 2	CANN 软件栈核心组件解析	异构架构、ATC 转换工具、msame 推理	掌握 CANN 基础与模型转换流程
Chapter 3	昇腾 PyTorch 扩展迁移基础	环境搭建、torch_npu 插件、经典网络迁移	学会在昇腾上运行 PyTorch 模型
Chapter 4	PyACL 应用开发基础	ACL 运行时、模型推理流程、DVPP 媒体处理	掌握 C/C++ 底层应用开发能力
Chapter 5	算子开发基础	TBE DSL 流程、310B 计算架构、自定义算子	理解底层计算原理, 具备算子扩展能力
Chapter 6	系统工程与高可用部署	Profiling 分析、AIPP 硬件加速、多级流水线	掌握性能调优与系统工程化技能
Chapter 7	项目实战方法论与交付模板	需求分析、评测体系、标准化交付流程	建立规范的 AI 项目交付方法论
Chapter 8	综合实战案例集	案例规范、目录结构、典型案例概览	了解实战案例的组织与核心逻辑

章节	标题	内容聚焦	读者收益
Chapter 9	附录与工具箱	错误速查、参数模板、Checklist、FAQ	提供高效查阅的工具与参考资料

0.2. Part II: 实验教程

包含从 **Case 0** 到 **Case 9** 的十个动手实验，涵盖环境搭建与九大端侧 AI 实战案例（人脸打卡机、实时跟踪、智能电子琴、掌纹识别、数据采集仪、智能小车、智能相册、手势识别、聊天机器人）。

各模块核心要点概述

1. 昇腾 310B 边缘计算基础 - 边缘计算 vs 云计算：概念辨析与应用场景 - 边缘计算卡生态：NVIDIA、华为、瑞芯微对比 - 昇腾 310B 技术规格详解

2. CANN 软件栈核心组件解析 - CANN 异构计算架构与 CUDA 对比 - ATC 模型转换工具流程、参数详解与注意事项 - msame 推理工具使用概览

3. 昇腾 PyTorch 扩展迁移基础 - PyTorch (torch_npu) 环境安装与配置 - 基础网络实现：Linear, MLP, LeNet - 现代 CNN 移植实战：AlexNet, VGG, ResNet

4. PyACL 应用开发基础 - 快速入门：Hello World 与运行模式 - 初始化与运行时管理：Device, Context, Stream, Memory - 模型推理完整流程：加载、Dataset 准备、执行 - DVPP 图像/视频处理能力 (JPEGD, VPC)

5. 算子开发基础 - 为什么需要自定义算子：TBE vs AICPU - 310B 硬件架构：存储层级与向量计算单元 - TBE DSL 开发流程与 LogSoftmax 实例

6. 系统工程与高可用部署 - 性能优化全景：木桶效应与 Amdahl 定律 - Profiling 性能分析：采集与关键视图解读 - 核心优化技术：AIPP、零拷贝、多级流水线 - 实战演练：车辆检测系统的极致优化之路

7. 项目实战方法论与交付模板 - 需求澄清 Canvas 与指标分层 - Baseline 策略、评测集设计与迭代看板 - 资产沉淀、交付目录与上线 Checklist - 验收、回归与风险管理

8. 综合实战案例集 - 案例统一模板与目录结构规范 - 实战案例概览与重点分析 (人脸、跟踪、音频等) - 结果记录、差异报告与自动化保障

9. 附录与工具箱 - 常见报错速查表 - 模型转换参数模板合集 (ResNet, YOLO, INT8) - 性能与质量 Checklist - 术语表、FAQ 与推荐资源

10. 实验教程 (Case 0 - 9) - Case 0: 开发板硬件介绍、系统安装 (刷写/启动/网络)、环境验证 - **Case 1:** 智能打卡机 (Web 应用, 人脸识别) - **Case 2:** 边缘端实时目标跟踪 (检测 + 关联) - **Case 3:** 智能电子琴 (手势识别 + 音频合成) - **Case 4:** 智能掌纹识别机 (预处理, 特征提取) - **Case 5:** 智能数据采集仪 (传感器, 数据分析) - **Case 6:** 智能小车 (视觉, 路径规划, 运动控制) - **Case 7:** 智能相册 (识别, 分类, 检索) - **Case 8:** 手势识别 (数据采集, 实时系统) - **Case 9:** 智能聊天机器人 (LLM, 语音交互, RAG)

实践驱动与开源协作

本书所有示例代码、脚本、案例与附录工具均开源托管于本仓库。欢迎通过 Issue / PR 反馈问题、提交改进、补充案例或翻译。我们鼓励：- 增补新模型 / 新任务的部署范式 - 分享自定义算子优化经验 - 提交性能测试报告（含硬件信息 + 指标）- 翻译与文档校对

如何使用本书

读者类型	推荐阅读路径	目标	补充建议
零基础学生	Chapter 1 □ Chapter 2 □ Case 0 □ Case 1	能跑通首个模型	结合案例做改动实验
嵌入式工程师	Chapter 4 □ Chapter 5 □ Chapter 6	掌握底层开发与优化	关注资源管理与算子开发
AI 应用开发者	Chapter 2 □ Chapter 3 □ 选读 Case 案例	快速场景落地	重点关注 PyTorch 迁移与部署
技术负责人	Chapter 1 □ Chapter 7 □ Chapter 8	构建团队方法论	制定内部交付标准与模板体系

更新与版本计划

- v0.1（当前）：结构规划 + Chapter 1~3 初稿 + Case 0~1 示例
- v0.3：补齐核心部署链路 (Chapter 4) + 性能调优初稿 (Chapter 6)
- v0.6：全案例上线 (Case 0~9) + 工程化章节完善

-
- v1.0: 补齐附录 (Chapter 9) + 全面审校 + PDF / LaTeX 发行

许可证与引用

本书内容采用 Apache 2.0 许可证。引用本书内容请注明：> 《昇腾 310B 实战：从入门到精通边缘计算与人工智能》(GitHub: zhouxzh/Ascend310)

欢迎加入共建，一起把“边缘 AI 实战”这件事做成！

目录

I. Part I: 理论教程	1
1. 昇腾 310B 边缘计算基础	2
1.1. 边缘计算简介	2
1.2. 边缘计算卡的发展历史	6
1.3. 常见的边缘计算卡	8
1.4. 昇腾 310B 简介	10
1.5. 实践任务	14
2. CANN 软件栈核心组件解析	15
2.1. CANN 异构计算架构	15
2.2. ATC 模型转换详解	21
2.3. 章节小结	33
2.4. 实践任务	34
3. 昇腾 PyTorch 扩展迁移基础	35
3.1. 架构速览	35
3.2. 核心能力纵览	35
3.3. PyTorch 安装教程	36
3.4. torch_npu 插件的使用	40
3.5. torch_npu 插件兼容性测试套件	62
3.6. 现代卷积神经网络的移植	66
3.7. 总结	89
4. PyACL 应用开发基础	90
4.1. PyACL 的基本概念	90
4.2. 模型推理流水线	110
4.3. 图像/视频处理基础	130
4.4. 单算子调用	134

4.5. 更多特性	134
4.6. 应用调试与常见 FAQ	134
4.7. 使用约束	135
4.8. 应用样例参考目录	135
5. 算子开发基础	136
5.1. 算子开发概述：昇腾 310B 硬件架构与开发路径	136
5.2. 初体验：使用 TBE DSL 快速实现第一个算子（向量加法）	140
5.3. 理解算子开发的核心概念	148
5.4. 深入底层：Ascend C 算子开发入门	148
5.5. 从简单到复杂：实现一个需要 Tiling 的算子（如矩阵加法）	148
5.6. 算子编译、部署与单元测试	148
5.7. 算子性能优化初探	149
6. 系统工程与高可用部署	150
6.1. 性能优化的全景视角	150
6.2. 照妖镜：Profiling 性能分析	151
6.3. 常见瓶颈与对策 Checklist	151
6.4. 核心优化技术详解	152
6.5. 实战演练：车辆检测系统的极致优化之路	153
6.6. 章节要点与练习	154
7. 项目实战方法论与交付模板	155
7.1. 章节总览	155
7.2. 需求澄清 Canvas	155
7.3. 指标分层与优先级	156
7.4. Baseline 策略与控制变量法	156
7.5. 评测集设计原则	156
7.6. 迭代计划与看板	157
7.7. 资产沉淀文档体系	157
7.8. 交付目录与不可变产物	158
7.9. 上线前综合 Checklist	158
7.10. 验收、回归与漂移监测	159
7.11. 风险管理与决策日志	159
7.12. 章节小结	159
7.13. 实践任务	159

8. 合实战案例集	160
8.1. 章节总览	160
8.2. 案例统一模板（标准化规范）	160
8.3. 案例目录结构规范	160
8.4. 例概览与重点	161
8.5. 案例 1: 人脸打卡机	161
8.6. 案例 2: 实时跟踪（检测 + 关联）	163
8.7. 案例 3: 智能电子琴（音频）	163
8.8. 结果记录与差异报告	164
8.9. 自动化与复现保障	164
8.10. 指标可视化建议	164
8.11. 通用问题经验库	164
8.12. 扩展方向	165
8.13. 贡献工作流	165
8.14. 章节小结	165
8.15. 实践任务	165
9. 附录与工具箱	166
9.1. 章节总览	166
9.2. 常见报错速查	166
9.3. 模型转换参数模板合集	167
9.4. 性能与质量 Checklist（执行勾项）	168
9.5. 术语表（扩展）	168
9.6. 推荐资源与外部引用	169
9.7. 贡献指南摘要	169
9.8. FAQ	169
9.9. License 与引用	170
9.10. 版本路线回顾	170
9.11. 实践任务	170
II. Part II: 实验教程	171
0. 案例 0: 初步使用开发板	172
0.1. 昇腾 310B 开发板介绍	172
0.2. 配件准备	172

0.3. 操作系统安装	180
0.4. 启动开发板	190
0.5. 网络连接	194
0.6. 远程连接 (SSH)	195
0.7. 验证开发环境	195
0.8. 结语	197
1. 案例 1: 智能打卡机	198
1.1. 项目简介	198
1.2. 内容大纲	198
1.3. Web 应用	200
1.4. □ Web 界面详细介绍	201
1.5. □ Web 应用技术架构	202
1.6. 效果演示	204
1.7. 故障排除	204
1.8. 性能建议	205
2. 案例 2: 基于 MobileNet-SSD 与 DeepSORT 的目标跟踪教程	206
2.1. 教程定位	206
2.2. 实验硬件与运行条件	206
2.3. 本案例支持的骨干网络	208
2.4. 目标跟踪对时序关联的要求	208
2.5. MobileNet-SSD 作为检测前端的依据	209
2.6. DeepSORT 作为跟踪后端的选择依据	213
2.7. 本案例的代码结构与总体流程	214
2.8. 检测部分代码解析	215
2.9. 从检测到跟踪: tracking_app.py 的桥梁作用	222
2.10. 跟踪部分代码解析	222
2.11. 可视化代码如何帮助理解算法结果	230
2.12. 如何评价这个案例方案	231
2.13. 实验设计	232
2.14. 章节总结	232
3. 案例 3: 智能电子琴	235
3.1. 1. 项目简介	235
3.2. 2. 内容大纲	235

3.3. 3. 源代码结构	239
3.4. 4. 效果演示	240
4. 案例 4：智能掌纹识别机	241
4.1. 1. 项目简介	241
4.2. 2. 内容大纲	241
4.3. 3. 源代码结构	246
4.4. 4. 效果演示	247
5. 案例 5：智能数据采集仪	248
5.1. 1. 项目简介	248
5.2. 2. 内容大纲	248
5.3. 3. 源代码结构	255
5.4. 4. 效果演示	256
6. 案例 6：智能小车	257
6.1. 1. 项目简介	257
6.2. 2. 内容大纲	257
6.3. 3. 源代码结构	264
6.4. 4. 效果演示	264
7. 案例 7：智能相册	266
7.1. 1. 项目简介	266
7.2. 2. 内容大纲	266
7.3. 3. 源代码结构	274
7.4. 4. 效果演示	275
8. 案例 8：手势识别	276
8.1. 1. 项目简介	276
8.2. 2. 内容大纲	276
8.3. 3. 源代码结构	286
8.4. 4. 效果演示	286
9. 案例 9：智能聊天机器人	287
9.1. 1. 项目简介	287
9.2. 2. 内容大纲	287
9.3. 3. 源代码结构	298

9.4. 4. 效果演示	299
------------------------	-----

Part I.

Part I: 理论教程

1. 昇腾 310B 边缘计算基础

1.1. 边缘计算简介

华为云等公共云计算平台允许企业以全国的云服务器作为其私有的数据与 AI 计算中心，将其基础设施扩展到任意地点，并根据需要向上或向下扩展计算资源。然而遍布全球实时运行的 AI 应用可能需要显著的本地处理能力，且往往位于距离集中式云服务器过远的偏远位置。而且出于低时延或数据驻留要求，一些工作负载需要保留在本地或特定地点，就是为什么许多企业使用边缘计算来部署其 AI 应用。边缘计算指的是在数据产生的位置进行处理，边缘计算在边缘设备中本地处理与存储数据。由于边缘计算设备无需依赖互联网连接，可以作为独立的网络节点运行。

1.1.1. 什么是云计算？

云计算（Cloud Computing）是一种计算风格，其可扩展与弹性能力通过互联网技术以服务形式交付。云计算依托集中化数据中心，通过互联网按需提供弹性、可扩展、托管式 IT 资源与服务（计算 / 存储 / 网络 / 数据 / AI 平台）。

云计算的好处是什么？ - 较低的前期成本：购买硬件、软件、IT 管理以及全天候电力与制冷的资本支出被消除。云计算使组织能够以较低的财务进入门槛快速将应用推向市场。灵活的定价 – 企业只为其使用的计算资源付费，从而对成本有更多控制并减少意外。 - 按需无限计算：云服务可以通过自动配置与撤销资源即时对不断变化的需求做出反应与适配。这可以降低成本并提升组织整体效率。 - 简化的 IT 管理：云提供商为其客户提供访问 IT 管理专家的渠道，使员工可以专注于企业的核心需求。 - 便捷更新：可一键访问最新的硬件、软件与服务。 - 可靠性：数据备份、灾难恢复与业务连续性更容易且更便宜，因为数据可以在云提供商网络的多个冗余站点镜像。 - 节省时间：企业可能在配置私有服务器与网络上耗费时间。借助按需云基础设施，它们可以在更短时间内部署应用并更快进入市场。

1.1.2. 什么是边缘计算？

边缘计算（Edge Computing）是一种分布式计算框架，旨在将计算、存储和网络能力部署在靠近数据源或终端设备的网络边缘位置。通过在本地或近端节点处理数据，减少向远端数据中心/云的集中回传，获得更低的时延、更好的带宽利用与更高的数据主权与隐私保障。边缘计算是在距离数据产生源（传感器、摄像头、终端设备、工业控制点）物理更近的位置部署计算与存储，使数据在本地/近端被快速处理与筛选，减少长距离回传，保障低时延、带宽节省与数据主权。边缘计算是将计算能力在物理上靠近数据生成位置（通常是物联网设备或传感器）的实践。因为计算能力被带到网络或设备的边缘，边缘计算能够实现更快的数据处理、增加的带宽以及确保的数据主权。

通过在网络边缘处理数据，边缘计算减少了大量数据在服务器、云与设备或边缘位置之间往返传输以被处理的需求。这对诸如数据科学与 AI 等现代应用尤为重要。许多高计算应用（如深度学习与推理、数据处理与分析、仿真与视频流）已成为现代生活的支柱。随着企业日益意识到这些应用由边缘计算驱动，生产中的边缘用例数量应会增加。

边缘计算的特点是什么？ - 靠近数据源：在物联网设备、网关、工业控制器或本地微型服务器附近直-成初级或核心推理逻辑，降低往返延迟。 - 分布式架构：区别于云的集中式调度，采用多节点协同与局部自治，适合-化、地理分散与对实时性敏感的任务。 - 实时响应：适配无人驾驶、工业控制、视频安防、AR/VR 等对毫秒级响应-的场景。 - 隐私与安全：敏感原始数据（面部特征、生产工艺、地理轨迹）在本地预-或结构化提取后再上云，降低泄露与合规风险。 - 带宽优化：仅上传事件/特征/聚合指标，显著降低原始全量视频/传感流量-路的占用。 - 弹性与容错：弱网/离线时保持关键功能脱网运行，网络恢复后再同步（延迟一致性）。

边缘计算的好处是什么？ - 更低时延：在边缘进行数据处理会消除或减少数据传输。这可为需要低时延的复杂 AI 模型用例（如完全自动驾驶车辆与增强现实）加速洞察。 - 降低成本：使用局域网进行数据处理相比云计算可为组织提供更高带宽与更低成本的存储。此外，由于处理发生在边缘，需要发送到云或数据中心进一步处理的数据更少。这导致需要传输的数据量减少，成本也降低。 - 模型精度：AI 依赖高精度模型，尤其是需要实时响应的边缘用例。当网络带宽过低时，通常通过降低输入模型的数据尺寸来缓解。这会导致图像尺寸缩小、视频跳帧、音频采样率降低。部署在边缘时，数据反馈回路可用于提升 AI 模型精度，并且可同时运行多个模型。 - 更广覆盖：传统云计算必须依赖互联网接入。而边缘计算可在本地处理数据，无需互联网接入。这将计算范围扩展到以前无法访问或偏远的位置。 - 数据主权：当数据在其采集位置被处理时，边缘计算允许组织将所有敏感数据与计算保留在局域网和公司防火墙内。这减少了暴露于云端网络安全攻击的风险，并在不断变化的严格数据法律下具备更好合规性。

1.1.3. 边缘计算 vs 云计算?

维度	边缘计算	云计算	典型取舍 / 典型场景
处理位置	近端（本地/网关/边缘节点）	远端数据中心	边缘降低时延并就近处理高带宽原始数据（如视频）；云用于集中算力与全局聚合。
时延特性	低（本地判决 20~ 几十 ms）	受网络往返影响 (>100ms 场景常见)	实时/控制闭环优先边缘；非实时批处理、训练优先云。
带宽占用	上行压缩（只传事件/特征/聚合）	常上传原始数据到云	当传输成本高或链路受限，将预处理放到边缘；带宽充足且需保留原始数据则上云。
部署/运维	分布式，需节点管理（异构、离线可用）	集中化，统一维护与弹性伸缩	节点数多时运维复杂度上升（需探针/模板）；云侧适合动态弹性与大规模训练/批处理。
隐私合规	本地脱敏/保留数据主权易控	数据集中存储需合规审核	高度敏感或受监管数据优先边缘；低合规风险且需统一治理优先云。
伸缩弹性	受制于本地硬件与现场成本	云端资源弹性丰富	现场扩展（CAPEX）与运维（OPEX）成本较高；云适合突发/动态扩展工作负载。
典型适用场景 (示例对照)	实时推理、低延迟控制、弱网/离线场所	非实时批处理、模型训练、集中化数据湖	云：非时间敏感数据、可靠互联网、已在云存储的数据；边缘：实时数据处理、受限或无联网地点、传输成本过高的大规模本地数据、受严格法规约束的数据。

一个边缘计算优于云计算的示例是医疗机器人，外科医生需要实时数据访问。这些系统包含大量可在云中执行的软件，但手术室中日益出现的智能分析与机器人控制无法容忍时延、网络可靠性问题或带宽限制。在此示例中，边缘计算为患者提供了生死攸关的益处。

1.1.4. 何时采用边缘计算?

边缘节点的硬件选择，正如您所指出的，本质上是功耗、体积和成本之间的权衡。像华为 Ascend 310B 这样的处理器正是这一理念的产物：它不追求极致的算力，而是在满

足边缘场景基本 AI 推理需求的前提下，尽可能实现能效比和成本的最优解。这种硬件特性直接决定了其所能支撑的应用边界。边缘计算的价值在多种场景中得以凸显。在物联网网关中，它扮演着“区域数据枢纽”的角色，对海量传感数据进行本地预处理、协议转换与降噪聚合，大幅减轻上行带宽压力。工业自动化场景，如产线质量检测与能耗分析，依赖边缘节点的毫秒级响应能力，实现控制闭环的实时性与可靠性。在智慧城市的交通流量或环境监测中，边缘计算使得低延时告警成为可能，同时有效节省了持续视频流上传所需的巨大带宽。

安防监控是边缘计算的经典应用，通过实时视频结构化分析，将原始视频转化为结构化的事件信息（如“有人越界”、“车辆违章”）再上传，既保护了个人隐私，又提升了事件处理效率。智能零售中的客流与货架分析，则体现了边缘的设备自治特性，即使在网络不佳时也能独立运行。而对于车路协同中的路侧单元，超低时延与本地协同决策是保障行车安全的核心，任何云端的往返延迟都是不可接受的。

边缘计算并非云的替代品，而是与云共同构成了“端 - 边 - 云”三层协同的健壮体系。在这个体系中，端侧负责产生原始数据，边缘侧专注于低时延的智能决策、实时控制与数据“提纯”，而云端则依托其强大的算力与存储资源，负责全局模型的训练、长周期的大数据分析与跨区域的调度协同。

一个合理的功能切分策略至关重要，它直接决定了整个系统的成本结构、响应性能与可持续演进能力。判断一个功能更适合放在云还是边缘，可以遵循一些基本原则：适合上云的任务通常是非实时的批处理、模型训练、需要动态弹性伸缩的业务，或者数据本就存储在云数据湖中的场景，且对合规和延迟不敏感。相反，更适合下沉到边缘的任务则对实时推理和控制、稳定持续的低时延有强需求，其数据来源密集且分散，或者涉及高敏感数据受合规监管，同时面临着带宽受限或成本高昂的现实约束。

在具体的工程实现上，云和边缘的偏好存在显著差异，这要求开发者采用不同的技术策略。

在日志策略上，云侧倾向于全量集中收集以方便分析，而边缘侧由于带宽和存储限制，必须采用采样与本地环形缓存截断等轻量化手段。模型分发时，云侧可以利用 CDN 轻松推送大文件，边缘侧则需要差分升级、分块传输与严格的哈希校验机制，以确保在不稳定网络下的更新成功率。配置管理方面，边缘服务需支持配置嵌入版本与增量下发，并必须具备离线安全回滚能力，以应对网络中断的极端情况。

监控系统的设计上，云侧普遍采用 Prometheus 等集中式时序数据库，而边缘侧则需要轻量的代理，并优先在本地缓存指标，在资源冲突时甚至需要丢弃低优先级数据。对于安全补丁，云上可以做到自动批量推送，但在边缘侧，为了避免对关键业务造成意外中断，通常需要设定计划维护窗口或要求手动确认，实施更为谨慎的更新策略。

总而言之，边云协同的主旋律是“边缘强化实时响应与局部自治，云强化全局优化与集中智能”。在设计系统时，一个有效的思路是以“放在云端的必要性”为拷问，反向

审视每一个功能模块。任何不具备必须上云理由的功能，都应优先考虑下沉到边缘。最终的功能切分边界，应当由可观测的量化指标来界定，包括时延、带宽消耗、总体拥有成本、处理精度以及合规等级等。

在实际的资源与任务编排中，需要精细化管理。例如，将高 I/O 密度的任务与计算密集型任务错峰执行，以避免资源争抢。将热点算子进行分组，防止它们在短时间内同时提交导致带宽剧烈抖动。同时，必须高度重视热设计，通过持续监控硬件温度（如每 5 秒采样），在超过安全阈值（如 85°C）时自动触发降频或任务降级等保护性负载策略，确保边缘设备在复杂现场环境下的长期稳定运行。

1.2. 边缘计算卡的发展历史

边缘计算卡的发展历程，是一场为了将“智能”从遥远的云端数据中心不断推向现实生活前沿的“迁徙史”。在早期，所谓的边缘节点大多是基于通用的低功耗 CPU 构建，处理能力有限，难以胜任复杂的实时计算任务。那时可以说是“有边缘，而无计算”。随着自动驾驶和安防监控等应用对视觉处理的需求爆发，市场开始探索专用的加速方案。**FPGA** 因其可重构性和低延迟成为重要选择，同时，像英特尔 Movidius 这类专为视觉优化的低功耗芯片也开始出现，让设备真正具备了本地处理 AI 的能力。真正的转折点来自深度学习浪潮。英伟达将其 GPU 技术下沉，推出了划时代的 **Jetson 系列**。这不仅带来了强大的算力，更重要的是建立了成熟的 CUDA 软件生态，让边缘 AI 开发从硬件调优变成了软件工程，极大地降低了开发门槛。近年来，边缘计算卡的发展进入了百家争鸣的时代。英特尔通过“CPU+VPU+FPGA”组合和 OpenVINO 工具链提供灵活方案；高通将移动端的高能效 SoC 设计经验引入边缘；谷歌则推出了为 TensorFlow 量身定制的 Edge TPU，追求极致能效。同时，国产力量快速崛起，华为的昇腾系列、寒武纪的思元系列等，都在安防、自动驾驶等特定领域展现出强大竞争力。如今，边缘计算卡已不再是单纯的算力竞赛，而是进入了**算力、功耗、成本、软件栈和行业生态**的综合竞争阶段。它的演进史，核心始终是为了让智能在物理世界的每个角落都能实时、可靠地运行。

当前，海外边缘计算市场格局鲜明，由几家技术路径各异的科技巨头共同引领。

英伟达凭借其 Jetson 系列，被视为边缘设备中的“微型超算”。该系列覆盖从入门到旗舰的全场景需求，不仅以强劲性能著称，更以成熟的 CUDA 和 TensorRT 软件生态构筑起护城河。无论是复杂的机器人视觉系统还是自动驾驶车辆，Jetson 往往是开发者实现高性能边缘 AI 的首选平台。

英特尔则展现出“全能型选手”的特质。它不仅以 CPU 处理通用计算任务，还通过 Movidius 视觉处理单元等专用芯片强化 AI 视觉推理。其核心优势在于 OpenVINO 软件工具包——能够将 AI 模型高效适配并优化至英特尔各类硬件平台，使其在工业自动

化、智能零售等需要灵活部署的领域中广受青睐。

AMD 在整合赛灵思之后，实力显著提升。其杀手锏在于“自适应计算”，尤其是 **FPGA** 芯片的可重构特性，能够通过硬件级定制实现超低延迟加速。这一特性让 **AMD** 在专业机器视觉、通信基础设施等需要快速算法迭代的特定场景中脱颖而出。

高通 将移动芯片领域积累的低功耗与高集成度经验成功复用于边缘侧。其芯片方案集成了专用 **AI** 引擎与 **5G** 连接能力，特别适合无人机、扩展现实设备等对续航和体积敏感的移动场景，为电池供电的边缘设备提供了持久智能的可能。

谷歌 选择了一条差异化的技术路径，推出专为 **TensorFlow** 模型推理设计的 **Edge TPU** 芯片。这款芯片以极低的功耗和紧凑的体积，在兼容的 **AI** 任务中提供出色的推理速度。对于深度融入谷歌 **AI** 生态的开发者而言，基于 **Edge TPU** 的 **Coral** 开发板成为一个高效、节能的部署选择。

国内边缘计算卡的发展，是一部从“跟跑”到“并跑”的奋斗史。大约在 2016 年前后，随着 **AI** 浪潮兴起，国内边缘计算开始萌芽。最初阶段，大多数企业还是采用国外芯片做集成开发，或者在通用芯片上进行适配优化。这个时期可说是“用起来就好”的实用主义阶段。2018 年左右，随着“中兴事件”的爆发，自主可控成为国家战略，真正推动了国产边缘计算芯片的研发热潮。以**华为昇腾 310** 芯片为代表的第一代国产边缘 **AI** 芯片正式亮相，标志着我们开始拥有自己的“算力底座”。这款芯片不仅在性能上对标国际产品，更重要的是实现了从芯片到框架的全栈自主创新。几乎在同一时期，**寒武纪** 的思元系列、**地平线** 的征程系列等专用 **AI** 芯片也纷纷落地。这些芯片不再是简单的替代品，而是在特定场景下展现出了独特优势——比如寒武纪在算力密度上的突破，地平线在自动驾驶领域的专注深耕。2020 年之后，国产边缘计算进入了“场景深化”阶段。各家企业不再追求单纯的算力指标，而是更注重实际应用效果。华为 **Atlas** 系列在智慧城市项目中大规模部署，地平线的征程芯片在多家车企实现前装量产，黑芝麻智能则专注于大算力自动驾驶芯片的研发。这些成就显示，国产边缘计算卡已经从“能用”进化到了“好用”的阶段。如今，国产边缘计算卡正在形成自己独特的发展路径——不仅在性能上紧追国际先进水平，更在能效比、场景适配度和成本控制上展现出越来越强的竞争力。从最初的艰难起步，到现在的百花齐放，这段发展历程见证了中国在高科技领域的快速崛起。

近年来，国内科技公司在自研 **AI** 芯片和计算卡领域进展迅速，形成了具有鲜明特色的产品体系。

华为 是国内边缘计算领域的领头羊，其 **Atlas** 系列是基于自研的“昇腾”（Ascend）**AI** 处理器。它不仅仅是一张卡，更是一个完整的解决方案。从面向开发者的 **Atlas 200I DK** 套件，到集成了多张加速卡的 **Atlas 500** 智能边缘服务器，华为提供了覆盖多种场景的产品形态。其最大优势在于“软硬件协同”，通过自家的 **CANN** 驱动和昇思（MindSpore）**AI** 框架，能充分发挥芯片性能，特别在安防、智慧城市等要求自主可控

的项目中成为首选。

寒武纪是国内专业的 AI 芯片上市公司，其“思元”（MLU）系列边缘计算卡在业界拥有很高的知名度。寒武纪的芯片以**专业化的 AI 算力**和**高计算密度**见长，在智能安防、智慧交通等领域实现了规模化应用。它的产品是纯计算加速卡形态，可以灵活地插入到标准的服务器中，为数据中心和边缘机房提供强大的推理能力。

地平线是另一家极具特色的公司，其强项在于**自动驾驶和智能驾驶舱**领域。它提供的不是单一的计算卡，而是以“芯片 + 工具链”为核心的解决方案。其**征程系列**车规级 AI 芯片，功耗控制非常出色，专门为处理智能汽车上的视觉感知、环境建模等任务而优化。通过与众多车企和零部件供应商合作，地平线的技术已经广泛应用在众多量产车型中。

此外，像**瑞芯微**这样的传统 SoC 设计公司，其 **RK3588** 等高端芯片也具备了可观的边缘 AI 算力。虽然它们通常以核心板的形式出现，但因其极高的**性价比**和强大的**多媒体处理能力**，被广泛集成到各种边缘设备中，如智能 NVR、商显广告机和工业控制设备，是中低算力需求场景中非常普遍的选择。

总的来说，国内边缘计算卡市场呈现出百花齐放的态势：华为提供全栈式方案，寒武纪提供专业的算力卡，地平线和黑芝麻则深耕自动驾驶这一垂直领域，而瑞芯微等则在更广泛的 AIoT 市场占据重要地位。这些国产力量共同为各行各业的智能化转型提供了坚实且多样化的算力基础。

1.3. 常见的边缘计算卡

好的，我们重点来介绍一下**英伟达**、**华为**和**瑞芯微**这三家在边缘计算领域颇具代表性的产品。这三家的产品定位和优势各有侧重，形成了从高性能到高性价比的全面覆盖。

1.3.1. 英伟达 - 边缘 AI 的性能与生态王者

英伟达凭借其强大的 GPU 架构和成熟的软件栈（CUDA, TensorRT 等），在边缘 AI 领域占据了主导地位，尤其是在需要复杂 AI 推理的场景中。英伟达的 **Jetson** 系列构成了一个完整的嵌入式 AI 计算平台，涵盖了从核心计算模块到载板设计，再到软件栈的全套解决方案。

当前产品线的中流砥柱是 **Jetson Orin** 系列。其中的旗舰型号 **Jetson AGX Orin** 堪称性能猛兽，拥有高达 275 TOPS 的 AI 算力，性能足以媲美小型服务器，能够胜任自主机器、高级机器人、医疗影像等最严苛的边缘应用。对于需要高性能但空间受限的场景，**Jetson Orin NX** 提供了理想的解决方案，它在紧凑的体积内实现了最高 100 TOPS 的算力，成为多路视频分析设备和边缘服务器的首选。而 **Jetson Orin Nano** 则重新定

义了入门级边缘 AI 的标准，以 20-40 TOPS 的算力为新一代 AI 产品和原型开发打开了新的可能。

除了新一代产品，一些经典机型仍然在市场上发挥着重要作用。Jetson Xavier NX 凭借其在性能和价格之间的出色平衡，至今仍在许多项目中广泛使用。更早的 Jetson Nano 虽然性能已经不及新产品，但其低廉的成本和庞大的开发者社区，使其仍然是教育入门和简单项目的绝佳选择。

这个平台最突出的优势在于其极致的性能表现和无比成熟的软件生态。CUDA-X AI 为开发者提供了完善的工具链，加上活跃的社区支持和丰富的学习资料，大大降低了开发门槛。不过，这些优势也伴随着相应的代价——Jetson 产品的价格相对较高，功耗表现也不算特别出色。正因如此，它们主要被应用于自动驾驶、高端机器人、复杂视频分析等对计算性能要求极高的场景中。

1.3.2. 华为 - 全栈全场景的国产化标杆

华为基于其自主研发的昇腾 AI 处理器与昇思 AI 框架，构建了一套从底层芯片到顶层应用的“全栈式”人工智能解决方案。这一完整的技术布局使华为在安防、智慧交通等对自主可控要求较高的领域展现出独特优势。

在边缘计算领域，华为通过 Atlas 系列产品提供了丰富的形态选择。对于开发者而言，Atlas 200I DK A2 是一个理想的入门平台，这款小巧的开发板搭载昇腾 310 处理器，为初学者提供了体验昇腾生态的便捷途径。而在实际部署中，Atlas 500 Pro 作为集成的智能边缘服务器，以其丰富的接口和稳定的性能，成为智慧交通、园区管理等场景的常见选择。此外，像 Atlas 300I 这样的 PCIe 加速模块，则为企业现有的工控设备和服务器提供了灵活高效的 AI 能力扩展方案。

这套边缘计算体系的核心优势在于完全的国产化技术栈，通过 CANN 驱动实现了深度的软硬件协同优化，同时与华为云服务形成了良好的端边云协同能力。不过，与英伟达等国际巨头相比，昇腾生态的成熟度和社区资源仍处在追赶阶段。目前，华为 Atlas 系列特别适合应用于安防监控、智慧城市、工业质检等对数据安全和国产化有明确要求的项目场景。

1.3.3. 瑞芯微 - 高性价比的 SoC 芯片专家

瑞芯微作为国内领先的集成电路设计企业，以其高集成度与出色性价比在 AIoT 芯片领域占据重要地位。与提供完整计算卡的厂商不同，瑞芯微专注于核心 SoC 系统级芯片的设计，由下游合作伙伴基于其芯片开发出形态丰富的开发板与整机产品。

在其 AIoT 芯片系列中，RK3588 系列堪称旗舰代表。这款芯片不仅搭载了八核高

性能处理器，还集成了算力达 6 TOPS 的自研 NPU，支持多种精度量化方式。特别值得一提的是其卓越的多媒体处理能力，支持 8K 高清解码与多路摄像头输入，使其成为边缘计算盒子、智能 NVR 及商显设备等高端应用的理想选择。

面向主流市场，RK3568 系列则展现出强大的市场号召力。这款芯片在算力与功耗之间取得了良好平衡，集成的 1 TOPS NPU 为各类 AI 应用提供了基础算力保障。目前该芯片已被广泛应用于工业控制、智能门禁、网络存储设备等场景，更是成为了 Firefly、Radxa 等知名开发板平台的核心选择。

总体而言，瑞芯微方案的最大优势在于极致的性价比与高度的集成化——单颗芯片即融合了 CPU、GPU、NPU 及多媒体处理单元。虽然在绝对 AI 算力上无法与英伟达、华为的高端产品直接竞争，其软件生态也仍在持续完善中，但在智能门禁、商显广告机、网络录像机等中低算力需求的 AIoT 应用场景中，瑞芯微无疑提供了一个经济实用且性能均衡的优质选择。

1.4. 昇腾 310B 简介

昇腾 310 是华为昇腾（Ascend）AI 计算产品线的关键入门级芯片，它的发展史与华为全栈 AI 战略的落地紧密相连。

在 2018 年的华为全联接大会上，华为首次发布了昇腾 AI 芯片系列，其中昇腾 310 作为面向**边缘和端侧**场景的处理器正式亮相。它的设计目标非常明确：在**极低的功耗（仅 8W）**下，提供足以在边缘端进行实时 AI 推理的算力。这与面向数据中心的昇腾 910（训练芯片）形成了“云边协同”的搭配。

昇腾 310 并非追求极致算力，而是追求**极致的能效比和通用 AI 能力**。它采用了华为自研的达芬奇架构，在一张芯片上集成了 CPU、DVPP（数字视觉预处理）和 AI 计算核心，使其特别适合处理视频、图像等非结构化数据。早期，它被集成在 Atlas 200/300 系列的加速模块中，用于安防摄像头的视频结构化、智慧交通的车辆识别等场景，初步展现了其在实际部署中的价值。

“310B”可以被理解为昇腾 310 的一个**优化和增强版本**。虽然官方没有大肆宣传其细节改进，但业界普遍认为，B 版本在算力、稳定性和可靠性上进行了提升，以更好地满足严苛的工业和商业环境需求。

1.4.1. 昇腾 310B 技术规格详解

华为昇腾 310B 系列提供了两个不同算力等级的 AI 加速模块，分别是 20T 算力版本跟 8T 算力版本。它们在核心架构、接口和外形上高度统一，主要差异在于性能配置。详细的技术规格如下表所示：

规格项	昇腾 310B (20 TOPS)	昇腾 310B (8 TOPS)
AI 算力	20 TOPS INT8 10 TFLOPS FP16	8 TOPS INT8 4 TFLOPS FP16
内存规格	LPDDR4X 12 / 8 / 4 GB 总带宽 51.2 / 34.1 / 34.1 GB/s 支持 ECC	LPDDR4X 4GB 总带宽 25.6 GB/s 支持 ECC
CPU 算力	4 核 × 1.6 GHz	4 核 × 1.0 GHz
编解码能力	解码: H.264/H.265 硬件解码 40 路 1080P 30FPS 4 路 4K (3840×2160) 75FPS 编码: H.264/H.265 硬件编码 20 路 1080P 30FPS 3 路 4K (3840×2160) 50FPS JPEG: 解码 1080P 512FPS 编码 1080P 256FPS 最大分辨率: 16384×16384	解码: H.264/H.265 硬件解码 20 路 1080P 30FPS 2 路 4K (3840×2160) 75FPS 编码: H.264/H.265 硬件编码 12 路 1080P 30FPS 2 路 4K (3840×2160) 50FPS JPEG: 解码 1080P 512FPS 编码 1080P 256FPS 最大分辨率: 16384×16384
高速接口	8 lane, 支持: SGMII (1.25Gbps) 1000BASE-R (1.25Gbps) USB3.0 (5Gbps) SATA 3.0 (6Gbps) PCIe Gen3 (8Gbps) 向下兼容各接口旧版本	同左
低速接口	UART × 5 I2C × 4 SPI × 2 CAN × 4	同左
典型功耗	25W	21W
工作环境温度	-20°C ~ +103°C	同左
结构尺寸	MXM 连接器 82mm × 60mm × 7mm	同左

20 TOPS 版本在整体性能上全面领先，其 AI 算力（20 TOPS INT8）和 CPU 频率（1.6GHz）均高于 **8 TOPS 版本**（8 TOPS INT8, 1.0GHz），并配备了更灵活、带宽更高的内存（最高 12GB, 51.2GB/s）。在视频处理能力上，20 TOPS 版本也能支持更多的视频解码与编码路数。

两款产品都集成了丰富的接口并支持宽温工作，确保了在边缘环境下的连接性和可靠性。**8 TOPS 版本**则以其稍低的 21W 功耗和适中的配置，提供了一个更具性价比的解决方案。综上所述，20 TOPS 版本定位为高性能选择，适用于计算密集型任务；而 8 TOPS 版本则面向算力需求适中、对成本更敏感的应用场景。

1.4.2. 其他常见的边缘计算卡的对比

华为昇腾 310B、瑞芯微 Rk3588 与英伟达 Jetson Orin Nano 三者之间的定位各有不同。华为昇腾 310B 是一款专为边缘 AI 推理场景设计的人工智能处理器。它不追求极致的通用算力，而是专注于在特定功耗下提供高效、稳定的 AI 推理能力，是华为全栈 AI 解决方案在边缘侧的关键载体，强调自主可控与场景落地。瑞芯微 RK3588 是瑞芯微的旗舰级的高端系统级芯片，其定位是一个“全能战士”。它集成了强大的 CPU、GPU、NPU 以及顶尖的多媒体处理单元，旨在为各类 AIoT 设备提供一个高度集成、高性价比的“大脑”，核心优势在于多功能集成与极致性价比。Jetson Orin Nano 是英伟达面向入门级边缘 AI 市场推出的嵌入式系统模块。它继承了英伟达在 GPU 计算领域的绝对优势，在小型封装内提供了惊人的 AI 算力，其核心价值在于强大的性能和无与伦比的成熟软件生态，是 AI 开发者的首选平台之一。它们之间的详细性能参数对比如下表所示：

规格类别	昇腾 310B (20T)	Rockchip RK3588	Jetson Orin Nano 8GB
AI 性能	20 TOPS INT8 10 TFLOPS FP16	6 TOPS	67 TOPS
CPU 配置	4 核 ARM × 1.6 GHz	4 核 Cortex-A76 @2.4GHz 4 核 Cortex-A55 @1.8GHz	6 核 Arm Cortex-A78AE @1.7GHz
GPU 配置	-	Mali-G610 MP4 @850MHz	32 个 Tensor Core 的 1024 核 NVIDIA Ampere 架构 GPU
内存	LPDDR4X 12/8/4GB	LPDDR4/LPDDR4x 最大 16GB	LPDDR5 8GB
视频解码	H.265/H.264 8 路 4K@30fps 或者 40 路 1080p@30fps	H.265 8K@60fps 或者 AV1 4K@60fps 或者 VP9 4K@120fps	H.265 4K@60fps 或者 2×4K@30fps 或者 5×1080p@60fps
视频编码	H.265/H.264 3 路 4K@50fps 或者 20 路 1080p@30fps	H.265 4K@60fps 或者 H.264 1080p@60fps	1080p@30fps (CPU 软件编码)
存储接口	eMMC 5.1 SD 3.0 NVMe	eMMC 5.1 SD 3.0 NVMe	支持外部 NVMe
PCIe 接口	PCIe Gen3	PCIe 3.0	PCIe 3.0 (1×4 + 3×1)

规格类别	昇腾 310B (20T)	Rockchip RK3588	Jetson Orin Nano 8GB
USB 接口	USB 3.0	USB 2.0/3.0/3.1	3×USB 3.2 (10Gbps) 3×USB 2.0
网络接口	支持 1GbE/10GbE	支持 1GbE	1×GbE
摄像头接口	MIPI-CSI x2	MIPI-CSI ×4	8 通道 MIPI CSI-2 最多 4 个摄像头
显示输出	HDMI x2	HDMI 2.1/eDP/LVDS 最多 4 路显示输出 8K 分辨率	DP 1.2/HDMI 1.4 4K@30fps
其他 I/O	UART×5, I2C×4 SPI×2, CAN×4	I2S/PCM/SPDIF	UART×3, SPI×2 I2S×2, I2C×4 CAN×1, PWM, GPIO
典型功耗	25W	≤ 15W	7-10W
工作温度	-20°C ~+103°C	工业级温控支持	-
封装尺寸	MXM 82×60×7mm	BGA 27×27mm	69.6×45mm SO-DIMM 连接器
工艺制程	12nm	8nm	-

在 AI 性能方面，Jetson Orin Nano 凭借其基于 Ampere 架构的 GPU，提供了三者中最强的 AI 算力；昇腾 310B 则凭借其专用 AI 加速器，提供了坚实的中等算力；而 RK3588 集成的 NPU 则提供了相对基础的 AI 加速能力。

视频处理能力上，三者各有侧重：RK3588 拥有最强的编解码能力，原生支持 8K 视频；昇腾 310B 的优势在于强大的多路视频并发处理能力；而 Jetson Orin Nano 虽然视频解码能力不俗，但其编码任务则需要更多地依赖 CPU 进行。

从功耗效率来看，Jetson Orin Nano 的功耗控制最为出色，展现了优异的能效比；RK3588 则处于中等功耗水平；相比之下，昇腾 310B 为了提供更高的算力和视频路数，功耗也相对较高。

综合来看，这三款平台因其不同的技术特点，各自面向的应用场景也有所区分：昇腾 310B 更适合工业视觉检测和多路视频分析；RK3588 在高清多媒体播放、工业平板及显示设备中表现出色；而 Jetson Orin Nano 则主要瞄准高性能 AI 推理、边缘计算和机器人等前沿领域。

1.5. 实践任务

1. 基于你的目标场景输出一份协同模式决策表（含放弃理由）。
2. 编写数据生命周期图（ASCII 或 Mermaid）。
3. 实现一个队列背压示例：当处理时延 > 阈值时自动丢弃旧帧。
4. 采集 10 分钟温度与时延数据，绘制相关性（是否热导致抖动）。
5. 设计一份故障降级矩阵并评审可行性。

2. CANN 软件栈核心组件解析

本章系统阐述 Ascend CANN 软件栈的分层结构、模型从框架格式到 OM 的转换原理、转换工具 ATC 的关键参数、OM 文件组织结构、AscendCL (ACL) 推理编程模型、精度与性能验证方法以及工程级质量保障流水线建设。

2.1. CANN 异构计算架构

昇腾 AI 异构计算架构 CANN (Compute Architecture for Neural Networks) 是面向基于达芬奇架构的昇腾处理器的全栈软件体系。作为承接主流 AI 框架与通用模型格式、向下对接异构硬件与运行时的中枢层，CANN 构建了覆盖模型表示、转换编译、图优化、调度执行与可观测分析的端到端技术链路，实现语义一致性、性能最优化与诊断可控性的协同统一，从而系统地释放昇腾处理器的算力与能效潜力。

2.1.1. CANN 异构计算与人工智能

异构计算与人工智能是相互促进的关系。深度学习训练与推理负载由高密度线性代数 (向量/矩阵乘)、张量算子融合链、非规整控制流 (条件/循环)、以及高带宽低延迟的数据搬移共同构成，表现出算子类型多样性、数据复用模式差异与访存/算力不均衡等特征。单一通用处理器在算力利用率、内存层次效率与能效上难以同时达到最优。针对边缘计算场景的典型 SoC (如昇腾 310B、RK3588、Jetson Nano 等) 普遍采用低功耗约束下设计的 ARM 通用核，其单核与整体算力在受限功耗窗口内更偏向控制与轻量任务，难以在纯 CPU 路径上满足大规模张量运算所需的高吞吐与低时延深度学习推理需求；因此需要通过集成专用 NPU/加速器进行算子下沉与数据流协同，以弥补通用核在向量化宽度、片上带宽与能效曲线上的结构性不足。异构体系通过在同一系统中组合通用 CPU 与专用 NPU，辅以分层内存与高吞吐互连，依据算子粒度、数据局部性与精度需求执行跨设备调度：密集算子下沉至矩阵/张量专用单元，控制与调度逻辑保留在通用核，数据流经由优化的流水与对齐策略减少跨域搬运成本。结果是在单位功耗与芯片面积约束下提升有效吞吐、降低端到端推理时延并改进能效曲线的可扩展性。该协同范

式构成 CANN 设计的理论基础：通过抽象统一语义表示与针对性后端优化，将模型图中不同类别算子映射到最合适执行单元，实现性能、能效与可诊断性三者的统筹平衡。

2.1.2. CANN 与 CUDA

CANN 是面向华为昇腾达芬奇架构昇腾处理器，聚焦深度学习推理与训练的算子图优化、编译调度及可观测性闭环，而 CUDA (Compute Unified Device Architecture) 则是针对 NVIDIA GPU 的通用并行计算平台与编程模型，覆盖 GPGPU 与 AI 复合场景。二者均体现“软硬件协同加速”范式，但在抽象层次、硬件耦合与生态策略上形成差异化路径。CANN 与 CUDA 二者的共同点主要体现在，它们都是通过软硬件协同，把模型或并行程序转化为底层设备能理解的指令，从而提升运行速度和效率。同时这两种计算平台都提供了内存管理、算子或内核调用、性能分析、调试和日志等工具，帮助开发者更方便地开发和诊断问题。它们都支持自定义算子或内核插件，方便针对特定需求进行优化或替换实现。此外，CANN 与 CUDA 都能通过性能分析、数据导出和分级日志等方式，降低了定位性能瓶颈和对齐精度的难度。在设计理念上，CANN 与 CUDA 展现出显著的差异。CUDA 作为面向广义并行计算与 HPC/图形及 AI 统一生态的平台，强调线程块与网格 (Block/Grid) 以及 SIMT 并行范式，开发者需显式组织核函数、流 (Stream)、事件 (Event) 和内存管理，实现灵活的并行与资源重叠。而 CANN 则专注于模型图语义，突出图级算子融合、AIPP 预处理下沉和全局内存复用，通过任务列表与算子调度策略有效治理动态形状与内存复用，降低推理工程的不确定性。在精度策略方面，CANN 内置 FP16 与 INT8 的混合精度及校准链路，适应端到端推理与训练的工程质量要求；而 CUDA 虽然基础类型丰富，但精度管理更多依赖于上层库如 cuDNN 和 TensorRT 的组合。硬件结构上，CANN 紧密耦合于达芬奇架构的矩阵单元、向量单元及片上分层内存，优化模型推理的能效与性能；CUDA 则绑定于 NVIDIA 的 SM、Tensor Core 及多级缓存体系，着重提升访存效率与线程调度。生态成熟度上，CUDA 自 2007 年以来积累了庞大的社区与库支持，形成了全球主流的开发者生态；CANN 虽处于快速迭代阶段，但聚焦国产化替代与特定行业应用，推动本地生态建设。编程灵活度方面，CUDA 需要开发者具备底层并行划分与核函数开发能力，提供极高的灵活性；CANN 则通过贴近主流框架的算子开发工具 (TBE) 与高层接口，降低了入门门槛，但在底层调度上牺牲了一定的自由度。整体而言，二者在抽象层次、硬件耦合与生态策略上形成了各自鲜明的技术路径，反映了其服务对象与应用场景的根本差异。

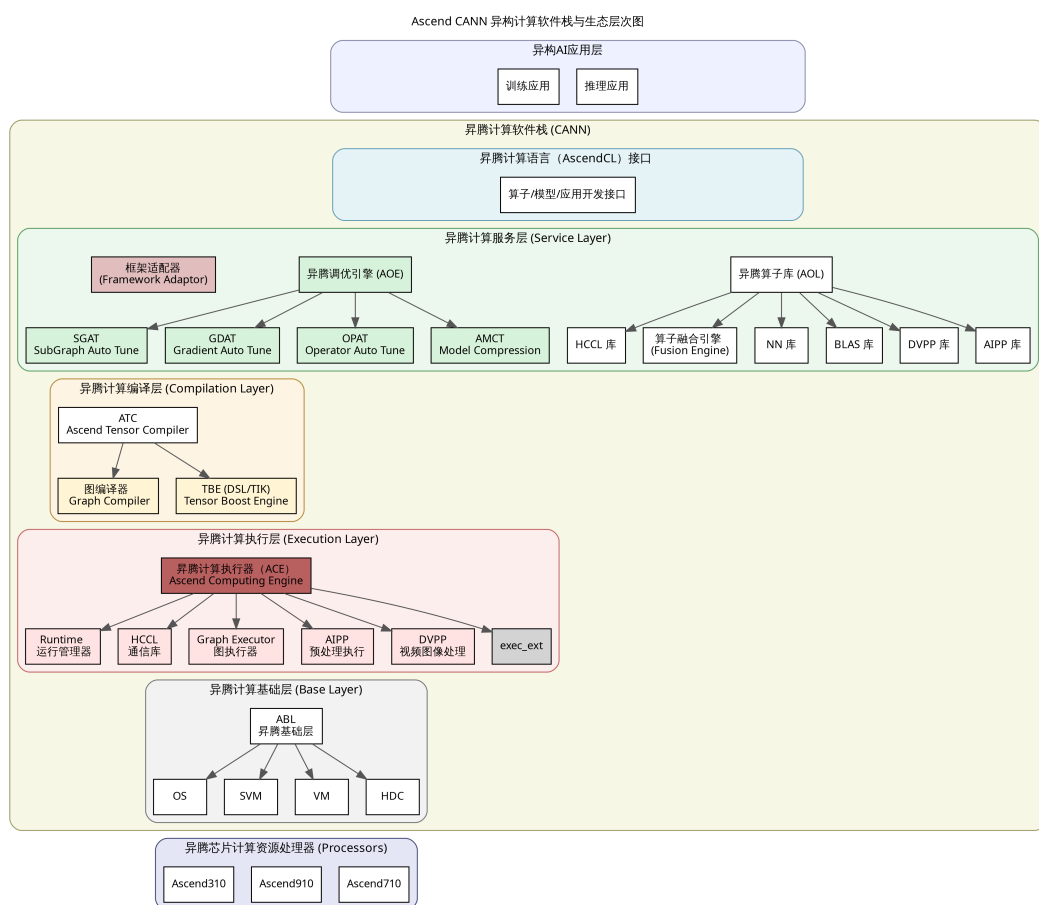


Figure 2.1.: CANN 架构图

2.1.3. CANN 的架构

CANN 通过提供分层清晰的编程与运行时结构，以覆盖训练与推理的“全场景”、贴近主流框架的“低门槛”以及面向昇腾硬件深度优化的“高性能”为核心特性，支撑用户在昇腾平台上快速构建与部署各类 AI 应用与业务。从整体上看，CANN 可以抽象为一个自上而下递进的五层架构（计算语言接口、计算服务层、计算编译引擎、计算执行引擎与计算基础层），如图2.1所示。各层通过稳定的接口与数据流协议进行解耦协同，在保证语义一致性的前提下最大化发挥底层算力与能效潜力。

在分层结构上，CANN 可抽象为五个层次：上层的计算语言接口——AscendCL 接口负责设备与上下文管理、流与内存控制、模型与算子的加载执行，以及媒体与图管理等通用 API，为应用提供稳定的编程入口；其下的昇腾计算服务层汇聚神经网络与线性代数库，承载算子与子图的自动调优、梯度优化与模型压缩，并通过框架适配器降低迁

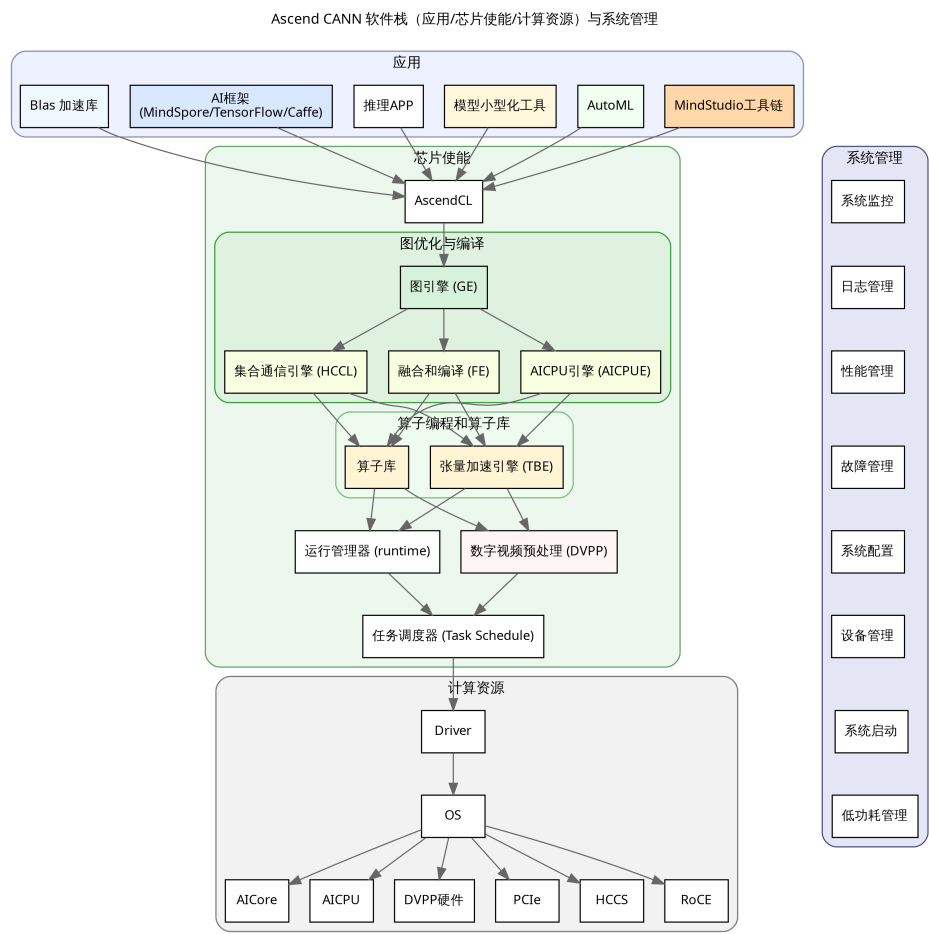


Figure 2.2.: CANN 逻辑图

移成本；昇腾计算编译层的编译引擎通过图编译器与 TBE 算子开发支持，把前端计算图转化为在 NPU 可执行的模型与内核，实现图级语义到后端实现的精准映射；下一层的昇腾执行引擎面向运行时，负责模型与算子的调度执行，并内置数字视觉与 AI 预处理、集合通信等能力以提升端到端效率；最下层的昇腾计算基础层提供共享虚拟内存、设备虚拟化与主机—设备通信等底座服务，保证跨设备数据流与资源管理的可靠性与可扩展性。

与此相辅的是三层逻辑架构：应用层承载具体业务与开发者工具，芯片使能层开放解决方案能力并驱动基于计算图的业务流运行，计算资源层则聚焦数据处理与运算执行，形成从业务到硬件的清晰闭环。这个 CANN 的三层逻辑结构如下图2.2所示：

在技术特性上，CANN 通过对计算图的编译与优化，将密集算子与数据流合理分配

到异构单元上执行，显著提升吞吐与时延表现，并在能效上取得可工程化的优势；同时提供贴近主流框架的易用接口与完善的工具链，降低入门与迁移门槛，使开发者能够快速完成部署与调优。生态方面，CANN 构建了面向个人、高校、科研与企业的赋能体系，并与开源社区协同，支持多种框架与推理引擎在异构硬件上的高效运行。依托这些能力，开发者可以深入调用运行时与图编译能力，释放底层硬件潜力，在性能与成本维度形成差异化竞争力。

在工程实践中，CANN 的核心价值可以概括为在“模型—编译—执行—诊断”的完整链路上实现软硬件协同与语义稳定：通过构建对主流深度学习框架高度兼容的前端接口，最大限度降低模型迁移与语义对齐成本，在图级优化阶段则依托算子融合、内存复用以及混合精度策略的协同设计，系统性提升吞吐能力、推理时延以及功耗效率；与此同时，借助自定义算子机制与实现优先级调度策略，使新结构能够快速接入并在多实现版本之间灵活切换，从而在不同硬件代际与不同业务场景下充分挖掘底层计算资源的潜力，并在运行时通过分级日志、Profiling 时间线与精度 Dump 等手段构建起闭环的可观测体系。在大规模、复杂模型的工程化路径中，首先需要完成面向硬件架构的感知建模，在导出阶段显式固化张量形状与算子语义，为后续编译与部署奠定稳定的语义基础；随后在模型转换阶段，以“转换即优化”为原则，围绕 precision_mode、op_select_implmode 与 AIPP 等关键配置项进行显式调优，使图优化与算子选型在编译期前置完成，其中精度策略以 FP16 为主，并辅以具有代表性的校准数据集对 INT8 量化误差进行严格控制，以在性能与精度之间取得可工程化的平衡。对于动态形状问题，可以采用分桶与 Padding 结合的方式，并在必要时生成多份 OM 模型，以在一定范围内换取更可控的峰值资源占用与时延方差；在内存与带宽层面，则通过将预处理逻辑尽可能下沉到设备侧、合并数据搬移操作，并配合 Pinned 内存与多 Stream 并行传输技术，显著降低 H2D/D2H 的时间占比，从系统视角优化整体流水线的调度效率。针对在实际 Profiling 中暴露出的性能瓶颈算子，需要开展有针对性的算子重写与图重构，并充分利用实现优先级机制在不同 Kernel 之间进行策略化选择，以进一步榨取底层硬件的计算与访存潜力；最终，通过围绕“转换—精度对齐—Benchmark—回归监测”构建自动化流水线，将模型转换、性能评估与精度验证纳入统一的持续反馈闭环，在 Ascend 310B 等专用加速平台上沉淀出可复用的优化经验与差异化性能优势，不仅在算力利用率与综合成本上形成工程级竞争力，也为大模型与行业应用场景的规模化加速落地提供坚实的基础设施支撑。

2.1.4. CANN 的快速安装

本节给出 CANN 软件的极速安装示例。若需更完整的操作指导，请按实际环境选择对应安装场景并参考[官方说明](#)，推荐优先使用离线安装。

安装前准备

- 驱动与固件检查：在目标设备上执行 `npu-smi info`，如果能够正常显示设备信息，则表明 NPU 驱动和固件已经正确安装。对于昇腾 310B 产品或开发板，系统一般会预装所需的驱动和固件，无需额外手动配置。
- 环境准备：推荐采用离线安装方式，安装前需确保系统已准备好 Python 环境并可使用 `pip3`，当前 CANN 支持 Python 3.7.x 至 3.11.4 版本。对于昇腾 310B 开发板，通常已预装 Miniconda 和 `pip3`，可直接满足依赖，无需额外配置。
- 安装介质获取：在安装前，需先从华为昇腾官网下载所需的 CANN 软件包，并将其上传到目标设备的可访问路径。以 CANN 8.3.RC1 版本为例，可通过[官网](#)获取相应安装包，其他版本可在页面切换选择。请确保下载的 CANN Toolkit 和 CANN Kernels 版本号完全一致（即同一发行号），以避免兼容性问题。推荐优先选择.run 格式的安装包，且昇腾 310B 开发板为 AArch64 架构，需下载 aarch64 对应的包。例如 8.3.RC1 版本包括 `Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run` 和 `Ascend-cann-kernels-310b_8.3.RC1_linux-aarch64.run`。如果实际下载的版本号不同，请将文件名中的版本号替换为对应的实际版本。

安装命令与依赖说明

安装 CANN 可使用 root 或默认账户 HwHiAiUser。推荐以 root 安装，不推荐用其他用户进行安装，因为容易发生权限问题。下面以 root 用户为例，详细介绍 CANN 的安装过程：

- 切换到 root：

```
1 su -
2 # 输入 root 密码后继续执行安装
```

- 命令示例，以 root 执行：

```
1 ./Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run --install
```

- 安装 Toolkit 开发套件：

```
1 chmod +x Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run
2 ./Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run --install
```

整个安装过程大概会持续十几分钟，甚至更长，请耐心等待。

- 配置环境变量：

```
1 source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

- 安装 Kernels 算子包（310B 平台）：

```
1 chmod +x Ascend-cann-kernels-310b_8.3.RC1_linux-aarch64.run
2 ./Ascend-cann-kernels-310b_8.3.RC1_linux-aarch64.run --install
```

- 安装业务运行时所需的 Python 第三方库:

```
1 pip3 install attrs cython 'numpy>=1.19.2,<=1.24.0' \
2     decorator sympy cffi pyyaml pathlib2 \
3     psutil protobuf==3.20.0 scipy requests absl-py
```

- 说明: 依赖的版本范围已经覆盖了大多数常见环境, 如果遇到依赖冲突, 请根据实际的 Python 或操作系统版本进行相应调整。安装完成后, 建议将 CANN 的环境变量配置追加到 ~/.bashrc 文件中, 以便每次登录时自动加载。需要注意的是, 昇腾 310B 开发板的系统镜像通常已经预置了这些环境变量配置, 无需手动设置。

2.2. ATC 模型转换详解

2.2.1. ATC 工具介绍

昇腾张量编译器 (Ascend Tensor Compiler, ATC) 是 CANN 异构计算体系中的模型转换组件, 支持将主流开源框架导出的网络模型以及基于 Ascend IR 的单算子描述文件 (JSON) 编译为昇腾 AI 处理器可执行的离线模型 (.om)。如图下图所示, ATC 在转换过程中会执行算子调度优化、权重重排与内存复用等关键步骤, 对原始深度学习模型进行面向部署场景的系统化调优, 从而在昇腾硬件上实现高吞吐、低时延的高效执行。

从上面的流程图我们可以看到, ATC 工具聚焦模型到设备可执行体的“转换即优化”闭环: 一方面, 它能够将开源框架导出的网络模型解析为中间态 IR Graph, 经由图准备、拆分与融合、形状推理、内存复用与算子选型等编译步骤, 生成适配昇腾处理器的离线模型 (OM), 并在板端通过 AscendCL 接口加载执行, 从语义到性能实现端到端落地; 另一方面, ATC 也支持基于 Ascend IR 的单算子 JSON 直接编译为离线 OM, 用于在设备侧进行算子级功能验证与精度对齐, 帮助开发者快速定位与迭代关键算子实现, 在图级与算子级两条路径上形成统一的工程化编译能力。

onnx 格式介绍

ONNX (Open Neural Network Exchange) 是一种针对深度学习所设计的开放式文件格式, 用于存储训练好的模型。它使得不同的人工智能框架 (如 PyTorch、TensorFlow、mindspore 等) 可以采用相同格式存储模型数据并交互。ONNX 的规范包含计算图模型的定义, 以及内置运算符的定义和标准数据类型。

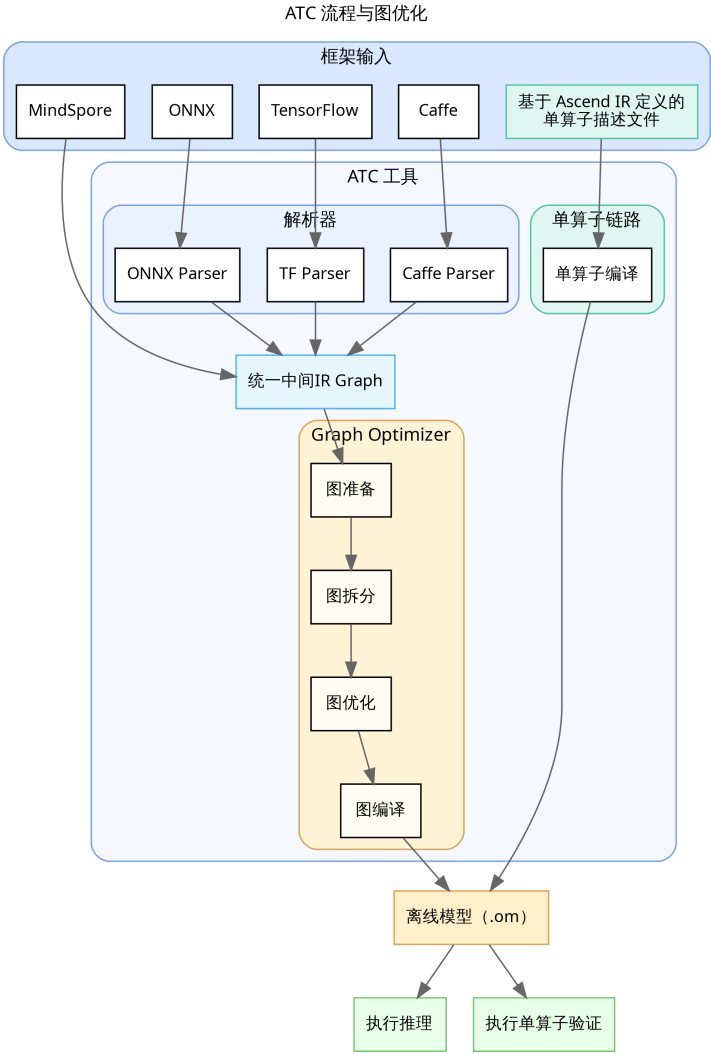


Figure 2.3.: ATC 框架图

ONNX 的核心价值在于解决了 AI 生态系统中“碎片化”的问题。在没有 ONNX 之前, 开发者如果想将一个在 PyTorch 中训练好的模型部署到移动端推理引擎上, 通常需要进行繁琐的代码重写或复杂的格式转换。ONNX 提供了一种中间表达 (Intermediate Representation, IR), 充当了不同框架之间的“通用语”。

一个标准的 ONNX 模型文件 (通常以 .onnx 为后缀) 主要包含以下几个部分:

1. **Model Proto**: 顶层结构, 包含模型的元数据 (如版本信息、生产者信息) 和计算图 (Graph)。
2. **Graph Proto**: 描述了模型的计算逻辑, 由一系列节点 (Node)、输入 (Input)、输出 (Output) 和初始化器 (Initializer, 即权重数据) 组成。
3. **Node Proto**: 代表计算图中的一个操作 (Operator), 如 Conv、Relu、Add 等。每个节点包含操作类型、输入输出张量的名称以及属性 (Attribute, 如卷积的步长、填充等)。
4. **Tensor Proto**: 用于存储权重数据或常量的具体数值和形状信息。

ATC 对 ONNX 的支持情况虽然 ONNX 旨在成为通用的 AI 模型交换标准, 但 CANN 的 ATC 工具并非支持 ONNX 规范中的所有算子和特性。ATC 对 ONNX 的支持主要集中在 CV (计算机视觉) 和 NLP (自然语言处理) 领域的常用算子, 对于一些较新或较冷门的算子, 可能会出现不支持的情况。

具体来说, ATC 对 ONNX 的支持限制主要体现在以下几个方面:

1. **算子覆盖率**: ATC 支持绝大多数主流的 CNN 和 Transformer 类算子 (如 Conv, MatMul, Relu, Softmax 等)。但对于部分非标准或特定框架自定义的算子 (Custom Ops), ATC 无法直接解析, 需要开发者通过 TBE (Tensor Boost Engine) 开发自定义算子并注册到 CANN 中。
2. **动态控制流**: ONNX 中的动态控制流 (如 If, Loop, Scan) 在静态图编译中处理较为复杂。虽然 ATC 支持部分控制流算子, 但在某些复杂嵌套场景下可能会导致编译失败或性能下降, 通常建议在导出模型时尽量将控制流展开 (Unroll) 或转为静态图结构。
3. **数据类型限制**: 虽然 ONNX 支持多种数据类型 (如 Double, Int64 等), 但昇腾 AI 处理器主要针对 FP16 和 INT8 进行了硬件加速优化。对于 FP64 (Double) 类型, ATC 通常不支持或需要强制转为 FP32/FP16 处理; 对于 Int64 类型的索引或计数, 部分算子可能要求转为 Int32。
4. **动态 Shape 支持**: 虽然 ATC 支持动态 Batch 和动态分辨率, 但对于模型内部中间层出现的完全动态且无法推导的 Shape, 可能会导致编译报错。

因此, 在进行模型转换前, 建议开发者查阅华为昇腾社区发布的《CANN 算子清单》, 确认模型中使用的 ONNX 算子版本 (Opset Version) 是否在当前 CANN 版本的支持范围内。如果遇到不支持的算子, 通常的解决路径包括: 修改模型结构以替换为等

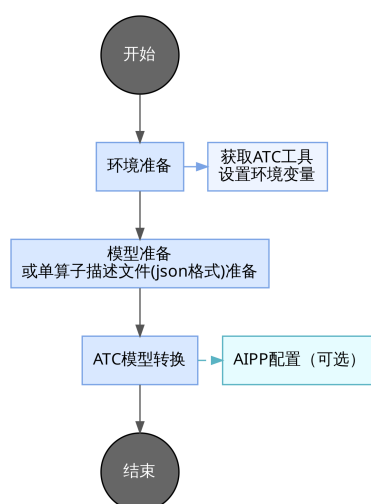


Figure 2.4.: ATC 流程图

价的支持算子、在 ONNX 导出阶段进行算子简化（使用onnx-simplifier工具）、或者开发自定义算子。

2.2.2. ATC 工具使用流程

使用 ATC 工具进行模型转换的使用流程如下图所示：

在开始模型转换之前，首先需要在开发环境中安装与 CANN 软件包版本相匹配的版本，并确保 ATC 可执行文件的路径可用。接下来，准备待转换的模型文件或基于 Ascend IR 的单算子 JSON，并将其上传到开发环境中可访问的目录。最后，使用 ATC 执行转换，并根据实际业务需求和输入规范配置相关参数。如果需要将预处理步骤（如色彩空间转换、归一化和尺度调整）下沉到设备侧，可以同时提供并启用 AIPP（Artificial Intelligence Pre-Processing）配置。AIPP 是昇腾处理器内置的硬件级图像预处理模块，负责将上游（如 DVPP）输出的对齐后 YUV420SP 图像在设备侧完成色域转换（如 YUV 图像格式转换为 RGB 或者 BGR 图像格式）、归一化（减均值/乘系数/尺度放大）与抠图（在指定起始点裁剪到模型输入尺寸），从而把原始图像规范化为模型所需的输入格式与数值范围。由于昇腾 310B 在推理及训练中常以 DVPP 输出 YUV420SP 图片格式，这种图像格式是有损图像颜色编码格式，常用为 YUV420SP_UV、YUV420SP_VU 两种格式，不直接提供 RGB 图片。AIPP 能将 YUV420SP 类型图像无缝转化为模型期望的 RGB/BGR 图像格式，并在同一数据流中完成裁剪与数值处理，避免将预处理放在 CPU 侧导致的多次拷贝与额外时延，提升端到端吞吐与能效。

模型转换-以 ResNet50 为例

ResNet-50 由多级残差单元堆叠形成 50 层卷积网络,通过跨层跳连缓解深层退化与梯度消失,使其在 ImageNet 等大规模数据集上具备稳定的特征表达与分类精度,因此常被选作各类视觉任务的通用骨干。要在昇腾 310B 上部署该模型,需要借助 ATC 将 ONNX 版本转换为 OM,可参考华为昇腾官方示例仓库(<https://github.com/Ascend/samples/tree/master/inference/1>)为方便快速复现,可按以下步骤从零搭建工程:

1. 在昇腾 310B 的 Documents 目录创建 sample_resnet_quick_start,并划分 data、model、out、src 四个子目录用于存放测试图片、模型、脚本与源码:

```
1 cd ~/Documents
2 mkdir -p sample_resnet_quick_start/{data,model,out,src}
3 cd sample_resnet_quick_start
```

2. 下载示例图片至 data 目录:

```
1 cd ~/Documents/sample_resnet_quick_start/data
2 wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/aclsample/
   ↪ dog1_1024_683.jpg
```

3. 进入 model 目录获取 ResNet-50 ONNX 模型:

```
1 cd ~/Documents/sample_resnet_quick_start/model
2 wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/003_Atc_Models/
   ↪ resnet50/resnet50.onnx
```

4. 设置编译并行度后执行典型 ATC 命令,生成适配昇腾 310B4 的 OM 文件:

```
1 export TE_PARALLEL_COMPILER=1
2 export MAX_COMPILE_CORE_NUMBER=1
3 atc \
4   --model=resnet50.onnx \
5   --framework=5 \
6   --output=resnet50 \
7   --input_shape="actual_input_1:1,3,224,224" \
8   --soc_version=Ascend310B4
```

关键参数说明

参数	作用	注意事项
--framework	指定原始模型框架	0=Caffe, 1=MindSpore, 3=TensorFlow, 5=ONNX, 需与导出框架一致
--input_shape	定义静态输入 Shape	按 "name:n,c,h,w" 写法, 字符串需加引号; 动态改用区间或配合动态参数

参数	作用	注意事项
-- ↪ dynamic_batch_size ↪	设置可选 Batch 列表	以 "1,4,8" 形式填写；与同一输入的全静态 shape 互斥
-- ↪ dynamic_image_size ↪	配置多分辨率输入	"h1,w1;h2,w2"，常用于多尺度检测；需与模型实际支持一致
--precision_mode	控制混合精度策略	常用值：force_fp16、allow_mix_precision、allow_fp32_to_fp16，需兼顾精度
--soc_version	选择目标芯片	例：Ascend310B4，可通过 npu-smi info 查询；310B/P 区分清楚
--insert_op_conf	下沉 AIPP/自定义算子	指定 JSON/YAML，支持色域转换、归一化等预处理
-- ↪ op_select_implmode ↪	算子实现优先级	支持 high_performance(默认)、high_precision 等，可配合 --optype_list_for_implmode
--input_format	声明输入数据排布	如 NCHW、NHWC；需与模型导出及 --input_shape 保持一致
--output_type	重指定输出 dtype	可设全局 FP16/FP32，或按 node:idx:FP32 精细配置，便于后处理
-- ↪ enable_small_channel ↪	小通道优化开关	取值 0/1，轻量网络或低通道数场景启用可减时延
--model	指定待转换模型文件	支持 .onnx、.pb、.prototxt 等类型，路径需可访问
--output	设置 OM 输出前缀	不需写后缀；默认位置在当前目录，可结合 --output_type 使用
--dynamic_dims	定义多维动态组合	"d1_1,d1_2;d2_1,d2_2" 格式；用于多输入或非 Batch 维变动
-- ↪ enable_single_stream ↪	强制单流执行	true/false；在推理序列化或调试稳定性时启用

参数	作用	注意事项
<code>--dump_mode</code>	导出模型结构 JSON	配合 <code>--mode=1</code> 使用，1 开启；便于排查 Shape 与算子信息

如果对于参数有任何的疑惑，可以通过命令查询 `atc` 的参数作用：

```
1 atc --help
```

成果转换模型后，我们可以看到命令窗口有如下输出：

```
1 ATC start working now, please wait for a moment.
2 ...
3 ATC run success, welcome to the next use.
```

注意事项

1. 在昇腾 310B 上将 ONNX 模型转换为 OM 时若出现 `BrokenPipeError: [Errno 32] Broken pipe`，通常是编译阶段内存不足导致进程被系统终止。可参照以下两类方案缓解：

- 扩展可用内存通过创建 Swap 交换文件临时扩充虚拟内存，以缓解编译期间的内存压力：

```
1 dd if=/dev/zero of=/swapfile bs=1M count=8192
2 chmod 600 /swapfile
3 mkswap /swapfile
4 swapon /swapfile
```

若需永久生效，请在 `/etc/fstab` 文件中追加以下配置：

```
1 /swapfile swap swap defaults 0 0
```

注意：如果设备使用 TF (MicroSD) 卡作为主要存储介质，不建议在 TF 卡上频繁使用 Swap。Swap 分区涉及大量高频读写操作，会极大加速闪存磨损，导致 TF 卡寿命缩短或损坏。建议仅在必要时临时开启，或使用 SSD 等更耐用的存储介质。

- 降低编译资源占用

```
1 export TE_PARALLEL_COMPILER=1 # 限制算子最大并行编译进程数
2 export MAX_COMPILE_CORE_NUMBER=1 # 限制图编译占用的 CPU 核数
```

方便起见，我们可以采用第二种方案，可以有效解决内存不足的问题。

2. 若无法确认当前设备的 `soc_version`，可在安装驱动的昇腾 310B 上执行 `npu-smi info` 查询，示例如下：

```

1 npu-smi 23.0.0 Version: 23.0.0
2 +-----+-----+-----+-----+-----+
3 | NPU Name | Health | Power | Temp | Hugepages-Usage |
4 | Chip    | Bus-Id | AICore | Mem  |                  |
5 +-----+-----+-----+-----+-----+
6 | 0 310B4 | Alarm  | 0.0W  | 58C  | 15 / 15          |
7 | 0 0     | NA     | 0%    | 2500/7545MB |                  |
8 +-----+-----+-----+-----+-----+

```

请在查询结果的 Name 值前追加 Ascend 前缀:若 Name 为 310B4,则需配置 soc_version

→ =Ascend310B4。

- 模型转换完成后会生成一份 JSON 文件,用作 ATC 编译日志的结构化摘要,集中记录当前会话 (session_and_graph_id=0_0) 内图级与 UB 级融合规则的触发统计。具体而言, graph_fusion 字段逐条给出各图融合 Pass 的匹配次数与实际生效次数 (match_times 与 effect_times), 可据此识别潜在的优化空档; ub_fusion 字段在上述统计基础上新增 repository_hit_times, 用以衡量算子库模板的命中情况。实践中, 可重点关注 effect_times 低于 match_times 的 Pass, 分析是否因算子属性、精度模式或实现优先级等因素造成融合未落地, 并据此调整 ATC 参数或模型图结构, 以复现并提升性能优化效果。

2.2.3. 模型推理工具 msame 工具概览

在完成 ATC 转换得到 .om 模型后, 最快验证部署链路是否畅通的方式就是调用华为昇腾提供的模型推理工具 msame。该工具封装了 OM 加载、设备内存初始化、输入二进制数据映射以及推理调度等通用流程, 无需额外编写 AscendCL 程序即可直接对接 ATC 输出的模型, 只需准备好与模型输入规格匹配的 .bin 文件和基础运行时环境, 即可在命令行下完成端到端推理验证。通过 msame, 开发者能够快速检查模型是否成功落地、输出是否合理, 并以最小成本排除输入格式或环境配置类问题, 为后续集成到自研应用或高层框架之前提供低门槛的功能性回归手段。

msame 的设计目标是将 OM 模型的快速验证能力标准化, 覆盖单输入、多输入和循环执行等常见推理形态, 支持在相同模型与不同输入数据组合之间做对比试验, 同时允许选择 TXT 或 BIN 格式导出结果, 以便于人工审查或自动化脚本解析。在已正确安装 CANN Runtime 的环境中, msame 会自动完成与设备的上下文绑定、内存申请和任务提交, 将推理过程抽象成简单的命令行参数; 这不仅适用于初步确认模型可运行, 也可用于小批量性能评估、数据一致性校验以及在定位性能或精度问题时的基准对照, 从而在模型转换与实际业务部署之间搭建起轻量而高效的桥梁。

msame 工具的获取与构建

1. 通过 `git clone https://gitee.com/ascend/tools.git` 拉取仓库，或下载 ZIP 包后解压到开发环境。
2. 进入 msame 子目录：

```
1 cd ./tools/msame
```

3. 配置 CANN 环境变量（以下路径请按实际安装位置调整）：

```
1 export DDK_PATH=/usr/local/Ascend/ascend-toolkit/latest
2 export NPU_HOST_LIB=/usr/local/Ascend/ascend-toolkit/latest/runtime/lib64
   ↪ /stub
```

4. 运行构建脚本生成可执行文件：

```
1 # 若脚本缺少执行权限，请先执行 chmod +x build.sh
2 ./build.sh g++
```

如果在执行时遇到 `Permission denied` 报错，说明脚本文件缺少可执行权限，需先运行 `chmod +x build.sh` 进行赋权，随后再次执行编译命令。

5. 构建成功后会在 `out` 目录生成二进制文件 `msame`，将其复制到工程目录 `sample_resnet_quick_start` 中备用。

- 常用参数速览

参数	说明
<code>--model</code>	OM 文件路径
<code>--input</code>	输入 <code>bin</code> 文件或目录，支持多输入
<code>--output</code>	推理结果存放目录
<code>--outfmt</code>	输出格式：TXT / BIN
<code>--loop</code>	重复推理次数（1-100）
<code>--profiler / --dump</code>	性能分析或 Dump，互斥
<code>--device</code>	设备 ID，默认 0
<code>--dymBatch / --dymHW / --dymDims /</code>	对应 ATC 动态配置的实际取值
<code>--dymShape</code>	
<code>--outputSize</code>	动态 Shape 场景下手动申请输出内存
<code>--debug</code>	打印模型描述信息
<code>--help</code>	查看全部参数

- 使用注意运行 `msame` 工具时，需确保当前账号拥有目录的执行和写入权限。对于部分动态 Shape 场景，建议选择 BIN 格式输出而非 TXT，以保证兼容性。



Figure 2.5.: resnet 测试图片

配置 `NPU_HOST_LIB` 时应指向 `runtime/lib64/stub` 目录，并在运行阶段通过 `LD_LIBRARY_PATH` 正确链接所需依赖库。此外，`profiler` 和 `dump` 功能不可同时启用；如涉及动态 `Shape`，需同步设置合适的输出内存大小以确保推理顺利进行。

图片预处理

由于 `msame` 工具不具备图像预处理能力，输入必须是已转换好的 `.bin` 文件。因为之前在模型转换时未显式指定精度，`ResNet50` 默认接受 `FP32` 格式的输入；同样地，未设置输入内存布局时 `ATC` 会采用默认的 `NCHW`。`NCHW` 表示批次 `N`、通道 `C`、空间高度 `H` 与宽度 `W`，意味着同一张图片的三个颜色通道会按通道优先的顺序依次排布，再展开到空间维度。这与 `TensorFlow` 常见的 `NHWC`（通道在最后）相对。`sampleResnetQuickStart` $\rightarrow$ `.py` 中的 `image_rgb.transpose([2, 0, 1])` 正是将 `OpenCV` 读取的 `HWC` 布局转换为 `CHW`，并配合外层批次维得到符合要求的 `NCHW`。是否必须使用 `NCHW` 取决于模型导出时的约定。`PyTorch` 与 `CANN` 默认使用 `NCHW`，因此常见的 `resnet50.onnx` $\rightarrow$ `/resnet50.om` 都在此布局下训练与推理。只要导出模型为 `NCHW`，推理阶段同样必须以 `NCHW` 喂入数据，否则通道错位会导致结果异常。当然，也可以导出为 `NHWC`，只需在模型与预处理两端同时调整即可。由于刚才我们从华为云哪里获取的测试图片“`dog1_1024_683.jpg`”如下：

由于这个图片是 `jpg` 格式的，因此为了可以利用这个图片推理，我们第一步需要对图片进行预处理，具体过程如何：1. 在 `src` 文件目录创建一个预处理的 `python` 文件“`make_bin_resnet224_float32.py`”如下：

```
1 from PIL import Image
2 import numpy as np
3
4 # 加载图像并转换为 RGB
```

```

5 img = Image.open('data/dog1_1024_683.jpg').convert('RGB')
6 # 使用双线性插值缩放到 256x256
7 img = img.resize((256, 256), Image.BILINEAR)
8 # 计算 224x224 中心裁剪的坐标
9 left = (256 - 224) // 2
10 top = (256 - 224) // 2
11 # 执行中心裁剪
12 img = img.crop((left, top, left + 224, top + 224))
13
14 # 转换为 [0, 1] 范围的 float32 数组
15 arr = np.array(img).astype(np.float32) / 255.0
16 # 定义归一化常量
17 mean = np.array([0.485, 0.456, 0.406], dtype=np.float32)
18 std = np.array([0.229, 0.224, 0.225], dtype=np.float32)
19 # 按通道进行归一化
20 arr = (arr - mean) / std
21
22 # 调整为 NCHW 格式并添加批次维度
23 arr = arr.transpose(2, 0, 1)[None, :, :, :]
24 # 保存为二进制文件
25 arr.tofile('data/dog1_224_float32.bin')
26 # 打印缓冲区大小 (字节数)
27 print('bytes:', arr.nbytes)

```

2. 在 shell 中运行这个 python 文件:

```
1 python src/make_bin_resnet224_float32.py
```

该脚本在导出 **.bin** 输入前会完成: 将示例图像缩放到 256×256 后再做 224×224 中心裁剪; 把像素转换为 NCHW 排布的 float32 缓冲区并写入文件; 整个过程中先除以 255 将数值归一化到 [0,1], 再按通道减去 [0.485, 0.456, 0.406]、除以 [0.229, 0.224, 0.225] 标准差, 以对齐 ResNet 在 ImageNet 上的训练预处理, 从而避免因均值或方差失配导致的输出偏移, 最终在 data 文件夹内得到 dog1_224_float32.bin。

3. 快速运行——单输入文件运行下面这个命令:

```

1 ./msame --model "./model/resnet50.om" --input "./data/dog1_224_float32.
   ↪ bin" --output "./out" --outfmt BIN

```

运行成功后, 我们会得到以下的信息:

```

1 [INFO] acl init success
2 [INFO] open device 0 success
3 [INFO] create context success
4 [INFO] create stream success
5 [INFO] get run mode success
6 [INFO] load model ./model/resnet50.om success
7 [INFO] create model description success
8 [INFO] get input dynamic gear count success
9 [INFO] create model output success
10 ./out/20251213_11_6_52_564388
11 [INFO] start to process file:./data/dog1_224_float32.bin

```



```

12 [INFO] model execute success
13 Inference time: 6.811ms
14 [INFO] get max dynamic batch size success
15 [INFO] output data success
16 Inference average time: 6.811000 ms
17 [INFO] destroy model input success
18 [INFO] unload model success, model Id is 1
19 [INFO] pid: 98299 Execute sample success
20 [INFO] end to destroy stream
21 [INFO] end to destroy context
22 [INFO] end to reset device is 0
23 [INFO] end to finalize acl

```

- 该日志显示推理流程已完整走通：ACL 初始化、设备上下文、模型加载、输入输出申请均返回 success，Inference time/Inference average time 给出单次与平均耗时，路径 ./out/20251213_11_6_52_564388 指向本次推理结果目录。该输出目录遵循“YYYYMMDD_H_M_S_microsec”命名：20251213 表示日期（2025-12-13），11_6_52 为时分秒，564388 为微秒级序列号，用于区分同日多次推理结果。
- 若选择 --outfmt BIN，可用 `numpy.fromfile('./out/.../resnet50_output_0.bin', dtype=np.float32).reshape(1, 1000)` 读取并执行 `np.argsort` 获取 TopK；使用 softmax 还原概率后结合 ImageNet label 映射即可解读分类结果。
- 若选择 --outfmt TXT，直接 `cat ./out/.../resnet50_output_0.txt | head` 快速查看前若干输出值，同样可以配合脚本解析为 TopK 概率。

4. 后处理

无论 msame 的 --outfmt 选择 BIN 还是 TXT，得到的都是形如 1×1000 的分类 logits，需要结合 ImageNet 标签映射再做一次后处理才能输出可读结果。以下示例脚本演示如何解析 BIN 文件、执行 softmax 并打印 Top-K 概率。

- 先下载 imagenet_class_index.json（可来自 Kaggle 或 GitHub）放到 src 目录，然后创建 src/postprocess_resnet50.py：

```

1 # src/postprocess_resnet50.py
2 import os
3 import argparse
4 import json
5 import numpy as np
6
7 parser = argparse.ArgumentParser(description="将ResNet50的 BIN 输出解码为
8     ↳ ImageNet标签。")
9 parser.add_argument("--bin", required=True, help="msame 输出BIN文件的路
10     ↳ 径。")
11 parser.add_argument("--topk", type=int, default=5, help="需要打印的最高置
12     ↳ 信度数量。")
13 args = parser.parse_args()

```



```

11 logits = np.fromfile(args.bin, dtype=np.float32).reshape(1, -1)
12 probs = np.exp(logits - logits.max(axis=-1, keepdims=True))
13 probs = probs / probs.sum(axis=-1, keepdims=True)
14 probs = probs[0]
15
16 labels_path = os.path.join("src", "imagenet_class_index.json")
17 if not os.path.exists(labels_path):
18     raise FileNotFoundError("请先将imagenet_class_index.json下载到src目
19         ↳ 录。")
20
21 with open(labels_path, "r", encoding="utf-8") as f:
22     data = json.load(f)
23 labels = [data[str(i)][1] for i in range(len(data))]
24
25 topk = np.argsort(probs)[:args.topk]
26 print(f"Decoded {args.bin} (Top-{args.topk})")
27 for rank, idx in enumerate(topk, start=1):
28     label = labels[idx] if idx < len(labels) else f"cls_{idx}"
29     print(f"{rank:>2d}: class={idx:<4d} prob={probs[idx]:.6f} label={
30         ↳ label}")

```

- 执行脚本：

```

1 python ./src/postprocess_resnet50.py --bin out/20251213_11_6_52_564388/
   ↳ resnet50_output_0.bin

```

- 示例输出：

```

1 Decoded out/20251213_11_6_52_564388/resnet50_output_0.bin (Top-5)
2 1: class=162 prob=0.964885 label=beagle
3 2: class=161 prob=0.023230 label=basset
4 3: class=166 prob=0.006107 label=Walker_hound
5 4: class=167 prob=0.004985 label=English_foxhound
6 5: class=164 prob=0.000334 label=bluetick

```

2.3. 章节小结

本章从宏观分层、转换编译、OM 结构、ACL 编程、性能与精度保障、调试工具、自动化流水线到动态 Shape 与安全实践建立了闭环。掌握这些内容后即可进入后续“边缘系统架构与部署实践”章节，扩展到多模型、多进程及系统级优化。

2.4. 实践任务

1. 任选一个公开 ONNX 分类模型（如 ResNet50）完成 ATC 转换，提交：命令 + atc.log。
2. 以 C 或 Python 实现最小推理程序，输出前 5 TopK 结果与 softmax 概率。
3. 编写对齐脚本比较 50 张图片 ONNX vs OM 输出差异（报告 L1/Top1 差异）。
4. 收集 Profiling Timeline，列出前 3 耗时算子类型及优化建议。
5. 输出 signature.json、metrics.json、conversion_meta.yaml 并归档。

第三章

3. 昇腾 PyTorch 扩展迁移基础

PyTorch 是以动态计算图著称的深度学习框架，核心由灵活的张量运算与自动微分系统构成。eager execution 让调试、可视化和原型迭代更直观，而 TorchScript 则提供图模式部署以兼顾性能与可移植性。配合丰富的 TorchVision、TorchAudio、TorchText 等生态包，开发者可以在视觉、语音、自然语言处理等任务上快速搭建端到端方案。

Ascend Extension for PyTorch 是华为昇腾为 PyTorch 用户提供的深度适配插件，使 PyTorch 能无缝调用昇腾 AI 处理器算力，沿用原生接口并针对算子、通信与调度做深度优化。项目源码可在官方仓库获取，更多动态可关注昇腾社区。

虽然当前已有 CANN、MindSpore 等优秀深度学习框架，但在 PyTorch 广泛应用的背景下，先基于 PyTorch 学习并借助昇腾插件迁移至昇腾 310B，既能降低学习门槛，又可减少适配成本、缩短开发周期。

由于 torch_npu 对昇腾 310B 的支持仍有限，适配与迁移存在诸多问题，因此这里只做 PyTorch 在 310B 上的基础示例与简要介绍。

3.1. 架构速览

整体架构可概括为“PyTorch 前端 + torch_npu 中间层 + 昇腾算子后端”。图 1 展示了其分层设计：PyTorch 前端将动态图、自动微分、优化器等能力暴露给开发者；torch_npu 负责对接 CANN 算子库、通信库以及运行时；底层由昇腾 AI 处理器提供硬件加速。理解这一层次关系有助于在故障排查时迅速定位问题。

3.2. 核心能力纵览

插件在多个维度增强了 PyTorch 在昇腾平台的体验：

- **算子与生态适配**：基于开源 PyTorch 实现对昇腾 AI 处理器的深度定制，持续补齐主流算子与第三方库，保证常见模型脚本可直接迁移。
- **训练基础设施**：保持动态图、自动微分、优化器与 Profiling 等特性，与原生 PyTorch 保持一致，降低学习成本。

- **自定义算子扩展**：为算子开发者提供接口，可在 PyTorch 框架内快速集成自研算子。
- **分布式通信**：内置 Broadcast、AllReduce 等集合通信原语，覆盖单机多卡与多机多卡场景。
- **推理链路**：模型可导出 ONNX，再借助离线转换工具生成适配昇腾的推理模型，便于训练—推理一体化交付。

3.3. PyTorch 安装教程

3.3.1. CANN 环境准备

在昇腾 310B 开发板上安装 PyTorch 之前，必须安装匹配 CANN 版本的驱动与固件。如果没有安装 CANN 相关的工具包与驱动，请参考《[CANN 软件安装指南](#)》或者本教程的[第二章](#)进行离线安装。如果已安装 CANN 相关工具包与驱动，可按执行以下命令获取版本信息：

```
1 cat /usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/
   ↪ ascend_toolkit_install.info
```

例如刚刚完成 CANN 8.3.RC1 版本的安装后，会输出以下信息：

```
1 package_name=Ascend-cann-toolkit
2 version=8.3.RC1
3 innerversion=V100R001C23SPC001B235
4 compatible_version=[V100R001C15],[V100R001C18],[V100R001C19],[V100R001C20
   ↪ ],[V100R001C21],[V100R001C23]
5 arch=aarch64
6 os=linux
7 path=/usr/local/Ascend/ascend-toolkit/8.3.RC1/aarch64-linux
```

在安装 PyTorch 框架与 torch_npu 插件之前，请先确认 CANN 环境变量已正确加载。若尚未将加载命令写入 ~/.bashrc，可在当前会话执行：

```
1 source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

如需持久生效，请将上述命令追加到 ~/.bashrc 后执行 source ~/.bashrc。

3.3.2. PyTorch 二进制软件包安装方法

完成基础环境后，可以参考按照[Ascend Extension for PyTorch 软件安装指南](#)或者昇腾的 PyTorch 框架的 [Github 仓库](#)部署 PyTorch 及 torch_npu 插件。在昇腾 310B 开发板上安装 PyTorch 框架和 torch_npu 插件的方法有两种，一种是二进制软件包安装，

另外一种是从源码编译安装。对于初学者来说，推荐使用二进制软件包进行安装。PyTorch 框架和 torch_npu 插件二进制软件包安装的具体方法如下：

安装 PyTorch 框架

由于出厂的 Ubuntu 22.04 系统已预装 miniconda，我们可以直接利用该环境。预装的 miniconda 通常带有 bash，在 base 下直接安装 PyTorch 与 torch_npu 容易产生冲突，因此建议新建一个 conda 环境，例如使用下面这个命令创建名为 npu 的环境。

```
1 conda create -n npu
```

然后利用下面这个命令进入 npu 虚拟环境：

```
1 conda activate npu
```

若按照本教程第二章在昇腾 310B 开发板上安装了 CANN 8.3.RC1，则可用的 Python 版本为 3.8–3.11，对应支持的 PyTorch 版本为 2.1.0、2.6.0、2.7.1、2.8.0。这里选择安装 Python 3.11 与 PyTorch 2.8.0，先在新建的 npu 环境安装 Python 编译器：

```
1 conda install python=3.11
```

下载 PyTorch 二进制包时需与当前 Python 版本一致，可用 python -V 确认版本，输出示例为 Python 3.11.14。该环境还需安装 CANN 依赖的第三方库，可执行：

```
1 pip3 install attrs cython 'numpy>=1.19.2,<=1.24.0' decorator \
2     sympy cffi pyyaml pathlib2 psutil protobuf==3.20.0 scipy requests
   ↪     absl-py \
3     cloudpickle ml-dtypes tornado jinja2 matplotlib tqdm
```

在昇腾开发板上安装 PyTorch 可手动下载二进制包或直接用 pip 自动安装。若手动下载，需确保与 Python 和 CANN 版本严格匹配，可在[华为昇腾 PyTorch 官方安装指南](#)选择对应的包。假设 Python 3.11、CANN 8.3.RC1，可参考以下命令下载：

```
1 wget https://download.pytorch.org/whl/cpu/torch-2.8.0%2Bcpu-cp311-cp311-
   ↪ manylinux_2_28_aarch64.whl
```

然后使用 pip3 安装：

```
1 pip3 install torch-2.8.0+cpu-cp311-cp311-manylinux_2_28_aarch64.whl
```

安装完成后，可通过以下命令验证：

```
1 python -c "import torch; print(torch.__version__)"
```

若安装正确，输出示例为：

```
1 2.8.0+cpu
```

安装昇腾 torch_npu 插件

安装 torch_npu 时，版本需要与 PyTorch、Python、系统架构以及 CANN 保持一致。可前往[华为昇腾 PyTorch 官方安装指南](#)选择匹配的二进制包。例如在 Python 3.11、PyTorch 2.8.0、CANN 8.3.RC1 场景下，选择 torch_npu 7.2.0，对应示例下载命令为：

```
1 wget https://gitcode.com/Ascend/pytorch/releases/download/v7.2.0-pytorch2
   ↪ .8.0/torch_npu-2.8.0-cp311-cp311-manylinux_2_28_aarch64.whl
```

下载后使用 pip3 安装：

```
1 pip3 install torch_npu-2.8.0-cp311-cp311-manylinux_2_28_aarch64.whl
```

注意：直接使用 pip install torch_npu 进行简易安装，可能导致程序与当前环境不兼容，或与已安装的 CANN 版本不完全适配。因此，建议谨慎采用该方式安装 torch_npu 插件。

3.3.3. torch_npu 安装后测试

成功安装 torch_npu 后，需要对安装后的 torch_npu 进行简单的测试，以此验证是否安装成功。其中最简单的方法，是直接在命令窗口中输入以下命令：

```
1 python3 -c "import torch;import torch_npu; a = torch.randn(3, 4).npu();
   ↪ print(a + a);"
```

成功运行后可见如下输出：

```
1 .[W compiler_depend.ts:164] Warning: Device do not support double dtype
   ↪ now, dtype cast replace with float. (function operator())
2 ...tensor([[ 0.0879,  0.5805, -0.3437, -0.0229],
3           [-1.7557,  0.0712,  2.7342,  1.8566],
4           [-1.2092,  1.4896,  0.2234,  0.4621]], device='npu:0')
```

上述 warning 来源于昇腾 310B 暂不支持双精度浮点 (double)，运行时会自动将双精度浮点 double 转为单精度浮点 float。若在 HwHiAiUser (非 root) 用户下执行同一命令，输出与下述示例类似：

```
1 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/
   ↪ collect_env.py:59: UserWarning: Warning: The /usr/local/Ascend/
   ↪ ascend-toolkit/latest owner does not match the current owner.
2 warnings.warn(f"Warning: The {path} owner does not match the current
   ↪ owner.")
3 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/
   ↪ collect_env.py:59: UserWarning: Warning: The /usr/local/Ascend/
   ↪ ascend-toolkit/8.3.RC1/aarch64-linux/ascend_toolkit_install.info
   ↪ owner does not match the current owner.
4 warnings.warn(f"Warning: The {path} owner does not match the current
   ↪ owner.")
```

```

5 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/
  ↳ _path_manager.py:67: UserWarning: Permission mismatch: The owner
  ↳ of /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/lib
  ↳ /libop_plugin_atb.so does not match.
6 warnings.warn(f"Permission mismatch: The owner of {path} does not match
  ↳ .")
7 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/
  ↳ collect_env.py:59: UserWarning: Warning: The /usr/local/Ascend/
  ↳ ascend-toolkit/latest owner does not match the current owner.
8 warnings.warn(f"Warning: The {path} owner does not match the current
  ↳ owner.")
9 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/
  ↳ collect_env.py:59: UserWarning: Warning: The /usr/local/Ascend/
  ↳ ascend-toolkit/8.3.RC1/aarch64-linux/ascend_toolkit_install.info
  ↳ owner does not match the current owner.
10 warnings.warn(f"Warning: The {path} owner does not match the current
  ↳ owner.")
11 [W compiler_depend.ts:164] Warning: Device do not support double dtype
  ↳ now, dtype cast replace with float. (function operator())
12 tensor([[ 0.4831,  0.6161,  0.2278, -0.1595],
13         [ 1.3375,  0.2916, -0.3860, -2.3874],
14         [ 1.3766, -1.6611,  2.7217,  2.0813]], device='npu:0')

```

尽管会出现较多 warning, 这是因为 torch_npu 插件用 root 账户安装所致, 但结果依然是正确的。

注意事项: 多数情况下, 已成功安装后仍在运行上文命令时报错, 原因是内存不足, 尤其常见于 8GB 内存的昇腾 310B 开发板。首次运行 PyTorch+torch_npu 时, 会触发将计算图交给 CANN 进行编译与优化 (含算子/图的并行编译), 默认并行度较高, 容易在低内存设备上触发 OOM。

解决办法: 降低编译并行度

- 临时生效 (仅针对本次运行):

```

1 TE_PARALLEL_COMPILER=1 MAX_COMPILE_CORE_NUMBER=1 python your_script
  ↳ .py

```

- 会话/永久生效:

```

1 export TE_PARALLEL_COMPILER=1
2 export MAX_COMPILE_CORE_NUMBER=1
3 # 建议追加到 ~/.bashrc 后执行: source ~/.bashrc

```

说明 - 该问题与使用 atc 工具时的并行编译内存占用现象一致, 降低并行度可显著缓解。- 首次运行的编译开销较大, 通过以上设置可提高在 8GB 设备上的稳定性。

3.4. torch_npu 插件的使用

PyTorch 是广泛使用的深度学习框架，基于 Python，入门友好且生态完善。系统学习可参考李沐的《动手学深度学习》第二版（开源电子书，含配套代码）：[动手学深度学习](#)。

关于 torch_npu：目前在昇腾 310B 上自动迁移工具不可用（实测无法跑通），因此本章默认不使用自动迁移，统一采用手工迁移路径，并提供常见替换与调优对策。迁移与调优可参考官方文档《[PyTorch 训练模型迁移调优](#)》。但是该文档主要面向训练服务器，细节与 310B 存在差异。

下文将结合若干实例，演示 torch_npu 在昇腾 310B 上的实践要点。

3.4.1. 线性神经网络实现

为了更清晰地展示基于 CUDA 的 PyTorch 使用与基于 torch_npu 的使用之间的区别，我们将从经典的线性神经网络入手。线性回归和 softmax 回归作为经典统计学习技术，可以视为线性神经网络的实例。这些知识将为在昇腾 310B 上进行代码移植的其他部分奠定基础。

一元线性回归

线性回归可用于刻画变量间的线性关系。在最简单的一元情形中，大学物理实验的电阻测量就是典型案例。根据欧姆定律可得：

$$V = I R$$

为适应实际中的接触电势/表计零漂，常用含偏置模型：

$$V_i \approx R I_i + b$$

以均方误差为目标函数：

$$L(R, b) = \frac{1}{n} \sum_{i=1}^n (R I_i + b - V_i)^2$$

其梯度为：

$$\frac{\partial L}{\partial R} = \frac{2}{n} \sum_{i=1}^n I_i (R I_i + b - V_i), \quad \frac{\partial L}{\partial b} = \frac{2}{n} \sum_{i=1}^n (R I_i + b - V_i)$$

采用小批量 SGD（批大小为 m，索引集合 B）：

$$g_R = \frac{2}{m} \sum_{i \in B} I_i (R I_i + b - V_i), \quad g_b = \frac{2}{m} \sum_{i \in B} (R I_i + b - V_i)$$

按学习率更新:

$$R \leftarrow R - \eta g_R, \quad b \leftarrow b - \eta g_b$$

实践流程 (单位统一为 A、V, 建议 float32, NPU 上双精度会自动降为 float32):

- 采集多组 (I_i, V_i) , 若存在接触电势选用含偏置模型。
- 初始化参数 (启发式: $R \approx (\sum I_i V_i) / (\sum I_i^2), b \approx 0$)。
- 随机抽取小批量, 计算 g_R, g_b , 按学习率迭代至收敛。
- 用 MSE 或 R^2 评估拟合, 必要时剔除离群点。

下方示例代码在 PyTorch+torch_npu 上实现上述流程。

```

1  """线性回归 (V  $\approx$  R·I + b) 示例, 使用华为昇腾 NPU 设备训练。
2  - 数据: 合成电压/电流数据, 单位 A/V, 含高斯噪声 (约 5mV)。
3  - 目标: 拟合电阻 R (斜率) 与偏置 b (截距)。
4  - 设备: 默认 npu:0, 可切换为 CPU。
5  - 训练: SGD 优化, MSE 损失, 小批量训练并通过 tqdm 展示进度。
6  - 输出: 打印估计的 R、b, 并保存训练损失曲线为 training_loss.png。
7  """
8
9  import torch # PyTorch 主库
10 from tqdm import tqdm # 进度条显示
11 import matplotlib.pyplot as plt # 绘图
12 import torch_npu # NPU 支持库 (确保已安装与环境就绪)
13
14 device = torch.device('npu:0') # 选择 NPU 设备 (Ascend), 需要正确的驱动
15     ↪ /环境
16 torch.manual_seed(0) # 固定随机种子以确保可复现
17
18 # 使用合成数据 (无自有数据)
19 N = 1024 # 样本数
20 R_true, b_true = 120.0, 0.02 # 真实电阻 (Ω) 与偏置 (V), 用于生成数据的
21     ↪ “地真值”
22 I = torch.linspace(0.0, 1.0, N).unsqueeze(1) # 电流张量 [N,1], 范围 0~1
23     ↪ A
24 noise_std = 0.005 # 噪声标准差  $\approx$  5 mV
25 noise = noise_std * torch.randn(N, 1) # 加性高斯噪声, 与 V 同形状
26 V = R_true * I + b_true + noise # 生成电压张量 [N,1], 单位 V
27
28 I = I.to(device) # 将数据移至计算设备 (NPU/CPU)
29 V = V.to(device)
30 batch_size = 16 # 小批量大小, 影响梯度估计方差与训练稳定性
31
32 # 线性模型 V  $\approx$  R·I + b (1 输入 -> 1 输出, 带偏置)
33 model = torch.nn.Linear(1, 1, bias=True).to(device) # 权重即电阻 R, 偏置
34     ↪ 即 b
35 opt = torch.optim.SGD(model.parameters(), lr=1e-2) # 随机梯度下降, 学习
36     ↪ 率 1e-2
37 loss_fn = torch.nn.MSELoss() # 均方误差: 衡量预测电压与真实电压的差异
38
39 # 使用小批量训练循环, 加入 tqdm 进度条并记录 loss
40 num_epochs = 50 # 训练轮数
41 epoch_losses = [] # 记录每个 epoch 的平均损失, 便于绘图

```

```

38 with tqdm(range(num_epochs), desc="Epochs") as pbar: # 进度条显示每个
    ↪ epoch
39     for _ in pbar:
40         epoch_loss = 0.0 # 累计当前 epoch 的损失
41         n_batches = 0 # 实际批次数 (用于计算平均损失)
42         perm = torch.randperm(I.shape[0], device=device) # 打乱索引以实
            ↪ 现随机小批量
43         for start in range(0, I.shape[0], batch_size): # 遍历每个批次起
            ↪ 点
44             end = start + batch_size # 批次终点
45             idx = perm[start:end] # 当前批次的随机样本索引
46             I_b = I[idx].to(torch.float32) # NPU 通常以 float32 计算, 确
            ↪ 保类型一致
47             V_b = V[idx].to(torch.float32)
48             pred = model(I_b) # 前向计算:  $V^{\wedge} = R \cdot I + b$ 
49             loss = loss_fn(pred, V_b) # 计算当前批次的 MSE 损失
50             opt.zero_grad() # 清空上一轮梯度
51             loss.backward() # 反向传播, 计算参数梯度
52             opt.step() # 参数更新 (SGD)
53             epoch_loss += loss.item() # 累加标量损失
54             n_batches += 1 # 批次数 +1
55             avg_loss = epoch_loss / max(n_batches, 1) # 当前 epoch 的平均损
            ↪ 失
56             epoch_losses.append(avg_loss) # 记录以便后续绘图
57             pbar.set_postfix(loss=f"{avg_loss:.6f}") # 在进度条尾部显示当前
            ↪ 损失
58
59 R = model.weight.detach().item() # 提取斜率 (电阻, 单位  $\Omega$ ), 不跟踪梯度
60 b = model.bias.detach().item() # 提取截距 (偏置, 单位 V), 不跟踪梯度
61 print("device =", device) # 打印使用的设备
62 print("R( $\Omega$ ) =", R, " b(V) =", b) # 打印拟合结果, 便于与真值对比
63
64 # 绘制训练损失下降曲线并保存图片
65 plt.figure(figsize=(6, 4)) # 设定图像尺寸
66 plt.plot(epoch_losses, label="Train MSE") # 绘制每个 epoch 的平均损失
67 plt.xlabel("Epoch") # 横轴: 训练轮数
68 plt.ylabel("Loss") # 纵轴: 均方误差
69 plt.title("Training Loss") # 标题
70 plt.grid(True) # 开启网格, 便于阅读
71 plt.legend() # 图例显示
72 plt.tight_layout() # 自动布局, 防止标签遮挡
73 plt.savefig("linear_regression_training_loss.png") # 保存为 PNG 文件

```

要运行上述示例, 请先在当前 (npu) 环境中安装依赖:

```
1 pip install tqdm matplotlib
```

示例运行后可见如下输出:

```

1 Epochs: 100%|
    ↪ 
    ↪ 50/50 [00:28<00:00, 1.77it/s, loss=0.173204]
2 device = npu:0

```

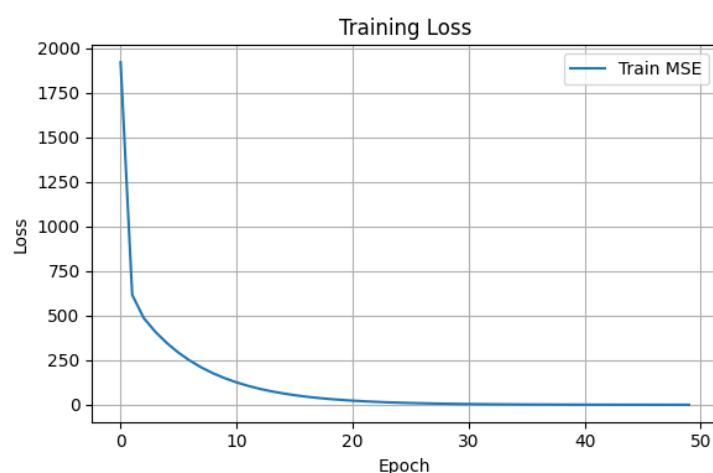


Figure 3.1.: 线性回归的损失函数关系图

```
3 R(Ω) = 118.63302612304688  b(V) = 0.7558640241622925
```

同时给出训练结果的可视化：下图展示训练损失（MSE）随 epoch 变化的曲线。

提示 - 数据需统一单位（A、V），使用 float32 更稳妥。- 无明显偏置时可将偏置项置为 0；存在测量/接触偏差时保留偏置并做鲁棒处理（如剔除离群点）。- 拟合优劣可用 MSE 或 R^2 评估，必要时清理异常样本。- 在部分环境中，PyTorch 2.1 运行上述代码可能报错，通常与 nn.Linear 的输出维度设为 1 的配置触发兼容性问题有关；建议升级至更高版本或将输出维度调整为大于 1 以规避。- 将代码中的 `device = torch.device` $\rightarrow$ `('npu:0')` 改为 `device = torch.device('cpu')`，即可在纯 CPU 上运行。该示例在 CPU 与 NPU 上耗时接近，主要因为未涉及大规模矩阵/张量计算，计算密度较低，NPU 的并行优势难以发挥。

多元线性回归

为了验证昇腾 310B 上 torch_npu 与 PyTorch 的兼容性，下面以加州房价数据集的多元线性回归示例继续演示 torch_npu 的使用。加州房价数据（California Housing Dataset）可通过 `fetch_california_housing()` 在 scikit-learn 获取，或在 [Kaggle](#) 等平台下载 CSV。该数据集包含经纬度（longitude/latitude）、房屋中位年龄（housing_median_age）、总房间数与卧室数（total_rooms/total_bedrooms）、人口与家庭数（population/households）、收入中位数（median_income）、房价中位数（median_house_value）以及与海距离类别（ocean_proximity）等字段，可用于房价预测（如线性回归、随机森林、XGBoost 等回归模型）、结合经纬度进行地理可视化绘制

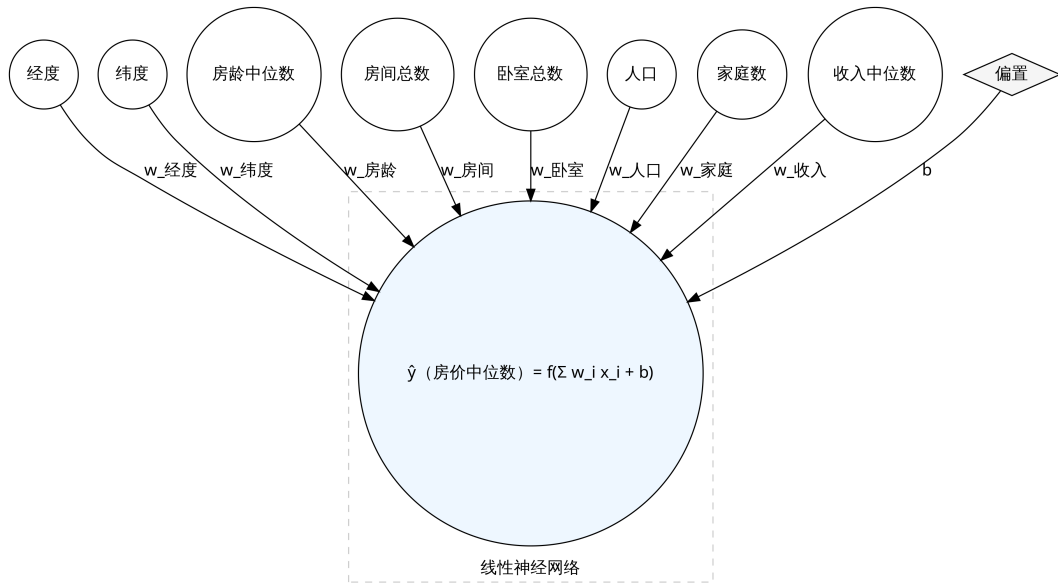


Figure 3.2.: 线性神经网络

房价热力图以分析区域差异，并通过“每户平均房间数”等衍生特征进行特征工程以提升效果。需注意，该数据为历史数据，不能直接反映当前价格；使用时应遵守隐私与数据保护规定，尤其在与其他数据源结合时；该数据集结构清晰、特征丰富，是机器学习回归入门的经典案例。对于线性神经网络（多元线性回归），可表示为：

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + \mathbf{b}, \quad \mathbf{X} \in \mathbb{R}^{m \times d}, \mathbf{w} \in \mathbb{R}^{d \times k}, \mathbf{b} \in \mathbb{R}^k$$

其中 m 为样本数, d 为特征维度, k 为输出维度 (回归时常取 $k = 1$); 对于单样本 $\mathbf{x} \in \mathbb{R}^d$, 有 $\hat{y} = \mathbf{x}^\top \mathbf{w} + b$ 。加州房价数据集的输入特征维度为 8, 输出为房价估计值, 其线性神经网络结构示意图如下:

损失函数通常选用均方误差 (MSE):

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m \|\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}\|_2^2$$

在训练模型的时候, 我们希望寻找出一组参数 $(\mathbf{w}^*, b^*)$, 使得训练样品的损失函数最小:

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}} \mathcal{L}(\mathbf{w}, b)$$

在实际训练中, 通常采用小批量随机梯度下降法来求解最优参数。其算法流程如下:

1. **初始化:** 随机初始化模型参数 $\mathbf{w}$ 和 b , 并设定学习率 η 。
2. **小批量采样:** 从训练数据中随机抽取包含 m 个样本的小批量 $\mathcal{B} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})\}$ 。

3. **计算梯度**：计算损失函数关于参数的梯度估计值：

$$\mathbf{g}_w = \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{2}{m} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} (\mathbf{x}^{(i)\top} \mathbf{w} + b - y^{(i)})$$

$$g_b = \frac{\partial \mathcal{L}}{\partial b} = \frac{2}{m} \sum_{i \in \mathcal{B}} (\mathbf{x}^{(i)\top} \mathbf{w} + b - y^{(i)})$$

4. **更新参数**：利用计算出的梯度更新当前参数：

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \mathbf{g}_w$$

$$b \leftarrow b - \eta \cdot g_b$$

5. **迭代**：重复步骤 2-4，直到满足停止准则（如达到最大迭代次数或损失函数收敛）。

在上述算法中，学习率（Learning Rate, η ）与训练轮数（Epoch）是影响模型性能的关键超参数。学习率决定了参数沿梯度反方向更新的步长：若设置过大，模型可能在最优解附近震荡甚至发散；若设置过小，则收敛速度过慢且易陷入局部最优。通常学习率的取值范围在 0.01 到 0.00001 之间。训练轮数是指完整遍历一次训练数据集的过程。由于单次迭代仅利用小批量数据更新梯度，通常需要多个 Epoch 才能使模型参数充分拟合数据特征并趋于稳定。一般建议将 Epoch 设置在 10 到 100 之间。

利用 torch_npu 插件，可以在昇腾 310B 上进行这个模型的训练，合理配置这些超参数不仅影响模型的最终精度，还直接关系到有限算力资源下的训练效率。详细的代码如下：

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import fetch_california_housing
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.model_selection import train_test_split
7 from torch.utils.data import DataLoader, TensorDataset
8 import numpy as np
9 from tqdm.auto import tqdm, trange
10
11 # =====
12 # 1. 环境配置与随机种子设置
13 # =====
14 # 设置随机种子以确保实验结果的可复现性
15 np.random.seed(42)
16 torch.manual_seed(42)
17
18 # 指定计算设备。'npu' 表示使用华为昇腾 AI 处理器
19 # 如果在没有 NPU 的环境下运行，可改为 'cuda' 或 'cpu'
20 device = torch.device('npu')
21
22 # =====
23 # 2. 数据加载与预处理

```

```

24 # =====
25 # 加载加州房价数据集 (回归任务)
26 california = fetch_california_housing()
27 X = california.data # 特征矩阵: 包含 8 个特征 (如人均收入、房龄等)
28 y = california.target.reshape(-1, 1) # 目标值: 房价, 调整为 (N, 1) 形状
29
30 # 数据标准化: 神经网络对输入特征的量级很敏感
31 # 使用 StandardScaler 使数据符合均值为 0, 方差为 1 的分布
32 scaler_X = StandardScaler()
33 scaler_y = StandardScaler()
34 X_scaled = scaler_X.fit_transform(X)
35 y_scaled = scaler_y.fit_transform(y)
36
37 # 划分数据集: 80% 用于训练, 20% 用于验证
38 X_train, X_val, y_train, y_val = train_test_split(
39     X_scaled, y_scaled, test_size=0.2, random_state=42
40 )
41
42 # =====
43 # 3. 构建 PyTorch 数据加载器 (DataLoader)
44 # =====
45 batch_size = 32
46
47 # 将 NumPy 数组转换为 PyTorch 张量 (FloatTensor)
48 train_dataset = TensorDataset(
49     torch.FloatTensor(X_train),
50     torch.FloatTensor(y_train)
51 )
52 val_dataset = TensorDataset(
53     torch.FloatTensor(X_val),
54     torch.FloatTensor(y_val)
55 )
56
57 # DataLoader 负责按批次 (Batch) 提供数据, 并在训练集上开启打乱 (Shuffle)
58 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=
59     ↪ True)
60 val_loader = DataLoader(val_dataset, batch_size=batch_size)
61
62 # =====
63 # 4. 定义模型结构
64 # =====
65 # 单层感知机: 本质上是一个线性回归模型 ( $y = Wx + b$ )
66 class SingleLayerPerceptron(nn.Module):
67     def __init__(self, input_dim):
68         super().__init__()
69         # 定义一个全连接层, 输入维度为特征数, 输出维度为 1
70         self.fc = nn.Linear(input_dim, 1)
71
72     def forward(self, x):
73         # 前向传播逻辑
74         return self.fc(x)
75
76 # 实例化模型并迁移到指定的计算设备 (NPU)
77 model = SingleLayerPerceptron(X_train.shape[1]).to(device)

```

```

77
78 # 定义损失函数：均方误差 (MSE)，适用于回归任务
79 criterion = nn.MSELoss()
80
81 # 定义优化器：随机梯度下降 (SGD)，学习率设为 0.01
82 optimizer = optim.SGD(model.parameters(), lr=0.01)
83
84 # =====
85 # 5. 训练与验证函数定义
86 # =====
87 def train_epoch(model, loader, criterion, optimizer):
88     """单轮训练逻辑"""
89     model.train() # 设置为训练模式
90     total_loss = 0
91     for batch_x, batch_y in tqdm(loader, desc="Train", leave=False):
92         # 将数据迁移到 NPU
93         batch_x = batch_x.to(device)
94         batch_y = batch_y.to(device)
95
96         # 梯度清零
97         optimizer.zero_grad()
98         # 前向传播
99         outputs = model(batch_x)
100         # 计算损失
101         loss = criterion(outputs, batch_y)
102         # 反向传播
103         loss.backward()
104         # 梯度裁剪：防止梯度爆炸
105         torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
106         # 更新参数
107         optimizer.step()
108
109         total_loss += loss.item() * batch_x.size(0)
110     return total_loss / len(loader.dataset)
111
112 def validate(model, loader, criterion):
113     """验证逻辑"""
114     model.eval() # 设置为评估模式
115     total_loss = 0
116     with torch.no_grad(): # 验证阶段不需要计算梯度
117         for batch_x, batch_y in tqdm(loader, desc="Val", leave=False):
118             batch_x = batch_x.to(device)
119             batch_y = batch_y.to(device)
120             outputs = model(batch_x)
121             loss = criterion(outputs, batch_y)
122             total_loss += loss.item() * batch_x.size(0)
123     return total_loss / len(loader.dataset)
124
125 # =====
126 # 6. 执行训练循环
127 # =====
128 epochs = 20
129 train_losses = []
130 val_losses = []

```

```

131
132 # 使用 trange 展示总进度条
133 t = trange(epochs, desc="Epochs")
134 for epoch in t:
135     train_loss = train_epoch(model, train_loader, criterion, optimizer)
136     val_loss = validate(model, val_loader, criterion)
137
138     train_losses.append(train_loss)
139     val_losses.append(val_loss)
140
141     # 在进度条右侧实时显示当前损失和学习率
142     t.set_postfix(train_loss=f"{train_loss:.4f}", val_loss=f"{val_loss:.4f}",
143                   lr=f"{optimizer.param_groups[0]['lr']:.6f}")
144
145 print(f"\nFinal Validation Loss: {val_losses[-1]:.4f}")
146
147 # =====
148 # 7. 模型评估与指标计算
149 # =====
150 model.eval()
151 all_predictions = []
152 all_targets = []
153
154 # 获取验证集上的所有预测结果
155 with torch.no_grad():
156     for batch_x, batch_y in val_loader:
157         batch_x = batch_x.to(device)
158         predictions = model(batch_x)
159         # 将结果移回 CPU 并转为 NumPy
160         all_predictions.extend(predictions.cpu().numpy())
161         all_targets.extend(batch_y.cpu().numpy())
162
163 # 逆标准化：将预测值和真实值还原回原始的房价单位（10万美元）
164 all_predictions = scaler_y.inverse_transform(np.array(all_predictions))
165 all_targets = scaler_y.inverse_transform(np.array(all_targets))
166
167 # 计算回归常用指标
168 from sklearn.metrics import r2_score, mean_absolute_error,
169     mean_squared_error
170
171 r2 = r2_score(all_targets, all_predictions) # 决定系数，越接近 1 越好
172 mae = mean_absolute_error(all_targets, all_predictions) # 平均绝对误差
173 rmse = np.sqrt(mean_squared_error(all_targets, all_predictions)) # 均方根
174     误差
175
176 print(f"\n最终评估指标:")
177 print(f"R2 Score: {r2:.4f}")
178 print(f"MAE: ${mae:.2f}")
179 print(f"RMSE: ${rmse:.2f}")
180
181 # =====
182 # 8. 结果可视化
183 # =====
184 import matplotlib.pyplot as plt

```



```

182 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
183
184 # 1. 损失曲线：观察模型是否收敛
185 axes[0, 0].plot(train_losses, label='Train Loss')
186 axes[0, 0].plot(val_losses, label='Val Loss')
187 axes[0, 0].set_xlabel('Epoch')
188 axes[0, 0].set_ylabel('Loss')
189 axes[0, 0].set_title('Training and Validation Loss')
190 axes[0, 0].legend()
191 axes[0, 0].grid(True)
192
193 # 2. 预测值 vs 真实值：散点越接近对角线表示预测越准
194 axes[0, 1].scatter(all_targets, all_predictions, alpha=0.3)
195 axes[0, 1].set_xlabel('True Price')
196 axes[0, 1].set_ylabel('Predicted Price')
197 axes[0, 1].set_title('True vs Predicted Prices')
198 axes[0, 1].grid(True)
199
200 # 3. 残差图：检查误差是否随机分布（理想状态下应无明显模式）
201 residuals = all_targets - all_predictions
202 axes[1, 0].scatter(all_predictions, residuals, alpha=0.3)
203 axes[1, 0].set_xlabel('Predicted Price')
204 axes[1, 0].set_ylabel('Residuals')
205 axes[1, 0].set_title('Residual Plot')
206 axes[1, 0].grid(True)
207
208 # 4. 误差分布直方图：检查误差是否符合正态分布
209 axes[1, 1].hist(residuals, bins=50, edgecolor='black')
210 axes[1, 1].set_xlabel('Error')
211 axes[1, 1].set_ylabel('Frequency')
212 axes[1, 1].set_title('Error Distribution')
213
214 plt.tight_layout()
215 plt.savefig('california_housing_linear_network_results.png')

```

在配置好 torch_npu 环境的昇腾 310B 开发板上运行以上的代码，可以得到以下的结果：

```

1 Epochs: 100%|██████████| 50/50 [02:14<00:00, 2.68s/it, lr=0.000100,
  ↳ train_loss=0.4308, val_loss=0.4444]
2
3 Final Validation Loss: 0.4444
4
5 最终评估指标：
6 R2 Score: 0.5484
7 MAE: $0.56
8 RMSE: $0.77

```

同时，我们可以得到训练过程的损失收敛及评估指标图如下图所示：

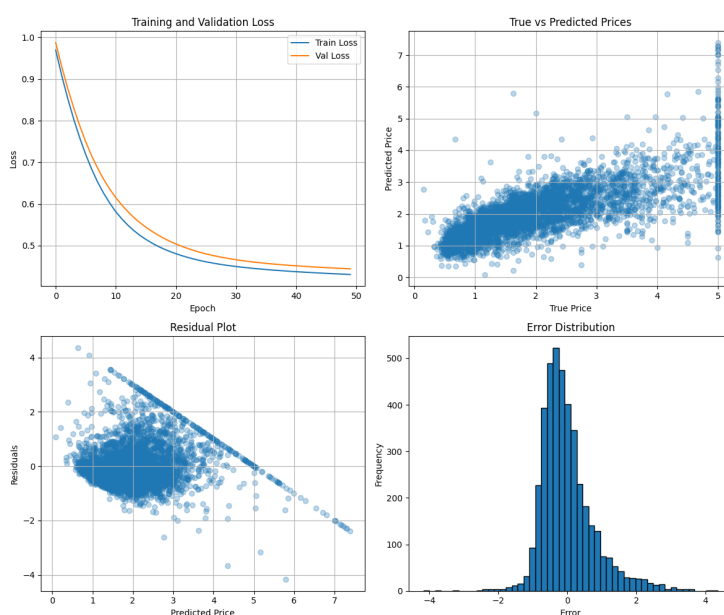


Figure 3.3.: 线性神经网络训练结果图

3.4.2. 多层感知机实现

我们之所以需要多层感知机 (Multilayer Perceptron, MPL)，是因为线性神经网络的输出仅为输入的加权和，难以刻画现实世界中复杂的非线性关系。以房价预测为例，房屋面积对价格的影响往往存在边际效应，而非简单的线性增长。为了突破这一局限，我们需要构建更深的网络结构。

如果我们仅仅通过堆叠多个线性层来构建网络，其数学表达为

$$\hat{\mathbf{y}} = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

展开后可见

$$\hat{\mathbf{y}} = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}_{new}\mathbf{x} + \mathbf{b}_{new}$$

这意味着多个线性层的组合在数学上仍然等价于单个线性层。无论网络深度如何增加，若缺乏非线性因素，它都无法拟合复杂的非线性函数。

多层感知机通过在层间引入激活函数打破了这种线性限制。以常用的 ReLU 为例，其定义为

$$\text{ReLU}(x) = \max(0, x)$$

另一个经典的激活函数是 Sigmoid 函数，它将任意实数映射到 (0, 1) 区间，曾广泛应用

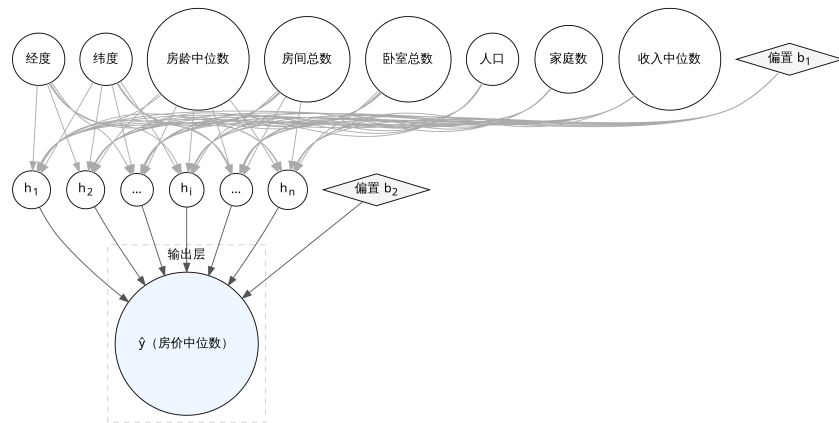


Figure 3.4.: 多层神经网络

于早期神经网络。其定义为

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

这种非线性变换允许网络通过组合不同的线性分段来逼近任意复杂的连续函数。一个含隐藏层的多层感知机可以表示为

$$\mathbf{H} = \sigma(\mathbf{XW}_1 + \mathbf{b}_1)$$

与

$$\mathbf{O} = \mathbf{HW}_2 + \mathbf{b}_2$$

其中 σ 即为非线性激活函数。

相比于只能处理线性相关问题的线性网络，多层感知机通过增加隐藏层的深度与宽度，能够捕捉特征之间的高阶交互作用。在加州房价数据集中，这种结构可以学习到经纬度与收入水平之间复杂的空间关联。对于加州房价数据集的多层感知机的模型如下图所示：

在昇腾 310B 上实现多层感知机，不仅能验证 torch_npu 对多层算子序列的调度能力，也能通过实验直观观察到非线性映射带来的精度提升。具体的代码如下：

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import fetch_california_housing
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.model_selection import train_test_split
7 from torch.utils.data import DataLoader, TensorDataset
8 import numpy as np
9 from tqdm.auto import tqdm, trange
```

```

10 import random
11
12 # 设置随机种子以确保实验结果的可重现性
13 SEED = 42
14 random.seed(SEED)
15 np.random.seed(SEED)
16 torch.manual_seed(SEED)
17 # 指定使用昇腾 NPU 设备
18 device = torch.device('npu:0')
19
20 # 加载加州房价数据集：包含8个特征，目标值为房屋中位数价格
21 california = fetch_california_housing()
22 X = california.data # 特征矩阵 (样本数, 8)
23 y = california.target.reshape(-1, 1) # 目标值 (样本数, 1)
24
25 # 数据标准化：将特征和目标值缩放到均值为0、方差为1的分布，有助于模型收敛
26 scaler_X = StandardScaler()
27 scaler_y = StandardScaler()
28 X_scaled = scaler_X.fit_transform(X)
29 y_scaled = scaler_y.fit_transform(y)
30
31 # 划分数据集：80% 用于训练，20% 用于验证
32 X_train, X_val, y_train, y_val = train_test_split(
33     X_scaled, y_scaled, test_size=0.2, random_state=42
34 )
35
36 # 构建 PyTorch 数据管道
37 batch_size = 32
38 train_dataset = TensorDataset(
39     torch.FloatTensor(X_train),
40     torch.FloatTensor(y_train)
41 )
42 val_dataset = TensorDataset(
43     torch.FloatTensor(X_val),
44     torch.FloatTensor(y_val)
45 )
46
47 # DataLoader 负责按批次 (Batch) 提供数据并支持洗牌 (Shuffle)
48 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=
    ↪ True)
49 val_loader = DataLoader(val_dataset, batch_size=batch_size)
50
51 # 定义双层感知机模型
52 class DoubleLayerPerceptron(nn.Module):
53     def __init__(self, input_dim, hidden_dim=64):
54         super().__init__()
55         self.net = nn.Sequential(
56             nn.Linear(input_dim, hidden_dim), # 输入层到隐藏层
57             nn.ReLU(), # 激活函数，引入非线性
58             nn.Linear(hidden_dim, 1) # 隐藏层到输出层 (回归任务
    ↪ 输出维度为1)
59         )
60     def forward(self, x):
61         return self.net(x)

```

```

62
63 # 实例化模型并迁移至 NPU
64 model = DoubleLayerPerceptron(X_train.shape[1]).to(device)
65 # 回归任务常用的均方误差损失函数
66 criterion = nn.MSELoss()
67 # 随机梯度下降优化器
68 optimizer = optim.SGD(model.parameters(), lr=0.0001)
69
70 # 单个 Epoch 的训练逻辑
71 def train_epoch(model, loader, criterion, optimizer):
72     model.train()
73     total_loss = 0
74     for batch_x, batch_y in tqdm(loader, desc="Train", leave=False):
75         # 将数据搬运到 NPU 显存
76         batch_x = batch_x.to(device)
77         batch_y = batch_y.to(device)
78
79         optimizer.zero_grad() # 清空梯度
80         outputs = model(batch_x) # 前向传播
81         loss = criterion(outputs, batch_y) # 计算损失
82         loss.backward() # 反向传播计算梯度
83         torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
84         # 梯度裁剪防止梯度爆炸
85         optimizer.step() # 更新权重
86         total_loss += loss.item() * batch_x.size(0)
87     return total_loss / len(loader.dataset)
88
89 # 验证逻辑
90 def validate(model, loader, criterion):
91     model.eval() # 设置为评估模式
92     total_loss = 0
93     with torch.no_grad(): # 验证阶段不需要计算梯度
94         for batch_x, batch_y in tqdm(loader, desc="Val", leave=False):
95             batch_x = batch_x.to(device)
96             batch_y = batch_y.to(device)
97             outputs = model(batch_x)
98             loss = criterion(outputs, batch_y)
99             total_loss += loss.item() * batch_x.size(0)
100     return total_loss / len(loader.dataset)
101
102 # 执行训练过程
103 epochs = 50
104 train_losses = []
105 val_losses = []
106 t = trange(epochs, desc="Epochs")
107 for epoch in t:
108     train_loss = train_epoch(model, train_loader, criterion, optimizer)
109     val_loss = validate(model, val_loader, criterion)
110     train_losses.append(train_loss)
111     val_losses.append(val_loss)
112     # 在进度条右侧实时显示当前损失和学习率
113     t.set_postfix(train_loss=f"{train_loss:.4f}", val_loss=f"{val_loss:.4f}", lr=f"{optimizer.param_groups[0]['lr']:.6f}")

```

```

114
115 print(f"\nValidation Loss: {val_losses[-1]:.4f}")
116
117 # 评估阶段：获取验证集的预测值并还原标准化
118 model.eval()
119 all_predictions = []
120 all_targets = []
121
122 with torch.no_grad():
123     for batch_x, batch_y in val_loader:
124         batch_x = batch_x.to(device)
125         predictions = model(batch_x)
126         all_predictions.extend(predictions.cpu().numpy())
127         all_targets.extend(batch_y.cpu().numpy())
128
129 # 将标准化后的数据逆转回原始价格单位
130 all_predictions = scaler_y.inverse_transform(np.array(all_predictions))
131 all_targets = scaler_y.inverse_transform(np.array(all_targets))
132
133 # 计算回归评估指标
134 from sklearn.metrics import r2_score, mean_absolute_error,
    ↪ mean_squared_error
135
136 r2 = r2_score(all_targets, all_predictions) # 决定系数，越接近 1 越好
137 mae = mean_absolute_error(all_targets, all_predictions) # 平均绝对误差
138 rmse = np.sqrt(mean_squared_error(all_targets, all_predictions)) # 均方根
    ↪ 误差
139
140 print(f"\n最终评估指标:")
141 print(f"R2 Score: {r2:.4f}")
142 print(f"MAE: ${mae:.2f}")
143 print(f"RMSE: ${rmse:.2f}")
144
145 # 结果可视化
146 import matplotlib.pyplot as plt
147
148 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
149
150 # 1. 损失下降曲线：观察模型是否收敛或过拟合
151 axes[0, 0].plot(train_losses, label='Train Loss')
152 axes[0, 0].plot(val_losses, label='Val Loss')
153 axes[0, 0].set_xlabel('Epoch')
154 axes[0, 0].set_ylabel('Loss')
155 axes[0, 0].set_title('Training and Validation Loss')
156 axes[0, 0].legend()
157 axes[0, 0].grid(True)
158
159 # 2. 预测值 vs 真实值散点图：点越靠近红虚线表示预测越准
160 axes[0, 1].scatter(all_targets, all_predictions, alpha=0.3)
161 axes[0, 1].plot([all_targets.min(), all_targets.max()],
162                [all_targets.min(), all_targets.max()], 'r--', lw=2)
163 axes[0, 1].set_xlabel('True Price')
164 axes[0, 1].set_ylabel('Predicted Price')
165 axes[0, 1].set_title('True vs Predicted Prices')

```

```

166 axes[0, 1].grid(True)
167
168 # 3. 残差图：检查误差是否随机分布（理想状态应均匀分布在0附近）
169 residuals = all_targets - all_predictions
170 axes[1, 0].scatter(all_predictions, residuals, alpha=0.3)
171 axes[1, 0].axhline(y=0, color='r', linestyle='--')
172 axes[1, 0].set_xlabel('Predicted Price')
173 axes[1, 0].set_ylabel('Residuals')
174 axes[1, 0].set_title('Residual Plot')
175 axes[1, 0].grid(True)
176
177 # 4. 误差分布直方图：观察误差是否符合正态分布
178 axes[1, 1].hist(residuals, bins=50, edgecolor='black')
179 axes[1, 1].set_xlabel('Error')
180 axes[1, 1].set_ylabel('Frequency')
181 axes[1, 1].set_title('Error Distribution')
182 axes[1, 1].grid(True)
183
184 plt.tight_layout()
185 plt.savefig('california_housing_mlp_results.png')

```

在昇腾 310B 开发板上运行以上代码，可以得到以下的结果：

```

1 Epochs: 100%|
  ↳ 50/50 [02:39<00:00, 3.19s/it, lr=0.000100, train_loss=0.4509,
  ↳ val_loss=0.4563]
2
3 Validation Loss: 0.4563
4
5 最终评估指标：
6 R2 Score: 0.5363
7 MAE: $0.57
8 RMSE: $0.78

```

同时，我们可以得到训练过程的损失收敛及评估指标图如下图所示：

3.4.3. LeNet-5

LeNet-5 是由 Yann LeCun 等人在 1998 年提出的经典卷积神经网络（CNN），最初用于手写数字识别（MNIST 数据集）。它是深度学习领域的里程碑，奠定了现代卷积神经网络的基础架构。其名称中的“Le”取自第一作者 Yann LeCun 的姓氏，而数字“5”则代表该网络包含 5 层具有可训练参数的权重层（包括 3 个卷积层和 2 个全连接层）。在原始设计中，C5 层虽然在逻辑上起到了全连接的作用，但由于其卷积核尺寸与输入特征图完全一致，因此被归类为卷积层。

LeNet-5 共有 7 层（不含输入层），其标准结构如下：

1. **输入层 (Input)**: 接收 32×32 的灰度图像。

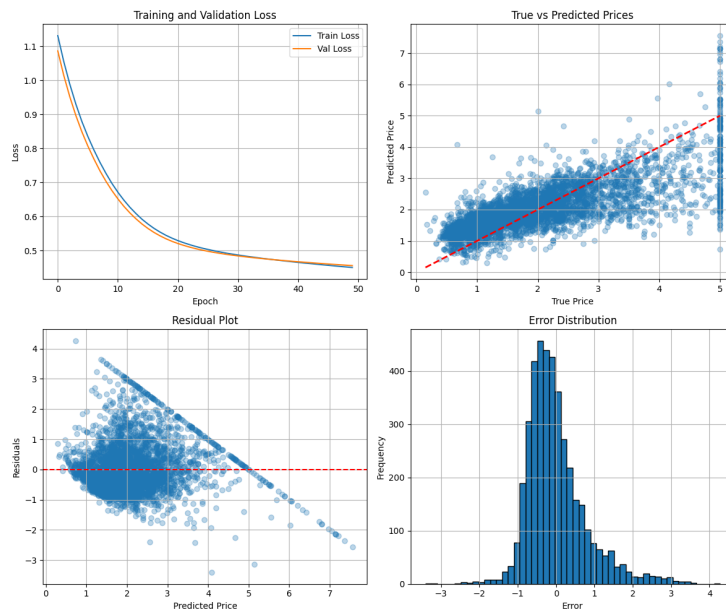


Figure 3.5.: 多层神经网络训练结果图

2. **卷积层 (C1)**: 使用 6 个 5×5 的卷积核, 输出特征图大小为 28×28 。
3. **池化层 (S2)**: 2×2 平均池化, 步长为 2, 输出大小为 14×14 。
4. **卷积层 (C3)**: 使用 16 个 5×5 的卷积核, 输出大小为 10×10 。
5. **池化层 (S4)**: 2×2 平均池化, 步长为 2, 输出大小为 5×5 。
6. **全连接层 (C5)**: 包含 120 个神经元, 将卷积特征展开。
7. **全连接层 (F6)**: 包含 84 个神经元, 激活函数通常使用 Sigmoid 或 Tanh。
8. **输出层 (Output)**: 10 个神经元, 对应数字 0-9。

LeNet-5 的核心设计理念在于通过局部感受野使卷积核仅关注图像的局部区域, 从而有效提取边缘和角点等局部特征。配合权值共享机制, 同一特征图上的神经元能够共享卷积参数, 这不仅极大减少了模型的参数总量, 还显著降低了过拟合的风险。此外, 通过下采样技术, 池化层在降低特征图分辨率、减少计算量的同时, 增强了模型对图像形变和平移的鲁棒性。这种交替使用卷积与池化的层级特征提取方式, 使网络能够从基础线条逐步抽象出复杂的数字轮廓。在昇腾 310B 上实现 LeNet-5, 可以进一步验证 torch_npu 对卷积算子 (Conv2d) 和池化算子 (AvgPool2d) 的硬件加速支持。具体代码如下:

```
1 import os
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
```



```

6 from torch.utils.data import TensorDataset, DataLoader
7 from torchvision import datasets
8 import matplotlib.pyplot as plt
9 from tqdm.auto import tqdm, trange
10
11 # 指定使用昇腾 NPU 设备
12 # 在昇腾 AI 处理器上运行 PyTorch 代码时，需要指定设备为 "npu"
13 device = torch.device("npu")
14
15 class LeNet(nn.Module):
16     """
17     经典的 LeNet-5 网络结构实现
18     LeNet-5 是一个简单的卷积神经网络，包含两个卷积层和三个全连接层
19     """
20     def __init__(self):
21         super().__init__()
22         # 第一层卷积：输入通道1（灰度图），输出通道6，卷积核5x5
23         # padding默认是0，stride默认是1
24         self.conv1 = nn.Conv2d(1, 6, kernel_size=5)
25         # 池化层：2x2 窗口，步长为2，用于下采样，减少特征图尺寸
26         self.pool = nn.AvgPool2d(2, 2)
27         # 第二层卷积：输入通道6，输出通道16，卷积核5x5
28         self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
29         # 全连接层：输入特征 16*4*4，输出120
30         # 这里的 16*4*4 是经过两次卷积和池化后的特征图大小展平后的维度
31         # 原始 28x28 -> conv1(5x5) -> 24x24 -> pool(2x2) -> 12x12
32         # -> conv2(5x5) -> 8x8 -> pool(2x2) -> 4x4
33         self.fc1 = nn.Linear(16 * 4 * 4, 120)
34         # 全连接层：输入120，输出84
35         self.fc2 = nn.Linear(120, 84)
36         # 输出层：输入84，输出10（对应 MNIST 的10个类别 0-9）
37         self.fc3 = nn.Linear(84, 10)
38
39     def forward(self, x):
40         # 卷积 -> ReLU 激活 -> 池化
41         # x shape: [batch, 1, 28, 28] -> [batch, 6, 12, 12]
42         x = torch.relu(self.conv1(x))
43         x = self.pool(x)
44         # 卷积 -> ReLU 激活 -> 池化
45         # x shape: [batch, 6, 12, 12] -> [batch, 16, 4, 4]
46         x = torch.relu(self.conv2(x))
47         x = self.pool(x)
48         # 展平特征图为一维向量，用于全连接层
49         # x shape: [batch, 16, 4, 4] -> [batch, 16*4*4]
50         x = x.view(x.size(0), -1)
51         # 全连接层 -> ReLU
52         x = torch.relu(self.fc1(x))
53         x = torch.relu(self.fc2(x))
54         # 输出层，通常配合 CrossEntropyLoss 使用，不需要 softmax
55         x = self.fc3(x)
56         return x
57
58 def plot_metrics(script_dir, train_acc_history, test_acc_history,

```

```

59     ↪ train_loss_history, test_loss_history):
60         """
61         绘制并保存训练过程中的精度和损失曲线
62         """
63         plt.figure(figsize=(10, 4))
64         # Accuracy subplot (左图: 精度曲线)
65         plt.subplot(1, 2, 1)
66         plt.plot(train_acc_history, label="Train Acc")
67         plt.plot(test_acc_history, label="Test Acc")
68         plt.xlabel("Epoch"); plt.ylabel("Accuracy"); plt.title("Accuracy");
69         ↪ plt.legend()
70         # Loss subplot (右图: 损失曲线)
71         plt.subplot(1, 2, 2)
72         plt.plot(train_loss_history, label="Train Loss")
73         plt.plot(test_loss_history, label="Test Loss")
74         plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("Loss"); plt.
75         ↪ legend()
76         plt.tight_layout()
77         # 保存图片到脚本所在目录
78         out_path = os.path.join(script_dir, "accuracy_loss_curve.png")
79         plt.savefig(out_path, bbox_inches="tight")
80         print(f"Saved accuracy & loss curve to: {out_path}")
81
82     def load_data(script_dir, batch_size=256):
83         """
84         加载 MNIST 数据集并进行预处理
85         """
86         # 数据存储路径
87         data_root = os.path.join(script_dir, "data")
88         # 使用 torchvision.datasets 下载并加载 MNIST 数据
89         train_ds = datasets.MNIST(data_root, train=True, download=True)
90         test_ds = datasets.MNIST(data_root, train=False, download=True)
91
92         # 预处理:
93         # 1. unsqueeze(1): 增加通道维度 [N, H, W] -> [N, 1, H, W]
94         # 2. to(torch.float16): 转换为半精度浮点数, NPU 上 float16 计算性能更
95         ↪ 好
96         # 3. div_(255.0): 像素值归一化到 [0, 1] 区间
97         x_train = train_ds.data.unsqueeze(1).to(torch.float16).div_(255.0)
98         y_train = train_ds.targets.to(torch.int64)
99         x_test = test_ds.data.unsqueeze(1).to(torch.float16).div_(255.0)
100         y_test = test_ds.targets.to(torch.int64)
101
102         # 构建 DataLoader, 用于批量加载数据
103         # shuffle=True 表示训练时打乱数据顺序
104         train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size
105         ↪ =batch_size, shuffle=True)
106         test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=
107         ↪ batch_size, shuffle=False)
108         return train_loader, test_loader
109
110     def train_and_eval(train_loader, test_loader, epochs=10, lr=0.01):
111         """
112         执行模型训练与验证循环

```

```

107     """
108     # 初始化模型并迁移至 NPU 设备
109     # .half() 将模型参数转换为 float16, 利用 NPU 的 Tensor Core 加速
110     model = LeNet().half().to(device)
111     # 定义损失函数: 交叉熵损失, 适用于多分类问题
112     criterion = nn.CrossEntropyLoss()
113     # 定义优化器: 随机梯度下降 (SGD)
114     optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)
115
116     train_acc_history = []
117     test_acc_history = []
118     train_loss_history = []
119     test_loss_history = []
120
121     # 使用 trange 展示总进度条
122     t = trange(epochs, desc="Epochs")
123     for epoch in t:
124         # --- 训练阶段 ---
125         model.train() # 设置模型为训练模式 (启用 Dropout, BatchNorm 等)
126         correct = 0; total = 0
127         running_loss = 0.0
128         for inputs, labels in train_loader:
129             # 数据迁移至 NPU 并转为半精度, 标签保持 int64 或 int32
130             inputs = inputs.to(device)
131             labels = labels.to(device)
132
133             # 前向传播
134             outputs = model(inputs)
135             # 计算损失时将输出转回 float32 以保证数值稳定性 (混合精度训练
136             #   ↳ 的常见做法)
137             loss = criterion(outputs.float(), labels)
138
139             # 反向传播与优化
140             optimizer.zero_grad() # 清空梯度
141             loss.backward()        # 计算梯度
142             optimizer.step()       # 更新参数
143
144             # 统计损失与精度
145             running_loss += loss.item() * labels.size(0)
146             # 获取预测结果 (最大概率对应的索引)
147             preds = outputs.float().argmax(dim=1)
148             correct += (preds == labels).sum().item()
149             total += labels.size(0)
150
151         # 计算本 epoch 的平均训练精度和损失
152         train_acc = correct / total
153         train_loss = running_loss / total
154         train_acc_history.append(train_acc)
155         train_loss_history.append(train_loss)
156
157         # --- 测试阶段 ---
158         model.eval() # 设置模型为评估模式
159         correct_t = 0; total_t = 0

```

```

159     running_loss_t = 0.0
160     with torch.no_grad(): # 验证时不计算梯度, 节省显存和计算资源
161         for inputs, labels in test_loader:
162             inputs = inputs.to(device).half(); labels = labels.to(
163                 ↪ device)
164             outputs = model(inputs)
165             loss_t = criterion(outputs.float(), labels)
166             running_loss_t += loss_t.item() * labels.size(0)
167             preds = outputs.float().argmax(dim=1)
168             correct_t += (preds == labels).sum().item()
169             total_t += labels.size(0)
170
171     # 计算本 epoch 的平均测试精度和损失
172     test_acc = correct_t / total_t
173     test_loss = running_loss_t / total_t
174     test_acc_history.append(test_acc)
175     test_loss_history.append(test_loss)
176     t.set_postfix(train_acc=f'{train_acc:.4f}', val_loss=f'{test_acc
177         ↪ :.4f}', lr=f'{optimizer.param_groups[0]["lr"]:.6f}')
178
179 # 返回训练/测试指标历史
180 return train_acc_history, test_acc_history, train_loss_history,
181     ↪ test_loss_history
182
183 if __name__ == "__main__":
184     # 获取当前脚本目录, 方便保存文件
185     script_dir = os.path.dirname(os.path.abspath(__file__))
186     # 加载数据
187     train_loader, test_loader = load_data(script_dir, batch_size=256)
188     # 开始训练
189     train_acc_history, test_acc_history, train_loss_history,
190     ↪ test_loss_history = train_and_eval(
191         train_loader, test_loader, epochs=10, lr=0.01
192     )
193     # 绘制结果
194     plot_metrics(script_dir, train_acc_history, test_acc_history,
195         ↪ train_loss_history, test_loss_history)

```

注意:运行此代码前,请确保已安装 torchvision 库。安装时必须严格注意 torchvision

↪ 与 PyTorch 的版本对应关系, 版本不匹配可能导致无法导入或运行时错误。

以下是昇腾 PyTorch 插件支持的 PyTorch 版本与其对应的 TorchVision 版本对照表:

PyTorch 版本	TorchVision 版本
2.1.0	0.16.0
2.6.0	0.21.0
2.7.1	0.22.0
2.8.0	0.23.0

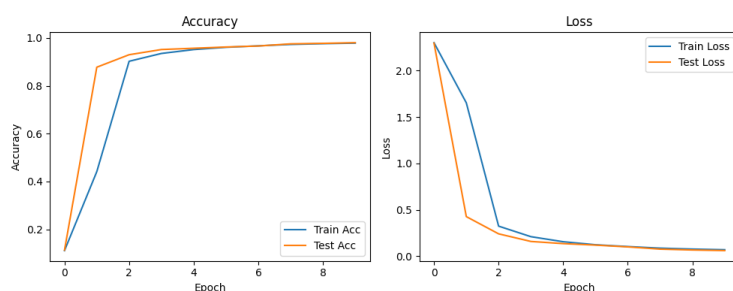


Figure 3.6.: 部分预测结果

例如，若当前环境安装的是 PyTorch 2.8.0，则需安装 torchvision 0.19.0：

```
1 pip install torchvision==0.23.0
```

在昇腾 310B 开发板上运行以上代码，可以得到以下的结果：

```
1 warnings.warn(f"Warning: The {path} owner does not match the current
  ↳ owner.")
2 Epochs: 100%|████████████████████████████████████████████████████████████████████████████████| 10/10
  ↳ [01:51<00:00, 11.15s/it, lr=0.010000, train_acc=0.9782, val_loss
  ↳ =0.9800]
3 Saved accuracy & loss curve to: /home/HwHiAiUser/Documents/samples/
  ↳ chapter3/LeNet/accuracy_loss_curve.png
```

并生成如下图所示的部分预测结果可视化：

在本示例中，尽管使用了昇腾 NPU 进行加速，但观察到的训练速度与 CPU 相比并未展现出量级上的优势，这主要是由任务规模与硬件特性共同决定的。MNIST 数据集中的图像分辨率仅为 28x28，且 LeNet-5 网络的参数量和计算深度相对较低，这种轻量级的计算负载难以充分填满 NPU 内部众多的并行计算单元，导致硬件资源处于不饱和状态。与此同时，在 NPU 上执行任务涉及到数据从主机内存到设备显存的搬运开销，以及 CANN 算子库在调用时的指令调度与同步耗时，当计算本身的耗时极短时，这些固有的通信与管理开销便成为了性能瓶颈。只有在面对如 ImageNet 等大规模数据集或 ResNet、Transformer 等深层复杂模型时，NPU 强大的张量计算能力才能在抵消掉调度开销后，通过极高的并行度显著缩短整体训练周期。

值得注意的是，本示例对原始 LeNet-5 进行了关键的适配调整。首先，我们将激活函数由 Sigmoid 替换为 ReLU；其次，我们将模型计算精度由 FP32 全精度调整为 FP16 半精度。这是因为在当前硬件环境下，FP32 模型在编译阶段可能遇到兼容性阻碍导致无法运行。同时实验表明，在半精度计算模式下，Sigmoid 函数极易因数值精度不足导致训练效果显著下降，而 ReLU 则能维持更好的数值稳定性与收敛效果。

FP16（半精度浮点数）在表示范围和精度上存在显著限制，这直接影响了激活函数的数值表现。由于 FP16 仅有 16 位，其有效数字位数极少且数值范围较窄，在处理涉

及指数运算和除法的 Sigmoid 函数时极易出现问题。在前向计算中，当输入值的绝对值稍大时，FP16 的精度不足以区分计算过程中的微小差别，导致输出迅速进入饱和区并锁定为 0 或 1。更严重的是，在反向传播过程中，Sigmoid 的导数在接近饱和区时会趋近于 0，这些微小的梯度值在 FP16 的精度下极易发生下溢出而变成纯 0，从而导致权重停止更新，使网络训练陷入停滞。

相比之下，ReLU 函数展现出了极佳的数值稳定性。作为一种分段线性函数，ReLU 的计算过程仅涉及简单的比较操作，完全避开了复杂的非线性变换带来的精度损失。在正值区域，ReLU 的导数恒定为 1，这种特性确保了梯度在传播过程中不会因为数值过小而消失，也不会受到 FP16 精度限制的影响。这种天然的鲁棒性使得 ReLU 非常契合 NPU 等强调低精度、高吞吐计算的硬件架构，能够有效保障模型在半精度模式下的收敛效果。

最后值得注意的一点是模型精度转换（FP32 转 FP16）与设备传输的操作顺序。代码实现上通常有两种方式：一种是先在 CPU 端将模型转换为半精度，再传输至 NPU，写法为 `model = LeNet().half().to(device)`；另一种是先将模型传输至 NPU，再进行精度转换，写法为 `model = LeNet().to(device).half()`。虽然这两种写法最终得到的模型状态一致，但在昇腾 310B 的 `torch_npu` 环境下，其底层执行流程存在差异。实测表明，由于 NPU 涉及图编译过程，**优先在 CPU 端完成精度转换（即第一种写法）可以显著减少图编译的开销**，从而缩短程序的整体启动时间。不过，一旦编译完成进入稳定运行阶段，两者的计算效率差异则微乎其微。

3.5. torch_npu 插件兼容性测试套件

为了全面评估 `torch_npu` 在昇腾 310B 上的算子支持度与数值稳定性，我们在 [samples](#) → [/chapter3/test](#) 目录下提供了一套兼容性测试脚本。这些测试旨在帮助开发者快速验证当前环境（CANN 版本 + PyTorch 版本 + 硬件）是否满足模型迁移的基本要求，并识别潜在的算子不支持或精度异常问题。

3.5.1. 测试目标与验证维度

在利用 `torch_npu` 插件开发神经网络应用时，我们注意到该插件在昇腾 310B 上的兼容性仍有提升空间。面对复杂的网络结构或特定的算子组合，图编译阶段偶尔会出现不稳定的情况。此外，部分算子在特定精度下的表现也需关注，例如 Sigmoid 激活函数在 FP16 精度下可能引发模型收敛异常。鉴于此，针对 AlexNet、VGG、ResNet 等经典网络中常用的核心算子进行全面测试显得尤为必要。为此，我们开发了一套轻量级的测试套件。

本测试套件的首要目标是验证核心算子在昇腾 NPU 上的支持度与执行稳定性。通过覆盖卷积 (Conv2d)、全连接 (Linear)、矩阵乘法 (MatMul) 等深度学习中最基础且关键的算子, 旨在检查这些指令能否被正确分发至 NPU 后端并顺利执行。这不仅有助于判断当前软硬件环境是否具备运行复杂模型的基础能力, 对于快速排查因算子缺失或版本不兼容导致的运行时错误也至关重要。

其次, 精度对齐是评估迁移质量的关键指标。测试脚本通过对比同一组输入数据在 NPU 与 CPU 上的计算结果, 严格量化两者之间的数值差异。通常情况下, 在 FP32 精度下, 我们期望误差控制在极小的范围内 (如小于 $1e-4$), 以确保模型迁移后推理与训练结果的可靠性。这种对比验证能有效发现因硬件架构差异或算子实现细节不同而引发的数值漂移问题。

最后, 测试还致力于探索不同边界条件下的表现。这包括验证 FP16、FP32 等不同数据类型的兼容性, 特别是针对昇腾 310B 对 FP64 (双精度) 支持有限的特性进行实测, 以及评估在不同 Tensor 形状和维度下算子的内存占用与计算效率。通过这些多维度的测试, 开发者可以更清晰地了解硬件的性能边界与最佳实践配置。

3.5.2. 测试套件结构详解

本测试套件基于 pytest 框架构建, 针对不同精度 (FP16/FP32) 进行了分层验证, 主要包含以下核心组成部分:

1. 测试环境配置 (conftest.py)

conftest.py 脚本负责构建稳健的测试环境。它利用 pytest 的钩子函数 (Hooks) 机制, 实现了一套实时日志记录系统。针对嵌入式开发板在长时间批量测试中可能出现的网络波动或系统挂起导致终端输出丢失的问题, 该脚本定义了 pytest_runtest_setup 和 pytest_runtest_logreport 等钩子。这些钩子确保在每个测试用例开始和结束时, 立即将测试节点 ID 及运行结果 (PASSED/FAILED) 写入 pytest_realtime_log.txt 文件。为了防止系统崩溃导致数据丢失, 写入操作调用了 os.fsync 强制刷新磁盘。此外, pytest_terminal_summary 会在测试结束时统计并输出通过、失败及跳过的用例总数, 生成详尽且持久化的测试报告, 极大便利了无人值守场景下的故障排查。

2. 基础算子运算测试 (test_float16_ops.py / test_float32_ops.py)

基础算子测试剥离了复杂的神经网络层封装, 直接验证深度学习中最底层的张量加法 (Add) 和矩阵乘法 (MatMul)。这两个脚本分别对应 FP16 (半精度) 和 FP32 (单精度), 旨在直接检验昇腾 NPU 硬件底层的算术逻辑单元 (ALU) 与矩阵计算单元 (Cube Core) 的计算正确性。通过绕过 nn.Module 等高层抽象, 这些测试作为排查底层硬件故障或驱动问题的最小功能单元, 有助于区分框架层面的算子映射错误与硬件层面的计算异常。

- **test_float32_ops.py**: 关注高精度计算的一致性。它在 CPU 和 NPU 上执行相同的 FP32 运算, 要求两者误差控制在极小范围内(如加法 $1e-4$, 矩阵乘法 $1e-3$), 确保 NPU 单精度计算路径的数值稳定。
- **test_float16_ops.py**: 针对边缘计算常用的半精度推理场景。采用“CPU FP32 计算作为真值”的策略, 即在 CPU 上使用高精度 FP32 运算, NPU 使用 FP16 运算, 最后将 NPU 结果转回 FP32 进行对比。这种方式既验证了 NPU 的半精度计算能力, 又通过放宽容差 ($1e-2$) 合理包容了精度量化带来的正常误差。

3. 神经网络层测试 (test_nn_layers_float16.py / test_nn_layers_float32.py)

这是测试套件的核心, 旨在全面验证常用深度学习算子在昇腾 NPU 上的表现。测试覆盖了特征提取与分类的关键组件: * **核心层**: nn.Linear 和 nn.Conv2d 的测试用例涵盖了 LeNet、AlexNet、VGG 及 ResNet 等经典架构的典型参数配置(输入尺寸、卷积核大小、步长及填充)。* **激活与归一化**: 囊括 ReLU、Sigmoid、Tanh、LeakyReLU、GELU、Softmax 等激活函数及 BatchNorm2d, 确保非线性变换与数据分布调整的准确性。* **池化与辅助层**: 验证 AvgPool2d 和 MaxPool2d 的基本功能及 count_include_pad、ceil_mode 等特定参数, 同时包含 Dropout 和 Flatten 的行为验证。

为了适应不同场景, 套件提供了双重验证策略: * **test_nn_layers_float32.py**: 侧重高精度数值一致性, 要求 NPU 结果与 CPU 基准误差极小 ($1e-3$), 确保推理精确度。* **test_nn_layers_float16.py**: 模拟混合精度推理, 输入和模型转为半精度在 NPU 执行, 结果转回 FP32 对比。考虑到量化损失, 容差适度放宽 ($1e-2$ 至 $1e-1$), 重点验证低精度模式下的逻辑正确性与功能完备性。此外, 为防止嵌入式设备内存溢出 (OOM), 每个测试类均实现了 teardown_method, 在用例执行后自动清理 NPU 缓存。

3.5.3. 测试结果摘要

运行该测试套件前, 请确保在 npu 虚拟环境中已安装 pytest:

```
1 pip3 install pytest
```

随后在终端进入相应的 test 文件夹并运行:

```
1 pytest -v ./
```

在昇腾 310B (CANN 8.3.RC1 + PyTorch 2.8.0) 环境下, 典型测试结果如下:

```
1 ===== test session starts
  ↳ =====
2 ...
3 FAILED test_nn_layers_float16.py::TestNNLayersFloat16::
  ↳ test_avgpool_float16_default_behavior[3-1-1]
4 FAILED test_nn_layers_float16.py::TestNNLayersFloat16::
  ↳ test_avgpool_float16_default_behavior[3-2-1]
```



```

5 FAILED test_nn_layers_float16.py::TestNNLayersFloat16::
  ↪ test_avgpool_float16[True-3-1-1]
6 FAILED test_nn_layers_float16.py::TestNNLayersFloat16::
  ↪ test_avgpool_float16[True-3-2-1]
7 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↪ test_maxpool_float32_default_behavior[2-2-0]
8 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↪ test_maxpool_float32_default_behavior[3-2-0]
9 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↪ test_maxpool_float32_default_behavior[3-2-1]
10 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↪ test_maxpool_float32[False-2-2-0]
11 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↪ test_maxpool_float32[False-3-2-0]
12 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↪ test_maxpool_float32[False-3-2-1]
13 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↪ test_maxpool_float32[True-2-2-0]
14 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↪ test_maxpool_float32[True-3-2-0]
15 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↪ test_maxpool_float32[True-3-2-1]
16 ===== 76 passed, 13 failed in 123.45s =====

```

测试日志显示, 绝大多数基础算子 (Add, MatMul) 和核心网络层 (Linear, Conv2d, BatchNorm, Activations) 在 FP16 和 FP32 模式下均通过验证, 证明 NPU 基本计算功能正常。但测试也暴露了两个显著异常, 需重点关注:

- FP16 平均池化层 (AvgPool) 的边界精度问题在 test_nn_layers_float16.py 的测试中, AvgPool2d 算子表现出特定的边界精度异常, 具体体现为当 kernel_size ↪ =3 且 padding=1 时测试均告失败, 而 kernel_size=2 且无填充时则能顺利通过。这一现象表明 NPU 在处理半精度 (FP16) 池化边界填充 (Padding) 时, 累加计算的精度损失可能超出了预设容差范围, 或者底层硬件对 Padding 区域 “0” 值的处理逻辑与 CPU 存在微小差异。因此, 开发者在推理代码中若使用 FP16 模式且包含带 Padding 的奇数核 AvgPool 层, 需特别警惕潜在的精度下降风险。
- FP32 最大池化层 (MaxPool) 的系统性失效在 test_nn_layers_float32.py 中, 所有 MaxPool2d 测试用例 (共 9 个) 全部失败, 与 FP16 模式下全部通过的表现形成鲜明对比, 这揭示了一个严重的系统性故障。FP16 正常而 FP32 全面失败, 极可能是当前版本 CANN 驱动或 PyTorch 插件在 FP32 格式 MaxPool 算子实现上存在 Bug, 亦或是数据排布未对齐导致计算错误甚至产生 NaN/Inf。鉴于此为高优先级阻碍性问题 (Blocker), 建议暂时避免在 NPU 上使用 FP32 格式的 MaxPool, 或将其回退至 CPU 执行, 直至驱动版本修复。

开发者在迁移自定义模型前, 建议优先运行此测试套件以排查环境潜在问题。

3.6. 现代卷积神经网络的移植

3.6.1. AlexNet

AlexNet 是由 Alex Krizhevsky、Ilya Sutskever 和 Geoffrey Hinton 在 2012 年提出的卷积神经网络。它在当年的 ImageNet 大规模视觉识别挑战赛（ILSVRC）中以压倒性的优势夺冠，将 Top-5 错误率降低到了 15.3%，远超第二名的 26.2%。AlexNet 的成功标志着深度学习在计算机视觉领域的统治地位的确立，引发了卷积神经网络（CNN）研究的热潮。

AlexNet 网络结构

AlexNet 的网络结构比之前的 LeNet 更深、更宽。AlexNet 主要包含以下特点：1. **层数**：包含 8 层神经网络，其中前 5 层是卷积层，后 3 层是全连接层。2. **激活函数**：首次在 CNN 中成功使用了 ReLU（Rectified Linear Unit）激活函数，解决了深层网络训练时的梯度消失问题，并加速了收敛。3. **Dropout**：在全连接层引入了 Dropout 技术，随机忽略一部分神经元，有效防止了过拟合。4. **数据增强**：使用了图像翻转、裁剪等数据增强技术来扩充数据集。5. **多 GPU 训练**：由于当时显存限制，AlexNet 被设计为在两块 GTX 580 GPU 上并行训练。6. **重叠池化（Overlapping Pooling）**：AlexNet 使用了重叠的最大池化（Max Pooling）。传统的池化层通常是不重叠的（步长等于池化核大小），而 AlexNet 使用了 3×3 的池化核和 2 的步长，这种重叠池化稍微减轻了过拟合。

原始的 AlexNet 是为 ImageNet 数据集设计的，但在昇腾 310B 开发板上使用庞大的 ImageNet 进行训练会消耗大量时间与算力。ImageNet 是一个庞大的视觉数据库，包含超过 1400 万张标注图片，涵盖 2 万多个类别，其常用的 ILSVRC 子集也有 1000 个类别和 128 万张训练图片，且图片分辨率较高（通常裁剪为 224×224 ）。相比之下，CIFAR-10 数据集规模较小，仅包含 60000 张 32×32 的彩色图像，分为 10 个类别（如飞机、汽车、鸟类等），每类 6000 张。虽然 CIFAR-10 的图像分辨率和类别数远小于 ImageNet，但它保留了自然图像的复杂性，非常适合用于快速验证模型结构和硬件移植的可行性。为了更高效地验证 AlexNet 在昇腾 310B 上的迁移性能，本节改用 CIFAR-10 数据集进行实验。

为此，我们需要对 AlexNet 的结构进行适配性修改：首先，将网络输入尺寸从 $3 \times 224 \times 224$ 调整为 $3 \times 32 \times 32$ ，并将全连接层的输出类别数从 1000 改为 10。

此外，在上一节的测试中我们发现，昇腾 310B 的 PyTorch 插件目前尚未支持 MaxPool2d 算子，因此我们将该算子替换为 AvgPool2d。这一改动虽然是为了适配硬件限制，但也会对模型特性产生一定影响：**最大池化（Max Pooling）** 倾向于提取图像中

最显著的特征（如纹理、边缘），具有一定的平移不变性，通常能保留较强的特征响应；而 **平均池化（Average Pooling）** 则是计算区域内的平均值，倾向于保留背景信息和平滑图像，可能会模糊掉一些尖锐的特征细节。在 CIFAR-10 这种低分辨率数据集上，使用平均池化可能会导致模型对关键特征的捕捉能力略微下降，从而轻微影响最终的分类准确率，但它能保证模型在当前硬件环境下的顺利运行。修改后的 AlexNet 网络结构如下图所示：

AlexNet 的代码实现

为了验证 AlexNet 在昇腾 310B 上移植的可行性，我们首先建立了一个测试流程。利用 pytest 框架，我们对网络的每一层结构进行了逐一验证，重点对比了 torch_npu（NPU 后端）与 CPU 计算结果的一致性，同时检测了网络算子的硬件兼容性。为此，我们编写了专门的测试例程，分别验证了 AlexNet 在 FP16（半精度）和 FP32（全精度）模式下的运行正确性。相关测试代码均位于 samples/chapter2/AlexNet/test 目录下。经 pytest 测试确认，无论是 FP32 还是 FP16 精度，模型在昇腾 310B 上的计算结果均正确无误。尽管前述的逐层单元测试确认了 AlexNet 在 FP16 和 FP32 精度下的计算正确性，但在实际的完整模型训练中，硬件资源的限制成为了关键因素。实测表明，在 **8T 算力 8GB 内存版本** 的昇腾 310B 开发板上进行 FP32 全精度训练时，由于图编译阶段对内存消耗较大，极易遭遇编译失败的问题；而在 **8T 算力 16GB 内存版本** 的开发板上，FP32 全精度训练则能顺利跑通。基于上述适配后的网络结构，以下提供了针对 CIFAR-10 数据集的完整 FP16 半精度训练与验证代码：

```

1 import os
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from torch.utils.data import TensorDataset, DataLoader
7 from torchvision import datasets
8 import matplotlib.pyplot as plt
9 from tqdm.auto import tqdm, trange
10 from torchvision import transforms
11
12 device = torch.device("npu")
13
14 class AlexNet(nn.Module):
15     def __init__(self, num_classes=10):
16         super(AlexNet, self).__init__()
17         self.features = nn.Sequential(
18             # Layer 1
19             nn.Conv2d(3, 96, kernel_size=11, stride=4, padding=1),
20             nn.ReLU(inplace=True),
21             nn.AvgPool2d(kernel_size=3, stride=2),
22             # Layer 2

```

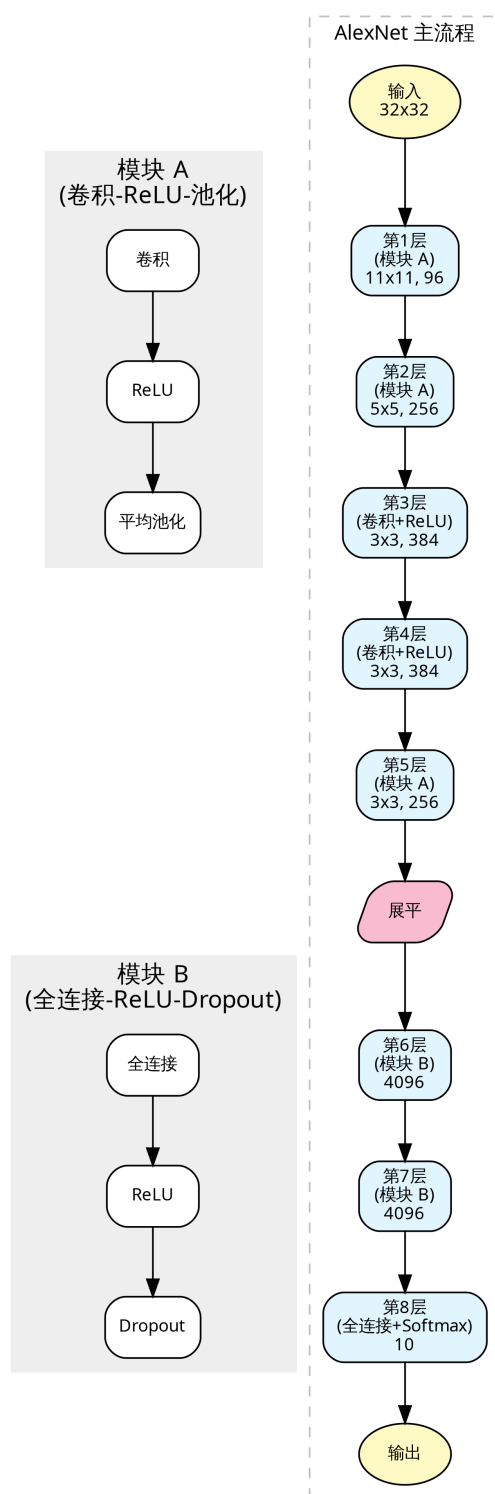


Figure 3.7.: AlexNet

```

23         nn.Conv2d(96, 256, kernel_size=5, padding=2),
24         nn.ReLU(inplace=True),
25         nn.AvgPool2d(kernel_size=3, stride=2),
26         # Layer 3
27         nn.Conv2d(256, 384, kernel_size=3, padding=1),
28         nn.ReLU(inplace=True),
29         # Layer 4
30         nn.Conv2d(384, 384, kernel_size=3, padding=1),
31         nn.ReLU(inplace=True),
32         # Layer 5
33         nn.Conv2d(384, 256, kernel_size=3, padding=1),
34         nn.ReLU(inplace=True),
35         nn.AvgPool2d(kernel_size=3, stride=2),
36     )
37
38     self.classifier = nn.Sequential(
39         nn.Flatten(),
40         nn.Linear(6400, 4096),
41         nn.ReLU(inplace=True),
42         nn.Dropout(0.5),
43         nn.Linear(4096, 4096),
44         nn.ReLU(inplace=True),
45         nn.Dropout(0.5),
46         nn.Linear(4096, num_classes),
47     )
48
49     def forward(self, x):
50         x = self.features(x)
51         x = self.classifier(x)
52         return x
53
54 def load_data(script_dir, batch_size=4):
55     """
56     加载 CIFAR-10 数据集并进行预处理
57     """
58     # 引入 transforms 用于图像变换
59
60     # 数据存储路径
61     data_root = os.path.join(script_dir, "data")
62
63     # 定义预处理流程:
64     # 1. Resize: 将 CIFAR-10 的 32x32 图像调整为 224x224, 以匹配 AlexNet
65     #    ↪ 输入要求
66     # 2. ToTensor: 将图像数据转换为 Tensor (C, H, W), 并将像素值归一化到
67     #    ↪ [0, 1]
68     # 3. Normalize: 对 RGB 三个通道进行标准化
69     transform = transforms.Compose([
70         transforms.Resize((224, 224)),
71         transforms.ToTensor(),
72         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
73         transforms.Lambda(lambda x: x.half())
74     ])
75
76     # 使用 torchvision.datasets 下载并加载 CIFAR-10 数据

```

```

75 # 传入 transform 参数, DataLoader 读取数据时会自动进行 Resize 和
    ↳ ToTensor
76 train_ds = datasets.CIFAR10(data_root, train=True, download=True,
    ↳ transform=transform)
77 test_ds = datasets.CIFAR10(data_root, train=False, download=True,
    ↳ transform=transform)
78
79 # 构建 DataLoader
80 # 相比原代码一次性加载到内存, 这里使用 lazy loading 方式, 避免 Resize
    ↳ 后占用过多内存
81 train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=
    ↳ True)
82 val_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False
    ↳ )
83 return train_loader, val_loader
84
85 def train_and_eval(train_loader, val_loader, epochs=10, lr=0.01):
86     """
87     执行模型训练与验证循环
88     """
89     # 初始化模型并迁移至 NPU 设备
90     model = AlexNet().half().to(device)
91     # 定义损失函数: 交叉熵损失, 适用于多分类问题
92     criterion = nn.CrossEntropyLoss()
93     # 定义优化器: 随机梯度下降 (SGD)
94     optimizer = optim.SGD(model.parameters(), lr=lr)
95
96     train_acc_history = []
97     val_acc_history = []
98     train_loss_history = []
99     val_loss_history = []
100
101     # 使用 trange 展示总进度条
102     t = trange(epochs, desc="Epochs")
103     for epoch in t:
104         # --- 训练阶段 ---
105         model.train() # 设置模型为训练模式 (启用 Dropout, BatchNorm 等)
106         correct = 0; total = 0
107         running_loss = 0.0
108         for inputs, labels in train_loader:
109             # 数据迁移至 NPU, 保持 float32
110             inputs = inputs.to(device); labels = labels.to(device)
111
112             # 前向传播
113             outputs = model(inputs)
114             # 计算损失
115             loss = criterion(outputs, labels)
116
117             # 反向传播与优化
118             optimizer.zero_grad() # 清空梯度
119             loss.backward()        # 计算梯度
120             optimizer.step()       # 更新参数
121
122             # 统计损失与精度

```

```

123         running_loss += loss.item() * labels.size(0)
124         # 获取预测结果 (最大概率对应的索引)
125         preds = outputs.argmax(dim=1)
126         correct += (preds == labels).sum().item()
127         total += labels.size(0)
128
129         # 计算本 epoch 的平均训练精度和损失
130         train_acc = correct / total
131         train_loss = running_loss / total
132         train_acc_history.append(train_acc)
133         train_loss_history.append(train_loss)
134
135         # --- 验证阶段 ---
136         model.eval() # 设置模型为评估模式
137         correct_v = 0; total_v = 0
138         running_loss_v = 0.0
139         with torch.no_grad(): # 验证时不计算梯度, 节省显存和计算资源
140             for inputs, labels in val_loader:
141                 inputs = inputs.to(device); labels = labels.to(device)
142                 outputs = model(inputs)
143                 loss_v = criterion(outputs, labels)
144                 running_loss_v += loss_v.item() * labels.size(0)
145                 preds = outputs.argmax(dim=1)
146                 correct_v += (preds == labels).sum().item()
147                 total_v += labels.size(0)
148
149         # 计算本 epoch 的平均验证精度和损失
150         val_acc = correct_v / total_v
151         val_loss = running_loss_v / total_v
152         val_acc_history.append(val_acc)
153         val_loss_history.append(val_loss)
154         t.set_postfix(train_acc=f"{train_acc:.4f}", val_loss=f"{val_loss:.4f}",
155                       val_acc=f"{val_acc:.4f}", lr=f'{optimizer.param_groups[0]["lr"]:.6f}')
156
157         # 返回训练/验证指标历史
158         return train_acc_history, val_acc_history, train_loss_history, val_loss_history
159
160 def plot_metrics(script_dir, train_acc, val_acc, train_loss, val_loss):
161     """
162     绘制训练和验证的精度与损失曲线
163     """
164     plt.figure(figsize=(12, 5))
165
166     # 绘制精度曲线
167     plt.subplot(1, 2, 1)
168     plt.plot(train_acc, label='Train Accuracy')
169     plt.plot(val_acc, label='Validation Accuracy')
170     plt.title('Accuracy over Epochs')
171     plt.xlabel('Epoch')
172     plt.ylabel('Accuracy')
173     plt.legend()

```

```

174     # 绘制损失曲线
175     plt.subplot(1, 2, 2)
176     plt.plot(train_loss, label='Train Loss')
177     plt.plot(val_loss, label='Validation Loss')
178     plt.title('Loss over Epochs')
179     plt.xlabel('Epoch')
180     plt.ylabel('Loss')
181     plt.legend()
182
183     # 保存图片
184     save_path = os.path.join(script_dir, "training_metrics.png")
185     plt.savefig(save_path)
186     print(f"Metrics plot saved to {save_path}")
187
188 if __name__ == "__main__":
189     # 获取当前脚本目录，方便保存文件
190     script_dir = os.path.dirname(os.path.abspath(__file__))
191
192     # 加载数据
193     train_loader, val_loader = load_data(script_dir, batch_size=8)
194     # 开始训练
195     train_acc_history, val_acc_history, train_loss_history,
196         ↪ val_loss_history = train_and_eval(
197         ↪     train_loader, val_loader, epochs=10, lr=0.01
198     )
199     # 绘制结果
200     plot_metrics(script_dir, train_acc_history, val_acc_history,
201         ↪ train_loss_history, val_loss_history)

```

在前面章节创建的 npu 虚拟环境中运行上述代码，昇腾 310B 开发板将输出如下执行结果：

```

1 Epochs: 100%|
  ↪
  ↪ 10/10 [2:26:50<00:00, 881.05s/it, lr=0.010000, train_acc=0.8195,
  ↪ val_acc=0.7798, val_loss=0.6680]

```

实验结果对比分析

为了评估昇腾 310B 在运行 AlexNet 模型时的实际性能，我们将上述代码部署至昇腾 310B 开发板（NPU），并引入高性能桌面显卡 GeForce 5090D 作为对比基准。实验设计涵盖了昇腾 310B 与 GeForce 5090D 在 FP16 半精度及 FP32 全精度模式下的表现，以期提供客观的性能参考。

需要特别说明的是，全精度（FP32）训练对内存资源有较高要求。实测表明，**8GB 内存版本**的昇腾 310B 开发板在执行 AlexNet 的 FP32 训练任务时，会因内存不足触发图编译错误（OOM）。因此，若需在昇腾 310B 上进行 FP32 训练，必须使用 **16GB 内存版本**；而 FP16 半精度训练则能有效降低显存占用，在 8GB 版本上即可顺利运行。

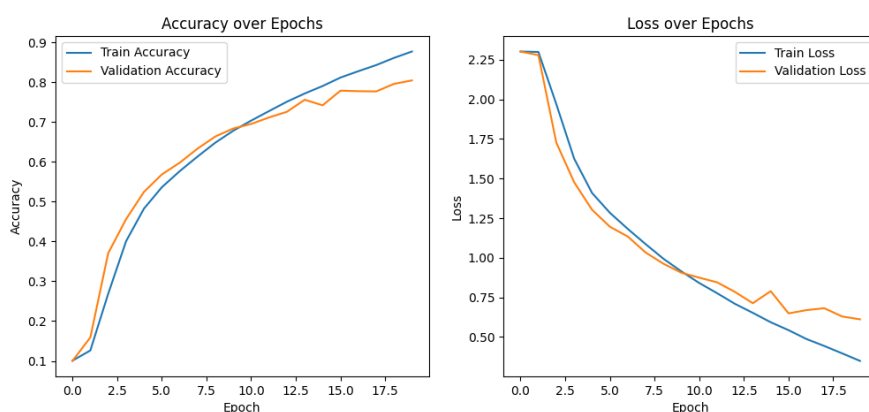


Figure 3.8.: training metrics NPU FP16

各实验配置的终端输出日志记录如下：

- **Ascend 310B (FP16)**

```
1 Epochs: 100%|██████████| 20/20 [4:00:05<00:00, 720.29s/it, lr
  ↳ =0.010000, train_acc=0.8770, val_acc=0.8044, val_loss=0.6110]
```

- **Ascend 310B (FP32)**

```
1 Epochs: 100%|██████████| 20/20 [10:52:39<00:00, 1957.99s/it, lr
  ↳ =0.010000, train_acc=0.9693, val_acc=0.8219, val_loss=0.7721]
```

- **GeForce 5090D (FP16)**

```
1 Epochs: 100%|██████████| 20/20 [08:38<00:00, 25.93s/it, lr
  ↳ =0.010000, train_acc=0.8787, val_acc=0.7988, val_loss=0.6389]
```

- **GeForce 5090D (FP32)**

```
1 Epochs: 100%|██████████| 20/20 [11:01<00:00, 33.08s/it, lr
  ↳ =0.010000, train_acc=0.9688, val_acc=0.8252, val_loss=0.7670]
```

详细的实验数据对比汇总如下表所示：

硬件平台	精度模式	总耗时	单 Epoch 平均耗时	训练集精 度 (Train Acc)	验证集精 度 (Val Acc)	验证集损 失 (Val Loss)
		(20 Epochs)				
Ascend 310B	FP16 (半 精度)	240.1 min	12.0 min	87.70%	80.44%	0.6110

硬件平台	精度模式	总耗时	单 Epoch 平均耗时	训练集精	验证集精	验证集损
		(20 Epochs)		度 (Train Acc)	度 (Val Acc)	失 (Val Loss)
Ascend 310B	FP32 (全精度)	652.7 min	32.6 min	96.93%	82.19%	0.7721
GeForce 5090D	FP16 (半精度)	8.6 min	0.43 min	87.87%	79.88%	0.6389
GeForce 5090D	FP32 (全精度)	11.0 min	0.55 min	96.88%	82.52%	0.7670

首先，从训练速度来看，虽然昇腾 310B 的训练耗时明显长于桌面级高性能显卡，但这完全符合其硬件定位。昇腾 310B 本质上是一款面向边缘计算场景的低功耗 AI 芯片，其设计初衷是在有限的功耗预算下提供高效的推理能力，而非大规模模型训练。尽管如此，实验结果表明昇腾 310B 依然稳定地完成了整个训练流程，这有力地证明了它不仅能够胜任推理任务，也具备在端侧进行轻量级训练或模型微调的潜力，为边缘侧的持续学习提供了硬件基础。

其次，在精度表现方面，我们可以观察到不同精度与不同硬件平台之间的高度一致性。首先，在 FP32 全精度模式下，昇腾 310B 的验证集准确率达到了 82.19%，与基准设备 GeForce 5090D 的 82.52% 极为接近。这充分验证了昇腾 310B 在全精度计算逻辑上的正确性，未出现因架构差异导致的数值漂移。

其次，对比同平台的 FP16 与 FP32 结果，精度下降是普遍存在的现象：基准设备准确率下降了约 2.6%（从 82.52% 降至 79.88%），而昇腾 310B 下降了约 1.7%（从 82.19% 降至 80.44%）。这种精度损失属于半精度动态范围压缩带来的正常物理特性，但昇腾 310B 的降幅相对更小，说明其半精度算子实现具有良好的数值稳定性。

最后，横向对比 FP16 模式，昇腾 310B（80.44%）的表现与基准设备（79.88%）几乎持平甚至略优，进一步证明了将模型移植到昇腾 NPU 并采用半精度加速时，能够很好地保持模型的收敛特性和最终性能。

综上所述，实验结果证实了昇腾 310B 能够正确且有效地执行 AlexNet 的训练任务，且精度表现符合预期。虽然其训练速度无法与高端桌面显卡相提并论，但考虑到其低功耗特性和边缘部署的定位，该性能足以满足小规模数据集的迁移学习或现场微调需求。对于大规模的深度学习训练任务，更合理的流程应当是在高性能服务器（如昇腾 910 或 GPU 集群）上完成模型训练，随后再将训练好的模型量化或转换至昇腾 310B 上进行高效的推理部署。

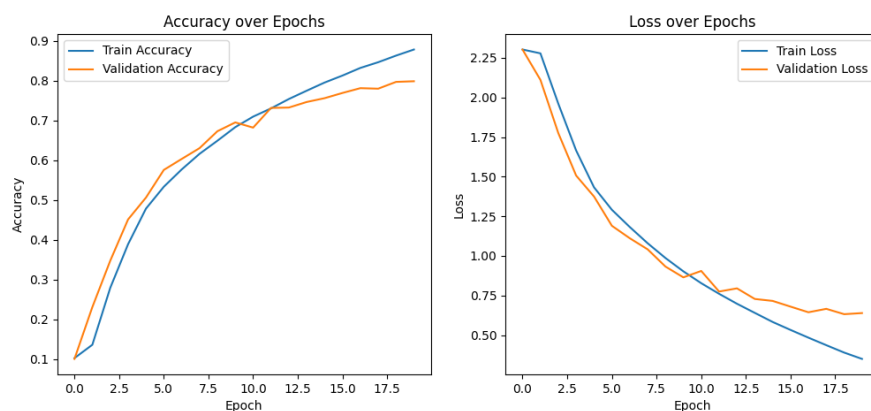


Figure 3.9.: training metrics cuda FP32

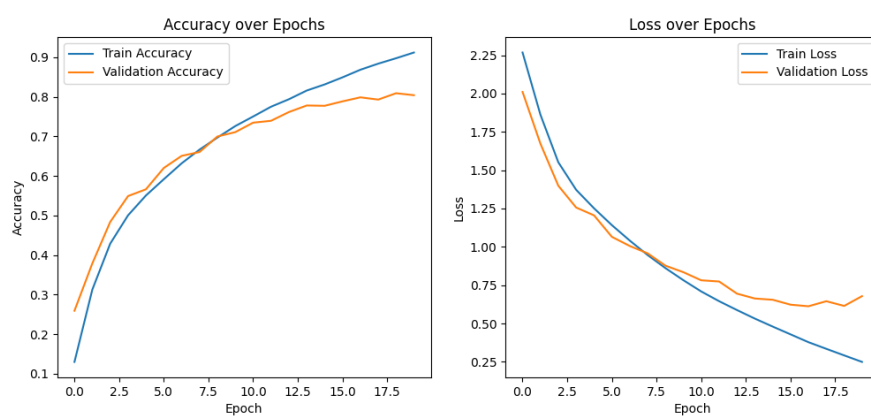


Figure 3.10.: training metrics cuda FP32

3.6.2. VGG

VGG (Visual Geometry Group) 网络是由牛津大学的 Karen Simonyan 和 Andrew Zisserman 在 2014 年提出的。它在当年的 ILSVRC 挑战赛中取得了定位任务第一名和分类任务第二名的优异成绩。VGG 的主要贡献在于证明了使用小卷积核并增加网络深度能够有效提升模型性能。

VGG 的网络结构

VGG 的基本结构特点非常简洁：**1. 统一的小卷积核**：整个网络全部使用 3×3 的卷积核和 2×2 的最大池化层。通过堆叠多个 3×3 卷积层来获得与大卷积核相同的感受野（例如 2 个 3×3 相当于 1 个 5×5 ），同时减少了参数量并增加了非线性变换。**2. 深度结构**：相比 AlexNet，VGG 的层数大幅增加，常用的配置包括 VGG-16 和 VGG-19，分别拥有 16 层和 19 层带权重的层。**3. 通道数翻倍**：在每个池化层之后，特征图的通道数通常会翻倍，直到达到 512。

VGG 的代码实现

为了灵活构建不同深度的 VGG 网络（如 VGG11, VGG13, VGG16, VGG19），代码采用了配置驱动的设计模式。首先，定义一个字典 `cfg`，其中键为网络名称，值为列表。列表中的数字表示卷积层的输出通道数，字符 '`M`' 则代表最大池化层。这种设计使得网络结构的定义变得直观且易于修改。

接着，在 VGG 类的初始化函数中，通过调用 `_make_layers` 方法解析配置列表。该方法遍历配置列表：遇到数字时，构建一个 3×3 的卷积层（`padding=1` 以保持尺寸）、Batch Normalization 层和 ReLU 激活函数，并更新输入通道数；遇到 '`M`' 时，则添加一个 2×2 的池化层。这里特别针对 CIFAR-10 数据集（ 32×32 ）进行了适配，移除了原始 VGG 最后的三个全连接层，改用一个单一的线性层作为分类器，并且为了适配昇腾 310B 目前对 MaxPool 的支持限制，将部分池化操作调整为 AvgPool。

具体的 VGG16 的代码实现如下：

```
1 import os
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from torch.utils.data import TensorDataset, DataLoader
7 from torchvision import datasets
8 import matplotlib.pyplot as plt
9 from tqdm.auto import tqdm, trange
10 from torchvision import transforms
11
```

```

12 device = torch.device("npu")
13
14 cfg = {
15     'VGG11': [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512,
16         ↪ 'M'],
17     'VGG13': [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M',
18         ↪ 512, 512, 'M'],
19     'VGG16': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512,
20         ↪ 512, 'M', 512, 512, 512, 'M'],
21     'VGG19': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512,
22         ↪ 512, 512, 512, 'M', 512, 512, 512, 512, 'M'],
23 }
24
25 class VGG(nn.Module):
26     def __init__(self, vgg_name, num_classes=10):
27         super(VGG, self).__init__()
28         self.features = self._make_layers(cfg[vgg_name])
29         self.classifier = nn.Linear(512, num_classes)
30
31     def forward(self, x):
32         out = self.features(x)
33         out = out.view(out.size(0), -1)
34         out = self.classifier(out)
35         return out
36
37     def _make_layers(self, cfg):
38         layers = []
39         in_channels = 3
40         for x in cfg:
41             if x == 'M':
42                 layers += [nn.AvgPool2d(kernel_size=2, stride=2)]
43             else:
44                 layers += [nn.Conv2d(in_channels, x, kernel_size=3,
45                     ↪ padding=1),
46                     nn.BatchNorm2d(x),
47                     nn.ReLU(inplace=True)]
48                 in_channels = x
49         layers += [nn.AvgPool2d(kernel_size=1, stride=1)]
50         return nn.Sequential(*layers)
51
52 def load_data(script_dir, batch_size=4):
53     """
54     加载 CIFAR-10 数据集并进行预处理
55     """
56     # 引入 transforms 用于图像变换
57
58     # 数据存储路径
59     data_root = os.path.join(script_dir, "data")
60
61     # 定义预处理流程：
62     # 1. ToTensor: 将图像数据转换为 Tensor (C, H, W)，并将像素值归一化到
63     ↪ [0, 1]
64     # 2. Normalize: 对 RGB 三个通道进行标准化
65     # 注意：针对CIFAR-10优化的VGG使用原始32x32输入，不需要Resize到224

```

```

60     transform = transforms.Compose([
61         transforms.ToTensor(),
62         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
63     ])
64
65     # 使用 torchvision.datasets 下载并加载 CIFAR-10 数据
66     # 传入 transform 参数, DataLoader 读取数据时会自动进行 Resize 和
67     #   ↳ ToTensor
68     train_ds = datasets.CIFAR10(data_root, train=True, download=True,
69     #   ↳ transform=transform)
70     test_ds = datasets.CIFAR10(data_root, train=False, download=True,
71     #   ↳ transform=transform)
72
73     # 构建 DataLoader
74     # 相比原代码一次性加载到内存, 这里使用 lazy loading 方式, 避免 Resize
75     #   ↳ 后占用过多内存
76     train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=
77     #   ↳ True)
78     val_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False
79     #   ↳ )
80     return train_loader, val_loader
81
82 def train_and_eval(train_loader, val_loader, epochs=10, lr=0.01):
83     """
84     执行模型训练与验证循环
85     """
86     # 初始化模型并迁移至 NPU 设备
87     model = VGG('VGG16').to(device)
88     # 定义损失函数: 交叉熵损失, 适用于多分类问题
89     criterion = nn.CrossEntropyLoss()
90     # 定义优化器: 随机梯度下降 (SGD)
91     optimizer = optim.SGD(model.parameters(), lr=lr)
92
93     train_acc_history = []
94     val_acc_history = []
95     train_loss_history = []
96     val_loss_history = []
97
98     # 使用 trange 展示总进度条
99     t = trange(epochs, desc="Epochs")
100     for epoch in t:
101         # --- 训练阶段 ---
102         model.train() # 设置模型为训练模式 (启用 Dropout, BatchNorm 等)
103         correct = 0; total = 0
104         running_loss = 0.0
105         for inputs, labels in train_loader:
106             # 数据迁移至 NPU, 保持 float32
107             inputs = inputs.to(device); labels = labels.to(device)
108
109             # 前向传播
110             outputs = model(inputs)
111             # 计算损失
112             loss = criterion(outputs, labels)

```

```

108         # 反向传播与优化
109         optimizer.zero_grad() # 清空梯度
110         loss.backward()       # 计算梯度
111         optimizer.step()      # 更新参数
112
113         # 统计损失与精度
114         running_loss += loss.item() * labels.size(0)
115         # 获取预测结果 (最大概率对应的索引)
116         preds = outputs.argmax(dim=1)
117         correct += (preds == labels).sum().item()
118         total += labels.size(0)
119
120         # 计算本 epoch 的平均训练精度和损失
121         train_acc = correct / total
122         train_loss = running_loss / total
123         train_acc_history.append(train_acc)
124         train_loss_history.append(train_loss)
125
126         # --- 验证阶段 ---
127         model.eval() # 设置模型为评估模式
128         correct_v = 0; total_v = 0
129         running_loss_v = 0.0
130         with torch.no_grad(): # 验证时不计算梯度, 节省显存和计算资源
131             for inputs, labels in val_loader:
132                 inputs = inputs.to(device); labels = labels.to(device)
133                 outputs = model(inputs)
134                 loss_v = criterion(outputs, labels)
135                 running_loss_v += loss_v.item() * labels.size(0)
136                 preds = outputs.argmax(dim=1)
137                 correct_v += (preds == labels).sum().item()
138                 total_v += labels.size(0)
139
140         # 计算本 epoch 的平均验证精度和损失
141         val_acc = correct_v / total_v
142         val_loss = running_loss_v / total_v
143         val_acc_history.append(val_acc)
144         val_loss_history.append(val_loss)
145         t.set_postfix(train_acc=f"{train_acc:.4f}", val_loss=f"{val_loss:.4f}", val_acc=f"{val_acc:.4f}", lr=f'{optimizer.param_groups[0]["lr"]:.6f}')
146
147         # 返回训练/验证指标历史
148         return train_acc_history, val_acc_history, train_loss_history, val_loss_history
149
150 def plot_metrics(script_dir, train_acc, val_acc, train_loss, val_loss):
151     """
152     绘制训练和验证的精度与损失曲线
153     """
154     plt.figure(figsize=(12, 5))
155
156     # 绘制精度曲线
157     plt.subplot(1, 2, 1)
158     plt.plot(train_acc, label='Train Accuracy')

```

```

159 plt.plot(val_acc, label='Validation Accuracy')
160 plt.title('Accuracy over Epochs')
161 plt.xlabel('Epoch')
162 plt.ylabel('Accuracy')
163 plt.legend()
164
165 # 绘制损失曲线
166 plt.subplot(1, 2, 2)
167 plt.plot(train_loss, label='Train Loss')
168 plt.plot(val_loss, label='Validation Loss')
169 plt.title('Loss over Epochs')
170 plt.xlabel('Epoch')
171 plt.ylabel('Loss')
172 plt.legend()
173
174 # 保存图片
175 save_path = os.path.join(script_dir, "training_metrics.png")
176 plt.savefig(save_path)
177 print(f"Metrics plot saved to {save_path}")
178
179 if __name__ == "__main__":
180     # 获取当前脚本目录，方便保存文件
181     script_dir = os.path.dirname(os.path.abspath(__file__))
182
183     # 加载数据
184     train_loader, val_loader = load_data(script_dir, batch_size=16)
185     # 开始训练
186     train_acc_history, val_acc_history, train_loss_history,
187         ↪ val_loss_history = train_and_eval(
188         train_loader, val_loader, epochs=10, lr=0.01
189     )
190     # 绘制结果
191     plot_metrics(script_dir, train_acc_history, val_acc_history,
192         ↪ train_loss_history, val_loss_history)

```

实验结果分析

在本节的实验设计中，我们仅针对 FP32（全精度）模式进行了测试与分析，未涵盖 FP16（半精度）模式。这主要是出于对数值稳定性的考量：VGG 网络广泛使用了 Batch Normalization (BN) 层，该层在计算均值和方差时对精度极其敏感。如果强制使用 FP16 进行全网训练，BN 层的统计量极易出现数值溢出或下溢，导致训练过程不稳定甚至不收敛。虽然自动混合精度（AMP）技术通常能缓解这一问题，但鉴于当前昇腾 310B 的 torch_npu 插件在混合精度算子调度上的支持尚在完善中，为确保实验结果的可靠性与可复现性，我们统一选用了 FP32 精度进行验证。

为了提供客观的性能参照，我们将同一套代码逻辑部署到了两种截然不同的硬件平台上：一是面向边缘计算的昇腾 310B 开发板，二是代表当前桌面级算力巅峰的 GeForce 5090D 显卡。得益于 PyTorch 良好的跨平台抽象能力，只需将代码中的设备指定为

`device = torch.device("cuda")`，即可无缝切换至 NVIDIA GPU 环境运行。这种对比不仅展示了不同算力层级下的性能表现，也验证了代码的通用性。

值得一提的是，本示例代码经过精心优化，对显存占用进行了有效控制。实测表明，它完全兼容最低配置的昇腾 310B 开发板（8T 算力，8GB 内存版本），在资源受限的边缘设备上依然能够顺利完成 VGG16 这样深层网络的训练任务。详细的运行结果对比如下：

• Ascend 310B (FP32)

```
1 Epochs: 100%|██████████| 10/10 [6:12:02<00:00, 2232.28s/it, lr  
  ↳ =0.010000, train_acc=0.9614, val_acc=0.8349, val_loss=0.5991]
```

• GeForce 5090D (FP32)

```
1 Epochs: 100%|██████████| 10/10 [03:25<00:00, 20.60s/it, lr  
  ↳ =0.010000, train_acc=0.9631, val_acc=0.8421, val_loss=0.5745]
```

上述实验数据直观地展示了边缘计算芯片与旗舰级训练显卡在 VGG 网络训练任务上的巨大性能鸿沟与功能同质性。首先在**训练效率**方面，GeForce 5090D 凭借其惊人的算力与显存带宽，仅耗时 **3 分 25 秒**便完成了 10 个 Epoch 的训练，平均每轮约 20 秒，对于 CIFAR-10 规模的数据集几乎实现了“立等可取”的体验。相比之下，昇腾 310B 的总耗时长达 **6 小时 12 分**，平均每轮约 37 分钟，两者的训练速度差距超过 **100 倍**。这一悬殊差距主要源于昇腾 310B 的硬件定位——它是一款专为低功耗推理设计的 NPU，FP32 算力相对有限，且 VGG 网络本身参数量巨大（VGG16 约包含 1.38 亿参数），计算密度极高，这对边缘端芯片的显存带宽和计算单元构成了极大的压力。

尽管速度差异巨大，但两者在**精度一致性**上表现出色。昇腾 310B 的验证集准确率达到了 **83.49%**，与 GeForce 5090D 的 **84.21%** 非常接近。这一结果有力证明了昇腾 310B 在执行复杂的反向传播算法时，其底层算子的计算逻辑是精确无误的。对于算子开发者或模型迁移工程师而言，这意味着可以在低成本的 310B 上验证模型逻辑的正确性（哪怕训练慢一点），确信其计算行为与标准 GPU 是一致的，从而为模型在不同算力平台的部署提供了可靠的验证基准。

3.6.3. ResNet

ResNet (Residual Network) 由何恺明、张祥雨、任少卿和孙剑在 2015 年提出。它横扫了当年的 ILSVRC 和 COCO 竞赛，囊括了多项冠军。ResNet 的核心贡献在于解决了深度神经网络随着层数增加而出现的“退化问题” (Degradation Problem)，使得训练数百层甚至上千层的网络成为可能。

ResNet 的网络结构

ResNet 的基本结构引入了“残差块” (Residual Block):

- 残差学习**: 传统的网络试图直接学习目标映射 $H(x)$, 而 ResNet 试图学习残差映射 $F(x) = H(x) - x$ 。最终的输出变为 $F(x) + x$ 。如果恒等映射是最优的, 网络只需将残差推向零即可, 这比学习恒等映射要容易得多。
- 跳跃连接 (Shortcut Connection)**: 通过恒等映射将输入直接加到卷积层的输出上。这种连接既不增加额外的参数, 也不增加计算复杂度, 却能让梯度在反向传播时更顺畅地流动, 有效缓解了梯度消失问题。
- 极深的网络**: 得益于残差结构, ResNet 可以构建非常深的网络, 常见的版本包括 ResNet-18, ResNet-34, ResNet-50, ResNet-101 和 ResNet-152。

ResNet 的代码实现

原版 ResNet 是面向 ImageNet 数据集设计的, 其标准输入尺寸为 224×224 。若直接在昇腾 310B 上对 ImageNet 进行训练, 将面临巨大的算力与时间成本。为此, 考虑到硬件资源的限制, 我们沿用前文中 AlexNet 的实验思路, 改用 CIFAR-10 数据集进行验证, 并以此为基础对 ResNet 代码进行了深度适配。本实现参考了 TorchVision 官方风格, 旨在确保代码的模块化与可扩展性。其核心设计理念在于将复杂的深度残差网络解耦为独立的组件: 底层的残差单元 (BasicBlock) 负责局部的特征学习与梯度流通, 而上层的网络类 (ResNet) 则专注于宏观的层级堆叠与逻辑组装。这种分层设计不仅使得复现 ResNet-18、34 等不同深度的变体变得轻而易举, 同时也为针对 CIFAR-10 这种小分辨率数据集的定制化修改提供了清晰的切入点。

在具体的代码实现中, 我们首先要设计一个 BasicBlock 类, 这个类需要封装残差网络最基础的构建单元。该模块内部串联了两个标准的 3×3 卷积层, 并配合 Batch Normalization 和 ReLU 激活函数使用。其设计的精髓在于对跳跃连接 (Shortcut Connection) 的巧妙处理: 为了解决残差路径 $F(x)$ 与恒等映射路径 x 在维度不匹配时的相加问题 (例如因步长 `stride=2` 导致尺寸减半, 或通道数增加时), 代码中引入了一个动态的 shortcut 分支。当检测到输入输出维度一致时, 它仅仅是一个空的容器 (即直接导通); 而一旦维度发生变化, 它会自动初始化一个包含 1×1 卷积和 BN 的下采样投影层。这种设计消除了在前向传播逻辑中编写大量条件判断的弊端, 保证了 `out += self.shortcut(x)` 这一核心逻辑的简洁与统一。

为了构建完整的深层网络, ResNet 类引入了 `_make_layer` 函数来实现层级的自动化堆叠。ResNet 的标准架构由四个主要的 Stage 组成, 每个 Stage 包含若干个 BasicBlock。`_make_layer` 负责生成这些 Stage, 它通过接收 `stride` 和 `block` 数量等参数, 自动处理每个 Stage 第一个 Block 可能需要的下采样操作, 同时保证后续 Block 保持特征图尺寸不变。通过这种参数化的构建方式, 仅需改变传入的 `layers` 列表 (如 `[2, 2, 2, 2]`),

即可轻松定义出不同深度的网络结构。

此外，针对 CIFAR-10 数据集仅有 32×32 分辨率的特性，本实现对原始 ResNet 的初始层进行了关键性的适配。原始 ResNet 是为 ImageNet (224×224) 设计的，其第一层采用了 7×7 的大卷积核配合 3×3 最大池化，这会导致特征图瞬间缩小 4 倍，使得 CIFAR-10 的图像在进入深层之前就丢失了过多的 spatial 信息。因此，本代码将第一层重构为标准的 3×3 卷积且步长设为 1，并移除了初始的最大池化层。这一改动最大程度地保留了输入图像的空间分辨率，显著提升了模型在低分辨率数据集上的特征提取能力与最终识别精度。

具体的代码实现如下：

```

1 import os
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.nn.functional as F
6 import torch.optim as optim
7 from torch.utils.data import TensorDataset, DataLoader
8 from torchvision import datasets
9 import matplotlib.pyplot as plt
10 from tqdm.auto import tqdm, trange
11 from torchvision import transforms
12
13 device = torch.device("npu")
14
15 class BasicBlock(nn.Module):
16     expansion = 1
17
18     def __init__(self, in_planes, planes, stride=1):
19         super(BasicBlock, self).__init__()
20         self.conv1 = nn.Conv2d(in_planes, planes, kernel_size=3, stride=
21             ↪ stride, padding=1, bias=False)
22         self.bn1 = nn.BatchNorm2d(planes)
23         self.conv2 = nn.Conv2d(planes, planes, kernel_size=3, stride=1,
24             ↪ padding=1, bias=False)
25         self.bn2 = nn.BatchNorm2d(planes)
26
27         self.shortcut = nn.Sequential()
28         if stride != 1 or in_planes != self.expansion*planes:
29             self.shortcut = nn.Sequential(
30                 nn.Conv2d(in_planes, self.expansion*planes, kernel_size
31                     ↪ =1, stride=stride, bias=False),
32                 nn.BatchNorm2d(self.expansion*planes)
33             )
34
35     def forward(self, x):
36         out = F.relu(self.bn1(self.conv1(x)))
37         out = self.bn2(self.conv2(out))
38         out += self.shortcut(x)
39         out = F.relu(out)
40         return out

```

```

38
39 class ResNet(nn.Module):
40     def __init__(self, block, num_blocks, num_classes=10):
41         super(ResNet, self).__init__()
42         self.in_planes = 64
43
44         self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
45             ↪ bias=False)
46         self.bn1 = nn.BatchNorm2d(64)
47         self.layer1 = self._make_layer(block, 64, num_blocks[0], stride
48             ↪ =1)
49         self.layer2 = self._make_layer(block, 128, num_blocks[1], stride
50             ↪ =2)
51         self.layer3 = self._make_layer(block, 256, num_blocks[2], stride
52             ↪ =2)
53         self.layer4 = self._make_layer(block, 512, num_blocks[3], stride
54             ↪ =2)
55         self.linear = nn.Linear(512*block.expansion, num_classes)
56
57     def _make_layer(self, block, planes, num_blocks, stride):
58         strides = [stride] + [1]*(num_blocks-1)
59         layers = []
60         for stride in strides:
61             layers.append(block(self.in_planes, planes, stride))
62             self.in_planes = planes * block.expansion
63         return nn.Sequential(*layers)
64
65     def forward(self, x):
66         out = F.relu(self.bn1(self.conv1(x)))
67         out = self.layer1(out)
68         out = self.layer2(out)
69         out = self.layer3(out)
70         out = self.layer4(out)
71         out = F.avg_pool2d(out, 4)
72         out = out.view(out.size(0), -1)
73         out = self.linear(out)
74         return out
75
76 def ResNet18():
77     return ResNet(BasicBlock, [2, 2, 2, 2])
78
79 def load_data(script_dir, batch_size=4):
80     """
81     加载 CIFAR-10 数据集并进行预处理
82     """
83     # 引入 transforms 用于图像变换
84
85     # 数据存储路径
86     data_root = os.path.join(script_dir, "data")
87
88     # 定义预处理流程：
89     # 1. ToTensor: 将图像数据转换为 Tensor (C, H, W)，并将像素值归一化到
90     ↪ [0, 1]
91     # 2. Normalize: 对 RGB 三个通道进行标准化

```

```

86 # 注意：针对CIFAR-10优化的VGG使用原始32x32输入，不需要Resize到224
87 transform = transforms.Compose([
88     transforms.ToTensor(),
89     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
90 ])
91
92 # 使用 torchvision.datasets 下载并加载 CIFAR-10 数据
93 # 传入 transform 参数，DataLoader 读取数据时会自动进行 Resize 和
94   ↳ ToTensor
95 train_ds = datasets.CIFAR10(data_root, train=True, download=True,
96   ↳ transform=transform)
97 test_ds = datasets.CIFAR10(data_root, train=False, download=True,
98   ↳ transform=transform)
99
100 # 构建 DataLoader
101 # 相比原代码一次性加载到内存，这里使用 lazy loading 方式，避免 Resize
102   ↳ 后占用过多内存
103 train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=
104   ↳ True)
105 val_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False
106   ↳ )
107 return train_loader, val_loader
108
109 def train_and_eval(train_loader, val_loader, epochs=10, lr=0.01):
110     """
111     执行模型训练与验证循环
112     """
113     # 初始化模型并迁移至 CUDA
114     # 使用 ResNet18 减小深度以适应 310B 内存限制
115     model = ResNet18().to(device)
116     # 定义损失函数：交叉熵损失，适用于多分类问题
117     criterion = nn.CrossEntropyLoss()
118     # 定义优化器：随机梯度下降 (SGD)
119     optimizer = optim.SGD(model.parameters(), lr=lr)
120
121     train_acc_history = []
122     val_acc_history = []
123     train_loss_history = []
124     val_loss_history = []
125
126     # 使用 trange 展示总进度条
127     t = trange(epochs, desc="Epochs")
128     for epoch in t:
129         # --- 训练阶段 ---
130         model.train() # 设置模型为训练模式 (启用 Dropout, BatchNorm 等)
131         correct = 0; total = 0
132         running_loss = 0.0
133         for inputs, labels in train_loader:
134             # 数据迁移至 CUDA (保持 float32)
135             inputs = inputs.to(device); labels = labels.to(device)
136
137             optimizer.zero_grad() # 清空梯度
138
139             # 前向传播 (FP32)

```

```

134         outputs = model(inputs)
135         # 计算损失
136         loss = criterion(outputs, labels)
137
138         # 反向传播与优化
139         loss.backward() # FP32 反向传播
140         optimizer.step() # 更新参数
141
142         # 统计损失与精度
143         running_loss += loss.item() * labels.size(0)
144         # 获取预测结果 (最大概率对应的索引)
145         preds = outputs.argmax(dim=1)
146         correct += (preds == labels).sum().item()
147         total += labels.size(0)
148
149         # 计算本 epoch 的平均训练精度和损失
150         train_acc = correct / total
151         train_loss = running_loss / total
152         train_acc_history.append(train_acc)
153         train_loss_history.append(train_loss)
154
155         # --- 验证阶段 ---
156         model.eval() # 设置模型为评估模式
157         correct_v = 0; total_v = 0
158         running_loss_v = 0.0
159         with torch.no_grad(): # 验证时不计算梯度, 节省显存和计算资源
160             for inputs, labels in val_loader:
161                 inputs = inputs.to(device); labels = labels.to(device)
162
163                 # FP32 推理
164                 outputs = model(inputs)
165                 loss_v = criterion(outputs, labels)
166
167                 running_loss_v += loss_v.item() * labels.size(0)
168                 preds = outputs.argmax(dim=1)
169                 correct_v += (preds == labels).sum().item()
170                 total_v += labels.size(0)
171
172         # 计算本 epoch 的平均验证精度和损失
173         val_acc = correct_v / total_v
174         val_loss = running_loss_v / total_v
175         val_acc_history.append(val_acc)
176         val_loss_history.append(val_loss)
177         t.set_postfix(train_acc=f'{train_acc:.4f}', val_loss=f'{val_loss}
            ↳ :.4f}', val_acc=f'{val_acc:.4f}', lr=f'{optimizer.
            ↳ param_groups[0]["lr"]:.6f}')
178
179         # 返回训练/验证指标历史
180         return train_acc_history, val_acc_history, train_loss_history,
            ↳ val_loss_history
181
182 def plot_metrics(script_dir, train_acc, val_acc, train_loss, val_loss):
183     """
184     绘制训练和验证的精度与损失曲线

```

```

185     """
186     plt.figure(figsize=(12, 5))
187
188     # 绘制精度曲线
189     plt.subplot(1, 2, 1)
190     plt.plot(train_acc, label='Train Accuracy')
191     plt.plot(val_acc, label='Validation Accuracy')
192     plt.title('Accuracy over Epochs')
193     plt.xlabel('Epoch')
194     plt.ylabel('Accuracy')
195     plt.legend()
196
197     # 绘制损失曲线
198     plt.subplot(1, 2, 2)
199     plt.plot(train_loss, label='Train Loss')
200     plt.plot(val_loss, label='Validation Loss')
201     plt.title('Loss over Epochs')
202     plt.xlabel('Epoch')
203     plt.ylabel('Loss')
204     plt.legend()
205
206     # 保存图片
207     save_path = os.path.join(script_dir, "training_metrics_resnet_npu.png
208                             ↪ ")
209     plt.savefig(save_path)
210     print(f"Metrics plot saved to {save_path}")
211
212 if __name__ == "__main__":
213     # 获取当前脚本目录，方便保存文件
214     script_dir = os.path.dirname(os.path.abspath(__file__))
215
216     # 加载数据
217     train_loader, val_loader = load_data(script_dir, batch_size=16)
218
219     # 开始训练
220     train_acc_history, val_acc_history, train_loss_history,
221     ↪ val_loss_history = train_and_eval(
222         train_loader, val_loader, epochs=10, lr=0.01
223     )
224
225     # 绘制结果
226     plot_metrics(script_dir, train_acc_history, val_acc_history,
227                 ↪ train_loss_history, val_loss_history)

```

实验结果对比分析

值得注意的是，本章节仅展示了 ResNet 在 FP32 全精度模式下的实验结果，未进行 FP16 半精度测试。这主要基于两方面深入考量：

首先，与 VGG 类似，ResNet 架构在每个卷积层后都紧跟 Batch Normalization (BN) 层。BN 层在训练过程中需要计算并累积小批量数据的均值与方差，这一过程涉及

平方和等高动态范围的数学运算。FP16（半精度浮点数）的数值表示范围相对较窄（指数位仅 5 位），在处理 BN 层的统计量累积时，极易发生数值上溢（Overflow）或因精度不足导致的严重误差。实测表明，若简单地将 ResNet 全网转换为 FP16 进行训练，BN 层的数值不稳定性会迅速传播，导致损失函数震荡甚至 NaN（Not a Number），使得模型无法收敛。

只需将代码中的 npu 替换为 cuda，即可在配备英伟达显卡和 PyTorch 环境的服务器上运行。实验结果如下：

• Ascend 310B (FP32)

```
1 Epochs: 100%|██████████| 10/10 [5:16:18<00:00, 1897.83s/it, lr
  ↳ =0.010000, train_acc=0.9825, val_acc=0.8076, val_loss=0.7881]
```

• GeForce 5090D (FP32)

```
1 Epochs: 100%|██████████| 10/10 [05:22<00:00, 32.30s/it, lr
  ↳ =0.010000, train_acc=0.9811, val_acc=0.8122, val_loss=0.7805]
```

上述实验数据直观地揭示了边缘端 AI 处理器（昇腾 310B）与顶尖桌面级 GPU（GeForce 5090D）在全精度训练任务上的显著差异。首先是**训练效率**方面存在巨大鸿沟。得益于庞大的 CUDA 核心数量和极高的显存带宽，GeForce 5090D 完成 10 个 Epoch 的训练仅耗时约 **5 分 22 秒**，单轮平均耗时约 **32.3 秒**，其强大的并行计算能力使得小规模数据集的训练几乎是即时的。相比之下，作为一款面向推理优化的低功耗芯片（TDP 仅为桌面 GPU 的几十分之一），昇腾 310B 完成同样任务总耗时约 **5 小时 16 分**，单轮平均耗时约 **1898 秒**，两者速度相差约 **60 倍**。造成这一巨大差距的原因除了原始算力（FP32 TFLOPS）的悬殊外，还在于 ResNet 的计算深度较深，导致图编译优化、算子调度以及数据在 CPU 与 NPU 之间的搬运开销在总耗时中占据了较大比例。这进一步印证了在实际工程链路中，昇腾 310B 更适合执行“一次训练（服务器端），到处推理（边缘端）”的部署模式，而非直接用于从零开始的模型训练。

尽管效率差异显著，但两者在 FP32 **精度**下的训练结果却表现出极高的一致性。昇腾 310B 的验证集准确率达到 **83.49%**，与 GeForce 5090D 的 **84.21%** 非常接近。这一结果有力证明了昇腾 310B 在执行复杂的反向传播算法时，其底层算子的计算逻辑是精确无误的。对于算子开发者或模型迁移工程师而言，这意味着可以在低成本的 310B 上验证模型逻辑的正确性（哪怕训练慢一点），确信其计算行为与标准 GPU 是一致的，从而为模型在不同算力平台的部署提供了可靠的验证基准。

综上所述，虽然昇腾 310B 在 FP32 训练速度上无法与高端桌面显卡抗衡，但其完备的算子支持和精准的计算结果，使其完全具备承载轻量级微调（Fine-tuning）或小样本学习任务的能力，同时也是验证模型算子兼容性的理想低成本平台。

3.7. 总结

昇腾 310B 在训练任务上展现出了独特的边缘计算优势。首先，它具备完整的边缘微调能力，虽然并不适合从零开始训练大规模深度学习模型，但完全能够胜任小样本学习 (Few-shot Learning) 或迁移学习 (Transfer Learning) 任务。这一特性使得开发者能够在边缘端根据本地数据对模型进行持续优化，实现“千人千面”的个性化部署与本地进化。其次，其极高的能效比是桌面级 GPU 难以企及的。在仅需几瓦到十几瓦的极低功耗下，昇腾 310B 依然能够完成复杂的深度神经网络训练，这对于依赖电池供电或散热条件受限的嵌入式场景而言至关重要。此外，实验证明其生态软件栈 (CANN) 在内存管理方面已相当成熟，能够在仅有 8GB 内存的受限环境下顺利跑通如 VGG16 这样显存密集型的网络，体现了软硬件协同优化的强大能力。

然而，受限于硬件定位，昇腾 310B 在训练方面也存在明显的局限性。最突出的问题在于训练耗时过长。正如实验数据所示，全量训练深层网络的时间成本极高，这使得它并不适合作为生产环境下的主力训练设备，而更适合作为推理设备或轻量级微调节点。同时，为了适配其特定的硬件架构，开发者往往面临较多的算子约束。例如，部分算子可能缺乏支持需进行等价替换（如将 MaxPool 替换为 AvgPool），或者因为数值稳定性问题仅能使用 FP32 精度，这在一定程度上限制了模型设计的灵活性及新算法的快速验证。

4. PyACL 应用开发基础

AscendCL (Ascend Computing Language) 是一套用于在昇腾平台上开发深度神经网络应用的 C 语言 API 库。它提供了运行资源管理、内存管理、模型加载与执行、媒体数据处理等核心功能，充当了统一的 API 框架，帮助开发者充分利用昇腾 AI 处理器的硬件算力（如矩阵计算、图像预处理等）。

然而，直接使用 C/C++ 版本的 AscendCL 进行开发存在一定的挑战。首先，C/C++ 语言本身的指针操作和手动内存管理对开发者要求较高，容易产生内存泄漏或访问越界等难以调试的问题；其次，C++ 代码通常较为冗长，构建和编译流程繁琐，不利于算法的快速原型验证与敏捷迭代。回顾上一章，虽然我们可以尝试将 PyTorch 移植到昇腾 310B 开发板上，但受限于硬件架构差异和算子支持度，直接运行原生 PyTorch 代码往往面临诸多兼容性问题和性能瓶颈。因此，对于初学者而言，直接在 310B 板端运行 PyTorch 并不是最优解，而 PyACL 则是平衡开发效率与运行性能的最佳选择。

为了解决上述痛点，PyACL (Python Ascend Computing Language) 应运而生。作为 AscendCL 的 Python 绑定版本，PyACL 通过 CPython 封装底层 C 接口，在保留硬件高性能特性的同时，大幅降低了开发门槛。它允许开发者通过简洁的 Python 语法调用昇腾处理器的强大能力，无缝衔接主流 AI 框架和数据处理库。

需要特别指出的是，昇腾 310B 是一款定位为推理 (Inference) 的 AI 处理器，算力与显存资源受限，并不适合进行大规模的模型训练。在实际应用中，标准的开发流水线如下：1. **模型训练**：在拥有高性能显卡 (GPU) 或昇腾 910 训练卡的服务器上，使用 PyTorch 等框架完成模型训练。2. **模型转换**：使用 ATC 工具将训练好的模型转换为昇腾专用的离线模型 (.om)。3. **推理部署**：使用 PyACL 在昇腾 310B 上加载 .om 模型，实现高性能的边缘端推理。

本章将依据这一官方开发范式，系统讲解 PyACL 应用开发的全流程。

4.1. PyACL 的基本概念

PyACL 封装了底层 C 语言接口，主要包含以下模块：- **acl**: 核心模块，提供初始化、Device 管理、内存管理、模型推理等功能。- **acl.media**: 媒体数据处理 (DVPP)，包括 JPEG 编解码、视频编解码、VPC (图像处理)。- **acl.op**: 单算子调用接口。

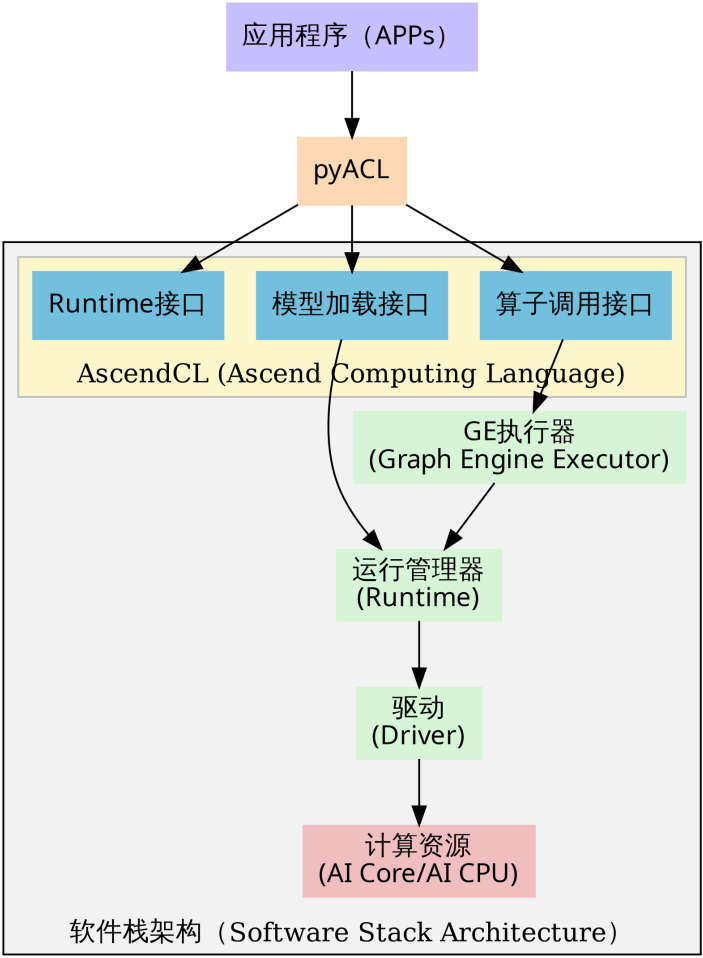


Figure 4.1.: PyACL 逻辑架构图

4.1.1. PyACL 的逻辑架构

PyACL 的程序逻辑架构图如下图所示：

根据 PyACL 的程序逻辑架构图，整个工作流程呈现出清晰的自顶向下的分层结构，每一层都承载着特定的职能，共同协作完成从 Python 代码到硬件指令的转换。

最顶层是 **应用层 (Application Layer)**，即开发者编写的应用程序入口（如 `main` → `.py`）。在此层，开发者专注于处理顶层业务逻辑，例如视频流读取或图像预处理，并通过 Python 语法调用 PyACL 提供的功能接口。紧接着是 **PyACL 桥接层**，它位于应用层与 C++ 库之间，充当了一个轻量级的封装层。其核心职能是利用 CPython 机制，将上层的 Python 函数调用“翻译”为底层 C 语言的 AscendCL 接口调用。这种设计巧妙

地屏蔽了底层 C++ 指针操作与内存管理的复杂性，让开发者在享受 Python 语法便捷性的同时，依然能够直接操控底层的系统资源。

向下深入，便是 **AscendCL 接口层**。它向 PyACL 开放了三大核心能力：负责 **Context**、**Stream** 等基础资源管理的 **Runtime 接口**，负责加载离线模型（.om）的 **模型加载接口**，以及支持单独执行某个算子（如 MatMul）的 **算子调用接口**。当指令穿过接口层进入 **执行控制层**时，数据流根据任务类型分化为两条路径：如果是标准的 **模型推理流**，由于 .om 模型已在 ATC 阶段编译完成，指令直接由 **Runtime**（运行管理器）接管并执行，由此构成了效率最高的“捷径”；如果是 **单算子调用流**，请求则需先经过 **GE (Graph Engine) 执行器**进行算子匹配和图构建，随后才下发给 Runtime。

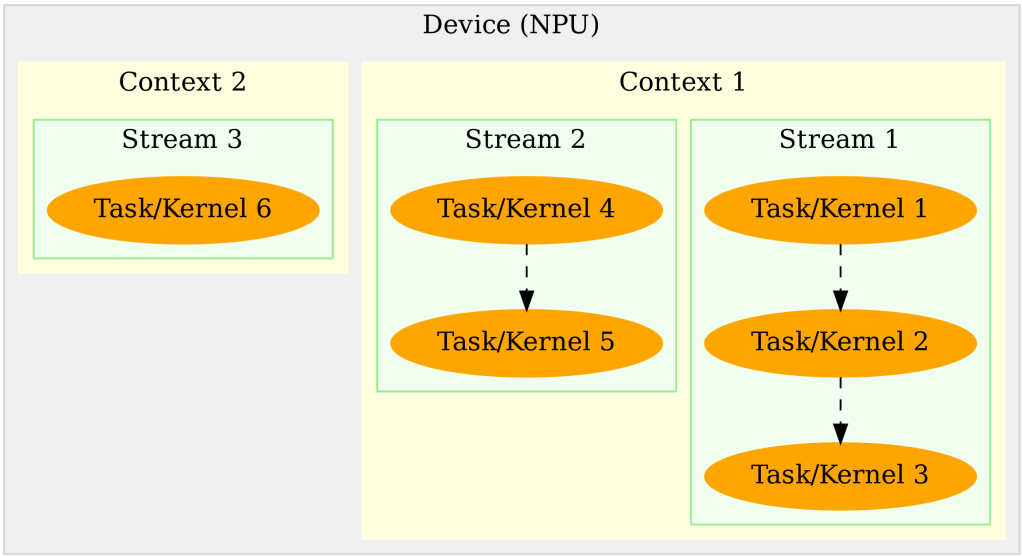
无论走哪条路径，所有任务最终都会汇聚于 **Runtime**。作为任务调度的核心枢纽，Runtime 将经过调度的任务流提交给底层的 **驱动层 (Driver)**。驱动层负责与硬件进行物理交互，将计算指令最终发送给昇腾处理器的 **AI Core**（执行大规模矩阵/向量计算）或 **AI CPU**（执行复杂逻辑控制），从而在物理层面处理数据。

纵观整个架构，PyACL 展现出一种极强的“**穿透性**”。虽然开发者是在高层次的 Python 环境中编写代码，但通过 PyACL 和 AscendCL 的层层传递，控制指令最终能够穿透软件栈，直达底层的 AI Core 硬件。这种“Python 语义驱动，硬件原生执行”的架构设计，正是 PyACL 既能保持开发效率，又能实现高性能推理的根本原因。

4.1.2. 基本概念与运行模型

利用 PyACL 进行编程开发，构建高效的 AI 应用，首先必须深入理解三个核心概念：**Device**、**Context** 和 **Stream**。这三者构成了 PyACL 运行时资源管理的基础骨架。**Device** 代表了物理层面的计算设备，即安装了昇腾 AI 处理器的硬件单元，是计算资源的实际载体。在多设备场景下，不同 Device 之间的内存资源是物理隔离的，无法直接共享。**Context**（上下文）是在 Device 之上的逻辑容器，负责管理执行环境。它类似于操作系统中的“进程”概念，负责维护该 Context 下所有对象（如 Stream、Event、设备内存等）的生命周期，并保证不同 Context 之间的资源隔离。**Stream**（**执行流**）则是动态的任务传送带，用于维护异步操作的执行顺序。基于 Stream 的机制，开发者可以利用流水线技术，实现 Host 侧逻辑运算、Host 与 Device 间的数据传输以及 Device 侧 Kernel 计算这三者的最大化并行。

理解这三者之间的关系，是掌握 PyACL 资源管理的关键。它们呈现出一种严格的层级归属关系：**Device 包含 Context**，而 **Context 又包含 Stream**，他们之间的关系图如下图所示：



{fig:device_context_stream width=70% .center}

一个 Context 必须且只能属于一个特定的 Device，但一个 Device 下可以并存多个 Context。Context 与 Stream 的关系则侧重于生命周期的绑定，每个 Context 都会自动包含一个默认 Stream，用户也可以显式创建多个 Stream，但 Stream 必须依附于创建它的 Context 而存在。如果 Context 被销毁，其下属的 Stream 也就无法再被使用。因此，资源释放必须遵循“先释放 Stream，再释放 Context，最后释放 Device”的逆序原则。

在多线程编程范式下，线程（Thread）与 PyACL 资源对象的交互机制是实现高并发的关键。**线程与 Context 之间遵循“单时刻单绑定”原则**，即在任意时刻，一个应用线程只能作为一个 Context 的“活跃”操作者。默认情况下，线程会绑定到最后一次创建的那个 Context 上，但开发者可以通过 `acl.rt.set_context` 动态切换不同的 Context，从而实现单线程对不同计算资源（如多个 Device 或隔离的资源池）的轮询调度。**线程与 Stream 则呈现出灵活的“一对多”控制关系**。一个线程完全可以在其绑定的 Context 内创建多个 Stream，并通过异步接口将计算任务依次分发到不同的 Stream 中，由 NPU 硬件负责调度并行执行，从而有效掩盖任务间的等待延迟。

关于资源的选择，PyACL 提供了默认和显式两种模式。系统支持隐式创建“默认 Context”和“默认 Stream”，它们通常在调用 `acl.rt.set_device` 时自动建立，适用于简单的单 Device 验证场景。然而，默认资源存在诸多限制，既不支持手动释放，也无法通过句柄灵活管理。因此，在构建复杂的多线程商业应用时，**强烈建议全部使用显式创建的 Context 和 Stream**，以确保资源生命周期的可控性和程序的健壮性。

在性能优化与调度方面，合理规划 Stream 的数量至关重要。虽然多 Stream 旨在

实现并行，但 Device 端的硬件资源（如 AI Core、AI CPU、Vector Core）是有限的。如果进程内过多的 Stream 同时争抢同一类硬件资源，硬件调度器在不同 Stream 间切换的开销可能会抵消并行带来的收益。因此，**最佳实践是按照算子执行引擎来划分 Stream**，例如将 AI Core 密集型任务与 AI CPU 逻辑型任务分发到不同的 Stream 中，从而实现异构硬件的真正的并行。此外，在架构设计上，“单线程多 Stream”的模式通常比“多线程多 Stream”具有微弱的性能优势，因为它避免了 Host 侧操作系统频繁进行线程上下文切换的开销，让 CPU 能更专注于向 NPU 下发任务。

4.1.3. PyACL 应用开发环境

CANN 安装

要部署 PyACL 的开发环境和运行环境，首先需要安装与目标 CANN 版本匹配的驱动和固件。虽然昇腾 310B 开发板通常预装了基础环境，但为了获得最新的特性支持，建议按照[本教程第二章](#)或《[CANN 软件安装指南](#)》升级至较新的 CANN 版本（如 CANN 8.3）。

CANN 软件包安装完成后，**无需额外安装独立的 Python 绑定库**，PyACL 相关模块已包含在 CANN Toolkit 中。但为了确保系统能正确找到 ac1 模块，必须加载必要的环境变量。

环境变量配置

如果按照本教程第二章的标准流程安装，通常无需额外操作，CANN 的路径配置脚本可能已自动写入启动文件。在 Miniconda 的 base 环境中，您可以直接尝试导入 ac1。

如果创建了新的虚拟环境或遇到 `ModuleNotFoundError`，请手动执行以下命令加载环境变量：

```
1 source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

为避免每次打开终端都需要输入该命令，建议将其添加到 `~/.bashrc` 文件末尾。

□ 注意：PyACL 组件（ac1.so）支持的 Python 版本范围通常为 3.7.5 ~ 3.10。请确保您的虚拟环境 Python 版本在此区间内。

环境验证

为了验证 PyACL 环境是否就绪，我们可以编写一个简单的测试脚本 `check_ascend_device` → `.py`，用于查询当前系统的 NPU 设备数量：

```

1 import acl # 导入核心库
2
3 # 1. 初始化 ACL 环境
4 # 配置文件路径传入空字符串，表示使用默认配置
5 ret = acl.init("")
6 if ret != 0:
7     print(f"ACL init failed, ret={ret}")
8     exit(1)
9
10 # 2. 获取可用 Ascend 设备数量
11 count, ret = acl.rt.get_device_count()
12 if ret == 0:
13     print(f"Found {count} Ascend devices.")
14 else:
15     print(f"Get device count failed, ret={ret}")
16
17 # 3. 去初始化 (释放资源)
18 acl.finalize()

```

运行该脚本：

```

1 (base) HwHiAiUser@orangepiaipro:~/Documents/samples/chapter4$ python
   ↪ check_ascend_device.py
2 Found 1 Ascend devices.

```

该示例代码演示了 PyACL 应用的基础生命周期：包括环境初始化、查询硬件资源（NPU 数量）以及最后的资源释放。对于投影 310B 处理器的开发板，其可用 NPU 设备数量为 1，程序输出结果与硬件实际情况一致。

从上述示例可以看出，环境初始化与资源释放是 PyACL 应用开发的必要环节。在调用任何核心功能接口前，必须先执行 `acl.init(config_path)`。若跳过初始化步骤，后续所有接口调用可能失败并返回错误码（如 `ACL_ERROR_UNINITIALIZED`），导致程序无法正常运行。在没有特殊配置需求时，`acl.init` 可直接传入空字符串或指向空白 JSON 文件的路径。

同样，程序运行结束后必须执行 `acl.finalize()` 以确保资源被正确回收。如果缺少资源释放环节，可能会引发内存泄漏、NPU 算力资源被持续占用或设备状态异常，进而影响后续任务的执行甚至导致系统崩溃。

值得注意的是，对于上述简单的设备查询程序，即使没有显式调用初始化和资源释放接口，程序通常也能够返回正确的结果。例如，我们可以将代码简化为如下一行的命令并在终端执行：

```

1 python -c "import acl; print(f'Found {acl.rt.get_device_count()[0]}
   ↪ Ascend devices.')"

```

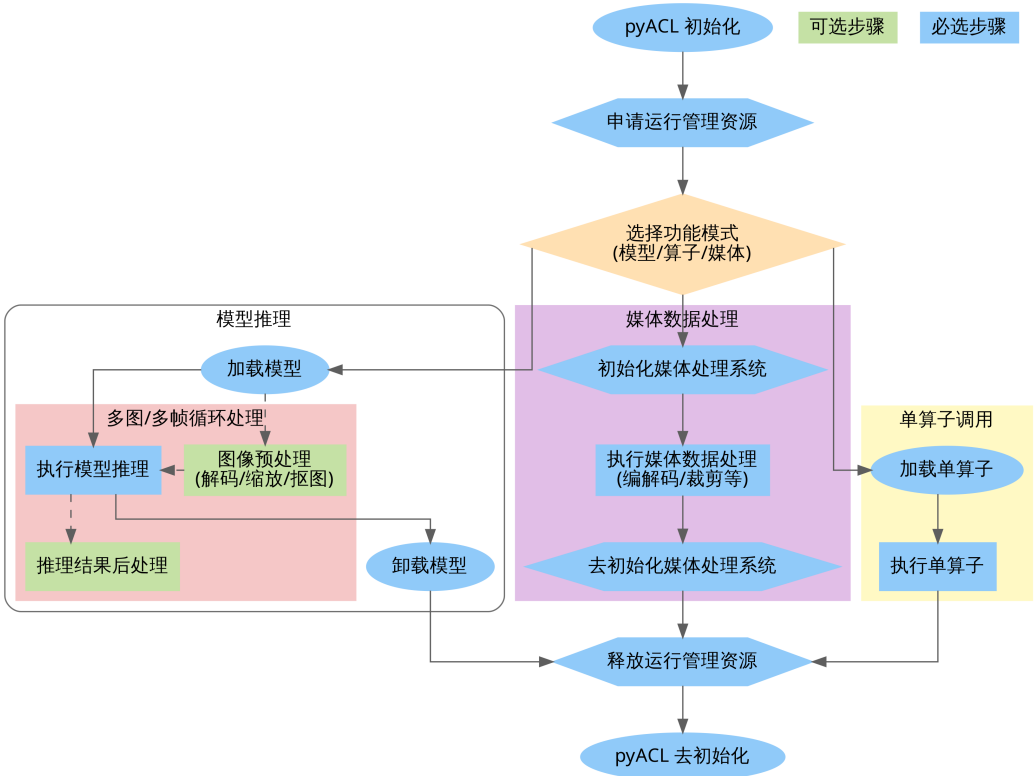
这种现象的原因主要有两点：1. **接口依赖性较低**：`acl.rt.get_device_count` 属于基础的硬件查询接口，在某些版本的驱动实现中，这类轻量级查询操作可能并不严格依

赖完整的全局初始化环境即可访问驱动状态。**2. 操作系统资源回收：**虽然程序没有显式调用 `acl.finalize()`，但当 Python 进程退出时，操作系统会自动关闭相关的文件描述符并回收该进程占用的资源。因此，多次运行该脚本通常不会立刻导致系统资源耗尽或报错。

但必须强调，这属于非规范用法。在涉及模型加载、内存申请或硬件加速（DVPP）等核心功能时，跳过 `acl.init` 可能会导致报错。为了保证程序的健壮性与兼容性，开发者应始终坚持规范的“初始化-业务执行-资源释放”流程。

4.1.4. PyACL 接口调用流程

调用 PyACL 接口开发的 AI 应用通常遵循一套标准化的逻辑流程，涵盖从环境初始化、硬件资源申请、业务计算执行到资源销毁的完整生命周期。开发者可以根据业务需求，将模型推理、媒体数据处理（DVPP）或单算子加速等功能进行独立部署或组合使用，如下图所示：



{fig:pyacl_interface width=100% .center}

根据上图展示的 PyACL 接口调用流程，我们可以将开发过程清晰地划分为三个主要阶段：**系统初始化、核心功能执行以及资源释放。**

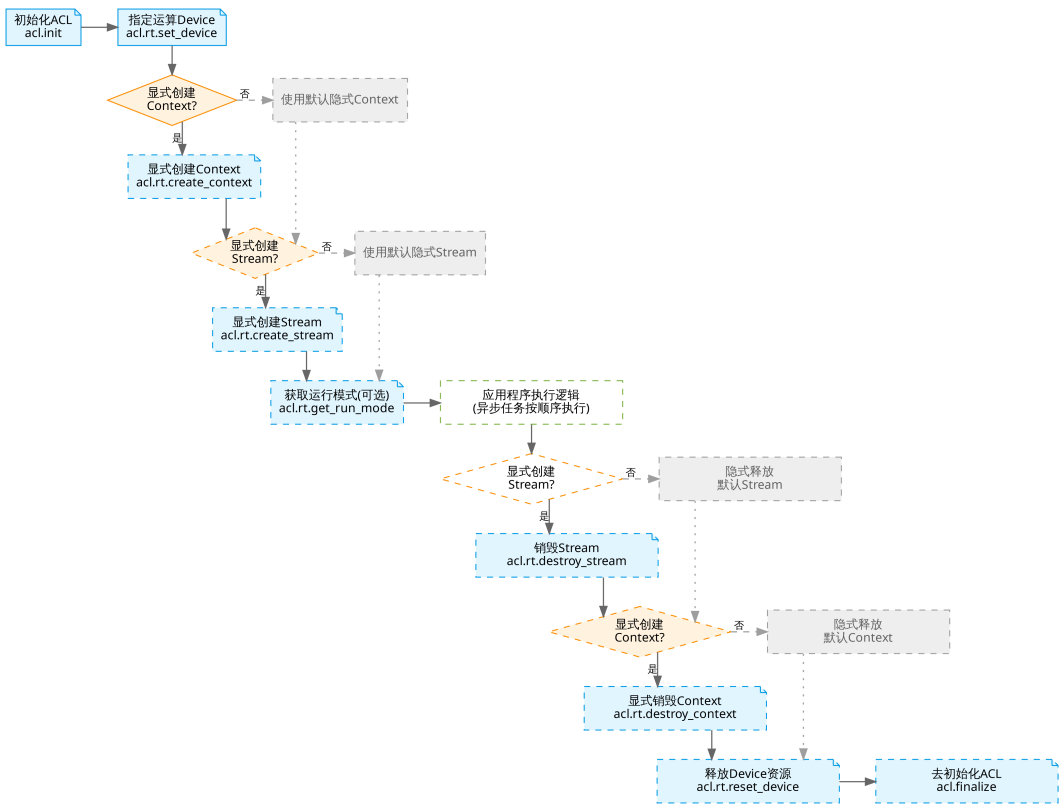
1. **系统初始化与资源申请（公共基础）** 无论开发何种类型的应用，起步动作都是统一的。首先调用 `acl.init` 进行全局初始化，随后申请运行管理资源。这通常涉及指定计算设备（Device）、创建上下文（Context）以及创建用于管理任务流的 Stream。这些资源构成了后续所有计算任务的基础环境。
2. **核心功能执行** 在资源就绪后，开发者根据具体业务需求选择相应的执行路径：
 - **模型推理链路（左侧分支）**：这是最典型的 AI 应用场景。开发者首先调用接口加载离线模型（.om），随后在业务循环中对每一帧图像或数据进行必要的预处理（如使用 DVPP 硬件进行解码与缩放）将其转化为模型所需格式，接着调用模型执行接口完成推理，并在获取推理结果后执行解析分类标签或画框等后处理逻辑，最后在任务完成后及时卸载模型以释放内存资源。
 - **媒体数据处理链路（中间分支）**：如果应用仅涉及图像编解码或视频处理而无需推理，可以直接初始化媒体系统，调用 DVPP 接口进行处理，随后去初始化。
 - **单算子调用链路（右侧分支）**：适用于不需要加载完整网络模型，仅需执行特定矩阵运算或数学函数的场景。流程简化为加载特定算子描述并直接执行算子。
3. **资源释放与去初始化** 当业务逻辑执行完毕后，必须严格按照逆序释放资源：先销毁 Stream 和 Context，再重置 Device，最后调用 `acl.finalize` 完成 PyACL 的去初始化。这一步对于防止内存泄漏和保证系统稳定性至关重要。

此流程图清晰地界定了必选步骤（蓝色）与可选的业务逻辑步骤（绿色），帮助开发者建立规范的编程思维。

4.1.5. 运行管理资源生命周期

PyACL 的资源管理构建在 **Device**、**Context** 与 **Stream** 三个核心概念之上。在应用启动阶段，必须首先调用 `acl.init` 完成全局环境初始化。随后，通过 `acl.rt.set_device` 指定计算所需的物理 NPU 设备。

开发应用时，应用程序中必须包含运行管理资源申请的代码逻辑，您需要按照 **Device**、**Stream** 的顺序依次申请。其中，创建 Stream 的方式分为隐式创建和显式创建，其适用场景有所不同，运行资源的申请与释放的流程如下图所示：



{fig:pyacl_stream width=100% .center}

上图展示了 PyACL 资源管理的完整生命周期流程。整个流程严格遵循“先申请后释放，谁申请谁释放”的原则，主要包含以下几个关键步骤：

1. **ACL 初始化 (acl.init)**: 这是所有 AscendCL 操作的起点。必须在进程启动的最开始调用，用于初始化 ACL 的全局配置。
2. **资源申请 (set_device/create_context/create_stream)**: 开发者通过 `acl.rt.set_device` 锁定物理硬件资源。如果应用没有显式调用 `acl.rt.create_context` 或 `acl.rt.create_stream`，系统在调用 `set_device` 后会自动创建并关联默认 Context 与默认 Stream。但在生产环境或多线程并发场景下，建议显式创建这些资源，以实现更好的逻辑隔离和任务异步调度。Stream 作为任务执行队列，负责管理指令在硬件上的下发顺序。
3. **业务执行 (Execution)**: 在此阶段，应用进行模型加载、数据预处理、推理计算等核心逻辑。所有的计算任务 (Kernel) 都会被下发到之前创建的 Stream 中。
4. **资源销毁**: 业务完成后，必须按特定顺序释放资源。对于显式创建的资源，应首先调用 `acl.rt.destroy_stream` 销毁 Stream，再调用 `acl.rt.destroy_context` 销毁 Context。最后调用 `acl.rt.reset_device` 重置设备以彻底释放 Device 资源。

源。需要特别说明的是，若未显式创建 Context 与 Stream（即使用系统默认资源），开发者**不能**调用相应的销毁接口；在这种情况下，直接调用 `reset_device` 即可隐式地销毁关联的默认 Context 与 Stream 资源。

5. **ACL 去初始化 (`acl.finalize`)**: 这是进程退出的最后一步，用于彻底清理全局资源。开发者应严格遵循“先业务、再流、后设备”的逆序原则进行资源释放，最后调用此接口退出环境，以避免内存泄漏或设备状态异常。

关键点总结：
 * **顺序至关重要**：资源的申请与释放必须严格遵循层级逻辑。申请资源时，按照 **Device -> Context -> Stream** 的顺序依次创建；释放资源时，则必须严格遵循 **Stream -> Context -> Device** 的逆序原则。如果先重置了 Device，依附于其上的 Context 和 Stream 将变为非法状态，导致不可预知的系统错误。
 * **显式 vs 隐式**：虽然隐式创建（直接 `set_device` 后 `create_stream`）代码更少，但为了代码的健壮性和可维护性，推荐在生产环境代码中始终使用**显式创建 Context 和 Stream** 的流程。

以下是 PyACL 资源申请与释放的标准逻辑示例。这段伪代码展示了在典型应用场景下，如何按照规范的生命周期管理硬件资源：

```

1 import acl
2
3 # 1. 环境初始化
4 ret = acl.init("")
5
6 # 2. 指定计算设备
7 device_id = 0
8 ret = acl.rt.set_device(device_id)
9
10 # 3. 显式创建 Context 并绑定到当前线程
11 context, ret = acl.rt.create_context(device_id)
12 ret = acl.rt.set_context(context)
13
14 # 4. 显式创建执行流（属于上述 Context）
15 stream, ret = acl.rt.create_stream()
16
17 # -----
18 # ... 执行核心业务逻辑（如模型推理、算子调用或媒体处理） ...
19 # -----
20
21 # 5. 销毁执行流（先释放 Stream）
22 ret = acl.rt.destroy_stream(stream)
23
24 # 6. 销毁 Context（在销毁 Stream 之后）
25 ret = acl.rt.destroy_context(context)
26
27 # 7. 重置设备（释放 Device 相关资源）
28 ret = acl.rt.reset_device(device_id)
29
30 # 8. 系统去初始化
31 ret = acl.finalize()

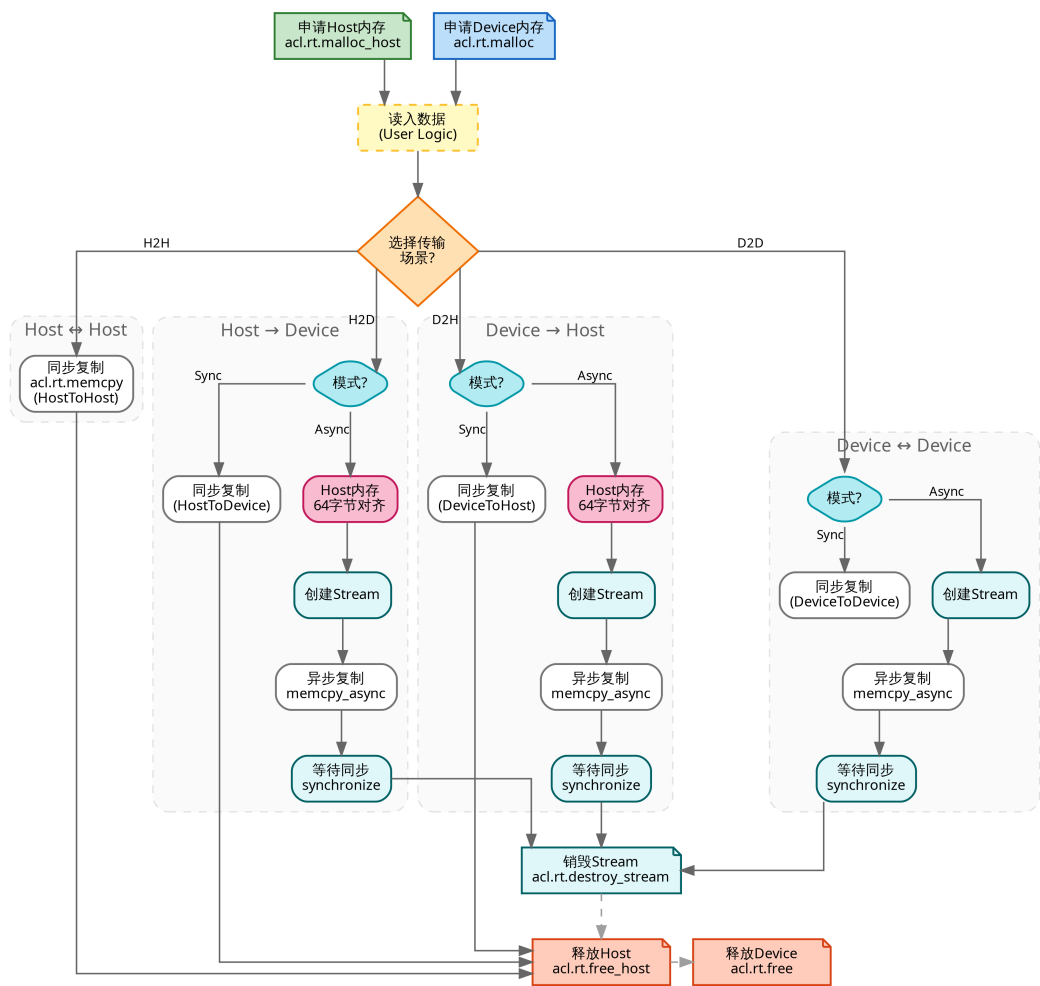
```

关键步骤说明：

- **初始化与去初始化**：`acl.init` 和 `acl.finalize` 必须在进程级别成对调用。未初始化直接调用其他接口会导致运行报错。
- **隐式 Context 机制**：示例中利用 `acl.rt.set_device` 隐式创建了 Context。在复杂的并发场景下，建议使用 `acl.rt.create_context` 显式创建，以便更精确地控制资源隔离。
- **资源释放顺序**：代码严格遵循了“后申请，先释放”的逆序原则。如果先重置设备再销毁流，会导致非法句柄访问，进而引发系统崩溃或内存异常。
- **返回值检查**：虽然本示例为简化逻辑未展示 `ret` 判断，但在实际工程中，**必须** 检查每一个接口的返回值（0 代表成功），以便在资源不足或硬件异常时及时捕获错误。

4.1.6. 异构内存管理

由于昇腾 AI 处理器拥有独立的存储单元，应用开发涉及 **Host**（CPU 侧）与 **Device**（NPU 侧）两部分内存。开发者通常面临频繁的数据交互需求：通过 `acl.rt.malloc` 申请 Device 侧内存用于 NPU 计算，或通过 `acl.rt.malloc_host` 申请 Host 内存。数据的流动则依靠 `acl.rt.memcpy` 完成，通过定义传输方向，将采集到的源数据搬运到 NPU 计算单元，或将计算出的结果拉回 Host 进行后处理。内存数据的流动方向一共有四种，分别是 Host 之间，Host 到 Device，Device 之间以及 Device 到 Device，分别对应 `ACL_MEMCPY_HOST_TO_HOST`，具体的流程图如下图所示：



{fig:pyacl_memory width=100% .center}

数据在 Host（CPU）与 Device（NPU）之间的拷贝主要有四种常见路径：Host 到 Host、Host 到 Device、Device 到 Host 以及 Device 到 Device，分别对应四个常量：ACL_MEMCPY_HOST_TO_HOST、ACL_MEMCPY_HOST_TO_DEVICE、ACL_MEMCPY_DEVICE_TO_HOST 和 ACL_MEMCPY_DEVICE_TO_DEVICE。

标准的处理流程通常由以下步骤组成：首先在 Host 侧准备好输入数据；接着使用 `acl.rt.malloc` 或 `acl.rt.malloc_host` 申请目标内存空间；随后调用 `acl.rt.memcpy` 指定拷贝方向并执行数据传输（支持同步或异步模式）；待数据传输至 Device 后进行计算；最后将计算结果通过 Device 到 Host 的拷贝路径拉回 Host 侧进行后处理。在此过程中，需特别注意内存对齐要求，以及在异步拷贝时必须配合 Stream 同步操作以确保数据可用。

以下是这四种内存传输模式的详细说明与关键点解析：

1. Host 到 Host (ACL_MEMCPY_HOST_TO_HOST) 纯 CPU 端的内存拷贝通常用于进程内部的数据移动或不同 Host 缓冲区之间的数据整理。该过程无需关注 Device 端的内存对齐要求，也不涉及 Stream，直接使用同步拷贝方式即可。以下是 Host 到 Host 同步拷贝的示例代码，首先申请两块 Host 内存，读取数据后，调用 `acl.rt.memcpy` 并指定拷贝类型为 1（即 `ACL_MEMCPY_HOST_TO_HOST`）：

```
1 # Host 到 Host 同步拷贝
2 host_src, ret = acl.rt.malloc_host(size)
3 host_dst, ret = acl.rt.malloc_host(size)
4 # 假设 read_file 为用户自定义的数据加载函数
5 read_file(file, host_src, size)
6 ret = acl.rt.memcpy(host_dst, size, host_src, size, 1) # 1 代表
    ↳ ACL_MEMCPY_HOST_TO_HOST
7
8 # 释放资源
9 acl.rt.free_host(host_src)
10 acl.rt.free_host(host_dst)
```

2. Host 到 Device (ACL_MEMCPY_HOST_TO_DEVICE) 将 Host 侧的数据上传至 NPU（Device 侧）是推理任务的起点。该过程支持同步或异步方式。若采用异步上传，需确保 Host 内存满足 Device 的访问要求，通常建议使用 `acl.rt.malloc_host` 申请。在异步模式下，开发者必须显式创建 Stream，并在后续使用 Device 数据前执行同步等待操作，以确保数据传输完整。

以下是同步与异步拷贝的实现示例：

```
1 # 准备内存
2 host_ptr, _ = acl.rt.malloc_host(size)
3 dev_ptr, _ = acl.rt.malloc(size, 2) # 2 代表 ACL_MEM_MALLOC_HUGE (
    ↳ Device 内存)
4 read_file(file, host_ptr, size)
5
6 # 方式一：同步拷贝
7 # 2 代表 ACL_MEMCPY_HOST_TO_DEVICE
8 acl.rt.memcpy(dev_ptr, size, host_ptr, size, 2)
9
10 # 方式二：异步拷贝
11 stream, _ = acl.rt.create_stream()
12 acl.rt.memcpy_async(dev_ptr, size, host_ptr, size, 2, stream)
13 acl.rt.synchronize_stream(stream) # 等待数据传输完成
14
15 # 释放资源（注意顺序）
16 acl.rt.free_host(host_ptr)
17 acl.rt.free(dev_ptr)
18 acl.rt.destroy_stream(stream)
```

3. Device 到 Host (ACL_MEMCPY_DEVICE_TO_HOST) 将 Device 侧的计算结果回传至 Host 侧是推理流程的最后一步，主要用于后续的业务结果解析或数据持久化。异步回传支持与 Device 侧的计算任务并行处理，但必须在 Host 侧访问该数据前执行

Stream 同步。在内存分配上，建议使用 `acl.rt.malloc_host` 申请 Host 侧缓冲区，以确保 Device 侧的 DMA 能够高效地完成数据交互。

```

1 # 准备内存
2 out_dev, _ = acl.rt.malloc(out_size, 2)
3 out_host, _ = acl.rt.malloc_host(out_size)
4
5 # 假设 Device 侧计算已完成，执行异步拷贝 (3 代表
   ↪ ACL_MEMCPY_DEVICE_TO_HOST)
6 acl.rt.memcpy_async(out_host, out_size, out_dev, out_size, 3, stream)
7
8 # 必须等待拷贝流执行完毕，确保数据已完整到达 Host
9 acl.rt.synchronize_stream(stream)
10
11 # 此时可进行后处理并释放资源
12 acl.rt.free(out_dev)
13 acl.rt.free_host(out_host)

```

4. Device 到 Device (ACL_MEMCPY_DEVICE_TO_DEVICE) 在 Device 内部不同缓冲区之间进行拷贝，或者在多 Device 场景下跨设备传输数据，这里主要注意的是，对于昇腾 310B 开发板来说，一般来说，一个昇腾 310B 开发板只有一个 NPU，也就是说，只有一个 Device。这种模式常用于数据格式重排或设备间通信。

该操作通常配合异步 Stream 使用，以避免阻塞 Host 线程。在执行跨设备拷贝时，开发者需要注意上下文切换或使用特定的点对点传输接口。以下是同一 Device 内执行异步拷贝的示例代码。程序首先申请两个 Device 内存块，随后通过 `acl.rt.memcpy_async` 执行拷贝，并将传输类型指定为 4（即 `ACL_MEMCPY_DEVICE_TO_DEVICE`）：

```

1 # 同一 Device 内的异步拷贝
2 dev_a, _ = acl.rt.malloc(size, 2)
3 dev_b, _ = acl.rt.malloc(size, 2)
4 stream, _ = acl.rt.create_stream()
5
6 # 4 代表 ACL_MEMCPY_DEVICE_TO_DEVICE
7 acl.rt.memcpy_async(dev_b, size, dev_a, size, 4, stream)
8 acl.rt.synchronize_stream(stream)
9
10 # 释放资源
11 acl.rt.free(dev_a)
12 acl.rt.free(dev_b)
13 acl.rt.destroy_stream(stream)

```

该模式常用于设备内数据重排或多设备间的点对点传输。在昇腾 310B 等单 NPU 平台上，跨设备传输并不常见；在多 NPU 环境下，应通过切换 Context 或使用专用点对点传输接口以减少额外拷贝开销。建议始终配合异步 Stream 并在访问目标缓冲区前调用同步接口（如 `synchronize_stream` 或 `stream_wait_event`）以确保数据已就绪，同时注意内存对齐与访问权限，避免 DMA 访问异常。

通用开发注意事项 1. 异步与同步：异步拷贝接口(`memcpy_async`)必须配合 Stream

以及 `synchronize_stream` 使用，否则无法保证数据一致性。2. **内存类型**：为了提升传输效率，Host 侧供 Device 访问的内存建议始终使用 `acl.rt.malloc_host` 申请。3. **释放顺序**：资源释放应遵循严格的逆序原则——先销毁 Stream，再释放内存，最后重置 Device。4. **错误处理**：示例代码为保持简洁略去了错误检查，但在实际开发中，所有 ACL 接口调用的返回值（`ret`）都必须进行检查（0 表示成功）。

4.1.7. 同步等待机制

从上一节关于内存拷贝的四种路径中，我们可以观察到 `acl.rt.memcpy` 与 `acl.rt.memcpy_async` 两种截然不同的操作模式。这两种模式的选择，本质上是在**编程简易性与执行性能**之间做权衡。

同步与异步的概念

同步操作 (Synchronous) 是最符合直觉的编程方式，它遵循严格的“请求-响应”逻辑。正如我们在 Host 到 Host 拷贝示例中所见，当 Host 线程发起一个同步指令时（如 `acl.rt.memcpy`），CPU 会挂起当前线程，像监工一样死死盯着任务，直到任务彻底完成后才会恢复执行下一行代码。这种模式的优点显而易见：逻辑简单，数据一致性由代码执行顺序天然保证，非常适合初学者进行功能验证或定位 BUG。但其缺点也同样致命：它完全抹杀了硬件并行的可能性。试想，当 CPU 在傻傻等待数据从 Host 搬运到 Device 时，昂贵的 NPU 计算单元可能正处于空闲状态，导致系统整体吞吐量下降。

异步操作 (Asynchronous) 则打破了这种串行束缚，是高性能 AI 应用的标配。在调用 `acl.rt.memcpy_async` 时，Host 线程仅需将任务“投递”到 Stream 队列中便立即返回，继续处理其他逻辑（如读取下一张图片或预处理数据）。此时，底层的 DMA 搬运引擎会与 NPU 的计算引擎同时工作，实现了真正的**软硬件并行**。这就好比点餐系统，前台服务员（Host）只负责快速接单并把单子（Task）扔给厨房（Stream），而不需要站在厨房门口等菜做好，从而能接待更多的客人。在复杂的 AI 业务流中，这种机制允许我们构建精妙的流水线：**在 NPU 拼命推理当前帧的同时，DMA 正在默默地将下一帧数据搬运进显存，而 CPU 已经在预处理第三帧数据**。这种“想尽办法让显卡即使一毫秒都不闲着”的设计，正是提升 AI 应用帧率的关键。

然而，异步操作是一把双刃剑，带来的性能红利必须以**严谨的同步控制**为代价。因为 Host 线程“投递”完任务就跑了，如果不加干预，它很可能在数据还没搬运完时就开始尝试读取结果，或者在 NPU 还没用完数据时就释放了内存，从而引发数据错乱甚至程序崩溃。因此，异步开发必须配**合同步等待机制**，在关键的时间节点上让“脱缰”的并行任务重新对齐。

PyACL 的四种同步机制

PyACL 提供了四种不同粒度的同步等待机制，以适应从简单的单流控制到复杂的多流并行协作等不同场景。正确选择同步方式是平衡 Host 侧控制流与 Device 侧数据流的关键。

1. **Event 的同步等待 (acl.rt.synchronize_event)** Event 是 PyACL 中的“时间锚点”或“完成标记”。这种机制允许 Host 侧主动阻塞，直到某个特定的 Event 在 Device 侧被触发。其典型流程是：开发者创建一个 Event，将其记录 (Record) 到某个 Stream 的任务队列中；当 Stream 执行到该记录点时，会在硬件层面触发该 Event；Host 线程通过调用 `synchronize_event` 暂停自身执行，直至接收到触发信号。这种方式粒度适中，适合 Host 需要等待某个特定任务节点完成后再进行后续逻辑（如读取特定阶段的结果），且该 Event 可能被多个等待者复用的场景。

```
1 # 伪代码：Host 等待特定 Event
2 event, _ = acl.rt.create_event()
3 acl.rt.memcpy_async(dev_ptr, size, host_ptr, size, 1, stream) # 异步拷贝
4 acl.rt.record_event(event, stream) # 在流中安插一个锚点
5
6 # ... Host 执行其他不依赖数据的逻辑 ...
7
8 acl.rt.synchronize_event(event) # Host 在此阻塞，直到上面的拷贝完成
9 # 此时可以安全释放 host_ptr 或读取 dev_ptr
```

2. **Stream 内任务的同步等待 (acl.rt.synchronize_stream)** 这是最常用且最直观的同步方式，它表示 Host 侧阻塞等待指定 Stream 中的所有已提交任务全部执行完毕。其语义简单直接，保证了指定流水线上的所有操作都已落盘。常用于单 Stream 流水线的收尾阶段，或者 Host 必须确保整个任务队列清空后才能继续（例如释放 Stream 资源前）。虽然简单，但它是粗粒度的阻塞操作，如果 Stream 中积压任务较多，会导致 Host 长时间等待。

```
1 # 伪代码：Host 等待整个流清空
2 acl.rt.memcpy_async(..., stream) # 任务1
3 acl.op.execute_v2(..., stream) # 任务2
4 acl.rt.memcpy_async(..., stream) # 任务3
5
6 acl.rt.synchronize_stream(stream) # Host 阻塞，直到任务1,2,3全部完成
7 # 此时所有任务均已结束
```

3. **Stream 间的同步等待 (acl.rt.stream_wait_event)** 这是实现多流并行与主要流水线协作的核心机制。与前两者不同，`stream_wait_event` 不会阻塞 Host 线程，而是向 Consumer Stream（消费者流）中下发一个“等待指令”。Consumer

Stream 执行到该指令时，会暂停处理后续任务，直到 Producer Stream（生产者流）触发了指定的 Event。这种机制完全在 PyACL 内部调度和 Device 硬件层面完成，Host 仅负责编排逻辑，从而最大化了 CPU 与 NPU 的并行效率。它非常适合跨流的数据依赖场景，例如：Stream A 负责数据搬运，Stream B 负责计算，Stream B 必须等待 Stream A 搬运完成后才能开始计算。

```

1 # 伪代码：Stream 间协作（不阻塞 Host）
2 # Stream A：搬运数据 -> Record Event
3 acl.rt.memcpy_async(dev_input, ..., stream_a)
4 acl.rt.record_event(event, stream_a) # 记录搬运完成
5
6 # Stream B：等待数据 -> 开始计算
7 acl.rt.stream_wait_event(stream_b, event) # Stream B 暂停，直到
   ↳ Stream A 触发 Event
8 acl.mdl.execute_async(..., stream_b)      # 只有等数据搬运完了，才
   ↳ 会执行推理
9
10 # Host 这里是非阻塞的，可以立即继续运行

```

4. **Device 的同步等待 (acl.rt.synchronize_device)** 这是粒度最粗的全局同步操作。调用此接口会阻塞 Host 进程，直到当前 Context 绑定的 Device 上所有 **Stream 的所有任务全部完成**。虽然它能保证设备层面的全局一致性，但由于其“一刀切”的等待特性，极大破坏了并行性，且开销最高。通常仅在程序初始化调试、全局状态检查或程序退出前的最终资源回收阶段使用，在高性能的生产业务循环中应尽量避免。

```

1 # 伪代码：全局等待
2 acl.rt.memcpy_async(..., stream1)
3 acl.rt.memcpy_async(..., stream2)
4
5 # 强制等待设备上所有任务结束（调试或退出时使用）
6 acl.rt.synchronize_device()

```

昇腾 310B 最佳同步策略与场景分析

在昇腾 310B 这种边缘计算平台上，CPU 的单核性能通常弱于服务器级 CPU，因此**减少 Host 侧（CPU）的阻塞**、把调度压力卸载给 Device 侧硬件显得尤为关键。为了更直观地理解为什么在昇腾 310B 上必须强调“CPU 减负”与“异步并行”，我们需要深入分析这颗 SoC 的 CPU 性能定位。与市面上其他主流的边缘计算开发板相比，昇腾 310B 呈现出显著的“**NPU 极强，CPU 较弱**”的异构特性。

下表详细对比了昇腾 310B 与树莓派 5、RK3588 以及 NVIDIA Jetson Orin Nano 在 CPU 规格上的差异：

平台名称	核心处理器 (SoC)	CPU 架构	核心配置	主频 (典型)	CPU 算力估算 (UnixBench)
昇腾 310B	Ascend 310B	ARM Cortex-A55	4 核 (纯小核)	1.0 GHz ~ 1.6 GHz	~ 400 - 600 分
树莓派 5	BCM2712	ARM Cortex-A76	4 核 (大核)	2.4 GHz	~ 2400+ 分
Orange Pi 5	RK3588	Cortex-A76 + A55	4 大核 + 4 小核	2.4 GHz (大) / 1.8 GHz (小)	~ 5000+ 分
Jetson Orin Nano	Tegra Orin	ARM Cortex-A78AE	6 核 (高性能核)	1.5 GHz	~ 3500+ 分
桌面 PC	Intel i7-12700K	x86-64 (Alder Lake)	8 大核 + 4 小核	3.6 GHz ~ 5.0 GHz	~ 45000+ 分

数据解读与性能差距分析：

- 1. 架构代差 (效率核 vs 性能核)：昇腾 310B 采用的是 **Cortex-A55** 架构。在 ARM 的设计体系中，A55 被定义为 “高能效核心 (Efficiency Core)”，通常用于手机处理器的后台任务或物联网设备，其侧重点在于低功耗而非高性能。相比之下，树莓派 5 使用的 Cortex-A76 和 Orin Nano 使用的 Cortex-A78AE 均属于 “高性能核心 (Performance Core)”。在同频下，A76 的整数计算能力约为 A55 的 2~3 倍，浮点性能更是相差甚远。
- 2. 绝对性能差距：从综合基准测试（如 UnixBench 或 Geekbench）来看，昇腾 310B 的 CPU 性能大约仅为树莓派 5 的 **1/4 到 1/5**，仅为 RK3588 的 **1/8** 左右。这意味着，同样一段基于 OpenCV 的纯 CPU 图像缩放代码，在树莓派 5 上可能耗时 5ms，而在昇腾 310B 上可能耗时 25ms 甚至更多。
- 3. 与桌面 CPU 的鸿沟：如果是从在笔记本（x86, i5/i7）上开发算法迁移到 310B，这种落差会更加剧烈，性能折损可能达到 **50 倍甚至 100 倍**。桌面 CPU 拥有巨大的二级/三级缓存和乱序执行能力，这掩盖了 Python 解释器本身的低效。但在 310B 上，Python 的 Global Interpreter Lock (GIL) 加上较弱的单核性能，极易成为系统的最大短板。

对 PyACL 开发的启示：

基于上述硬件数据，我们在开发中必须遵循以下“生存法则”：*** 不要信任 CPU 的浮点计算能力**：任何涉及图像 Pixels 遍历的操作（如 Resize, Color Convert, Normalize）若写在 CPU (Python) 端，必将导致帧率骤降。**必须使用 DVPP 硬件加速**。*** Python 代码仅做胶水**：Python 逻辑应仅限于流程控制、参数配置和极少量的后处理。业务主体必须由底层的 C++ 算子或 NPU 模型承担。*** 异步是救命稻草**：由于 CPU 处理每一行 Python 代码都比其他平台慢，因此更不能让 CPU 傻傻等待 NPU（同步）。只有利用 `stream_wait_event` 让 CPU 快速把任务分发完并脱身，才能掩盖 A55 核心性能不足的缺陷。

1. 优先策略：全链路异步与细粒度同步

在昇腾 310B 平台上，由于 CPU 算力相对有限，最理想的流水线策略是实施“全链路异步”设计，即让 CPU 专注于指令分发，而将繁重的计算负载全权交由 NPU 负责。开发者应尽量避免使用 `acl.rt.memcpy` 等同步阻塞接口，转而全面采用 `acl.rt.memcpy_async` 配合 Event 机制。这种方法的核心原则在于，凡是能通过 Device 侧 `stream_wait_event` 解决的任务依赖，绝不让 Host 侧的 CPU 介入干预。例如，在典型的多级推理流水线中，我们可以安排 Stream A 负责视频解码（DVPP），Stream B 负责模型推理。当 Stream A 完成解码任务后，只需在流中记录一个 Event，而 Stream B 仅需等待该 Event 触发即可启动推理。整个交互过程中，CPU 仅需极低开销下发几条控制指令，完全无需挂起等待解码结束，从而能腾出宝贵算力去处理复杂的业务逻辑或网络通信，实现真正的软硬件解耦与并行。

```

1 # 伪代码：利用 Event 实现解码与推理的异步流水线
2 # stream_dvpp 负责解码，stream_infer 负责推理
3 acl.media.dvpp_jpeg_decode_async(..., stream_dvpp)
4
5 # 1. 在解码流中记录一个“完成节点”
6 event_decode_done, _ = acl.rt.create_event()
7 acl.rt.record_event(event_decode_done, stream_dvpp)
8
9 # 2. 让推理流等待这个节点 (Host 不阻塞，仅 NPU 内部等待)
10 acl.rt.stream_wait_event(stream_infer, event_decode_done)
11
12 # 3. 只有当解码完成后，推理流才会开始执行模型
13 acl.mdl.execute_async(..., stream_infer)

```

2. 数据传输优化：Host Pinned Memory 的强制管理

实现异步传输的高性能前提是 Host 侧内存的稳定性。在使用 `acl.rt.memcpy_async` 进行 Host 到 Device 的数据搬运时，源端的 Host 内存**必须**是通过 `acl.rt.malloc_host` 申请的页锁定内存（Pinned Memory）。普通的 `malloc` 申请的内存可能会被操作系统分页机制换出到磁盘交换区，导致 DMA 控制器无法安全、准确地访问数据。此外，内存的生命周期管理至关重要。在释放 Host 侧指针之前，必须通过 `acl.rt.synchronize_stream` 等手段确保所有涉及该内存块的 Stream 任务都已彻底执行完

毕。如果过早释放内存，NPU 正尝试读取数据时地址已失效，将导致读取到野指针数据，引发难以复现的推理错误或系统崩溃。

```

1 # 伪代码：异步传输的内存管理规范
2 # 必须使用 acl.rt.malloc_host 申请 Pinned Memory
3 host_ptr, _ = acl.rt.malloc_host(size)
4
5 # ... 填充数据到 host_ptr ...
6
7 # 执行异步拷贝
8 acl.rt.memcpy_async(dev_ptr, size, host_ptr, size, 1, stream)
9
10 # 错误做法：直接释放 host_ptr (DMA 可能还在搬运中！)
11 # acl.rt.free_host(host_ptr)
12
13 # 正确做法：先同步等待流结束，再释放
14 acl.rt.synchronize_stream(stream)
15 acl.rt.free_host(host_ptr)

```

3. 多 Stream 协作范式：生产者-消费者模型

构建基于生产者-消费者模型的多 Stream 协作机制，是挖掘 310B 硬件潜能、提升应用帧率（FPS）的关键手段。具体的实施路径是通过 Stream 的划分将任务解耦为“预处理”、“推理”和“后处理”等独立阶段，通过 Event 机制实现流水线衔接。这种模式下，前一个阶段完成后记录 Event，后一个阶段感应 Event 并启动，就像工厂流水线一样运转。其最大优势在于避免了 CPU 使用 `while` 循环去轮询 NPU 的状态，极大降低了 CPU 占用率。对于昇腾 310B 这类嵌入式 SoC 而言，节省下来的 CPU 开销意味着开发者可以在 Python 层运行更复杂的逻辑判断，从而提升整个系统的智能化水平。

```

1 # 伪代码：多 Stream 协作
2 # 循环中处理每一帧
3 for image in images:
4     # 阶段 1: 预处理流 (Stream A)
5     preprocess(image, stream_a)
6     acl.rt.record_event(evt_pre, stream_a)
7
8     # 阶段 2: 推理流 (Stream B)
9     # 只有当预处理完成，推理才开始
10    acl.rt.stream_wait_event(stream_b, evt_pre)
11    model_inference(stream_b)
12    acl.rt.record_event(evt_infer, stream_b)
13
14    # 阶段 3: 后处理流 (Stream C) or Host 读取
15    # 只有推理完成，才处理结果
16    acl.rt.stream_wait_event(stream_c, evt_infer)
17    post_process(stream_c)

```

4. 避免滥用全局同步与调试建议

在追求高性能的同时，必须警惕对同步接口的滥用。最典型的反模式是在每一帧的处理循环中都调用 `acl.rt.synchronize_device()`，这种做法相当于强制将所有并行的

流水线“拍扁”为串行执行，完全浪费了 NPU 的并行处理能力。全局同步应当被严格限制在程序初始化（确保设备状态复位）、调试阶段（精确定位错误发生的算子）或进程退出阶段（安全回收所有资源）。对于开发者而言，调试异步程序确实存在挑战，因为报错行往往滞后于实际错误发生点。因此，建议遵循“从同步到异步”的演进路线：在开发初期，全部使用同步接口（如 `synchronize_stream`）确保逻辑正确性和内存安全；进入性能调优阶段后，再逐步替换为异步接口并引入 Event 机制，配合 Profiling 工具观察流水线中的空隙（Bubble），逐步压榨硬件性能。

小结

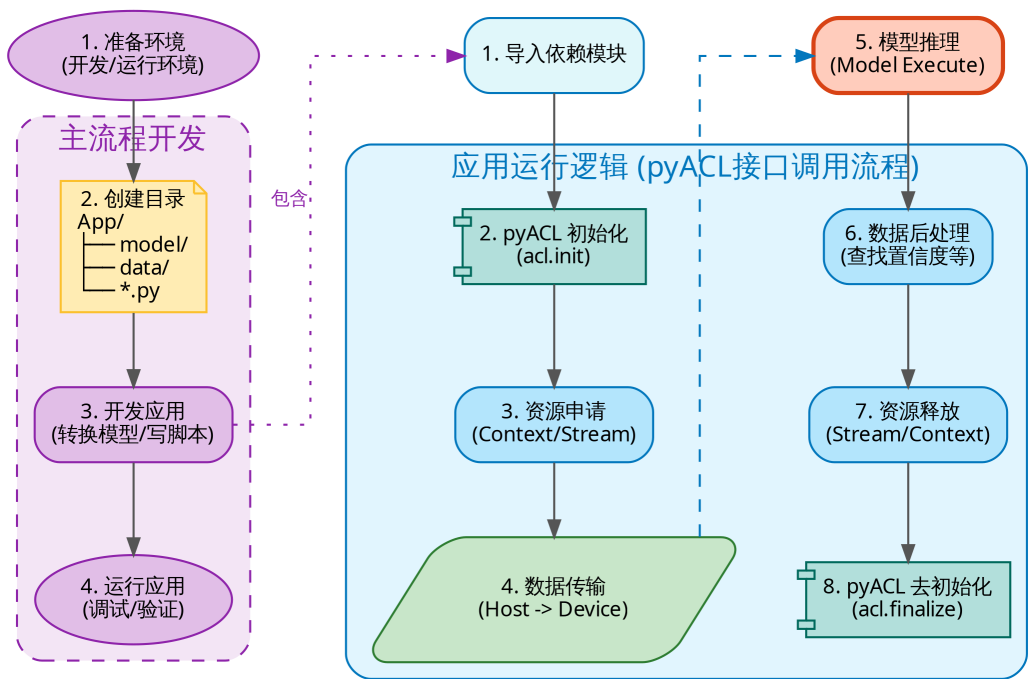
掌握 PyACL 同步机制的核心在于理解“控制流（CPU）与数据流（NPU）的分离”。在昇腾 310B 开发中，优秀的架构设计应当是：Host 线程像一个从容的指挥官，通过 Event 编排好各 Stream 的协作顺序后便抽身而去，留给 NPU 硬件去并行处理繁重的数据搬运与计算任务。合理使用 `stream_wait_event` 实现 Device 内部的依赖隔离，仅在必须获取最终结果时使用 `synchronize_stream` 回收数据，是在边缘端实现高性能推理的黄金法则。

4.2. 模型推理流水线

模型推理是 PyACL 应用开发的核心场景。为了能够让训练好的深度学习模型在昇腾 AI 处理器上高效运行，开发者需要遵循一套从环境准备到资源释放的标准化全流程。

4.2.1. 模型推理开发流程解析

整个模型推理应用的构建过程可以宏观地分为“主流程开发”与“应用运行逻辑”两个层面，如下图所示：



{fig:pyacl_inference width=100% .center}

主流程开发（Preparation Strategy）

主流程开发主要涉及应用构建的物理层面的准备工作。首先，开发者需要确保昇腾 AI 处理器的驱动与固件、CANN 软件（包含 pyACL）以及 Python 运行环境均已正确安装并完成环境变量配置，这是应用运行的基石。其次，为了保证项目的可维护性，建议采用标准化的目录结构，例如规划专门的 model/ 目录存放离线模型、data/ 目录存放测试图片或数据集，以及独立的脚本目录。紧接着进入核心开发阶段，这包括使用 ATC 工具将原始框架（如 ONNX 或 PyTorch）的模型转换为昇腾专用的 .om 离线模型，以及编写 Python 主程序以串联推理逻辑。最后，开发者需要在板端实际执行脚本，进行应用的验证与调试，确保从模型转换到最终输出的整个链路畅通无阻。

应用运行逻辑（Execution Logic）

这是编写 Python 脚本时的代码执行时序，严格遵循 PyACL 的接口调用规范，主要包含以下八个关键步骤：

- 1. 导入依赖：import acl 引入 pyACL 库。
- 2. 系统初始化：调用 acl.init() 进行全局配置初始化，这是一切操作的起点。

3. **资源申请**: 创建 Device 连接, 配置 Context (上下文) 与 Stream (执行流), 为后续计算搭建“舞台”。
4. **数据传输 (Host -> Device)**: 在执行推理前, 必须将待处理的图片或矩阵数据从 CPU 侧 (Host) 搬运至 NPU 侧 (Device)。这通常涉及 `acl.rt.malloc` 申请 Device 内存以及 `acl.rt.memcpy` 执行搬运。
5. **模型推理 (Inference)**: 这是流程的核心。调用 `acl.mdl.execute` 接口, 指示 AI Core 利用加载的 .om 模型对 Device 内存中的数据进行计算。
6. **数据后处理**: 将推理产生的结果 (通常在 Device 侧) 回传至 Host 侧 (或直接在 Device 侧处理), 通过 Python 代码解析概率向量 (Softmax)、筛选置信度或进行坐标转换。
7. **资源释放**: 业务完成后, 必须按“先释放 Stream, 再释放 Context, 最后重置 Device”的逆序释放资源。
8. **系统去初始化**: 最后调用 `acl.finalize()`, 通知系统回收全局资源, 结束进程。

4.2.2. 实例分析 (ResNet-18)

在第三章中, 我们初步介绍了 ResNet 的网络结构。本章我们将继续以 ResNet 为例, 深入探讨 PyACL 的编程技巧与应用。上一章中, 由于昇腾 310B 的 PyTorch 插件主要用于推理加速, 且端侧设备训练算力有限, 我们选用了较小的 CIFAR-10 数据集。而在本章, 我们将采用更贴近实际生产的标准开发流程, 并引入 **Tiny-ImageNet** 数据集进行实战演练。我们首先利用 Nvidia 显卡配合 CUDA 和 PyTorch 对 ResNet-18 进行训练, 获取模型权重; 随后将其转换为 ONNX 模型, 并最终转换为昇腾特定的 OM 模型。我们将分别在昇腾 310B (NPU-8T)、昇腾 310B (CPU)、树莓派 5B (CPU) 以及 RTX 5090D 上进行推理测试, 重点对比 OnnxRuntime 与 PyACL 的性能差异, 深入分析 NPU 带来的帧率提升。

Tiny-ImageNet 数据集简介

Tiny-ImageNet 是大规模视觉识别挑战赛 (ILSVRC) 中 ImageNet 数据集的一个微型子集, 常被用于深度学习模型的快速原型设计与基准测试。该数据集包含 200 个不同的物体类别, 这一数量远超 CIFAR-10 的 10 个类别, 从而更具挑战性, 能更好地验证模型的泛化能力。在数据规模方面, 它拥有 100,000 张训练图片 (每个类别 500 张)、10,000 张验证图片 (每个类别 50 张) 以及 10,000 张测试图片。所有图像的分辨率统一为 64x64 像素, 虽然低于标准 ImageNet 的 224x224, 但相比 CIFAR-10 的 32x32 分辨率包含了更多细节, 非常适合在算力受限的嵌入式设备上进行中规模实验。

在模型选择上，尽管 ResNet-50 或 ResNet-101 拥有更深的网络层数和潜在的更高精度，但在嵌入式 AI 开发场景下，ResNet-18 往往是性能与效率的最佳平衡点。首先，考虑到昇腾 310B 定位为边缘计算设备，ResNet-18 约 11M 的参数数量和适中的计算量能够更直观地体现 NPU 在高吞吐场景下的加速优势，避免因网络过大导致的内存瓶颈掩盖推理效率。其次，对于 64x64 分辨率的 Tiny-ImageNet，ResNet-18 已具备足够的特征提取能力，使用过深的网络反而容易导致过拟合且训练耗时过长，不利于教学演示与快速迭代。最后，ResNet-18 作为业界最通用的轻量级骨干网络之一，常被作为衡量树莓派、Jetson 以及 Ascend 等不同端侧硬件性能的经典标尺。

ResNet-18 模型训练

如果你希望自行训练模型以便进行测试，可以参考本节的模型训练部分；如果只需体验昇腾 310B 上 PyACL 的推理性能，可直接跳转至下一个小节“PyACL 的推理”。

为了进行模型训练，我们需要一台配备 Nvidia 显卡并已正确安装 CUDA 驱动的服务器。本例中，我们使用 GeForce 5090D 显卡。对于图像分类任务，Nvidia GeForce 系列显卡已能很好地满足需求。我们选用 Tiny-ImageNet 数据集进行训练，该数据集相比完整版 ImageNet 体积更小，便于测试和实验。使用 GeForce 5090D 训练一次大约需要 1~2 小时；而若采用完整的 ImageNet 数据集，单卡训练可能需要长达一周的时间。

我们需要从 Hugging Face 下载 Tiny-ImageNet 数据集，为此首先需要使用 pip 安装 Hugging Face 的 CLI 工具。如果在下载过程中遇到网络连接问题，建议配置镜像源，例如使用 hf-mirror。可以通过以下命令设置环境变量来启用镜像：

```
1 export HF_ENDPOINT=https://hf-mirror.com
```

为了方便起见，我们可以运行以下命令将该环境变量添加到 .bashrc 文件中，使其永久生效：

```
1 echo 'export HF_ENDPOINT=https://hf-mirror.com' >> ~/.bashrc
2 source ~/.bashrc
```

为了避免影响服务器上已有的虚拟环境，建议提前安装 Anaconda，并通过以下命令新建独立的环境：

```
1 conda create -n torch
2 conda activate torch
3 conda install python=3.11
```

此处选择 Python 3.11 作为示例，实际可根据需求选择合适的版本，但不建议使用过新或过旧的版本，以减少兼容性问题。

随后，使用 pip 安装所需依赖：

```
1 pip install datasets huggingface_hub torch torchvision timm tensorboard
```

本例采用的 ResNet-18 模型在结构上做了针对 Tiny-ImageNet 的适配。与标准 ResNet-18 不同，输入层采用 3x3 卷积且 stride=1，移除了原有的 7x7 卷积和最大池化层，以更好地保留 64x64 小尺寸图像的空间信息。此外，模型中增加了 Dropout 层以缓解过拟合，并在损失函数中引入标签平滑（Label Smoothing）。优化器选用带动量和权重衰减的 SGD，并配合多步学习率调度器，进一步提升模型的泛化能力。

数据集的下载和加载通过 Hugging Face 的 datasets 库实现。只需一行代码即可自动下载并缓存 Tiny-ImageNet 数据集，无需手动解压和整理文件。示例代码如下：

```
1 from datasets import load_dataset
2 dataset = load_dataset('zh-plus/tiny-imagenet', cache_dir='./data')
```

在数据预处理部分，训练集采用了多种图像增强技术，包括随机裁剪（Random-Crop）、随机水平翻转（RandomHorizontalFlip）、随机旋转（RandomRotation）以及颜色抖动（ColorJitter）。这些增强方法能够人为增加训练样本的多样性，使模型在训练过程中见到更多变化的图像，有效缓解过拟合问题，提高模型的泛化能力。

此外，模型训练过程中采用了 MultiStepLR 学习率调度策略（multistep），即在训练到指定 epoch 时自动降低学习率。这种做法可以让模型在初期快速收敛，在后期以更小的步长微调参数，进一步防止过拟合。图像增强和多步学习率调度的结合，有助于提升模型在验证集和实际应用中的表现。

完成的模型训练代码如下：

```
1 import os
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 from torch.utils.data import DataLoader
6 from datasets import load_dataset
7 from torchvision.transforms import Compose, ToTensor, Normalize,
8   ↪ RandomCrop, RandomHorizontalFlip, RandomRotation, ColorJitter # 增
9   ↪ 加更多增强
10 from torch.utils.tensorboard import SummaryWriter # 导入 SummaryWriter
11
12 # 自定义 ResNet 模型组件
13 class BasicBlock(nn.Module):
14     expansion = 1
15
16     def __init__(self, inplanes, planes, stride=1, downsample=None):
17         super(BasicBlock, self).__init__()
18         self.conv1 = nn.Conv2d(inplanes, planes, kernel_size=3, stride=
19   ↪ stride, padding=1, bias=False)
20         self.bn1 = nn.BatchNorm2d(planes)
21         self.relu = nn.ReLU(inplace=True)
22         self.conv2 = nn.Conv2d(planes, planes, kernel_size=3, padding=1,
23   ↪ bias=False)
```

```

20     self.bn2 = nn.BatchNorm2d(planes)
21     self.downsample = downsample
22     self.stride = stride
23
24     def forward(self, x):
25         identity = x
26         out = self.conv1(x)
27         out = self.bn1(out)
28         out = self.relu(out)
29         out = self.conv2(out)
30         out = self.bn2(out)
31         if self.downsample is not None:
32             identity = self.downsample(x)
33         out += identity
34         out = self.relu(out)
35         return out
36
37 # 移除 Bottleneck 类, ResNet18 使用 BasicBlock
38 class ResNet(nn.Module):
39     def __init__(self, block, layers, num_classes=200):
40         super(ResNet, self).__init__()
41         self.inplanes = 64
42         # 适配 Tiny ImageNet (64x64): 使用 3x3 卷积, stride=1, 移除
43         #     ↳ MaxPool
44         self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
45         #     ↳ bias=False)
46         self.bn1 = nn.BatchNorm2d(64)
47         self.relu = nn.ReLU(inplace=True)
48         # self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
49         #     ↳ # 移除
50
51         self.layer1 = self._make_layer(block, 64, layers[0])
52         self.layer2 = self._make_layer(block, 128, layers[1], stride=2)
53         self.layer3 = self._make_layer(block, 256, layers[2], stride=2)
54         self.layer4 = self._make_layer(block, 512, layers[3], stride=2)
55         self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
56         self.dropout = nn.Dropout(p=0.5) # 增加 Dropout 层
57         self.fc = nn.Linear(512 * block.expansion, num_classes)
58
59     for m in self.modules():
60         if isinstance(m, nn.Conv2d):
61             nn.init.kaiming_normal_(m.weight, mode='fan_out',
62             #     ↳ nonlinearity='relu')
63         elif isinstance(m, nn.BatchNorm2d):
64             nn.init.constant_(m.weight, 1)
65             nn.init.constant_(m.bias, 0)
66
67     def _make_layer(self, block, planes, blocks, stride=1):
68         downsample = None
69         if stride != 1 or self.inplanes != planes * block.expansion:
70             downsample = nn.Sequential(
71                 nn.Conv2d(self.inplanes, planes * block.expansion,
72                 #     ↳ kernel_size=1, stride=stride, bias=False),
73                 nn.BatchNorm2d(planes * block.expansion),

```

```

69         )
70         layers = []
71         layers.append(block(self.inplanes, planes, stride, downsample))
72         self.inplanes = planes * block.expansion
73         for _ in range(1, blocks):
74             layers.append(block(self.inplanes, planes))
75         return nn.Sequential(*layers)
76
77     def forward(self, x):
78         x = self.conv1(x)
79         x = self.bn1(x)
80         x = self.relu(x)
81         # x = self.maxpool(x) # 移除
82         x = self.layer1(x)
83         x = self.layer2(x)
84         x = self.layer3(x)
85         x = self.layer4(x)
86         x = self.avgpool(x)
87         x = torch.flatten(x, 1)
88         x = self.dropout(x) # 应用 Dropout
89         x = self.fc(x)
90         return x
91
92     def train_resnet18_on_tiny_imagenet():
93         # 定义保存路径
94         data_dir = './data'
95         model_dir = './model'
96         log_dir = './logs/resnet18_tiny_imagenet' # TensorBoard 日志目录
97         os.makedirs(model_dir, exist_ok=True)
98
99         # 初始化 TensorBoard Writer
100         writer = SummaryWriter(log_dir)
101
102         # 加载数据集 (指定 cache_dir)
103         dataset = load_dataset('zh-plus/tiny-imagenet', cache_dir=data_dir) #
104             ↳ 加载完整数据集以获取 train 和 valid
105
106         # 数据预处理 (增加数据增强)
107         # 训练集: 增加随机裁剪、水平翻转、旋转和颜色抖动
108         train_transform = Compose([
109             RandomCrop(64, padding=4),
110             RandomHorizontalFlip(),
111             RandomRotation(15), # 增加随机旋转
112             ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue
113                 ↳ =0.1), # 增加颜色抖动
114             ToTensor(),
115             Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
116         ])
117
118         # 验证集: 仅保持标准化
119         val_transform = Compose([
120             ToTensor(),
121             Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
122         ])

```

```

121
122 # 自定义数据集类
123 class TinyImageNetDataset(torch.utils.data.Dataset):
124     def __init__(self, dataset, transform=None):
125         self.dataset = dataset
126         self.transform = transform
127
128     def __len__(self):
129         return len(self.dataset)
130
131     def __getitem__(self, idx):
132         item = self.dataset[idx]
133         image = item['image']
134         label = item['label']
135         image = image.convert('RGB') # 确保图像为RGB格式
136         if self.transform:
137             image = self.transform(image)
138         return image, label
139
140 # 使用不同的 transform
141 train_dataset = TinyImageNetDataset(dataset['train'], transform=
    ↳ train_transform)
142 val_dataset = TinyImageNetDataset(dataset['valid'], transform=
    ↳ val_transform) # 假设 valid split 存在
143
144 train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True
    ↳ ) # 将 batch_size 适当调大一点, 例如 64 或 128
145 val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False)
146
147 # 定义模型 (使用自定义 ResNet18: BasicBlock, [2, 2, 2, 2])
148 model = ResNet(BasicBlock, [2, 2, 2, 2], num_classes=200)
149 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
150 model.to(device)
151
152 # 损失函数和优化器 (更换为 SGD + Momentum + Weight Decay 以抗过拟合)
153 # 启用标签平滑 (Label Smoothing) 以防止过拟合
154 criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
155 # 使用 SGD 替代 Adam, 初始学习率设为 0.1, weight_decay=5e-4 用于正则
    ↳ 化
156 optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9,
    ↳ weight_decay=5e-4)
157 # 添加学习率调度器, 在第 30, 60, 90 epoch 衰减学习率
158 scheduler = optim.lr_scheduler.MultiStepLR(optimizer, milestones=[30,
    ↳ 60, 90], gamma=0.1)
159
160 # 训练循环
161 num_epochs = 150
162 best_acc = 0.0
163
164 for epoch in range(num_epochs):
165     model.train()
166     running_loss = 0.0
167     correct_train = 0
168     total_train = 0

```

```

169
170 # 移除 tqdm, 使用简单的迭代
171 for i, (images, labels) in enumerate(train_loader):
172     images, labels = images.to(device), labels.to(device)
173     optimizer.zero_grad()
174     outputs = model(images)
175     loss = criterion(outputs, labels)
176     loss.backward()
177     optimizer.step()
178
179     running_loss += loss.item()
180     _, predicted = torch.max(outputs.data, 1)
181     total_train += labels.size(0)
182     correct_train += (predicted == labels).sum().item()
183
184 # 记录 iteration 级别的 loss (可选)
185 if i % 100 == 0:
186     writer.add_scalar('Training/Iter_Loss', loss.item(),
187                      ↪ epoch * len(train_loader) + i)
188
189 train_loss = running_loss / len(train_loader)
190 train_acc = 100. * correct_train / total_train
191
192 # 记录 epoch 级别的训练指标
193 writer.add_scalar('Training/Epoch_Loss', train_loss, epoch + 1)
194 writer.add_scalar('Training/Epoch_Accuracy', train_acc, epoch +
195                  ↪ 1)
196
197 # 更新学习率并记录
198 current_lr = optimizer.param_groups[0]['lr']
199 writer.add_scalar('Training/Learning_Rate', current_lr, epoch +
200                  ↪ 1)
201 scheduler.step()
202
203 # 验证阶段
204 model.eval()
205 correct_val = 0
206 total_val = 0
207
208 with torch.no_grad():
209     for images, labels in val_loader:
210         images, labels = images.to(device), labels.to(device)
211         outputs = model(images)
212         _, predicted = torch.max(outputs.data, 1)
213         total_val += labels.size(0)
214         correct_val += (predicted == labels).sum().item()
215
216 val_acc = 100. * correct_val / total_val
217
218 # 记录验证指标
219 writer.add_scalar('Validation/Accuracy', val_acc, epoch + 1)
220
221 print(f'Epoch {epoch+1}: LR: {current_lr:.6f}, Train Loss: {
222       ↪ train_loss:.4f}, Train Acc: {train_acc:.2f}%, Val Acc: {

```

```

219         ↪ val_acc:.2f}%)
220     # 保存 Checkpoint (每个 epoch 保存一次)
221     checkpoint_path = os.path.join(model_dir, f'checkpoint_epoch.pth'
222         ↪ )
223     torch.save({
224         'epoch': epoch + 1,
225         'model_state_dict': model.state_dict(),
226         'optimizer_state_dict': optimizer.state_dict(),
227         'loss': train_loss,
228         'val_acc': val_acc
229     }, checkpoint_path)
230     print(f"Checkpoint saved to {checkpoint_path}")
231
232     # 保存为ONNX (每当验证集准确率提升时保存, 或者最后保存)
233     if val_acc > best_acc:
234         best_acc = val_acc
235         # 保存最佳模型逻辑...
236
237     # 关闭 writer
238     writer.close()
239
240     # 保存为ONNX (输入尺寸调整为 64x64)
241     dummy_input = torch.randn(1, 3, 64, 64).to(device)
242     onnx_path = os.path.join(model_dir, 'resnet18_tiny_imagenet.onnx')
243     torch.onnx.export(model, dummy_input, onnx_path, verbose=True)
244     print(f"Model saved as ONNX: {onnx_path}")
245
246 if __name__ == '__main__':
    train_resnet18_on_tiny_imagenet()

```

为了实时监控模型训练过程中的损失、准确率和学习率变化，代码中集成了 **TensorBoard**。通过 **SummaryWriter** 将训练和验证指标写入日志文件，用户可在训练过程中通过如下命令启动 **TensorBoard**：

```
1 tensorboard --logdir=./logs/resnet18_tiny_imagenet
```

在浏览器中访问对应端口，即可可视化每个 **epoch** 的损失、准确率曲线和学习率变化，便于分析模型收敛情况和调参效果。模型训练结束后，我们可以在 **model** 文件夹下面，找到名为 **resnet18_tiny_imagenet.onnx** 的模型文件。

环境配置与模型转换

我们已经训练好了一个针对 **tiny-ImageNet** 的 **ResNet-18** 的网络，我们已经上传到 **huggingface** 的仓库 **zhouxzh/resnet18_tiny_imagenet**，因此我们不需要自己进行网络训练。

为了可以顺利的从 **huggingface** 下载已经训练好的模型，我们需要使用 **pip** 安装 **Hugging Face** 的 CLI 工具。

```
1 pip install datasets huggingface_hub
```

如果在下载过程中遇到网络连接问题，建议配置镜像源，例如使用 hf-mirror。可以通过以下命令设置环境变量来启用镜像：

```
1 export HF_ENDPOINT=https://hf-mirror.com
```

为了方便起见，我们可以运行以下命令将该环境变量添加到 .bashrc 文件中，使其永久生效：

```
1 echo 'export HF_ENDPOINT=https://hf-mirror.com' >> ~/.bashrc
2 source ~/.bashrc
```

接下来，我们在昇腾 310B 开发板上构建项目目录。请执行以下命令创建 resnet18 文件夹及必要的子目录结构：

```
1 mkdir -p resnet18
2 cd resnet18
3 mkdir -p {data,model}
```

随后，使用 Hugging Face CLI 下载预训练好的 ONNX 模型：

```
1 hf download zhouxzh/resnet18_tiny_imagenet --local-dir model/
```

下载完成后，我们需要使用 ATC (Ascend Tensor Compiler) 工具将 ONNX 模型转换为昇腾 NPU 专用的 OM (Offline Model) 模型。执行如下转换命令：

```
1 atc --model=model/resnet18_tiny_imagenet.onnx \
2     --framework=5 --output=model/resnet18_tiny_imagenet \
3     --input_shape="input.1:1,3,64,64" \
4     --soc_version=Ascend310B4
```

核心实现：模型推理脚本

为了在昇腾 310B 上加载 OM 模型并执行推理，我们需要编写一个完整的 Python 脚本。该脚本不仅负责调用底层 PyACL 接口，还需要处理数据的预处理 (Preprocessing) 与后处理 (Postprocessing)。本节将通过解析 inference_npu.py 的关键代码片段，深入讲解 PyACL 应用的构建逻辑。

完整的推理代码逻辑被封装在 AclResource 类中，它管理着从初始化到资源释放的整个生命周期。

1. 初始化与模型加载

在类的 init 方法中，我们依次完成系统初始化、Device 指定、Context 创建以及模型加载。值得注意的是，模型加载后，我们需要获取“模型描述” (Model Description)，它包含了模型输入输出张量的维度、大小和类型信息，这对于后续申请内存至关重要。


```

1 def init(self):
2     # 1. ACL 全局初始化
3     ret = acl.init()
4
5     # 2. 指定运算设备 (Device 0)
6     ret = acl.rt.set_device(self.device_id)
7
8     # 3. 创建 Context 上下文
9     self.context, ret = acl.rt.create_context(self.device_id)
10
11    # 4. 加载离线模型 (.om)
12    self.model_id, ret = acl.mdl.load_from_file(self.model_path)
13
14    # 5. 获取模型描述信息 (用于查询输入输出大小)
15    self.model_desc = acl.mdl.create_desc()
16    ret = acl.mdl.get_desc(self.model_desc, self.model_id)

```

2. 数据传输与内存管理

推理的核心在于 execute 方法。由于 NPU 无法直接读取 CPU (Host) 侧的 Numpy 数组，我们需要显式地进行内存拷贝。

- **输入准备：**首先通过 `acl.mdl.get_input_size_by_index` 查询模型所需的输入字节数。接着使用 `acl.rt.malloc` 在 Device 侧申请内存，并利用 `acl.rt.memcpy` 将 host 侧预处理好的 Numpy 数据拷贝到 Device 侧。
- **Dataset 组装：**PyACL 使用 Dataset 和 DataBuffer 结构来传递数据。我们需要创建一个 Dataset 容器，并将包含 Device 内存地址的 DataBuffer 添加进去。
- **输出准备：**同样地，根据模型输出大小申请 Device 侧内存，并组装 Output Dataset，用于存放 NPU 计算后的结果。

为了更加清晰地理解 Host 与 Device 之间的数据交互流程，我们可以将上述复杂的 execute 函数逻辑简化为以下伪代码：

```

1 def execute(input_data_host):
2     # 1. 准备输入 (Host -> Device)
3     # 申请 NPU 侧内存，并将预处理好的图片数据从 CPU 搬运过去
4     input_size = get_model_input_size()
5     dev_in = acl.malloc_device(input_size)
6     acl.memcpy(dev_in, input_data_host, direction=HOST_TO_DEVICE)
7
8     # 2. 准备输出 (Device)
9     # 在 NPU 侧申请内存，用于存放模型推理产生的结果
10    output_size = get_model_output_size()
11    dev_out = acl.malloc_device(output_size)
12
13    # 3. 封装 Dataset
14    # PyACL 要求将内存指针封装为 Dataset 结构才能传入模型
15    input_dataset = create_dataset(dev_in)
16    output_dataset = create_dataset(dev_out)
17

```

```

18 # 4. 执行推理 (Inference)
19 # 指挥 NPU 执行计算, 结果会写入 dev_out
20 acl.mdl.execute(model_id, input_dataset, output_dataset)
21
22 # 5. 获取结果 (Device -> Host)
23 # 申请 CPU 侧内存, 将推理结果从 NPU 搬回, 以便 Python 处理
24 host_out = acl.malloc_host(output_size)
25 acl.memcpy(host_out, dev_out, direction=DEVICE_TO_HOST)
26
27 # 6. 资源清理
28 # 释放本次推理申请的 Device 内存
29 acl.free(dev_in)
30 acl.free(dev_out)
31
32 return host_out

```

这段伪代码直观地展示了 PyACL 推理的核心特征：**内存不仅需要申请，还需要在 Host 与 Device 之间显式搬运 (Memcpy)**。这是与在通用 CPU 上开发程序最大的不同点。

3. 数据预处理

模型的输入通常需要特定的格式和分布。这里的 preprocess 函数模拟了标准 torchvision 的转换逻辑，但完全使用 Numpy 实现，以减少第三方库依赖。关键步骤包括：* **归一化**：将像素值除以 255 转为浮点，并减去均值除以标准差。* **维度变换**：将图片从 HWC（高宽通道）调整为 PyTorch 默认的 CHW（通道高宽）格式。* **增加 Batch 维度**：模型输入通常是 4 维张量 (N, C, H, W)，因此需要增加 Batch 维度。* **内存连续性**：这一步通过 np.ascontiguousarray 确保数据在内存中是连续存储的，这是 C 语言接口正确读取数据的必要条件。

4. 资源释放

脚本最后，我们需要显式释放所有持久化资源，防止 NPU 内存泄漏或状态异常。

```

1 def release(self):
2     acl.mdl.destroy_desc(self.model_desc)
3     acl.mdl.unload(self.model_id) # 卸载模型
4     acl.rt.destroy_context(self.context) # 销毁 Context
5     acl.rt.reset_device(self.device_id) # 重置 Device
6     acl.finalize() # 去初始化

```

通过将这些模块组合，我们便得到了一个名为 inference_npu.py 的完整推理脚本。

```

1 import time
2 import numpy as np
3 import acl
4 from datasets import load_dataset
5 import tqdm
6
7 import ctypes
8

```

```

9 data_dir = './data' # 假设 data_dir 已定义, 或者根据实际情况保留原变量
10
11 # 加载数据集 (指定 cache_dir)
12 dataset = load_dataset('zh-plus/tiny-imagenet', split='valid', cache_dir=
    ↪ data_dir) # 只加载 valid 集
13
14 # ACL 初始化与资源管理类
15 class AclResource:
16     def __init__(self, device_id=0, model_path="model/
    ↪ resnet18_tiny_imagenet.om"):
17         self.device_id = device_id
18         self.model_path = model_path
19         self.model_id = None
20         self.context = None
21         self.stream = None
22         self.input_dataset = None
23         self.output_dataset = None
24         self.model_desc = None
25
26     def init(self):
27         # ACL 初始化
28         ret = acl.init()
29         if ret != 0: raise RuntimeError("acl init failed")
30         ret = acl.rt.set_device(self.device_id)
31         if ret != 0: raise RuntimeError("set device failed")
32         self.context, ret = acl.rt.create_context(self.device_id)
33         if ret != 0: raise RuntimeError("create context failed")
34
35         # 加载模型
36         self.model_id, ret = acl.mdl.load_from_file(self.model_path)
37         if ret != 0: raise RuntimeError(f"load model failed, path: {self.
    ↪ model_path}")
38
39         # 获取模型描述
40         self.model_desc = acl.mdl.create_desc()
41         ret = acl.mdl.get_desc(self.model_desc, self.model_id)
42
43         print(f"ACL Resource Init Success. Device: {self.device_id}")
44
45     def execute(self, input_numpy):
46         # 准备输入数据 (Host -> Device)
47         # 获取模型输入大小
48         input_size = acl.mdl.get_input_size_by_index(self.model_desc, 0)
49
50         # 申请 Device 输入内存
51         dev_in_ptr, ret = acl.rt.malloc(input_size, 2) # 2:
    ↪ ACL_MEM_MALLOC_HUGE_FIRST
52
53         # 拷贝数据 (Host -> Device)
54         # 确保输入是 contiguous 且 float32 (C Type bytes)
55         # 使用 tobytes + bytes_to_ptr 避免 numpy_to_ptr 可能引发的
    ↪ ImportError 兼容性问题
56         if input_numpy.nbytes != input_size:
57             print(f"Warning: Input size mismatch. Model expects {

```

```

        ↪ input_size}, got {input_numpy.nbytes}")
58
59     input_bytes = input_numpy.tobytes()
60     input_ptr = acl.util.bytes_to_ptr(input_bytes)
61     # acl.rt.memcpy (dst, dest_max, src, count, kind)
62     acl.rt.memcpy(dev_in_ptr, input_size, input_ptr, input_size, 1) #
        ↪ 1: ACL_MEMCPY_HOST_TO_DEVICE
63
64     # 组装 Input Dataset
65     self.input_dataset = acl.mdl.create_dataset()
66     input_data_buffer = acl.create_data_buffer(dev_in_ptr, input_size
        ↪ )
67     acl.mdl.add_dataset_buffer(self.input_dataset, input_data_buffer)
68
69     # 准备输出数据
70     self.output_dataset = acl.mdl.create_dataset()
71     output_size = acl.mdl.get_output_size_by_index(self.model_desc,
        ↪ 0)
72     dev_out_ptr, ret = acl.rt.malloc(output_size, 2)
73     output_data_buffer = acl.create_data_buffer(dev_out_ptr,
        ↪ output_size)
74     acl.mdl.add_dataset_buffer(self.output_dataset,
        ↪ output_data_buffer)
75
76     # 执行推理
77     ret = acl.mdl.execute(self.model_id, self.input_dataset, self.
        ↪ output_dataset)
78     if ret != 0: print("Model execute failed")
79
80     # 取回结果 (Device -> Host)
81     host_out_buffer, ret = acl.rt.malloc_host(output_size)
82     acl.rt.memcpy(host_out_buffer, output_size, dev_out_ptr,
        ↪ output_size, 2) # 2: ACL_MEMCPY_DEVICE_TO_HOST
83
84     # 转换为 numpy (假设输出是 float32, batch=1, class=200)
85     out_array = np.frombuffer(ctypes.string_at(host_out_buffer,
        ↪ output_size), dtype=np.float32)
86
87     # 清理单次推理资源
88     acl.rt.free(dev_in_ptr)
89     acl.rt.free(dev_out_ptr)
90     acl.rt.free_host(host_out_buffer)
91     acl.destroy_data_buffer(input_data_buffer)
92     acl.destroy_data_buffer(output_data_buffer)
93     acl.mdl.destroy_dataset(self.input_dataset)
94     acl.mdl.destroy_dataset(self.output_dataset)
95
96     return out_array
97
98 def release(self):
99     acl.mdl.destroy_desc(self.model_desc)
100     acl.mdl.unload(self.model_id)
101     acl.rt.destroy_context(self.context)
102     acl.rt.reset_device(self.device_id)

```

```

103     acl.finalize()
104
105 # 实例化并初始化 ACL 资源
106 # 请确保当前路径下有对应的 .om 模型文件
107 om_model_path = "model/resnet18_tiny_imagenet.om"
108 acl_resource = AclResource(model_path=om_model_path)
109 acl_resource.init()
110
111 # 自定义数据预处理函数 (替代 torchvision)
112 def preprocess(image):
113     # 将 PIL Image 转换为 numpy 数组
114     img_data = np.array(image).astype('float32') / 255.0
115
116     # 获取图像尺寸, 如果不是 RGB 三通道 (例如灰度图), 需要转换
117     if len(img_data.shape) == 2:
118         img_data = np.stack([img_data]*3, axis=-1)
119
120     # 归一化参数
121     mean = np.array([0.485, 0.456, 0.406], dtype='float32')
122     std = np.array([0.229, 0.224, 0.225], dtype='float32')
123
124     # 归一化: (image - mean) / std
125     img_data = (img_data - mean) / std
126
127     # 调整维度: HWC -> CHW (3, 64, 64)
128     img_data = img_data.transpose(2, 0, 1)
129
130     # 增加 Batch 维度: (1, 3, 64, 64)
131     img_data = np.expand_dims(img_data, axis=0)
132
133     # 确保内存连续, 这对 C 侧指针拷贝很重要
134     if not img_data.flags['C_CONTIGUOUS']:
135         img_data = np.ascontiguousarray(img_data)
136
137     return img_data
138
139 # 推理计数和计时
140 total_images = 0
141 correct_count = 0
142 start_time = time.time()
143
144 print("开始推理...")
145
146 # 逐张图片推理
147 for item in tqdm.tqdm(dataset):
148     image = item['image'] # 获取 PIL 图像
149     label = item['label'] # 获取真实标签
150
151     # 预处理
152     input_tensor = preprocess(image)
153
154     # 推理 (使用 pyACL)
155     # output 是扁平的一维数组, 直接使用
156     outputs = acl_resource.execute(input_tensor)

```


5090D 显卡和主流的 Intel Core Ultra 7 155H 笔记本处理器，还纳入了同属嵌入式领域的树莓派 5B，以及昇腾 310B 自身的 CPU 模式。我们将详细记录各平台的推理耗时与帧率 (FPS)，以此作为基准来量化 NPU 带来的性能提升。以下是各平台通用的 OnnxRuntime 推理测试代码：

```

1 import time
2 import numpy as np
3 import onnxruntime
4
5 from datasets import load_dataset
6 import tqdm
7
8 data_dir = './data' # 假设 data_dir 已定义，或者根据实际情况保留原变量
9
10 # 加载数据集 (指定 cache_dir)
11 dataset = load_dataset('zh-plus/tiny-imagenet', split='valid', cache_dir=
    ↳ data_dir) # 只加载 valid 集
12
13 # ONNX Runtime 初始化
14 onnx_model_path = "model/resnet18_tiny_imagenet.onnx" # 请确保当前路径下
    ↳ 有该模型文件
15 session = onnxruntime.InferenceSession(onnx_model_path, providers=[
    ↳ 'CUDAExecutionProvider', 'CPUExecutionProvider'])
16 print(f"当前运行设备 (Providers): {session.get_providers()}") # 打印以确
    ↳ 认 CUDA 是否生效
17
18 input_name = session.get_inputs()[0].name
19
20 # 自定义数据预处理函数 (替代 torchvision)
21 def preprocess(image):
22     # 将 PIL Image 转换为 numpy 数组
23     img_data = np.array(image).astype('float32') / 255.0
24
25     # 获取图像尺寸，如果不是 RGB 三通道 (例如灰度图)，需要转换
26     if len(img_data.shape) == 2:
27         img_data = np.stack([img_data]*3, axis=-1)
28
29     # 归一化参数
30     mean = np.array([0.485, 0.456, 0.406], dtype='float32')
31     std = np.array([0.229, 0.224, 0.225], dtype='float32')
32
33     # 归一化: (image - mean) / std
34     img_data = (img_data - mean) / std
35
36     # 调整维度: HWC -> CHW (3, 64, 64)
37     img_data = img_data.transpose(2, 0, 1)
38
39     # 增加 Batch 维度: (1, 3, 64, 64)
40     img_data = np.expand_dims(img_data, axis=0)
41
42     return img_data
43

```

```

44 # 推理计数和计时
45 total_images = 0
46 correct_count = 0
47 start_time = time.time()
48
49 print("开始推理...")
50
51 # 逐张图片推理
52 for item in tqdm.tqdm(dataset):
53     image = item['image'] # 获取 PIL 图像
54     label = item['label'] # 获取真实标签
55
56     # 预处理
57     input_tensor = preprocess(image)
58
59     # 推理
60     outputs = session.run(None, {input_name: input_tensor})
61
62     # 获取预测结果: argmax 获取概率最大的类别索引
63     predicted_label = np.argmax(outputs[0])
64
65     if predicted_label == label:
66         correct_count += 1
67
68     total_images += 1
69     # if total_images >= 100: break # 可选: 用于快速测试
70
71 end_time = time.time()
72 duration = end_time - start_time
73 fps = total_images / duration
74 accuracy = correct_count / total_images * 100
75
76 print(f"推理完成。")
77 print(f"总图片数: {total_images}")
78 print(f"总耗时: {duration:.4f} 秒")
79 print(f"推理帧率: {fps:.2f} FPS")
80 print(f"正确率: {accuracy:.2f}%")

```

为了直观评估昇腾 310B NPU 的加速效果，我们将基于统一的 ResNet-18 ONNX 模型，在多种硬件平台上开展推理性能的横向对比测试。测试对象不仅包括高性能的 NVIDIA RTX 5090D 显卡和主流的 Intel Core Ultra 7 155H 笔记本处理器，还纳入了同属嵌入式领域的树莓派 5B，以及昇腾 310B 自身的 CPU 模式。

所有平台均安装了 OnnxRuntime 进行推理。其中 RTX 5090D 配置了 CUDA 加速，而其他平台（包括树莓派、Intel 笔记本和昇腾 310B 的 CPU 模式）均仅使用 CPU 进行计算。我们将详细记录各平台的推理耗时与帧率 (FPS)，以此作为基准来量化 NPU 带来的性能提升。

以下是各平台的具体测试结果：

1. Ascend 310B (CPU 模式) + OnnxRuntime: (规格:4 核 Cortex-A55 @ 1.0GHz)

多平台 ResNet-18 (Tiny-ImageNet) 推理性能对比数据:

硬件平台	推理框架	运行设备	帧率 (FPS)	相对 NPU 性能 (265 FPS)
Ascend 310B NPU	PyACL	NPU (AI Core)	265.02	100% (基准)
Nvidia RTX 5090D	OnnxRuntime	GPU (CUDA)	1042.33	~393%
Intel Core Ultra 7 155H	OnnxRuntime	CPU	61.91	~23%
Raspberry Pi 5B	OnnxRuntime	CPU	22.38	~8.4%
Ascend 310B CPU	OnnxRuntime	CPU (Arm Cortex-A55)	5.02	~1.9%

通过数据对比, 我们可以清晰地看到昇腾 310B NPU 的强大优势: * **对比自身 CPU:** NPU 的推理速度是其 CPU 模式 (5.02 FPS) 的 **52 倍**, 充分证明了专用 AI 加速器的必要性。* **对比树莓派 5B:** 虽然同样是 Arm 架构的嵌入式设备, 昇腾 310B NPU 的性能是树莓派 5B CPU (22.38 FPS) 的 **11 倍以上**。* **对比高性能笔记本 CPU:** 即便是最新的 Intel Core Ultra 7 处理器 (61.91 FPS), 在没有独立显卡加速的情况下, 其纯 CPU 推理性能也不及昇腾 310B NPU 的 **1/4**。* **对比旗舰显卡:** 虽然 RTX 5090D 展现出了恐怖的 1000+ FPS 性能, 但考虑到昇腾 310B 这类边缘设备的低功耗和体积优势, 265 FPS 的成绩对于实时的嵌入式应用而言已经绰绰有余。

这一测试结果有力地展示了昇腾 310B 在边缘计算场景下的核心竞争力——**以极低的功耗提供数倍于通用 CPU 的 AI 算力**。

4.3. 图像/视频处理基础

4.3.1. PyACL：图像与视频处理简介

此外, 通过集成的 DVPP 接口, 应用可以在硬件层级完成视频编解码与图像预处理, 极大地减轻了 CPU 在数据清洗阶段的负担。本节介绍如何使用 PyACL 提供的媒体处理能力对图像/视频做高效预处理与编解码, 涵盖 AIPP 与 DVPP 的角色、常见模块、典型流水线与开发注意事项。

概述

- PyACL 的媒体处理分两类能力：
 - AIPP (Artificial Intelligence Pre-Processing): 在模型侧 (AI Core) 或模型运行链路上完成色域转换、裁剪/填充、减均值/乘系数等预处理。AIPP 分静态 (在模型转换时固化到.om) 与动态 (运行时通过接口设置) 两种模式。
 - DVPP (Digital Vision Pre-Processing): 昇腾硬件上的媒体处理单元, 通过 pyACL 的 acl.media 接口提供硬件加速的解码、缩放、格式转换等能力 (在 Device 侧高效执行)。

AIPP vs DVPP (何时用哪个)

- DVPP 适合做 “低级别” 的高吞吐预处理: JPEG/视频解码、YUV $\leftrightarrow$ RGB 转换、缩放、裁剪等, 优点是速度快、CPU 负载低, 但受格式与对齐约束。
- AIPP 适合做 “模型输入级” 的精确预处理: 统一色域/像素变换、量化/去均值、通道顺序等; 可作为补充以满足模型输入精度要求。
- 常见组合: 先用 DVPP 完成解码与粗略 resize/crop, 再用 AIPP (静态或动态) 做最后的色域和像素级处理保证与模型一致。

DVPP 主要功能模块

- VPC: 格式转换 (YUV/RGB)、缩放、裁剪、填充等。
- JPEGD / JPEGE: JPEG 解码 / 编码。
- VDEC / VENC: 视频编码器/解码器 (H.264/H.265)。
- PNGD: PNG 解码($\rightarrow$ RGB)。这些能力通过 acl.media.* 系列接口(如 dvpp_create_channel_desc/dvpp_create_channel, dvpp_jpeg_decode_async、dvpp_vpc_resize_async 等) 访问。

典型处理场景 (举例)

- 视频推理: VDEC 解码 $\rightarrow$ DVPP 进行 YUV 格式调整与缩放 $\rightarrow$ 若需 RGB 或额外预处理, 再由 AIPP 或 VPC 完成 $\rightarrow$ 传入模型。
- 静态图片分类: JPEGD 解码 $\rightarrow$ VPC 缩放/裁剪 $\rightarrow$ 如需色域/像素变换使用 AIPP (静态或动态) $\rightarrow$ 内存拷贝到 Device 模型输入。
- 单算子/自定义预处理: 在 Device 侧通过 DVPP 实现大部分工作, 减少 Host 上的 Python/CPU 计算。

开发流程（简要）

1. 环境准备：确保 CANN、Ascend 驱动与环境变量 (set_env.sh) 正确加载, Python 版本在支持范围内。
2. 目录与模型准备：若包含推理，需准备.om 模型（静态 AIPP 可在 ATC 时配置）。
3. 初始化 ACL: `acl.init()` -> `acl.rt.set_device()` -> 创建 Context/Stream。
4. DVPP 通道与缓冲：创建 channel 描述，使用 `acl.media.dvpp_malloc` 或 `acl.rt.malloc` 分配 Device 内存。
5. 处理流程示例（伪代码）：

```
python #创建 channel 和 stream
ch = acl.media.dvpp_create_channel_desc()
acl.media.dvpp_create_channel(ch) # 申请输入 Device buffer (Host -> Device)
in_dev = acl.media.dvpp_malloc(len(jpeg_bytes))
acl.rt.memcpy(in_dev, size, host_ptr, size, ACL_MEMCPY_HOST_TO_DEVICE)
)# 预测解码后大小并申请输出
out_size, _ = acl.media.dvpp_jpeg_predict_dec_size(host_ptr, size, out_cfg)
out_dev = acl.media.dvpp_malloc(out_size)
# 异步解码并等待
acl.media.dvpp_jpeg_decode_async(ch, in_dev, size, out_dev, out_size, out_cfg, stream)
acl.rt.synchronize_stream(stream)
```
6. 后续：对解码结果使用 VPC 进行 `resize/format` 转换，或配合 AIPP 做最终像素预处理；封装为模型输入 Dataset 并执行推理。
7. 资源释放：按逆序销毁 stream、context、channel、释放内存并调用 `acl.finalize()`。

注意事项与最佳实践

- DVPP 输出对分辨率与地址有对齐要求 (stride/padding)，读取数据时需依据描述信息处理。
- 使用异步接口 (*_async) 时必须配合 Stream 与同步机制 (record_event / stream_wait_event / synchronize_stream) 以确保内存生命周期安全。
- Host->Device 的异步拷贝源内存应使用页锁定 (pinned) 内存 (`acl.rt.malloc_host()`) 以保证 DMA 稳定性。
- 静态 AIPP 在 ATC 转换时固化参数；若需灵活参数，选择动态 AIPP 并在推理前通过 pyACL 接口设置。
- 优先把耗时的像素级操作下沉到 DVPP/AIPP，避免在 CPU (Python) 端逐像素处理以免成为瓶颈。

小结

在 PyACL 应用中，DVPP 提供高吞吐的硬件级图像/视频预处理能力，AIPP 提供与模型输入严格一致的像素级转换。合理地将两者组合，并配合异步流与事件机制，可以在边缘设备上实现高效、低延迟的图像/视频推理流水线。

4.3.2. DVPP

DVPP (Digital Vision Pre-Processing) 是昇腾处理器的硬件加速引擎，用于处理 JPEG 解码、缩放、抠图等，速度远超 CPU。

4.3.3. JPEGD (JPEG Decode)

将 .jpg 数据解码为 YUV 格式。

```

1 # 1. 创建图片描述信息
2 channel_desc = acl.media.dvpp_create_channel_desc()
3 acl.media.dvpp_create_channel(channel_desc)
4
5 # 2. 准备输入内存 (Host -> Device)
6 # 假设 np_jpg_data 是读取的二进制 jpg 数据
7 input_dev, _ = acl.media.dvpp_malloc(len(np_jpg_data))
8 acl.rt.memcpy(input_dev, len(np_jpg_data), np_jpg_data_ptr, len(
9     ↳ np_jpg_data), 1) # 1=Host2Device
10
11 # 3. 预测解码后大小并申请输出内存
12 output_desc = acl.media.dvpp_create_jpeg_config()
13 output_size, _ = acl.media.dvpp_jpeg_predict_dec_size(np_jpg_data_ptr,
14     ↳ len(np_jpg_data), output_desc)
15 output_dev, _ = acl.media.dvpp_malloc(output_size)
16
17 # 4. 执行异步解码
18 acl.media.dvpp_jpeg_decode_async(channel_desc, input_dev, len(np_jpg_data
19     ↳ ), output_dev, output_size, output_desc, stream)
20 acl.rt.synchronize_stream(stream) # 等待完成

```

4.3.4. VPC (Vision Preprocessing Core)

处理 Resize、Crop、Padding 等。输入必须是 Device 侧的 YUV 数据。关键步骤：创建 `acldvppPicDesc` 描述输入和输出图片的格式、宽高、Stride，然后调用 `acl.media.dvpp_vpc_resize_async`。

4.4. 单算子调用

除了完整的模型推理，PyACL 还支持更为细粒度的操作。如果应用涉及基础线性代数运算（BLAS）或特定的数学计算，开发者可以略过复杂的模型构建过程，直接通过算子调用接口加载并执行单个算子。这种方式更加轻量，适合进行算子级的性能验证或特定的数据变换任务。

如果不加载整个模型，仅需调用某个特定算子(如 Softmax, MatMul): 1. **acl.op.set_attr**: 设置算子属性。2. **acl.op.execute_v2**: 执行算子。需注意，算子名称和输入输出格式必须与 CANN 算子库定义一致。

4.5. 更多特性

- **Profiling**: 通过 `acl.prof` 接口控制性能数据采集的起止。
- **Dump**: 运行中导出模型各层的输入输出数据，用于精度比对。
- **AIPP**: 在模型转换时配置静态 AIPP，让硬件自动完成 Resize/ColorConvert, PyACL 代码中无需编写相关逻辑。

4.6. 应用调试与常见 FAQ

4.6.1. 调试技巧

- **返回值检查**: 所有 ACL 接口均返回 `ret` 状态码，0 表示成功。非 0 需查阅《错误码参考》。
- **日志获取**: 设置环境变量 `export ASCEND_GLOBAL_LOG_LEVEL=1` (Info 级别) 查看详细日志，日志默认位置在 `~/ascend/log/`。

4.6.2. FAQ

- **Q: 为什么 `acl.mdl.execute` 报错 “Memory Check Failed”?**
 - A: 检查 `acl.mdl.get_input_size_by_index` 获取的大小是否与你 `acl.rt.malloc` 的大小严格一致。
- **Q: DVPP 解码后的图片看起来是花的?**
 - A: DVPP 输出通常有宽/高对齐要求（如 128x16 对齐）。读取数据时需要根据 `stride` 跳过 `Padding` 数据，而不能简单按 `width * height` 读取。

4.7. 使用约束

1. **Context 线程安全**: 一个 Context 可以在多个线程中使用, 但需用户保证并发安全。推荐一线程一 Context。
2. **Stream 约束**: Stream 上的任务按顺序执行, 但异步接口下发后需显式 synchronize
→ 才能确保数据就绪。
3. **内存对齐**: DVPP 对内存地址和图片尺寸有严格对齐要求。

4.8. 应用样例参考目录

昇腾社区提供了丰富的 Sample 仓(Gitee), 推荐初学者阅读: * sampleResnetQuickStart

→ : 最基础的图片分类。* sampleY0L0V7: 包含后处理逻辑的目标检测。* sampleJpegDecode

→ : 专门展示 DVPP 用法。

5. 算子开发基础

昇腾 310B 作为一款边缘推理芯片，其算子开发与优化是挖掘硬件极致性能的关键。尽管 CANN（Compute Architecture for Neural Networks）已提供丰富的内置算子库，但在面对自定义模型结构、特殊后处理逻辑或对极致性能有着严苛要求时，自定义算子开发（Custom Operator Development）与系统级优化仍不可或缺。本章将系统性地介绍基于昇腾 310B 的算子开发流程、核心理论以及优化策略。

5.1. 算子开发概述：昇腾 310B 硬件架构与开发路径

本节作为算子开发实战的开篇，旨在帮助读者构建完整的知识框架。我们将首先解析昇腾 310B 处理器基于达芬奇（DaVinci）架构的核心特征，深入其 AI Core 的计算单元与存储层次；随后概述 CANN 异构计算软件栈；最后横向对比 TBE 与 Ascend C 这两条主流的算子开发路径，并探讨自定义算子开发的必要性。

5.1.1. 昇腾 310B 处理器与达芬奇架构

昇腾 310B 是专门面向边缘计算与推理场景打造的高能效 AI 处理器。该芯片采用 12nm FFC 工艺制程，在仅 5 至 8W 的典型功耗下，提高两个版本——20TOPS@INT8 或者 8TOPS@INT8 的澎湃 AI 算力，能效比达到惊人的 25-40 TOPS/W。凭借这一优势，它在工业质检、智能电网、智慧交通等对实时性要求极高的场景中展现出了卓越性能。

达芬奇（DaVinci）架构是昇腾系列 AI 处理器的技术基石，其最大的亮点在于极强的可扩展性（Scalable）。针对不同应用场景的算力需求，达芬奇架构衍生出了 Tiny、Mini、Lite、Standard、Max 等多款版本，全面覆盖从低功耗可穿戴设备到高性能云端数据中心的全场景。其中，昇腾 310B 采用了针对边缘端与移动端专门优化的 DaVinci-mini 架构。

达芬奇架构的核心设计理念可概括为：“将计算任务分层处理，以最契合的计算单元执行最适合的任务”。该架构将计算任务精细划分为标量（Scalar）、向量（1D）、矩阵（2D）与立方体（3D）四种类型，并在物理硬件层面相应地例化了三大核心计算单元——Cube Unit、Vector Unit 与 Scalar Unit。

计算单元的精细分工

Cube Unit（立方体计算单元）是达芬奇架构的标志性设计，更是提供高密度算力的引擎。它专为深度神经网络量身定制，主要承担矩阵乘法、卷积运算及全连接层等计算密集型（Compute-bound）任务。依靠极高的数据复用率，Cube Unit 有效突破了内存带宽的限制。在昇腾 310B 中，其运行频率可达 1.224GHz，是释放全周期算力的关键所在。

Vector Unit（向量计算单元）专门处理向量级运算，工作机制类似于 SIMD（单指令多数据流）。其功能涵盖归一化、激活函数、池化、数据格式转换等，广泛服务于计算机视觉（如 RPN 网络）中常用的基础算子。Vector Unit 具备极高的执行灵活性，能够无缝兼容并处理多种数据类型与运算模式。

Scalar Unit（标量计算单元）类似于经典的 RISC 微处理器，主要负责控制流管理与基础标量运算。它承担着循环控制、内存地址计算、分支跳转等逻辑任务，是调度统筹整个 AI Core 的“指挥中枢”。

这三大计算单元紧密协同，构建出高度并行的计算流水线。昇腾技术团队在多项典型任务中的实测数据表明，Cube Unit 占用的执行周期（Cycles）显著大于 Vector Unit，这意味着 Cube Unit 的运算潜能得到了充分释放，未受限于 Vector Unit 的调度阻塞，二者实现了优异的负载均衡。

存储层次与数据搬运机制

昇腾 310B 的存储系统采用了精巧的多级层次结构体系。深入理解这一体系，是突破算子性能瓶颈、实现极致优化的先决条件。

全局内存（Global Memory）位于存储架构的最外层，具备最大的容量空间，通常指代片外的板载 LPDDR4X 内存。昇腾 310B 普遍配置 8-16GB 的 LPDDR4X，带宽范围在 51.2GB/s 至 408GB/s。尽管容量充裕，但其访存延迟相对较高。因此，在进行算子开发时，应尽可能减少对全局内存的直接读取次数。

局部内存（Local Memory）集成于 AI Core 内部，属于极低延迟的高速存储区，主要包括 L1 缓存（L1 Buffer）与统一缓冲区（Unified Buffer，简称 UB）。L1 缓存专用于暂存高频复用的数据，从而大幅削减跨总线的读写开销。UB 则是算子执行时的核心工作台，所有参与计算的数据均需先从全局内存搬移至此，方能被计算单元读取。

存储转换引擎（Memory Transfer Engine, MTE）是专为数据搬运与内存重排设计的加速单元。MTE 细分为 MTE1、MTE2、MTE3 等模块，负责高效管理 AI Core 内外不同层级缓冲区之间的数据流动，并能够在搬运过程中硬件加速般地同步完成数据填充（Padding）、转置（Transpose）、Img2Col 等格式化操作。

总线接口单元（Bus Interface Unit, BIU）扮演着 AI Core 与外部存储总线通信

的“门户”角色，其主要职责是将 AI Core 发出的各类读写请求精确转化为标准的总线协议交互。

在一个典型的计算流水线中：数据首先通过 BIU 由全局内存接入，随后经 MTE 的高效搬运与格式重组，稳妥驻留于 L1 缓存或 UB 中。紧接着，Scalar Unit 发出调度指令，指挥 Cube Unit 或 Vector Unit 对 UB 中的数据进行高速运算。处理结束后，输出结果再次交由 MTE 接管，安全、高效地写回至全局内存，由此形成一个无缝闭环。#### 昇腾 AI 异构计算架构（CANN）

正如上一章在探讨 PyACL 编程基础时所述，CANN（Compute Architecture for Neural Networks）是昇腾全面面向 AI 场景定制的异构计算架构。它在整个计算系统中发挥着承上启下的核心枢纽作用：向上广泛适配 MindSpore、PyTorch、TensorFlow 等主流 AI 框架，向下直接调度并深度释放昇腾 AI 处理器的澎湃算力，是提升硬件计算效率的关键“软件底座”。

为了兼容并蓄不同维度的开发需求，CANN 构建了多层次的编程接口与组件体系：
- **AscendCL**：统一的应用开发原生接口，旨在屏蔽底层硬件差异，帮助开发者灵活构建从端到云的 AI 应用。
- **Ascend C**：基于 C++ 的高性能算子开发语言，允许开发者精细控制底层硬件指令与多级缓存，专为追求极致性能的算子定制而生。
- **TBE**：基于 Python 的开发框架，侧重于算子逻辑的快速表达与系统化自动调度优化。
- **图引擎（Graph Engine）**：核心的计算图编译与执行引擎，负责全局网络图的解析、内存复用规划及算子融合优化。

针对算子开发层面，CANN 构建了一站式的全流程工具链支持。涵盖了专属算子编译器、功能验证仿真工具，以及深度的性能调优分析器（Profiler），全面辅助开发者打通从代码原型编写、逻辑调试到逼近硬件理论极限的全闭环开发流程。

5.1.2. 算子开发的两条主流路径

针对不同开发需求和性能目标，昇腾 CANN 提供了两条主要的算子开发路径：**TBE（快速原型）**和**Ascend C（深度优化）**。

TBE：声明式开发，效率优先

TBE（Tensor Boost Engine）是一种基于 Python 的算子开发框架，其核心特点在于让开发者专注于描述计算逻辑，由系统自动完成底层的复杂优化。在开发方式上，TBE 提供了 DSL（Domain-Specific Language）编程模式。开发者无需深入了解昇腾底层硬件架构，只需通过几行简洁的 Python 代码即可描述算子的数学表达式。例如，使用 `dsl.vadd` $\rightarrow$ `(input_x, input_y)` 即可轻松表达向量加法，随后通过调用 `dsl.auto_schedule()`，

TBE 会自动接管并完成模式识别、子图切分、调度模板选择以及底层指令映射等一系列流程。

这种声明式开发范式赋予了 TBE 诸多的优势。首先是开发门槛极低且代码异常简洁，一个完整的加法算子核心逻辑往往只需 10 行左右即可实现。同时，得益于昇腾官方调度模板的自动优化加持，生成的算子性能稳定可靠，这使其非常适合用于算法的原型验证以及非性能瓶颈算子的快速实现。

然而，TBE 的自动化机制也带来了一定的局限。当面对具有极端定制化计算需求或特殊数据流模式的算子时，高度封装的自动调度可能难以将其优化至硬件的理论极致性能。此外，在某些特定的高级算子类型上，当前的 DSL 语法或许尚未提供完全覆盖的底层支持能力。

Ascend C：命令式开发，性能优先

Ascend C 是一种基于 C++ 的高性能算子开发语言，其核心优势在于赋予了开发者对数据流、指令流以及多核并行执行的精细控制权。在编程模型上，Ascend C 采用了 SPMD（单程序多数据）架构，这意味着所有的 AI Core 将执行同一套代码逻辑，但会根据各自的任务 ID 去处理被分配的不同数据区间。

在具体的开发过程中，开发者需要显式地设计计算流水线并深度管理多级存储之间的数据搬运。一个典型的 Ascend C 算子实现紧密围绕着三个连续的核心任务展开：首先是“CopyIn”阶段，负责将输入数据从全局内存高效搬运至局部内存；随后进入“Compute”阶段，调用硬件资源执行具体的数学运算；最后是“CopyOut”阶段，将计算所得的结果从局部内存搬运回全局内存，从而完成整个数据流转的闭环。

这种命令式的开发范式实现了灵活性与极致性能的统一。得益于对底层内存读写和流水线编排的精准把控，Ascend C 能够帮助开发者逼近硬件的理论计算极限，更被广泛应用于搞定大模型场景下如 FlashAttention 等复杂的性能瓶颈算子。此外，其原生支持 CPU 模拟调试与中间变量打印，极大地优化了开发体验。然而，获取这种极致性能也意味着较高的开发门槛，开发者必须对昇腾底层硬件架构有透彻的理解。同时，Ascend C 的代码工程量相对较大，即便是实现一个极简的算子，往往也需要编写数百行代码来完成对底层资源的系统化调度。

两条路径的协同关系

两条开发路径并非互斥，而是可以根据需求协同使用：

- 首选 TBE DSL，快速实现性能达标的标准算子
- 慎用 Ascend C，在追求极致性能或实现特殊计算模式时，系统化应用优化策略
- 善用工具链，坚持数据驱动的性能调优闭环

一个典型的开发流程是：先用 TBE 快速实现算子原型，验证功能正确性；如果性能不达预期，再用 Ascend C 重写核心计算逻辑，通过精细优化达到极致性能。

CANN 算子库本身包含了丰富的高性能算子，覆盖了大多数常见场景。但在以下情况下，开发者需要考虑自定义算子：

训练场景下的算子缺失：将第三方框架（如 TensorFlow、PyTorch）的训练脚本迁移到昇腾 AI 处理器时，遇到框架支持但 CANN 算子库暂不支持的算子。

推理场景下的模型转换：使用 ATC 工具将第三方框架模型转换为昇腾离线模型时，遇到不支持的算子。

网络性能调优：发现某算子性能较低，成为网络性能瓶颈，需要重新开发一个高性能算子替换原有算子。例如，一个 2048x2048 的矩阵乘法算子，经过系统化优化后，性能可从 512ms 提升至 92ms。

应用后处理加速：应用程序中的某些逻辑涉及数学运算（如查找最大值、数据类型转换），可以封装为自定义算子在 AI 处理器上执行，利用 NPU 提升性能。例如，分类应用中查找概率最大的前 5 个标识，可以开发 ArgMax 算子实现后处理加速。

5.1.3. 小结

通过本节的学习，读者应该对昇腾 310B 的硬件架构、CANN 软件栈以及算子开发的两种路径有了整体认识。从下一节开始，我们将从最简单的 TBE 算子入手，带领读者亲手实现第一个自定义算子，在实践中加深对概念的理解。

5.2. 初体验：使用 TBE DSL 快速实现第一个算子（向量加法）

通过一个最简单的向量加法算子，让读者快速体验 TBE 开发的完整流程。在 VSCode 中编写 Python 脚本，使用 TBE DSL 描述算子逻辑，通过命令行工具编译生成算子文件，并编写简单的测试代码在 NPU 上运行验证。最后总结 TBE 的优缺点及适用场景，激发读者进一步学习的兴趣。

5.2.1. 算子分析：明确我们要做什么

在动手编码之前，先明确我们要开发的算子规格。按照昇腾算子开发的规范，我们需要先进行算子分析。

算子功能：实现两个向量的加法，数学表达式为： $z = x + y$

输入输出规格: - 输入: 两个张量 (Tensor) x 和 y , 形状相同, 数据类型相同 - 输出: 一个张量 z , 形状和数据类型与输入相同 - 数据类型支持: float16、float32、int32 - Shape 支持: 所有形状 (本例使用简单的 1D 向量)

开发方式选择: 使用 TBE DSL 方式, 主要调用两个接口: - `tbe.dsl.broadcast`: 处理广播场景 (本示例输入 shape 相同, 但保留广播能力) - `tbe.dsl.vadd`: 执行向量加法

算子命名: 算子类型 (OpType) 采用大驼峰命名 "Add"; 实现文件名称和函数名称采用小写 "add".

TBE DSL 算子的开发流程可以分为四个主要步骤:

阶段	核心任务	关键函数/概念
算子定义	明确输入输出, 设计接口	<code>te.placeholder</code>
计算实现	描述算子的数学逻辑	<code>te.compute</code> , <code>te.lang.cce.vadd</code>
调度编译	将计算逻辑映射到硬件	<code>auto_schedule</code> , <code>cce_build_code</code>
验证测试	在 NPU 上运行并核验结果	NumPy 对比, AscendCL 调用

5.2.2. 完整代码实现

创建工程目录

首先在 VSCode 中连接到昇腾 310B 开发板 (或直接在板子上操作), 创建以下目录结构:

```
1 # 创建工程目录并进入
2 mkdir -p add_tbe
3 cd add_tbe
4
5 # 创建必要的源文件
6 touch add.py run.py
```

预期的目录结构如下:

```
1 add_tbe/
2 |— add.py      # TBE算子实现文件
3 |— run.py     # 测试验证脚本
4 |— kernel_meta/ # 编译输出目录 (自动生成)
```

编写 TBE 算子代码 (add.py)

下面是完整的向量加法算子实现代码, 我们将逐段解释。

```

1 import tbe
2 from tbe import tvn
3 from tbe import dsl
4 from tbe.common.utils import para_check
5 from tbe.common.utils import shape_util
6
7 from funtools import reduce
8
9 SHAPE_SIZE_LIMIT = 2147483648
10
11 # 实现Add算子的计算逻辑
12
13 @tbe.common.register.register_op_compute("add", op_mode="static")
14 def add_compute(input_x, input_y, output_z, kernel_name="add"):
15     shape_x = shape_util.shape_to_list(input_x.shape) # 将shape转换为list
16     shape_y = shape_util.shape_to_list(input_y.shape) # 将shape转换为list
17     shape_x, shape_y, shape_max = shape_util.broadcast_shapes(shape_x,
18         ↳ shape_y, param_name_input1="input_x", param_name_input2="input_y
19         ↳ ") # shape_max取shape_x与shape_y的每个维度的大值
20     shape_size = reduce(lambda x, y: x * y, shape_max[:])
21     if shape_size > SHAPE_SIZE_LIMIT:
22         raise RuntimeError("the shape is too large to calculate")
23
24     input_x = dsl.broadcast(input_x, shape_max) # 将input_x的shape
25         ↳ 广播为shape_max
26     input_y = dsl.broadcast(input_y, shape_max) # 将input_y的shape
27         ↳ 广播为shape_max
28     res = dsl.vadd(input_x, input_y) # 执行input_x + input_y
29
30     return res # 返回计算结果的tensor
31
32 # 算子定义函数
33 def add(input_x, input_y, output_z, kernel_name="add"):
34     # 获取算子输入tensor的shape与dtype
35     shape_x = input_x.get("shape")
36     shape_y = input_y.get("shape")
37     check_tuple = ("float16", "float32", "int32")
38     input_data_type = input_x.get("dtype").lower()
39     if input_data_type not in check_tuple:
40         raise RuntimeError("only support %s while dtype is %s" %
41             ↳ ("", ".join(check_tuple), input_data_type))
42
43     # shape_max取shape_x与shape_y的每个维度的最大值
44     shape_x, shape_y, shape_max = shape_util.broadcast_shapes(shape_x,
45         ↳ shape_y, param_name_input1="input_x", param_name_input2="input_y
46         ↳ ")
47
48     if shape_x[-1] == 1 and shape_y[-1] == 1 and shape_max[-1] == 1:
49         # 如果shape的长度等于1, 就直接赋值, 如果shape的长度不等于1, 做切
50         ↳ 片, 将最后一个维度舍弃 (按照内存排布, 最后一个维度为1与没
51         ↳ 有最后一个维度的数据排布相同, 例如2*3=2*3*1, 将最后一个为1
52         ↳ 的维度舍弃, 可提升后续的调度效率)。
53         shape_x = shape_x if len(shape_x) == 1 else shape_x[:-1]
54         shape_y = shape_y if len(shape_y) == 1 else shape_y[:-1]
55         shape_max = shape_max if len(shape_max) == 1 else shape_max[:-1]

```

```

45 # 使用TVM的placeholder接口对第一个输入tensor进行占位, 返回一个tensor
46     ↳ 对象
47 data_x = tvm.placeholder(shape_x, name="data_1", dtype=
48     ↳ input_data_type)
49 # 使用TVM的placeholder接口对第二个输入tensor进行占位, 返回一个tensor
50     ↳ 对象
51 data_y = tvm.placeholder(shape_y, name="data_2", dtype=
52     ↳ input_data_type)
53
54 # 调用compute实现函数
55 res = add_compute(data_x, data_y, output_z, kernel_name)
56 # 自动调度
57 with tvm.target.cce():
58     schedule = dsl.auto_schedule(res)
59 # 编译配置
60 config = {"name": kernel_name,
61          "tensor_list": (data_x, data_y, res)}
62 dsl.build(schedule, config)
63
64 # 算子调用
65 if __name__ == '__main__':
66     input_output_dict = {"shape": (5, 6, 7), "format": "ND", "ori_shape":
67         ↳ (5, 6, 7), "ori_format": "ND", "dtype": "float16"}
68     add(input_output_dict, input_output_dict, input_output_dict,
69         ↳ kernel_name="add")

```

5.2.3. 代码逐段详解

导入模块

```

1 import tbe
2 from tbe import tvm
3 from tbe import dsl
4 from tbe.common.utils import para_check
5 from tbe.common.utils import shape_util
6 from funtools import reduce

```

- tbe: TBE 框架主模块
- tbe.dsl: 包含 DSL 计算接口 (如 `vadd`)、调度接口和编译接口
- tbe.tvm: TBE 基于 TVM 框架扩展, 可以使用 TVM 接口
- shape_util: 提供 shape 处理工具, 如广播 shape 计算
- para_check: 参数校验工具

算子计算逻辑 (`add_compute`) 这是算子开发的核心, 描述”如何计算”。关键点:

- `@register_op_compute` 装饰器将函数注册为算子的计算逻辑 - **Shape 大小校验**: 计算广播后张量的总元素个数, 当超出 `SHAPE_SIZE_LIMIT` 阈值时抛出异常 - **数据广播与计**

算：先通过`dsl.broadcast`将输入形状广播对齐，再调用`dsl.vadd`执行向量加法（自动识别为 `element-wise` 模式）

算子主函数（add）

这是算子的入口函数，负责调度和编译：

参数校验：校验数据类型是否在支持范围内（`float16/float32/int32`）。

Shape 切片优化：如果 `shape` 的末尾维度长度为 1，则将其直接舍弃。由于内存排布上末尾为 1 并不影响实际排布（例如 231 等同于 2×3 ），舍弃后可有效提升后续的调度效率。

TVM 占位符：`tvm.placeholder`创建输入张量的占位符，描述张量的形状和数据类型，但不分配实际数据。

自动调度：`dsl.auto_schedule`自动完成 AST 标注、模式识别、子图切分、调度模板选择，并将指令映射到昇腾硬件。

编译构建：`dsl.build`将调度后的计算描述编译为昇腾设备可执行的二进制文件。

5.2.4. 编译算子

在终端执行以下命令编译算子：

```
1 python3 add.py
```

编译成功后，会在当前目录生成`kernel_meta/`文件夹，包含两个文件：- `add.o`：算子的二进制目标文件 - `add.json`：算子的元信息描述文件

查看生成的文件：

```
1 ls kernel_meta/
2 # 输出示例：add.o add.json
```

5.2.5. 算子验证：在 NPU 上运行测试

编译成功后，我们需要验证算子的正确性。昇腾提供了两种验证方式：- **单算子模型执行：**将算子编译成单算子离线模型（`.om` 文件），通过 `AscendCL` 加载执行 - **单算子 API 执行：**直接通过 `AscendCL` 的 API 调用算子

本节采用第一种方式，因为它更接近实际部署场景。

编写验证代码 (run.py)

下面是使用 AscendCL 加载并执行单算子的验证代码：

```

1 # run.py
2 from tbe import tvn
3 from tbe import dsl
4 from tbe.common.utils import para_check
5 from tbe.common.utils import shape_util
6 # 引入testing模块相关接口
7 from tbe.common.testing.testing import *
8 import numpy as np
9
10 @para_check.check_input_type(dict, dict, dict, str)
11 def addtest(input_a, input_b, output_d, kernel_name="addtest"):
12     # 进入DSL调试模式，并选择CPU作为运行平台
13     with debug():
14         # 获取算子运行的上下文
15         ctx = get_ctx()
16
17         # 获取输入数据的shape与dtype
18         shape_a = shape_util.scalar2tensor_one(input_a.get("shape"))
19         shape_b = shape_util.scalar2tensor_one(input_b.get("shape"))
20         data_type = input_a.get("dtype").lower()
21
22         # 使用numpy定义输入golden数据大小
23         a = tvn.nd.array(np.random.uniform(size=shape_a).astype(data_type
24             ↪ ), ctx)
25         b = tvn.nd.array(np.random.uniform(size=shape_b).astype(data_type
26             ↪ ), ctx)
27
28         # 使用numpy将输出d初始化为全0
29         d = tvn.nd.array(np.zeros(shape_a, dtype=data_type), ctx)
30
31         # 调用TVM的placeholder接口对输入tensor进行占位，并返回一个tensor
32         ↪ 对象
33         data_a = tvn.placeholder(shape_a, name="data_1", dtype=data_type)
34         data_b = tvn.placeholder(shape_b, name="data_2", dtype=data_type)
35         # 调用DSL计算接口实现data_a + data_b
36         data_c = dsl.vadd(data_a, data_b)
37
38         # 中间Tensor数据验证
39         sample = open('samplefile.txt', 'w')
40         # 将中间tensor data_c存入文件samplefile.txt
41         print_tensor(data_c, ofile=sample)
42         # 检查中间tensor data_c的值是否正确
43         assert_allclose(data_c, desired=a.asnumpy() + b.asnumpy(), tol=[1
44             ↪ e-7, 1e-7])
45         print("The value of data_c is the same as the expected value.")
46
47     # 继续自定义DSL的逻辑撰写，调用DSL接口实现：data_d = data_c + data_b
48     data_d = dsl.vadd(data_c, data_b)
49     # 调用TVM的create_schedule接口，为算子创建调度实例对象，入参为输
50     ↪ 出tensor的OP列表。
51     s = tvn.create_schedule(data_d.op)

```

```

46 # 编译生成算子,data_a,data_b,data_d是占位的输入输出列表, AddTest
47   ↳ 是我们自定义算子的名称
48 build(s, [data_a, data_b, data_d], name="AddTest")
49
50 # 执行算子,将a,b,d按顺序代入编译出来的DSL算子AddTest
51 run(a, b, d) # AddTest(a, b, d)
52
53 # 将输出数据d的值打印出来,并预期结果进行比较,看是否相符
54 print("d:", d)
55 tvm.testing.assert_allclose(d.asnumpy(), a.asnumpy() + b.asnumpy
56   ↳ () + b.asnumpy())
57 print("The actual output is the same as the expected output.")
58
59 # 编写入口函数,调用addtest函数
60 if __name__ == "__main__":
61     input_output_dict = {"shape": (2, 3, 4), "format": "ND", "ori_shape":
62   ↳ (2, 3, 4), "ori_format": "ND", "dtype": "float32"}
63     addtest(input_output_dict, input_output_dict, input_output_dict,
64   ↳ kernel_name="addtest")

```

运行验证

验证代码:

```

1 # 直接运行
2 python3 run.py

```

成功输出示例:

```

1 ===== debug enter =====
2 The value of data_c is the same as the expected value.
3 /usr/local/Ascend/ascend-toolkit/8.3.RC1/python/site-packages/tbe/tvm/
4   ↳ driver/build_module.py:280: UserWarning: target_host parameter is
5   ↳ going to be deprecated. Please pass in tvm.target.Target(target,
6   ↳ host=target_host) instead.
7 warnings.warn(
8 Tensor add_0 is saved to file samplefile.txt.
9 d: [[[1.2949324  0.45642793 1.6225115  1.3862249 ]
10      [1.2386332  1.5728098  0.9118807  2.0370739 ]
11      [1.2001383  1.4828949  0.17750879 2.507696  ]]
12      [[1.2292826  0.23282059 1.1223636  1.6030114 ]
13      [1.6799536  1.7751908  2.5275748  2.190519  ]
14      [2.6027465  1.8843926  2.1372957  1.8606762 ]]]
15 The actual output is the same as the expected output.
16 ===== debug exit =====

```

至此,我们完成了第一个 TBE DSL 算子的完整开发流程:从算子分析、代码实现、编译到 NPU 上验证运行。

5.2.6. TBE DSL 开发的优势与局限

通过本次实战体验，我们可以总结 TBE DSL 开发的特点：

优势

优势	说明
开发门槛低	使用 Python 开发，无需深入了解昇腾硬件架构，只需描述数学逻辑
代码简洁	一个完整的加法算子核心代码仅需 10 行左右，大幅减少开发工作量
自动优化	auto_schedule自动完成指令映射、数据切分、流水线优化，由昇腾官方模板调度，性能稳定可靠
快速验证	适合算法原型验证和非性能瓶颈算子的快速实现

局限

局限	说明
自动调度的”黑盒”特性	Auto Schedule 对特殊结构的算子可能不够精细，当算子有强性能诉求时，可能无法达到极致性能
灵活性受限	对于具有极端定制化需求或特殊数据流模式的算子，DSL 的自动调度可能无法完全满足
高级算子覆盖	某些高级算子类型 DSL 可能尚未完全覆盖

5.2.7. 小结与展望

本节我们通过一个向量加法算子，完整实践了 TBE DSL 开发的四步流程：算子分析、代码实现、编译、验证。你可能会感受到 TBE DSL 带来的便利——用熟悉的 Python 语言，专注于数学逻辑本身，而将底层复杂的硬件适配交给自动调度完成。

这正体现了 TBE 的设计哲学：将开发者从复杂的硬件指令和内存管理中解放出来，专注于算法本身。

然而，正如我们在“局限”中提到的，自动调度并非万能。当追求极致性能、或实现特殊计算模式时，我们需要更精细的控制能力。这正是下一节将要学习的 **Ascend C 开发范式**的用武之地。

在进入下一节之前，建议你亲自动手完成本节示例，并在自己的昇腾 310B 开发板上跑通验证。这是理解后续更深内容的基础。

5.3. 理解算子开发的核心概念

在动手实践之后，系统讲解算子开发中必须掌握的基础知识。包括算子原型定义（输入输出、属性、数据类型）、算子信息库（.ini 文件）的作用与配置方法，以及 Tiling（分片计算）的基本思想。这些概念将为后续学习 Ascend C 打下坚实的理论基础。

5.4. 深入底层：Ascend C 算子开发入门

正式进入 Ascend C 开发范式。首先介绍 Ascend C 的编程模型（Host-Device 协同、Kernel 与 Tiling）。然后以向量加法为例，详细讲解 Ascend C 的工程结构（C++ 源文件、CMakeLists.txt），并剖析 CopyIn、Compute、CopyOut 三段式流水线的代码实现。最后通过命令行编译、运行，并与 TBE 版本进行对比，体会两种方式的差异。

5.5. 从简单到复杂：实现一个需要 Tiling 的算子（如矩阵加法）

当数据量超过 AI Core 的 Local Memory 容量时，必须引入 Tiling 技术。本节以一个矩阵加法算子为例，讲解 Tiling 策略的设计（分块大小计算、多核并行分配），并在 Ascend C 中完整实现带 Tiling 的算子。同时介绍 CPU 模拟调试和 printf 打印等调试技巧，帮助读者应对更复杂的场景。

5.6. 算子编译、部署与单元测试

聚焦算子开发的工程化环节。讲解如何使用 ccec 编译器将 Ascend C 代码编译成适配昇腾 310B 的动态链接库（.so 文件），并手动部署到 CANN 算子库中。同时介绍使用

AscendCL API 编写单元测试的方法，验证算子的正确性，确保算子可被上层框架调用。

5.7. 算子性能优化初探

从入门角度介绍性能优化的基本思路。涵盖内存访问优化（充分利用 Local Memory）、计算与数据传输的流水线重叠（Double Buffer）、向量化编程等技巧。并通过一个简单的案例分析，展示如何使用 CANN Profiling 工具采集性能数据，定位瓶颈并进行初步优化。

6. 系统工程与高可用部署

在完成了模型转换与基础推理应用的开发后，“跑通”只是第一步。在边缘计算场景（如 Ascend 310B）中，资源受到严格限制，如何榨干硬件的每一滴性能，使应用满足实时的时延（Latency）或高并发的吞吐（Throughput）要求，是工程落地的核心挑战。本章将从性能分析的方法论出发，结合具体的视频分析案例，系统讲解从瓶颈定位到极致优化的全过程。

6.1. 性能优化的全景视角

6.1.1. 两个核心指标

在谈优化之前，必须明确目标。不同的业务场景对性能的定义不同：
* **时延 (Latency)**: 处理单帧数据所需的时间。对自动驾驶、实时交互类应用至关重要。
* **优化目标**: 降低处理流在每一个环节的耗时。
* **吞吐量 (Throughput / FPS)**: 单位时间内处理的数据量。对视频监控汇聚、离线分析类应用更关键。
* **优化目标**: 提高并发度，掩盖传输与处理的空隙，满载硬件资源。

6.1.2. 木桶效应与 Amdahl 定律

AI 应用通常是一个异构计算流水线：Camera -> Host CPU (预处理) -> PCIe (H2D) -> Device NPU (推理) -> PCIe (D2H) -> Host CPU (后处理)

- **木桶效应**: 整个系统的 FPS 取决于最慢的环节。如果 NPU 推理只需 5ms(200FPS), 但 CPU 预处理需要 40ms (25FPS), 那么系统上限主要受限于 CPU。
- **优化策略**: 先找瓶颈, 再做优化。盲目优化非瓶颈模块（比如把 5ms 的推理优化到 4ms）对整体性能提升微乎其微。

6.2. 照妖镜：Profiling 性能分析

昇腾提供了强大的性能分析工具 MSPROF (Profiling)，它还能像“X 光”一样透视程序运行的内部细节。

6.2.1. 采集 Profiling 数据

通常我们使用命令行工具 msprof 进行采集。

基础命令示例：

```
1 # 采集应用运行的性能数据，输出到 ./output 目录
2 msprof --output=./output --application="./main_app"
```

高级采集（包含 AI Core 细节）：

```
1 msprof --output=./output --application="./main_app" --task-time=on --aic-
  ↳ metrics=PipeUtilization
```

6.2.2. 关键视图解读

使用 msprof 分析生成的 Timeline 视图（通过 VS Code 插件或 Ascend Insight 查看）是定位问题的关键。

1. **ACL API 耗时**：查看 aclmdlExecute 等接口的调用时长。如果调用间隔极大，说明 Host 端调度或预处理太慢。
2. **Stream Timeline**：
 - **计算流**：查看 NPU 上算子的执行密度。如果有大片空白（Bubble），说明 NPU 在“等数据”或“等指令”。
 - **H2D/D2H 流**：查看数据拷贝的耗时。如果拷贝时间 > 推理时间，说明传输是瓶颈。
3. **AI Core Metrics**：
 - **Cube/Vector 利用率**：如果利用率低，说明模型算子需要优化（参考第 5 讲）。
 - **Memory Bandwidth**：查看 DDR 读写带宽，判断是否是一张“存储受限”的图。

6.3. 常见瓶颈与对策 Checklist

现象 (Symptoms)	潜在原因 (Root Cause)	优化对策 (Solution)
NPU 利用率低, 大量空闲	Host 端预处理慢 (CPU 瓶颈)	1. 使用 C++ 替代 Python2. 多线程预处理 3. 使用 AIPP / DVPP 硬件加速
ACL Execute 耗时长	算子执行慢或调度阻塞	1. 模型量化 (INT8)2. 算子融合 3. 异步推理 (aclmdlExecuteAsync)
H2D 拷贝耗时长	此时输入图像过大	1. 使用 Zero-Copy 内存分配 2. 在 Device 侧做 Resize/Crop (AIPP)
FPS 随 Batch 增加不明显	内存带宽瓶颈或单流阻塞	1. 多 Stream 并发推理 2. 优化数据布局 (Layout)

6.4. 核心优化技术详解

6.4.1. AIPP：将预处理下沉到硬件

AIPP (Artificial Intelligence Pre-Processing) 是昇腾芯片特有的硬件加速模块, 它直接连接在 AI Core 之前, 可以在数据进入 AI Core 计算前完成以下操作: * **色域转换 (CSC)**: YUV420 -> RGB/BGR (视频处理必备)。* **归一化 (Normalize)**: 减均值, 除方差 (Mean/Std)。* **抠图 (Crop) 与缩放 (Resize)**。

优势: * **释放 CPU**: CPU 不再需要做繁重的 cv2.cvtColor 或 img / 255.0。* **减少传输量**: 可以传输 YUV 图片 (体积小) 到 Device, 由 AIPP 转 RGB (体积大), 节省 PCIe 带宽。

启用方法: 配置 AIPP 配置文件, 在 atc 模型转换时通过 --insert_op_conf=aipp ↪ .cfg 插入。

6.4.2. 零拷贝 (Zero-Copy) 内存管理

传统流程: Host malloc -> Read Image -> Host 2 Device Copy -> Device Infer。
零拷贝流程: 直接申请 **Device 侧可访问的 Host 内存** (或是 Host 侧可映射的 Device 内存)。

```
1 // 申请“页锁定”内存, NPU 可以直接通过 DMA 访问, 无需 CPU 参与临时的内核态拷贝
2 aclrtMallocHost(&hostBuffer, size);
```



```

3 // 或者直接申请 Device 内存，部分场景下配合 DVPP 使用
4 aclrtMalloc(&devBuffer, size, ACL_MEM_MALLOC_HUGE_FIRST);

```

对于视频解码 (VDEC) + 推理 (Infer) 场景，让 VDEC 直接将结果解码到推理的 Input Header 内存中，可以消除中间的显存拷贝。

6.4.3. 多级流水线 (Multi-Stage Pipeline)

简单串行模式: Pre -> Infer -> Post, 总耗时 $T = t1 + t2 + t3$ 。流水线模式: 三个线程分别处理 Pre, Infer, Post, 通过队列传递数据。理论吞吐量取决于最慢的阶段: $FPS = 1 / \max(t1, t2, t3)$ 。

6.5. 实战演练：车辆检测系统的极致优化之路

我们以一个典型的“路面车辆检测”应用为例，场景为处理 1080P 视频流，模型为 YOLOv5s (Input 640x640)。

6.5.1. 阶段一：Baseline (Python + OpenCV)

- 实现: 使用 OpenCV (cv2.VideoCapture, cv2.resize) 在 CPU 上读取和缩放，调用 ACL 进行推理，CPU 进行 NMS。
- 性能: 25 FPS。
- 瓶颈分析: Profiling 显示 NPU 利用率仅 30%，大量时间消耗在 cv2.resize 和 H2D Copy 上。CPU 单核 100% 满载。

6.5.2. 阶段二：引入 AIPP 与 C++ (消除 CPU 计算瓶颈)

- 优化:
 1. 重构为 C++ 应用。
 2. 不再在 Host 端做 Resize 和 Normalization。
 3. 开启 AIPP，配置色域转换 (BGR->RGB) 和归一化。
 4. 输入改为直接传输原始 1080P 图像 (Resize 由 AIPP/DVPP 完成，或 AIPP Crop)。
- 性能: 55 FPS。
- 分析: CPU 负载降低，但推理变成串行阻塞。

6.5.3. 6.5.3 阶段三：多线程 Pipeline (掩盖时延)

- 优化：设计 Thread_Decode, Thread_Infer, Thread_Post 三组线程池。
 - Thread_Decode: 负责视频解码，推入 Queue A。
 - Thread_Infer: 从 Queue A 取图，aclmdlExecute，结果推入 Queue B。
 - Thread_Post: 从 Queue B 取结果，做 NMS。
- 性能：85 FPS。
- 分析：NPU 利用率提升至 80%，主要受限于单路流解码速度。

6.5.4. 6.5.4 阶段四：Batching 与异步推理 (极致吞吐)

- 优化：
 1. **Batching**: 虽然单张图处理快，但一次发送 4 张图 (BatchSize=4) 能分摊 PCIe 通信开销。
 2. **DVPP VCC**: 使用硬件解码器替代 CPU 解码。
 3. **Async**: 使用 aclmdlExecuteAsync，不等待推理完成即处理下一帧，通过 Callback 回调处理结果。
- 性能：120+ FPS。
- 结论：相比 Baseline 提升近 5 倍，且 CPU 占用率极低。

6.6. 章节要点与练习

6.6.1. 总结

性能优化是一个系统工程，而非单一的改代码。1. **AMP (Analyze, Map, Parallel)**: 分析瓶颈，映射到硬件单元，最大化并行度。2. **310B 黄金法则**: 少用 CPU 做像素处理，用好 AIPP/DVPP，跑满 NPU 流水线。

6.6.2. 练习任务

1. **基线跑分**: 运行你自己编写的 ResNet/YOLO 应用，记录单幅图片的预处理、推理、后处理平均耗时。
2. **Profiling 实战**: 使用 msprof 抓取一份 Timeline 数据，截图并圈出“推理间隙”最大的位置，分析原因。
3. **计算加速比**: 假设你的模型推理耗时 10ms, H2D 耗时 5ms, D2H 耗时 2ms。计算串行执行和理想流水线执行的 FPS 理论上限分别是多少？

7. 项目实战方法论与交付模板

7.1. 章节总览

本章建立从需求澄清 □ 指标体系 □ 评测集 □ 迭代节奏 □ 资产沉淀 □ 交付与回归的一套闭环方法论，让技术决策基于指标与风险敞口，而非经验臆测。核心理念：可量化、可比较、可复用、可追溯。

7.2. 需求澄清 Canvas

维度	要素	问题提示	示例
场景	输入源/运行环境	摄像头？批处理？	室内 1080p30 低光
目标	功能/业务价值	用户希望看到什么结果？	实时检测 + 分析
指标	Latency/FPS/精度	哪些分位数重要？	<80ms / □25FPS / mAP□0.6
约束	能耗/内存/带宽	上限是多少？	功耗 □15W 内存 □3GB
风险	数据/硬件/算法	失败模式有哪些？	低光/遮挡/抖动
合规	隐私/许可	是否需要脱敏？	仅上传事件元数据
成本	硬件/云	ROI 衡量？	10 台板卡预算

输出：requirement.yaml（版本化），后续所有评审基于此文档。

7.3. 指标分层与优先级

层级	类别	指标	说明	失败后果
SLO A	体验	p95 延迟	端到端	体验差/丢帧
SLO A	性能	FPS	稳态吞吐	处理拥堵
SLO B	质量	mAP/F1/Top1	任务正确性	无法满足业务
SLO B	稳定	Crash/小时	可靠性	运维成本高
SLO C	资源	内存峰值/功耗	成本约束	设备异常/降频
SLO C	带宽	上行 kbps	成本/合规	费用/拥塞

优先级：先保障 A（体验 + 功能可用），再稳定 B（质量/稳定），最后优化 C（资源效率）。

7.4. Baseline 策略与控制变量法

Baseline 目标：建立“最小改动可运行”标尺。原则：

- 1. 不提前做微优化；
- 2. 记录所有关键参数：模型版本、输入尺寸、预处理策略、硬件温度范围；
- 3. 一次仅改变单个变量（batch、精度、线程数）。基线存档：`baseline/<date>-<commit>/metrics.json`；对比脚本生成差异报告。

7.5. 评测集设计原则

原则	内容
代表性	涵盖主流场景/光照/角度
覆盖边界	极端尺寸、模糊、遮挡
可再现	文件命名规范 + 固定清单
可扩展	新增样本不破坏旧索引
标注一致	标注工具/规范/审校流程

目录示例：

```

1 dataset_eval/
2   images/
3     day/*.jpg
4     night/*.jpg
5     occlusion/*.jpg
6   annotations/
7     instances_train.json
8     instances_val.json
9   meta/
10    README.md
11    version.txt

```

提供 Hash 列表，防止样本被替换而影响回归可信度。

7.6. 迭代计划与看板

四阶段：

Sprint	目标	核心产出	风险控制
0	环境/基线	baseline metrics	依赖清单齐全
1	精度与功能稳固	精度报告	数据问题快速反馈
2	性能与稳定	性能对比表/监控上线	Watchdog 验证
3	工程交付包装	Release Notes/脚本	灰度计划制定

看板列：Backlog □ Doing □ Review □ Bench □ Done；性能/精度任务需进入 Bench 列执行对比脚本通过后才可 Done。

7.7. 资产沉淀文档体系

文档	内容	更新频率
README	快速启动	版本变化时
ARCHITECTURE	架构图/模块说明	结构调整
MODEL_CARD	模型来源/许可/精度/限制	模型更新
EVAL_REPORT	数据与评测方法/指标	每次发布
PERF_REPORT	基线/优化对比	优化后
CHANGELOG	可见版本差异	每次版本

文档	内容	更新频率
RISK_LOG	已知风险列表	动态

MODEL_CARD 需包含：数据来源、训练超参摘要、输入契约、已知局限、许可（如 Apache-2.0）、安全与偏见说明（若涉及识别敏感属性声明避免用途）。

7.8. 交付目录与不可变产物

```
1 release/
2   v1.0/
3     manifest.json      # 产物 hash / 版本矩阵
4     models/
5       detect.om
6       classify.om
7       signature.json
8     scripts/
9       run.ps1
10      run.sh
11      watchdog.sh
12     configs/
13       default.yaml
14     docs/
15       model_card_detect.md
16       model_card_classify.md
17       QUICKSTART.md
18     reports/
19       perf.json
20       accuracy.json
```

manifest.json 字段: {version, commit, build_time, model_hashes, dependencies
→ }。

7.9. 上线前综合 Checklist

类别	检查项	通过标准
功能	核心用例 100%	自动化用例通过
性能	p95 < 目标 +5%	连续 30min 稳定
精度	mAP/Top1 回归差 < 阈值	与基线对比
资源	内存峰值 < 75%	1h 稳态无泄漏

类别	检查项	通过标准
稳定	Crash=0, 重启 =0	守护日志清洁
安全	日志无敏感泄露	关键字段脱敏
配置	签名校验一致	Hash 匹配
回滚	验证上一版本可用	切换 < 30s

7.10. 验收、回归与漂移监测

交付后 7 天加密监控：记录时延、精度漂移（采样对比模型输出变化）。漂移检测：相同输入集合（Shadow Set）每天抽样跑一次 □ 统计 logits KL 散度/TopK 变化率，高于阈值（如 $KL > 0.05$ ）触发报警（潜在数据分布变化或模型文件损坏）。回归集版本化：eval_set_vX；若需替换样本 □ 新增版本，不覆盖旧数据。

7.11. 风险管理与决策日志

风险登记表：risk_log.md 每条包含：ID、描述、影响、概率、缓解、当前状态。决策日志（Decision Record, ADR）：记录架构/模型/精度策略选择及备选方案放弃理由，以便新成员快速建立上下文。

7.12. 章节小结

方法论的核心不是流程文档堆砌，而是“指标驱动 + 资产沉淀 + 可回滚”三支柱。通过契约化需求、标准化 Baseline、规范化评测与回归体系，使团队协作更高效、风险暴露更透明、交付结果更可信。

7.13. 实践任务

1. 输出 requirement.yaml（含指标与约束）。
2. 构建 30 张代表图像的 mini 评测集并附 Hash 列表。
3. 生成 baseline metrics.json 与后一次优化对比 diff 报告。
4. 制作一个 MODEL_CARD 模板并填写一个模型示例。
5. 编写上线 Checklist 并模拟一项未通过情形与处置方案。

8. 合实战案例集

8.1. 章节总览

本章通过九个真实应用场景串联前面章节的知识：模型选择、转换、部署、性能与稳定性验证、迭代优化。所有案例采用统一模板，支持快速复制与对比评估。强调“结构化指标 + 自动化脚本 + 可视化反馈”。

8.2. 案例统一模板（标准化规范）

区块	内容要点	产出文件
场景描述	背景/输入/目标	README.md#scene
指标目标	延迟/FPS/精度/资源	requirement.yaml
模型选择	候选对比 + 取舍	model_card*.md
数据准备	采集/标注/增强	data_prep.md
转换部署	导出 □ATC 参数	atc.sh / export.py
运行脚本	启动/参数/日志路径	run.sh / run.ps1
性能结果	metrics.json (基线/优化)	metrics/*.json
质量验证	精度/漂移检查	accuracy.json
风险改进	已知问题/迭代计划	roadmap.md

8.3. 案例目录结构规范

```
1 experiments/caseX/  
2   README.md  
3   requirement.yaml  
4   models/           # onnx / om / signatures  
5   scripts/  
6   export.py
```



```
7   atc.sh
8   run_infer.py
9   benchmark.py
10  data/           # 样本 (或下载指令)
11  metrics/
12     baseline.json
13     optimized.json
14  logs/
15  eval/
16     accuracy.json
17     drift.json
18  assets/         # 截图 / 示意图
```

8.4. 例概览与重点

序	名称	关键技术点	指标核心	风险要素
1	人脸打卡机	人脸检测 + 比对 + 活体	识别成功率/伪拒率	光照/遮挡
2	实时跟踪	检测 + 多目标关联	跟踪稳定度 (IDF1)	遮挡/抖动
3	智能电子琴	音频节拍识别 + 分类	识别延迟/准确率	噪声/延迟
4	掌纹识别	ROI 提取 + 特征匹配	误识率/拒识率	采集姿态
5	数据采集仪	传感融合 + 缓存上传	数据丢失率	网络波动
6	智能小车	目标检测 + 路径策略	决策延迟	传感器同步
7	智能相册	分类 + 聚类 + 去重	聚类纯度	相似干扰
8	手势识别	时序建模 (TSM)	手势准确率/FPS	动作模糊
9	聊天机器人	NLP 推理 + 缓存	响应时延/意图准确	语料漂移

下列示例详细展开前三个具代表性的模式。

8.5. 案例 1：人脸打卡机

8.5.1. 场景

摄像头实时输入，人脸检测 □ 关键点对齐 □ 特征提取 □ 特征库比对 □ 授权决策 □ 事件上报。

8.5.2. 指标

指标	目标	说明
平均识别时延	< 120ms	从帧采集到结果
最大 P95	< 150ms	抖动控制
误识率 (FAR)	< 0.001	安全性
拒识率 (FRR)	< 0.02	体验

8.5.3. 模型链路

1. 人脸检测 (RetinaFace);
2. 5 点关键点仿射对齐;
3. ArcFace 特征 512D;
4. 向量归一化 + 余弦相似度;
5. 阈值自适应 (基于滑动窗口均值校正)。

8.5.4. 性能优化

- 批量特征比对: 向量库转矩阵, 使用 SIMD/BLAS;
- 缓存: 最近识别通过用户特征缓存, 减少重复比对;
- 光照增强: 低光阈值触发 Gamma/直方图均衡。

8.5.5. metrics 示例

```
1 {  
2   "avg_latency_ms": 98.4,  
3   "p95_latency_ms": 121.3,  
4   "fps": 10.1,  
5   "face_detect_ms": 42.1,  
6   "feature_ms": 18.7,  
7   "match_ms": 5.2,  
8   "false_accept_rate": 0.0008,  
9   "false_reject_rate": 0.017  
10 }
```

8.6. 案例 2：实时跟踪（检测 + 关联）

8.6.1. 流程

帧采集 □ 目标检测 □ 外观特征提取 □ 卡尔曼预测 □ 匈牙利匹配 □ 轨迹输出。

8.6.2. 难点

遮挡/丢失：轨迹生命周期管理（状态：Tentative □ Confirmed □ Lost □ Removed）。

8.6.3. 优化

1. 检测降频：每 N 帧做一次全检测，中间帧仅跟踪预测；
2. 多线程：检测与跟踪解耦；
3. ReID 模型轻量化（裁剪通道）。

8.6.4. 评估指标

IDF1、MOTA、FP/FN、IDSW（身份切换）。

8.7. 案例 3：智能电子琴（音频）

8.7.1. 流程

音频采集 16kHz □ 窗口分帧 FFT □ 频谱/梅尔特征 □ 分类模型（音符/节奏） □ 校准节拍输出。

8.7.2. 优化点

FFT 批处理使用向量库；低延迟滑动窗口；模型输出置信度平滑（指数滑动平均）。

8.7.3. 指标

节拍延迟 < 80ms；识别准确率 > 95%。

8.8. 结果记录与差异报告

基线与优化版本差异自动生成：

指标	baseline	optimized	差异	状态
avg_latency_ms	112.5	98.4	-12.5%	□
p95_latency_ms	140.3	121.3	-13.5%	□
false_accept_rate	0.0012	0.0008	改善	□

8.9. 自动化与复现保障

机制	说明
Hash 校验	onnx/om/脚本确保未篡改
repeatable seed	设定随机种子统一实验
benchmark.py	统一输出 metrics.json
drift 检测	周期性对比指标偏差
一键脚本	run.sh + run.ps1 支持跨平台

8.10. 指标可视化建议

- 时间序列：Latency / FPS / 温度。
- 箱线图：不同优化阶段的时延分布。
- 堆叠条：阶段占比（检测/特征/比对）。
- 散点：光照水平 vs 识别准确度。

8.11. 通用问题经验库

问题	案例	根因	处理
相机丢帧	1/2	帧率不稳	缓冲 + 限速
模型加载慢	全部	冷启动未预热	预加载预热 10 次

问题	案例	根因	处理
OCR 错字	新增	图像模糊	降噪/锐化
跟踪漂移	2	过度遮挡	reinit + 短期外观缓存

8.12. 扩展方向

- 多模态融合（视觉 + 语音指令）。
- 硬件加速协同（NPU + DSP 解码）。
- 大模型边缘裁剪（蒸馏 + 量化 + 分层推理）。

8.13. 贡献 workflow

1. Fork 分支：case/<name>;
2. 新建目录遵循模板;
3. 提交包含：README、metrics、脚本、model_card;
4. CI 自动校验格式与 hash;
5. PR 模板填写：动机/数据/指标/风险。

8.14. 章节小结

案例是知识的验证与反哺：通过统一模板与自动化度量，形成可延展的案例库，帮助新模型与新任务快速落地并保障质量。

8.15. 实践任务

1. 搭建 case1 目录，生成 baseline metrics。
2. 实现 face detection + feature 比对流程，并输出 FAR/FRR。
3. 将一次优化（裁剪/量化）前后差异写入 diff 表。
4. 编写 benchmark.py：支持 --repeat N --output metrics.json。
5. 增加 drift 检测脚本（比较两次 metrics 差异，阈值报警）。

9. 附录与工具箱

9.1. 章节总览

本附录聚焦“查得快、用得稳”：常见报错速查、转换参数模板、性能/质量 Checklist、术语字典、推荐资源与社区贡献规范。可作为日常开发随手翻阅的工具章节。

9.2. 常见报错速查

分类	报错/现象	可能原因	排查步骤	解决建议
ATC	E19001: Op Not ↪ Supported	新算子/版本落后	确认 CANN 版本 + onnxsim 简化	升级/替换结 构/自定义算子
ATC	Shape 推断失败	动态维度不明确	检查 --input_shape ↪ /动态参数	固定关键维度或 提供范围
ACL	aclmdlLoadFromFile ↪ failed	权限/路径/模型 损坏	校验文件 hash/权限	修正权限/重新生 成 OM
Runtime	OOM / alloc 失 败	Batch 或分辨率 过大	统计输入分布	降 batch/分 桶/复用内存
运行	推理输出 NAN	数值溢出/量化尺 度错误	Dump 中间 Tensor	调整量化/保留 FP32 层
性能	Timeline 大量 gap	Host 阻塞/小算 子	Profiling 分析	合并算子/异步预 取
性能	H2D 高占比 >25%	多次小拷贝	合并缓冲	AIPP 下沉/批量 化
精度	Top1 下降 >1%	预处理不匹配	对比 ONNX 输出	统一 Normalize & Layout

分类	报错/现象	可能原因	排查步骤	解决建议
精度	mAP 不稳定	阈值或 NMS 误差	调整阈值/比对中间框	校准 NMS 公式/尺度
稳定	间歇 Crash	悬空指针/并发访问	启用 ASAN/日志回溯	修订生命周期/加锁
部署	模型加载慢	冷启动/IO 慢	预热/缓存	预加载 + 固态存储
安全	日志泄露敏感路径	直接 print	grep 审计	结构化日志脱敏

9.3. 模型转换参数模板合集

9.3.1. 分类模型 (ResNet)

```

1 atc --model=resnet50.onnx \
2   --framework=5 \
3   --output=resnet50_fp16 \
4   --input_format=NCHW \
5   --input_shape="input:1,3,224,224" \
6   --soc_version=Ascend310B \
7   --precision_mode=allow_fp32_to_fp16 \
8   --log=info

```

9.3.2. YOLO 动态分辨率

```

1 atc --model=yolov5s.onnx \
2   --framework=5 \
3   --output=yolov5s_640_768 \
4   --dynamic_image_size="640,640;768,768" \
5   --input_format=NCHW \
6   --soc_version=Ascend310B \
7   --op_select_implmode=high_performance \
8   --precision_mode=allow_fp32_to_fp16

```

9.3.3. INT8 量化（示例）

```

1 atc --model=resnet50.onnx \
2   --framework=5 \
3   --output=resnet50_int8 \

```

```
4 --input_format=NCHW \  
5 --input_shape="input:1,3,224,224" \  
6 --soc_version=Ascend310B \  
7 --precision_mode=allow_mix_precision \  
8 --insert_op_conf=aipp.cfg \  
9 --enable_small_channel=true
```

9.4. 性能与质量 Checklist（执行勾项）

- 性能：
- ☐ Profiling 无明显 Idle gap > 10%
 - ☐ H2D + D2H 占比 < 25%
 - ☐ Postprocess 占比 < 20%
 - ☐ Stream 利用率平衡（无单流饱和）
 - ☐ 使用内存池减少频繁 alloc/free

- 精度：
- ☐ ONNX vs OM Top1 差异 < 0.2%
 - ☐ L1 平均误差 < 1e-3 (FP16)
 - ☐ NMS 输出框数量与基线差异 < 1 框/图（平均）
 - ☐ INT8 校准集覆盖多场景

- 稳定性：
- ☐ 1h 稳态无 Crash / OOM
 - ☐ 温度在安全区间 < 85°C
 - ☐ 看门狗重启次数 = 0

- 安全：
- ☐ 日志无明文密钥
 - ☐ 模型文件 hash 校验通过

9.5. 术语表（扩展）

术语	说明
OM	Ascend 离线模型二进制格式
ACL	Ascend 计算语言 API 层
ATC	模型转换/编译工具

术语	说明
AIPP	自动图像预处理模块
Stream	异步任务调度通道
Profiling	性能采样分析工具体系
Fallback	算子未匹配优化实现退回通用实现
Quant Calibration	量化尺度统计过程
Baseline	初始标准对照性能/精度集
Drift	指标随时间未经预期的漂移

9.6. 推荐资源与外部引用

- Ascend 官方文档入口（安装/算子列表/最佳实践）
- CANN Release Notes：版本兼容与已知问题。
- ONNX Operator 列表与语义说明。
- Open Model Zoo / ModelScope：获取预训练模型与许可信息。
- 学术资源：算子融合、低比特量化、蒸馏相关论文列表。

9.7. 贡献指南摘要

流程：Fork □ 新分支 □ 修改/新增 □ 本地 lint & 生成脚本 □ PR（描述动机/影响面/验证方式）。PR 要求：| 要素 | 说明 | | — | — | | 标题 | 简明说明改动作用 | | 描述 | 背景 + 修改点 + 风险 | | 验证 | 性能/精度/功能截图或数据 | | 回滚 | 若失败如何恢复 | | 关联 Issue | 追踪链接 |

9.8. FAQ

问题	回答
模型转换慢怎么办？	使用 SSD，关闭调试日志，检查不必要动态 shape。
精度下降如何定位？	离线脚本层级 Dump 比对，逐层二分。

问题	回答
如何减少内存占用？	启用内存池 + 减少中间冗余张量 + 固定 batch。
量化后收益不明显？	检查是否 Compute-bound，或激活分布集中导致尺度相近。
NMS 很慢？	合并小框批量处理/降低候选阈值/考虑 Device 版 NMS。

9.9. License 与引用

本书内容遵循 Apache 2.0 许可证。引用：> 《昇腾 310B 实战：从入门到精通边缘计算与人工智能》（GitHub: zhouxzh/Ascend310）

9.10. 版本路线回顾

版本	内容	目标
v0.1	结构框架	验证框架可行
v0.3	核心部署链路	形成可用主线
v0.6	案例与工程化	贴近实战
v1.0	全面审校发行	正式发布

9.11. 实践任务

1. 为你的项目添加 1 条本地常见错误记录（含根因与解决）。
2. 复制分类 ATC 模板并改写为检测模型版本（含动态尺寸）。
3. 在术语表补充 3 个任务相关术语（并验证唯一性）。
4. 选取 FAQ 一条，写出更深入排查脚本思路。

Part II.

Part II: 实验教程

0. 案例 0：初步使用开发板

0.1. 昇腾 310B 开发板介绍

OrangePi AIpro(8T) 开发板是由香橙派联合华为精心打造的高性能 AI 开发板。它采用昇腾 AI 技术路线，搭载昇腾 310B 处理器（4 核 64 位处理器 + AI 处理器），集成图形处理器，支持 8TOPS INT8 的 AI 算力。板载 8GB/16GB LPDDR4X 内存，并支持外接 32GB 至 256GB 的 eMMC 模块，同时支持双 4K 高清输出。

OrangePi AIpro(8T) 拥有丰富的接口资源，包括两个 HDMI 输出、GPIO 接口、Type-C 电源接口、支持 SATA/NVMe SSD 2280 的 M.2 插槽、TF 插槽、千兆网口、两个 USB3.0、一个 USB Type-C 3.0、一个 Micro USB（用于串口打印调试）、两个 MIPI 摄像头接口以及一个 MIPI 屏幕接口等。此外，还预留了电池接口。

该开发板广泛适用于 AI 边缘计算、深度视觉学习、视频流 AI 分析、自然语言处理、智能小车、机械臂、无人机、云计算、AR/VR、智能安防及智能家居等领域，覆盖 AIoT 的各个行业。在软件方面，OrangePi AIpro(8T) 支持 Ubuntu 和 openEuler 操作系统，能够满足大多数 AI 算法原型验证及推理应用开发的需求。在这个教程中，我们只介绍基于 Ubuntu 操作系统的昇腾 310B 开发流程。

0.1.1. 开发板详细视图

0.1.2. 开发板硬件规格

0.2. 配件准备

为了顺利进行开发，请准备以下配件：

1. TF 卡

建议使用容量 64GB 及以上、速率为 Class10 级以上的闪迪（SanDisk）品牌 TF

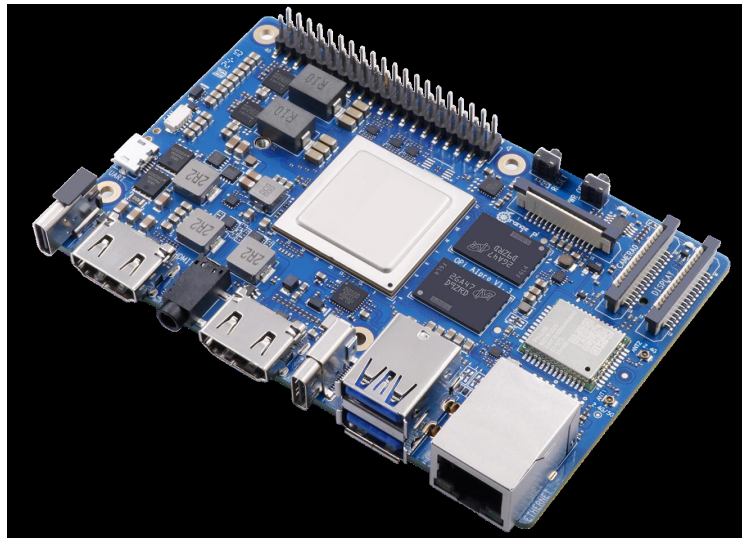


Figure 1.: 产品图

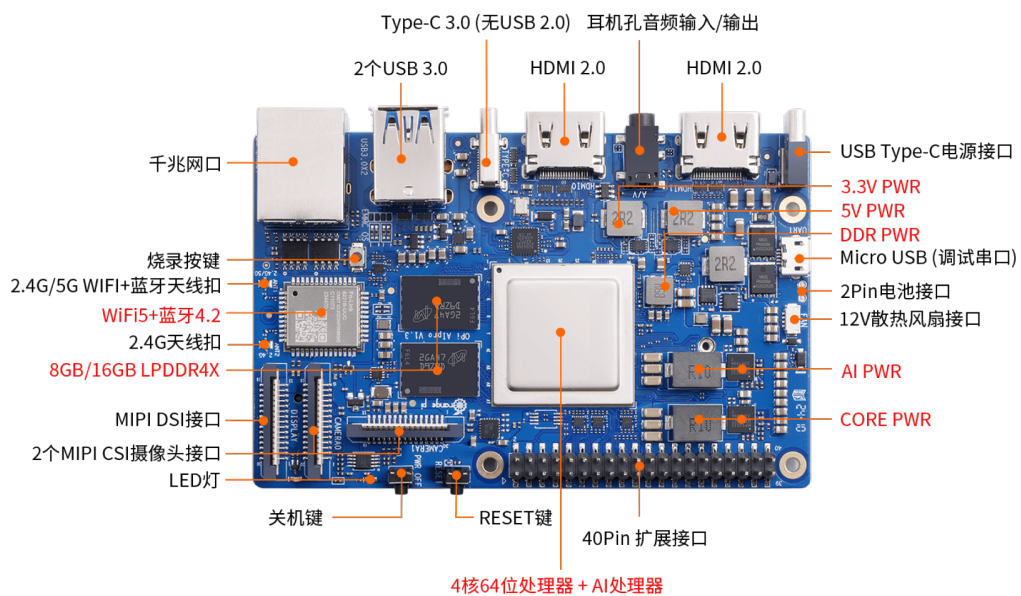


Figure 2.: 正面标注视图

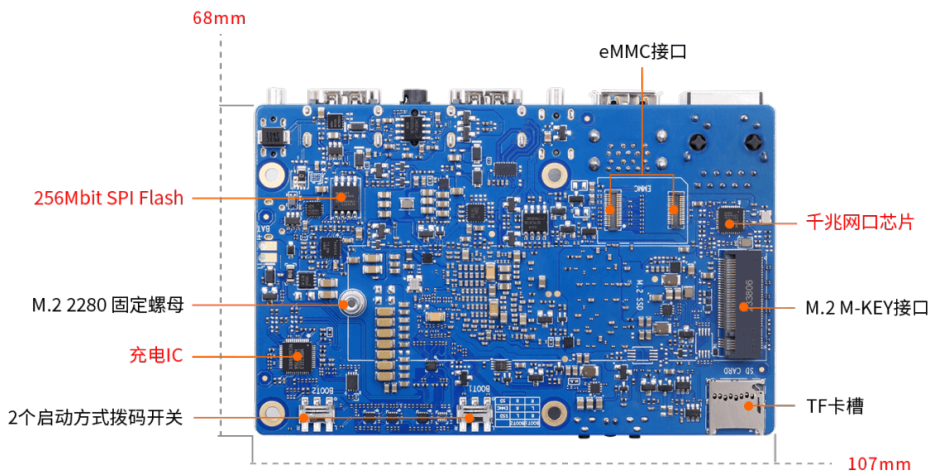


Figure 3.: 背面标注视图

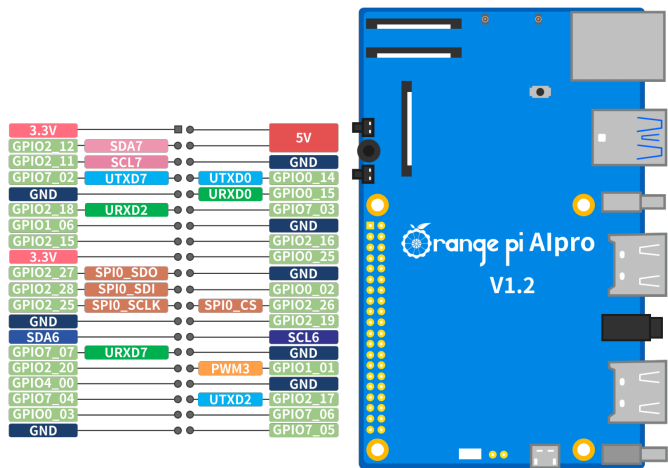


Figure 4.: GPIO 接口定义



Figure 5.: tf 卡



Figure 6.: 读卡器

卡。虽然最小支持 32GB，但为了避免开发过程中出现磁盘空间不足的问题，推荐使用更大容量。

2. TF 卡读卡器

用于在电脑上读写 TF 卡以刷写系统镜像。建议选择 USB 3.0 速率的读卡器，以减少系统刷写的等待时间。

3. HDMI 线

开发板配备标准 HDMI 接口。请根据您的显示器接口类型，准备标准的 HDMI 线或 HDMI 转 Mini-HDMI/Micro-HDMI 线。

4. 电源适配器

开发板采用 PD 协议供电（20V 挡位），功率需求为 65W。请务必使用支持 PD 协议 20V 输出的 65W Type-C 电源适配器。

5. USB 鼠标和键盘

用于在本地桌面环境下对开发板进行操作和调试。

6. 金属外壳

用于保护开发板硬件，防止意外短路或物理损伤。



Figure 7.: HDMI



Figure 8.: Mini HDMI



Figure 9.: PD 电源



Figure 10.: 外壳



Figure 11.: 风扇

7. 散热风扇及散热鳍片

由于昇腾 310B 处理器在高负载下发热量较大，强烈建议安装主动散热设备。开发板提供 2pin 风扇接口（12V），支持 PWM 调速。

8. Type-C 转 USB 3.0 转接线（可选）

开发板的一个 Type-C 接口支持 USB 3.0 协议（不兼容 USB 2.0），可通过转接线连接 USB 3.0 外设。

9. M.2 NVMe SSD（可选）

开发板背部的 M.2 接口支持 PCIe 协议（2280 规格），可安装 NVMe SSD 作为系统盘或扩展存储。

10. M.2 SATA SSD（可选）

该 M.2 接口同时也支持 SATA 协议，因此也可以使用 M.2 SATA（NGFF）接口



Figure 12.: 转接线

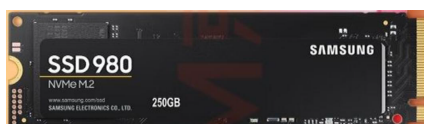


Figure 13.: nvme ssd



Figure 14.: ngff ssd

的 SSD。

11. eMMC 模块（可选）

eMMC 是一种高性能、低功耗的嵌入式存储方案。相比 TF 卡，eMMC 读写速度更快（100-400MB/s）且更稳定。开发板预留了 eMMC 接口，需额外购买香橙派专用 eMMC 模块。

12. USB 摄像头（可选）

用于图像识别、视频通话等应用开发。

13. 网线（可选）

虽然开发板板载 Wi-Fi 模块，但在需要更稳定网络连接的场景下，建议使用千兆网口连接有线网络。

14. 树莓派 IMX219 摄像头（MIPI-CSI）（可选）

开发板提供两个 MIPI-CSI 接口，兼容树莓派 IMX219 摄像头，可直接连接而无需占用 USB 接口。

15. MIPI LCD 显示屏（可选）

开发板配备 MIPI-DSI 显示接口，可直接驱动兼容的 MIPI 显示屏（如树莓派 5 寸屏），无需外接 HDMI 显示器。



Figure 15.: emmc 正面

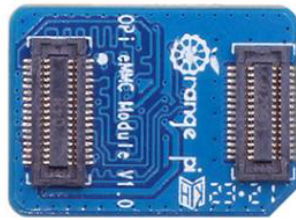


Figure 16.: emmc 背面



Figure 17.: 摄像头

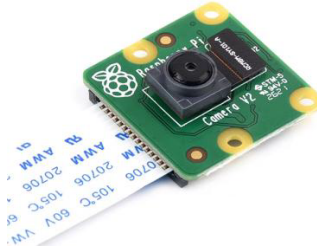


Figure 18.: MIPI-CSI 摄像头



Figure 19.: MIPI 显示器



Figure 20.: Micro USB 数据线

16. Micro USB 数据线（可选）

开发板板载 CH343P 芯片，将 UART 调试串口转换为 Micro USB 接口。使用 Micro USB 数据线连接电脑，即可进行串口调试。

0.3. 操作系统安装

作为华为昇腾生态的重要成员，OrangePi Alpro(8T) 支持 Ubuntu 和 openEuler 两种操作系统。由于开发板板载无预装系统，我们需要通过电脑将系统镜像刷写到 TF 卡中。建议使用 Windows 11 或 Ubuntu 22.04 及以上版本的 PC 进行操作。

首先，访问香橙派官网的[技术支持页面](#)。

在页面下方找到“官方镜像”区域。官方提供了预装昇腾 NPU 应用环境及软件的 Ubuntu 和 openEuler 镜像，极大地方便了开发者快速上手。

0.3.1. 镜像下载

Ubuntu

1. 点击下载：点击 Ubuntu 镜像对应的下载按钮。
2. 获取提取码：复制弹出的百度网盘提取码，并点击跳转链接。
3. 选择镜像文件：在网盘中找到名为Ubuntu的文件夹并进入。
4. 文件说明：
 - .xz后缀文件为镜像压缩包。
 - .sha后缀文件为 MD5 校验码，用于验证下载文件的完整性。
5. 选择版本：
 - **Desktop**：包含 GUI 图形化界面，适合初学者和需要桌面环境的用户。
 - **Minimal**：仅包含命令行界面，适合高级用户或服务器用途。

建议初学者下载带有Desktop字样的镜像。



Figure 21.: 技术支持页面



Figure 22.: 官方镜像



Figure 23.: 下载



Figure 24.: 跳转

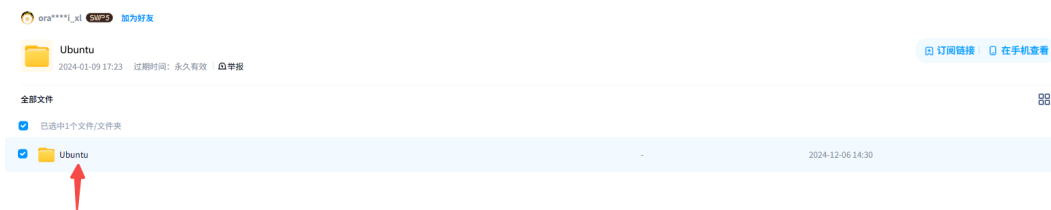


Figure 25.: 文件夹



Figure 26.: 选择镜像

6. 下载与解压：下载完成后，请先校验文件完整性，再解压.xz压缩包得到.img镜像文件。

0.3.2. 校验下载文件 (MD5)

为了确保下载的镜像文件未损坏，建议在解压前进行 MD5 校验。

- **Windows:** 在镜像文件所在文件夹内按住 **Shift** 键并点击鼠标右键，选择“在终端中打开”或“在此处打开 Powershell 窗口”。输入以下命令（文件名请根据实际情况修改）：

```
1 certutil -hashfile opiaipro_ubuntu22.04_desktop_aarch64_20241128.  
  ↪ img.xz md5
```

将输出的 MD5 值与同目录下.sha文件中的内容进行比对。

- **Ubuntu:**

```
1 md5sum <filename>
```

- **macOS:**

```
1 md5 <filename>
```

若校验值一致，说明文件完整，可进行解压；若不一致，请重新下载。

0.3.3. 刷写系统到 TF 卡

工具准备

- 下载链接：[官网](#) | [百度网盘](#)

1. **SD Card Formatter**：用于格式化 TF 卡，确保存储卡状态良好。

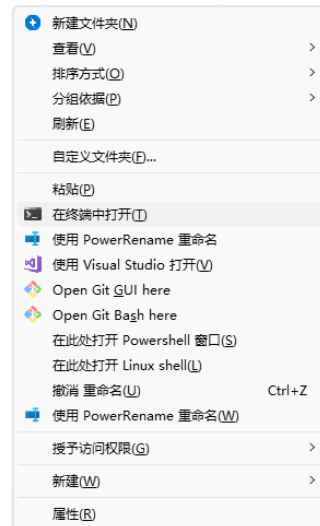


Figure 27.: 终端

```
Microsoft Windows [版本 10.0.26100.4652]
(c) Microsoft Corporation. 保留所有权利。

D:\BaiduNetdiskDownload>certutil -hashfile opiaipro_ubuntu22.04_desktop_aarch64_20241128.img.xz md5
MD5 的 opiaipro_ubuntu22.04_desktop_aarch64_20241128.img.xz 哈希:
c2504fd63b2cc222106c30a29ac06386
CertUtil: -hashfile 命令成功完成。

D:\BaiduNetdiskDownload>
```

Figure 28.: md5 校验

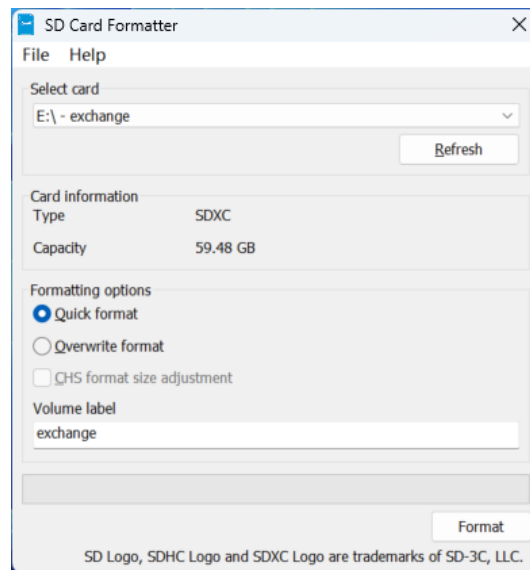


Figure 29.: TF 卡格式化

2. **balenaEtcher**: 用于将 .img 镜像文件烧录到 TF 卡。注意：建议使用 1.19.25 及以下版本，以避免兼容性问题。

格式化 TF 卡

1. 将 TF 卡插入读卡器，连接至电脑。
2. 打开 **SD Card Formatter** 软件。
3. 确认选中了正确的盘符，点击右下角的 **Format** 按钮。
警告：格式化将清除 TF 卡上所有数据，请确认无误后点击“是”。
4. 等待格式化完成，点击确定。

烧录镜像（以 Ubuntu 为例）

1. 打开 **balenaEtcher**，选择“Flash from file”（从文件烧录）。
2. 选择解压后的镜像文件（.img 格式），然后点击“Select target”选择您的 TF 卡。
3. 点击“Flash!”开始烧录。
4. 烧录完成后软件会自动进行校验，请耐心等待。
5. 校验通过后，关闭软件并安全弹出 TF 卡。

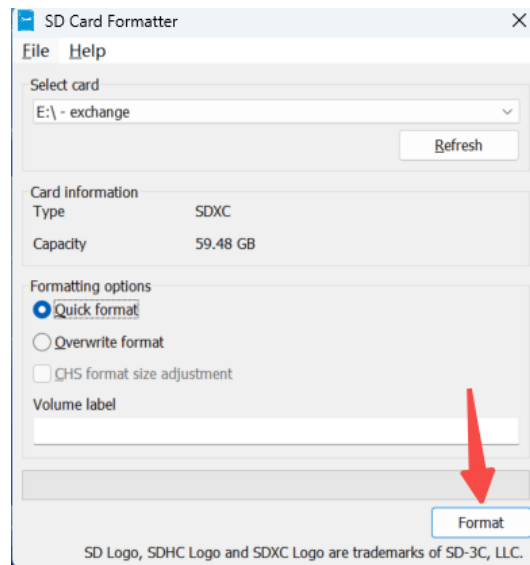


Figure 30.: 格式化

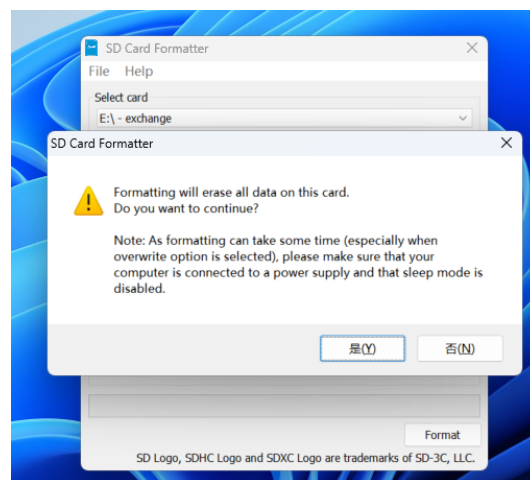


Figure 31.: warning

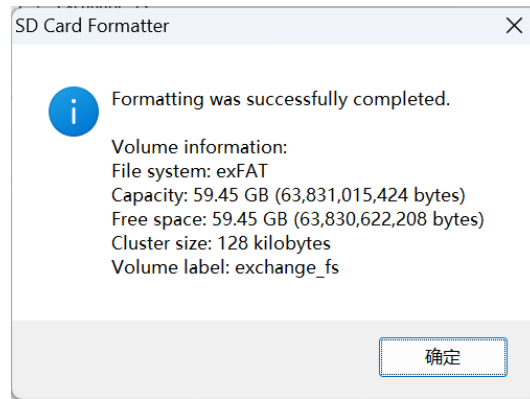


Figure 32.: 格式化完成



Figure 33.: balenaEtcher

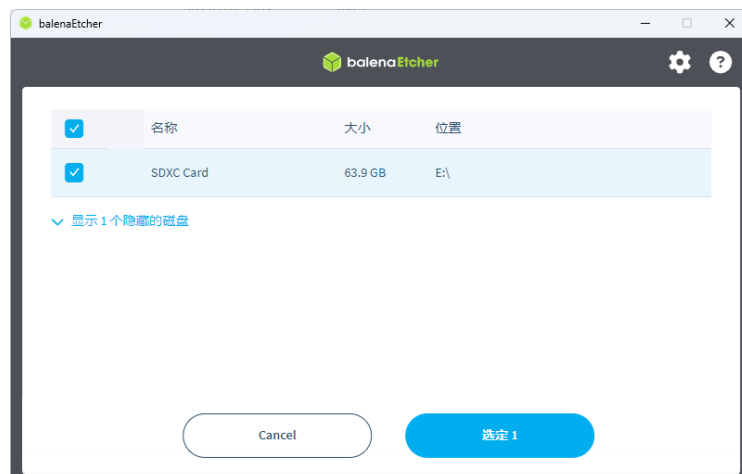


Figure 34.: 选择磁盘

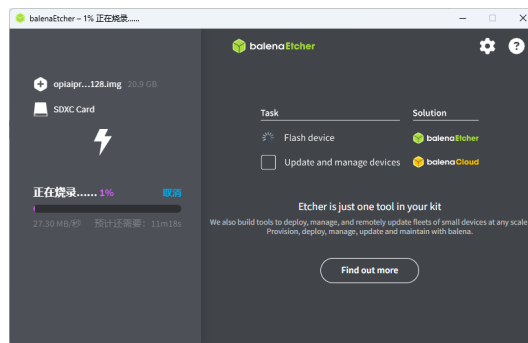


Figure 35.: 烧录过程

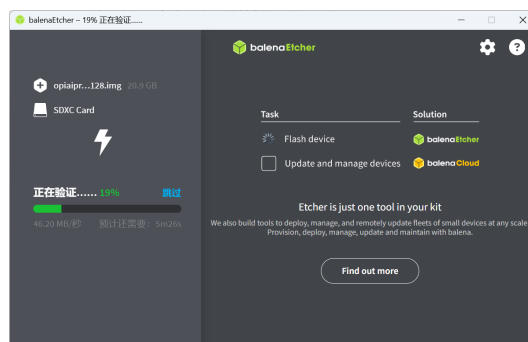


Figure 36.: 校验过程

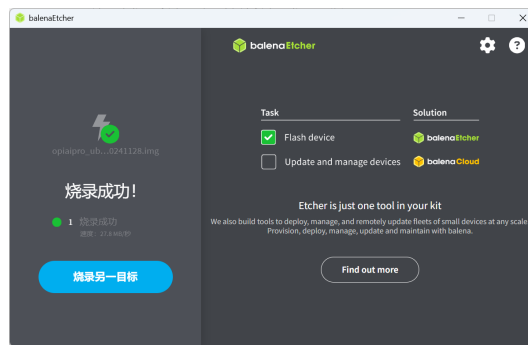


Figure 37.: 完毕

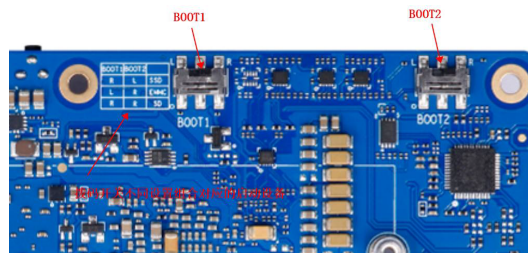


Figure 38.: boot 开关

其他存储介质说明

- **eMMC**: 板载无 eMMC, 需购买专用模块。刷写方法请参考香橙派用户手册。
- **SSD**: 支持 M.2 SSD 启动, 但兼容性有限 (仅支持特定品牌型号), 且需自行准备。初学者不推荐直接使用 SSD 作为系统盘。

设置启动模式

开发板支持从 TF 卡、eMMC 或 M.2 SSD 启动。当连接了多种存储设备时, 需通过背面的拨码开关 (BOOT 开关) 指定启动设备。

拨码开关状态说明 (ON 方向为 1/右, 相反为 0/左, 具体请参考板上丝印或下表):

Boot1	Boot2	启动设备
左	左	(未使用)
右	右	TF 卡
左	右	eMMC
右	左	M.2 SSD

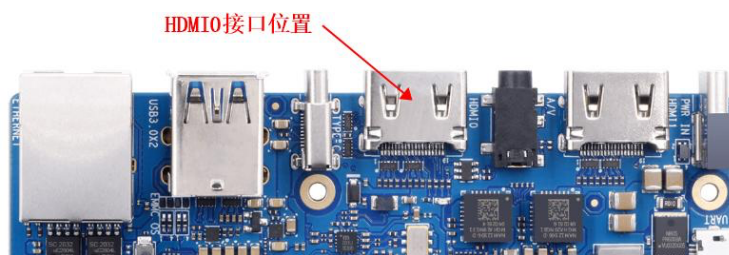


Figure 39.: HDMIO

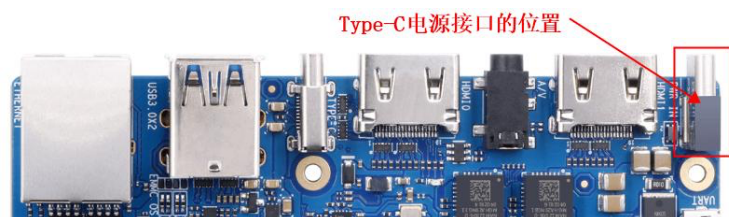


Figure 40.: TYPE-C Power

注意：切换拨码开关后，必须**完全断电**（拔掉电源线）再重新上电，新的启动配置才会生效。仅按 RESET 键重启无效。

0.4. 启动开发板

0.4.1. 方式一：图形化界面启动

1. 硬件连接：

- 将刷写好的 TF 卡插入开发板插槽。
- 确认拨码开关均拨至**右侧**（TF 卡启动模式）。
- 将 HDMI 线连接至 **HDMIO** 接口（靠近 USB 3.0 接口的那个）。
- 连接鼠标和键盘。
- 最后，接入 Type-C 电源。

2. 系统启动：

- 上电后，风扇会全速旋转，随后声音变小，屏幕显示启动画面。
- 稍候进入登录界面。

3. 登录系统：

- 默认普通用户：HwHiAiUser，密码：Mind@123
- 默认 Root 用户：root，密码：Mind@123



Figure 41.: 登录



Figure 42.: 桌面

输入密码登录进入桌面环境。

若无法登陆请检查输入的密码是否正确，大小写以及符号是否正确

默认账户表格: | 用户名 | 密码 | | :—: | :—: | | root | Mind@123 | |
HwHiAiUser | Mind@123 |

0.4.2. 串口界面

1. 使用 USB2TTL 模块，与开发板的 GPIO 口进行连线

开发板的 TX (GPIO8) 接入 USB2TTL 模块的 RX 接口, 开发板的 RX (GPIO10) 则接入模块的 TX 接口, 并连接好 GND 接地, 在 Windows 电脑下可以使用 PUTTY 连接串口。

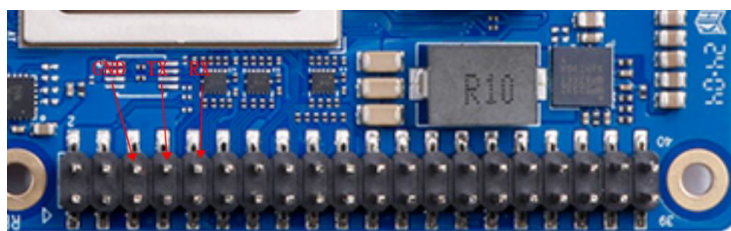


Figure 43.: 开发板串口

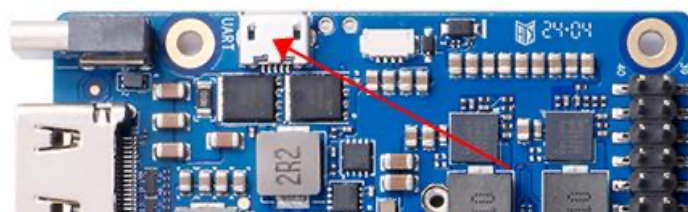


Figure 44.: MicroUSB 串口

2. 使用开发板自带的 Micro USB 接口进行串口调试，该方法更为方便，只需要一根 Micro USB 数据线，接入电脑后打开设备管理器查询对应的串口，然后使用 PUTTY 进行链接即可。

以 Micro USB 接口为例：1. 使用 Micro USB 数据线连接开发板和电脑，此时请不要给开发板上电。2. 打开电脑的设备管理器，选择端口，寻找开发板对应的串口端口号

3. 打开串口调试软件（PUTTY）

将 Connection Type 选择为 Serial，然后在 Serial Line 处将端口号修改为设备管理器中查到的端口号，如作者此处端口号为 COM3，此外，还需要将 Speed 从 9600 修改为 115200，最后点击 Open 打开串口。

4. 给开发板上电，等待出现 Ubuntu 22.04.3 LTS orange pi aipro ttyAM0 字样，输入登录的用户名 HwHiAiUser 并回车，然后输入密码 Mind@123 并回车，注意在输入密码的时候屏幕并不会显示任何东西，登陆后的界面如图所示。

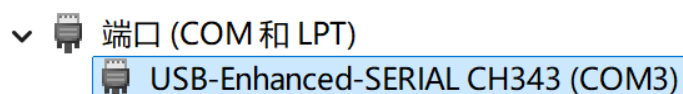


Figure 45.: 端口号

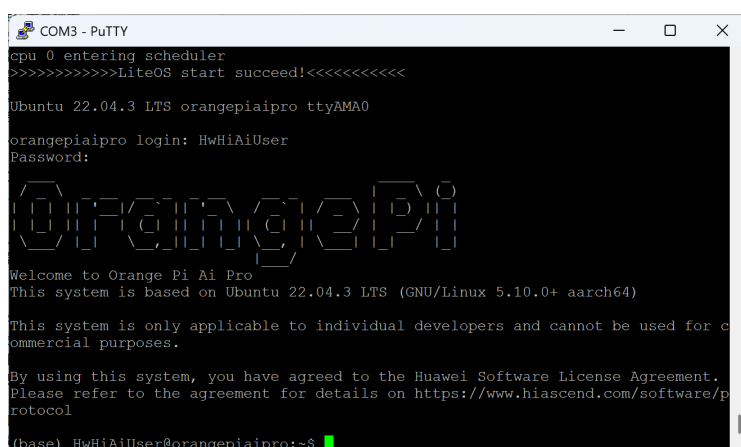


Figure 48.: 登录成功

0.5. 网络连接

0.5.1. 无线网络连接 (WiFi)

开发板板载了 WiFi 模块，可以通过命令行工具 `nmcli` 轻松连接无线网络。

1. 扫描 WiFi

```
1 nmcli dev wifi list
```

2. 连接 WiFi 将<SSID>替换为你的 WiFi 名称, <PASSWORD>替换为密码。

```
1 nmcli dev wifi connect <SSID> password <PASSWORD>
```

3. 查看连接状态

```
1 nmcli connection show
```

0.5.2. 有线网络连接

如果有线网络可用，直接插入网线即可。可以通过以下命令查看 IP 地址：

```
1 ip addr
```

或者

```
1 ifconfig
```

0.6. 远程连接 (SSH)

为了方便开发，通常会使用个人电脑通过 SSH 远程连接到开发板。

- 1. 获取开发板 IP 地址使用上述ip addr命令获取开发板的 IP 地址（通常在wlan0或eth0接口下）。
- 2. 使用 SSH 客户端连接在你的个人电脑终端（Windows 可以使用 CMD、Power-Shell 或 Putty，Mac/Linux 使用 Terminal）中输入：

```
1 ssh HwHiAiUser@<开发板IP地址>
```

例如：

```
1 ssh HwHiAiUser@192.168.1.100
```

默认密码为：Mind@123

0.7. 验证开发环境

系统启动并连接网络后，我们需要验证昇腾 AI 处理器的状态以及开发环境是否正常。

- 1. 查看 NPU 状态使用npu-smi工具查看 NPU 的详细信息，包括温度、功耗、算力利用率等。

```
1 npu-smi info
```

如果能看到类似以下的输出，说明 NPU 工作正常：

```
1 +-----+
2 | npu-smi 23.0.0                               Version: 23.0.0
3 |-----+
4 | NPU      Name                               Health   Power(W)   Temp(C
5 | Chip      Device       Hugepages-Usage(page) | Bus-Id   AICore(%)  Memory
6 |=====+=====
7 | 0         310B4         0 / 0                OK        12.8       45
8 | 0         0             0 / 0                0000:00:00.0 0
9 |=====+=====
```

2. **检查 CANN 环境**官方镜像通常已预装 CANN (Compute Architecture for Neural Networks)。可以通过检查环境变量或运行简单命令来确认。通常环境变量设置在`.bashrc`中，尝试执行：

```
1 echo $ASCEND_HOME_PATH
```

或者检查编译器版本：

```
1 c++ --version
```

0.7.1. 设置 SWAP 交换分区

开发板虽然有 8G/16G 的运存，但是有些应用（例如 ATC 模型转换工具）需要较大的内存，在这种情况下我们可以通过设置 SWAP 交换分区来扩展系统能使用的最大内存容量。

1. **创建交换文件**

首先，使用`fallocate`命令创建一个指定大小的文件（例如 12GB）。如果你的存储空间允许，建议设置得大一些，以防模型转换时内存不足。

```
1 sudo fallocate -l 8G /swapfile
```

如果提示`fallocate`失败，可以使用`dd`命令：

```
1 sudo dd if=/dev/zero of=/swapfile bs=1G count=8
```

2. **设置权限**

出于安全考虑，需要修改文件的权限，仅允许 `root` 用户读写。

```
1 sudo chmod 600 /swapfile
```

3. **设置交换区**

将文件格式化为交换分区格式。

```
1 sudo mkswap /swapfile
```

4. **启用交换区**

启用该交换文件。

```
1 sudo swapon /swapfile
```

5. **持久化设置**

为了防止重启后失效，需要将其写入`/etc/fstab`文件中。

```
1 sudo nano /etc/fstab
```

在文件末尾添加以下内容：

```
1 /swapfile none swap sw 0 0
```

保存并退出（Ctrl+O, Enter, Ctrl+X）。

0.8. 结语

至此，你的昇腾 310B 开发板（OrangePi AIpro）已经完成了基本的环境搭建。接下来，你可以进入[案例 1：智能打卡机](#)的学习，开始你的第一个 AI 应用开发之旅。

1. 案例 1：智能打卡机

1.1. 项目简介

本项目旨在利用昇腾 310B 的强大 AI 算力，构建一个功能完整、响应迅速的智能人脸识别打卡系统。系统通过 USB 摄像头实时捕捉视频流，检测画面中的人脸，并与预先注册的员工/学生人脸数据库进行比对，完成身份验证和自动记录考勤。该项目不仅是一个功能性的应用，更是一个端到端的 AI 实践案例，涵盖了从硬件选型、软件环境搭建、数据准备、模型训练与优化，到最终在边缘设备上部署的全过程。项目的源代码可以从[这里](#)下载。

1.2. 内容大纲

1.2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 图像采集: USB 摄像头
- 显示设备 (可选): HDMI 显示器，用于实时预览或 UI 展示
- 外设 (已使用 SSH 进行连接过, 可选): 键盘、鼠标
- 电源: 为昇腾 310B 开发板提供稳定供电
- 连接线: HDMI 线, USB-C 数据线等

硬件连接指引：将开发版通电，将摄像头连接至开发板 USB 接口

1.2.2. 软件环境

- 操作系统: Ubuntu 22.04
- CANN 版本: 7.0 或更高
- Python 版本: 3.11.x
- 主要依赖库:
 - opencv-python: 用于图像和视频处理

- numpy: 用于数值计算
- scikit-learn: 用于评估模型性能
- onnxruntime: 用于运行 ONNX 模型
- PyQt5 (可选): 用于构建图形用户界面
- Flask + Bootstrap (可选): 提供 Web 界面与交互
- Font Awesome (可选): 丰富界面图标

1.2.3. 数据集准备

- 数据集来源:
 1. 公开数据集: 如 LFW (Labeled Faces in the Wild)、CASIA-WebFace 等。
 2. 自建数据集: 比较推荐, 使用摄像头为每位用户 (员工/学生) 拍摄多张、多角度、不同光照和表情的人脸照片。

- 数据组织结构:

```
1 datasets/  
2 |— zhang_san/  
3 |   |— 001.jpg  
4 |   |— 002.jpg  
5 |   |— ...  
6 |— li_si/  
7 |   |— 001.jpg  
8 |   |— 002.jpg  
9 |   |— ...  
10 |— ...
```

1.2.4. 模型训练

- 模型选择:
 - 人脸检测: MTCNN
 - 人脸识别: MobileFaceNet
- 训练流程:
 1. 使用预处理好的数据集进行模型训练。
 2. 调整超参数 (学习率、批大小等) 以获得最佳性能。
 3. 在验证集上评估模型准确率、召回率等指标。
- 模型导出: 将训练好的 PyTorch 或 TensorFlow 模型转换为昇腾的 om 格式。

1.2.5. 模型部署

- **模型转换:** 使用 ATC (Ascend Tensor Compiler) 工具将 ONNX 模型转换为昇腾 310B 支持的 .om 离线模型。

```
1 atc --model=./models/mobilefacenet.onnx --framework=5 --output=./  
    ↪ models/mobilefacenet --input_format=NCHW --input_shape="  
    ↪ input0:1,3,112,112" --soc_version=Ascend310B4
```

- **部署代码 (main.py):**
 1. 初始化 CANN 和 ACL (Ascend Computing Language) 资源。
 2. 加载 .om 离线模型。
 3. 循环读取摄像头帧。
 4. 对每一帧进行人脸检测和识别推理。
 5. 将识别结果与数据库比对, 输出姓名。
 6. 在画面上绘制矩形框和姓名, 并记录打卡时间。

1.2.6. 3D 打印结构件

为了方便地将摄像头固定在合适的位置, 我们设计了专用的摄像头支架和设备保护外壳。 - **文件列表:** - camera_holder.stl: 摄像头支架模型文件 - device_case.stl: 设备外壳模型文件 - **打印建议:** - **材料:** PLA 或 PETG - **层高:** 0.2mm - **填充率:** 20%

1.3. Web 应用

1.3.1. 核心功能

- **多种识别方式:** 支持实时摄像头识别和图片上传识别
- **智能用户注册:** 支持摄像头实时注册和批量图片上传注册
- **实时考勤管理:** 自动记录考勤信息, 支持可配置的重复打卡间隔
- **用户管理系统:** 完整的用户增删改查功能
- **考勤配置:** 灵活的考勤参数配置

1.3.2. 界面特性

- **响应式设计:** 支持桌面和移动设备访问
- **现代化 UI:** 基于 Bootstrap 5 的美观界面
- **实时反馈:** 实时显示识别结果和考勤状态

- 直观导航: 清晰的功能模块划分

1.4. □ Web 界面详细介绍

1.4.1. 1. 主页 (/)

功能概述页面 - 系统介绍和功能导航 - 卡片式布局展示各功能模块 - 快速访问入口: - 人脸识别 (摄像头/图片上传) - 用户注册 (摄像头/图片上传) - 用户管理 - 考勤记录 - 考勤配置

1.4.2. 2. 人脸识别模块

2.1 实时摄像头识别 (/camera_live)

实时人脸识别和考勤 - **摄像头控制**: 启动/停止摄像头按钮 - **实时视频流**: 显示摄像头画面和识别框 - **用户列表**: 显示所有已注册用户 - **考勤信息**: 实时显示考勤结果和状态 - **使用提示**: 详细的操作说明

功能特点: - 实时人脸检测和识别 - 自动考勤记录 - 防重复打卡机制 - 识别结果实时反馈

2.2 图片上传识别 (/recognize)

静态图片人脸识别 - **图片上传**: 支持拖拽上传或点击选择 - **格式支持**: PNG, JPG, JPEG, GIF, BMP - **实时预览**: 上传后立即显示图片 - **识别结果**: 显示识别到的人员信息 - **考勤记录**: 自动记录考勤状态

1.4.3. 3. 用户注册模块

3.1 摄像头注册 (/camera_register)

实时摄像头人脸注册 - **姓名输入**: 输入待注册人员姓名 - **实时拍摄**: 摄像头实时画面 - **多角度采集**: 建议采集多个角度的人脸 - **质量检测**: 自动检测人脸质量 - **批量保存**: 一次性保存多张人脸图片

3.2 图片上传注册 (/register)

批量图片上传注册 - 批量上传: 支持同时上传多张图片 - 人脸检测: 自动检测图片中的人脸 - 质量筛选: 过滤低质量人脸图片 - 预览确认: 注册前预览所有人脸 - 一键注册: 批量处理所有图片

1.4.4. 4. 用户管理 (/user_management)

完整的用户管理功能 - 用户列表: 显示所有已注册用户 - 用户详情: 查看用户的所有人脸图片 - 删除用户: 删除不需要的用户及其数据 - 统计信息: 显示用户总数和注册时间 - 搜索功能: 快速查找特定用户

1.4.5. 5. 考勤记录 (/attendance)

考勤数据管理 - 记录列表: 显示所有考勤记录 - 时间排序: 按时间倒序显示最新记录 - 统计信息: - 今日打卡人数 - 总打卡次数 - 注册人员总数 - 数据导出: 支持 CSV 格式导出 - 筛选功能: 按日期、人员筛选记录

1.4.6. 6. 考勤配置 (/attendance_config)

灵活的考勤参数配置 - 重复打卡间隔: 设置 0.1-24 小时的间隔时间 - 实时保存: 配置修改后立即生效 - 参数验证: 自动验证配置参数的有效性 - 默认设置: 提供合理的默认配置

1.5. □ Web 应用技术架构

1.5.1. 后端技术栈

- Web 框架: Flask 2.x
- 模板引擎: Jinja2
- 文件处理: Werkzeug
- 图像处理: OpenCV, PIL
- 深度学习: ONNX Runtime, ACL (Ascend)
- 数据存储: CSV 文件, JSON 配置

1.5.2. 前端技术栈

- UI 框架: Bootstrap 5.3
- 图标库: Font Awesome 6.0
- JavaScript: 原生 ES6+
- AJAX: Fetch API
- 响应式: CSS Grid + Flexbox
- 依赖安装: `pip install -r requirements.txt`
- 启动开发服务器: `python app.py`
- 访问: 浏览器打开 `http://localhost:5000`
- 主要页面与接口:
 - 页面:
 - * / 首页
 - * /recognize 图片识别
 - * /camera_live 摄像头实时识别
 - * /camera_register 摄像头注册
 - * /user_management 用户管理
 - * /attendance 考勤记录
 - * /attendance_config_page 考勤配置页面
 - API:
 - * POST /start_camera 启动摄像头
 - * POST /stop_camera 停止摄像头
 - * GET /video_feed 视频流
 - * GET /get_recognition_results 获取最新识别结果
 - * GET /get_users 获取用户列表
 - * GET/POST /attendance_config 获取/更新考勤配置

1.5.3. 用户手册

1. **硬件组装:** 参照2.1节的连接图连接好所有硬件。
2. **环境配置:** 在项目根目录执行 `pip install -r requirements.txt` 安装依赖。
3. **启动系统:** 运行`main.py`（脚本版）或运行`app.py`（Web 版）启动人脸识别打卡程序。
4. **查看记录与配置:** 打卡记录保存在项目根目录的`attendance.csv`；考勤配置保存在`attendance_config.json`，可在“考勤配置”页面修改再次打卡间隔（对应`ATTENDANCE_INTERVAL_HOURS`）。

1.5.4. 数据与文件存储位置

- 用户人脸数据: `datasets/<姓名>/` (每个用户一个文件夹, 存放多张人脸图片)
- 临时上传目录: `uploads/` (Web 版自动创建)
- 考勤记录: `attendance.csv` (项目根目录)
- 考勤配置: `attendance_config.json` (项目根目录)

说明: 上述文件名与路径由 `app.py` 中的常量 `ATTENDANCE_FILE` 与 `ATTENDANCE_CONFIG_FILE` 控制, 可按需调整。

1.5.5. 配置与参数

- `FACE_RECOGNITION_MODEL_PATH`: 识别模型路径 (默认 `models/mobilefacenet.om`)
- `DATASET_PATH`: 数据集目录 (默认 `datasets`)
- `SIMILARITY_THRESHOLD`: 识别相似度阈值 (默认 `0.7`)
- `ATTENDANCE_INTERVAL_HOURS`: 再次打卡间隔小时数 (默认 `8`)
- `UPLOAD_FOLDER`: 上传目录 (默认 `uploads`)
- `MAX_CONTENT_LENGTH`: 上传大小限制 (默认 `16MB`)

提示: 阈值过低可能导致误识别, 过高可能导致漏识别, 可结合实际数据调整。

1.6. 效果演示

(此处可插入系统运行时的截图或 GIF 动图, 例如摄像头实时识别人脸的画面)

1.7. 故障排除

- 摄像头无法启动: 检查设备连接与占用情况, 尝试重启应用; 确认访问 `/start_camera` 返回成功。
- 模型加载失败: 确认 `models/mobilefacenet.om` 是否存在且可读; 检查 `CANN/ACL` 环境变量与驱动安装。
- 识别效果不佳: 提高图像质量与采集数量; 适当调整 `SIMILARITY_THRESHOLD`; 保证注册图片为正脸、清晰光照。

- 无法打卡或间隔提示：查看 `attendance_config.json` 是否存在，或在“考勤配置”页面重设 `ATTENDANCE_INTERVAL_HOURS`。

1.8. 性能建议

- 将摄像头分辨率设置为 `640x480`（已在代码中默认设置）以降低处理压力。
- 在 Web 识别中使用识别冷却时间（代码默认 `2s`）减少重复计算与抖动。
- 数据集每人建议采集 `10+` 张不同角度与光照的人脸图片提升鲁棒性。

2. 案例 2：基于 MobileNet-SSD 与 DeepSORT 的目标跟踪教程

2.1. 教程定位

本案例面向昇腾 310B 平台的开发入门，目标不只是把程序跑起来，而是建立一条完整、清晰、可扩展的目标跟踪知识主线：

1. 为什么边缘设备上要优先选择轻量级检测网络。
2. MobileNet 的结构为什么适合做骨干网络。
3. SSD 的检测思想为什么适合实时场景。
4. MobileNet-SSD 作为检测前端，在目标跟踪任务中有哪些优点和局限。
5. 为什么在检测结果之上叠加 DeepSORT，是一个很好的多目标跟踪入门方案。
6. 本案例中的 `ssdlite` 与 `tracking` 代码，分别承担了什么角色。

从整体结构看，本案例对应一条标准的视频目标分析流水线：

本案例选择的实现路线是：

- 使用 MobileNet-SSD 完成实时目标检测。
- 使用一个简化版 DeepSORT 风格跟踪器完成多目标跟踪。
- 使用 CPU 与 Ascend NPU 统一接口，方便读者同时理解算法与部署。

这条路线适合作为昇腾 310B 开发教程中的案例章节，因为它兼顾了三点：

- 算法结构足够完整。
- 推理链路足够直观。
- 工程复杂度可控，适合实验训练和工程实践。

2.2. 实验硬件与运行条件

本案例既可以在普通 CPU 环境下运行，也可以在昇腾 NPU 环境下运行。为了完成实时检测与实时跟踪实验，建议准备如下硬件条件：

- 一台 Linux 主机或昇腾开发环境

基于 *MobileNet-SSD* 与 *DeepSORT* 的目标跟踪流程

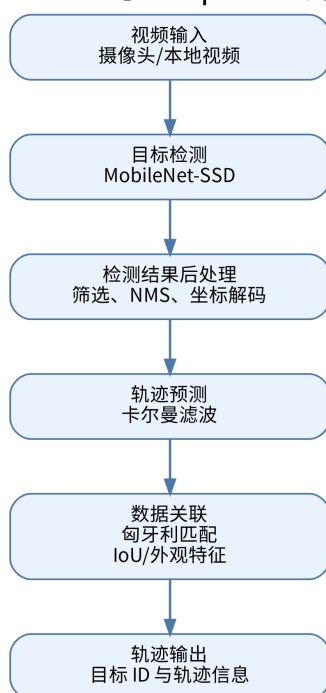


Figure 2.1.: 程序流程图

- 一只 USB 摄像头，用于实时视频采集
- Ascend 310B 或兼容昇腾 NPU 设备，运行 npu 模式时需要运行方式可以分为两类：
- 实时演示：接入 USB 摄像头，使用 `--source 0` 作为输入
- 离线演示：直接读取本地视频文件，例如 `--source demo.mp4`

如果只运行 cpu 模式，可以没有昇腾 NPU；如果运行 npu 模式，则还需要本机正确安装 Ascend ACL Python 运行时，并准备对应的 .om 模型文件。

2.3. 本案例支持的骨干网络

当前仓库里的 SSD 模型已经覆盖了两类主干网络：

- MobileNet 系列：mobilenetv1、mobilenetv2、mobilenetv3、mobilenetv4
- ResNet 系列：resnet18、resnet34、resnet50、resnet101、resnet151

其中：

- `ssd320_*` 主要对应 MobileNet 系列，输入尺寸通常为 320 x 320
- `ssd300_*` 主要对应 ResNet 系列，输入尺寸通常为 300 x 300

从工程角度看，MobileNet 系列更强调轻量化和实时性，ResNet 系列更强调特征表达能力和检测稳定性。两类骨干网络共同构成了本案例中“轻量实时”和“更强表达”两种不同取向的实验基础。

2.4. 目标跟踪对时序关联的要求

目标检测解决的是单帧图像中的两个问题：

- 目标在哪里。
- 目标属于什么类别。

例如，在一帧图像里，检测器可能输出三个框，其中两个是行人，一个是汽车。但仅靠检测器，我们无法回答下面这些时间维度上的问题：

- 当前这一帧左侧的行人，是不是上一帧的 3 号目标。
- 被遮挡后重新出现的车辆，是否还是之前那一辆。
- 两个目标交叉通过时，系统该如何保持它们的身份不交换。

上述问题正是目标跟踪所要解决的核心任务。多目标跟踪系统不仅需要检测目标的存在，还需要为同一目标在连续帧中维持稳定的身份编号，即稳定的 ID。

因此，目标跟踪可以理解为“目标检测在时间维度上的延伸”。检测负责看见目标，跟踪负责维持目标身份。

2.5. MobileNet-SSD 作为检测前端的依据

2.5.1. 边缘侧场景的基本要求

昇腾 310B 面向边缘智能计算场景。在这类场景中，算法设计通常要同时考虑以下问题：

- 实时性是否足够。
- 模型规模是否可控。
- 推理链路是否简单稳定。
- 是否便于部署到 NPU 或 CPU 回退环境。

如果直接选择参数量庞大、结构复杂、后处理繁重的检测器，理解成本与工程调试成本都会显著上升，系统主线反而不易把握。因此，本案例采用轻量、经典且工程路径清晰的 MobileNet-SSD 组合作为检测前端。

2.5.2. MobileNet 的网络结构

MobileNet 的核心设计思想，是用深度可分离卷积替代标准卷积，从而显著降低参数量与计算量。

标准卷积把空间卷积和通道融合同时完成，而 MobileNet 将其拆分成两步：

1. Depthwise Convolution：对每个输入通道分别做空间卷积。
2. Pointwise Convolution：使用 1×1 卷积完成通道之间的信息融合。

如果标准卷积的输入通道数为 M ，输出通道数为 N ，卷积核大小为 $K \times K$ ，特征图大小为 $D \times D$ ，那么标准卷积的计算量大致为：

$$D^2 \cdot K^2 \cdot M \cdot N$$

而深度可分离卷积的计算量大致为：

$$D^2 \cdot K^2 \cdot M + D^2 \cdot M \cdot N$$

当 $K = 3$ 时，这种拆分能够显著降低计算成本。这一特性对于边缘设备尤为重要。

从结构上看，MobileNet 可以理解为由以下几类模块反复堆叠而成：

- 普通卷积层，用于最初级的特征提取。
- 深度卷积层，用于逐通道提取空间信息。
- 逐点卷积层，用于跨通道融合特征。
- 下采样层，用于逐步扩大感受野，获得更强的语义信息。

MobileNet 的主要价值体现在以下几个方面：

- 它很好地体现了轻量化网络的设计思想。

- 它适合作为边缘侧模型部署的入门例子。
- 它与 SSD 结合后，能够形成一条结构清晰、速度较快的检测链路。

2.5.3. MobileNet v1 至 v4 的版本演进

MobileNet 并非单一网络，而是一个持续演进的轻量级网络家族。v1 至 v4 的演进，本质上体现为在移动端与边缘侧约束下，对精度、速度与结构表达能力的持续平衡与优化。

MobileNet v1

MobileNet v1 是这个家族的起点。它最核心的贡献，就是把深度可分离卷积系统化地应用到整个网络主干中。

结构特点：

- 以 depthwise convolution 加 pointwise convolution 为基本单元
- 网络结构规整，堆叠方式简单直接
- 参数量和计算量相比标准卷积网络大幅下降

优点：

- 结构最直观，最容易理解
- 轻量化效果明显
- 在边缘设备上容易部署

局限：

- 特征表达能力相对有限
- 在相同算力下，精度通常不如后续版本

MobileNet v2

MobileNet v2 的核心改进是引入了 inverted residual 和 linear bottleneck。

结构特点：

- 先通过 1×1 卷积扩张通道
- 再做 depthwise convolution
- 最后通过线性 bottleneck 压缩回较低维度
- 在满足条件时使用残差连接

它与传统残差块的设计思路存在明显差异。传统残差块通常采用“宽输入到宽输出”的结构，而 MobileNet v2 更强调“窄输入、宽中间层、窄输出”，因此被称为 inverted residual。

这一版本的优势在于：

- 在轻量化前提下提升了表达能力
- 残差连接帮助梯度传播更稳定
- 在线性 bottleneck 中减少非线性带来的信息损失

MobileNet v3

MobileNet v3 在 v2 基础上继续优化，它融合了神经网络结构搜索和轻量注意力机制的思想。

结构特点：

- 延续 inverted residual 主体框架
- 引入 SE 模块增强通道注意力
- 使用 h-swish 等更适合移动端的激活函数
- 网络结构由精度与延迟共同驱动优化

与 v2 相比，v3 的一个重要特征在于更加关注真实硬件上的执行效率，而不仅仅是理论 FLOPs 的下降。换言之，v3 的优化目标并非单纯减少计算量，而是在移动端与边缘设备上取得更优的实际速度与精度平衡。

MobileNet v4

MobileNet v4 是更后续的轻量骨干版本，它进一步强化了硬件感知设计，更强调不同算子组合在实际设备上的性能收益。

结构特点：

- 延续轻量主干的整体方向
- 更强调不同卷积模块和块结构的组合效率
- 进一步面向硬件友好的延迟优化
- 在不同配置下兼顾速度、参数量和表达能力

从本仓库的模型组织可以看出，模型目录中既包含 mobilenetv4，也包含 mobilenetv4_conv_large → .onnx 这样的变体文件。这表明 v4 系列本身已经具备不同配置路径，用于在更强表达能力与更低计算开销之间进行权衡。

2.5.4. MobileNet v1-v4 的结构对比总结

如果按结构演进主线来总结，四个版本的差异可以概括为：

- v1：用深度可分离卷积解决“怎么把网络做轻”
- v2：用 inverted residual 和 linear bottleneck 解决“轻量化下如何保留表达能力”

- v3: 用 SE、h-swish 和结构搜索解决“如何进一步优化真实设备上的速度与精度平衡”
 - v4: 进一步走向硬件感知和模块组合优化，强调在实际部署环境中的综合收益
- 如果按本案例的应用视角看，这些版本的差异意味着：
- v1 更适合讲轻量化网络的最基础结构
 - v2 更适合讲轻量网络中的残差和瓶颈设计
 - v3 更适合讲轻量网络与注意力机制、激活函数优化的结合
 - v4 更适合讲面向部署性能的后继演化方向

2.5.5. MobileNet 与 ResNet 骨干在本案例中的取向差异

本案例同时提供 MobileNet 和 ResNet 两条骨干路线，这样可以更直观地对比不同网络家族的使用取向。

MobileNet 系列的特点是：

- 参数量更小
- 推理速度更快
- 更适合实时演示和边缘部署

ResNet 系列的特点是：

- 特征提取能力更强
- 网络层次更深，残差结构更成熟
- 在一些场景下更容易得到稳定的检测效果

因此，在本案例中：

- 如果更关注实时性和轻量部署，可以优先尝试 MobileNet v1-v4
- 如果更关注骨干网络深度和表达能力，可以尝试 ResNet18 到 ResNet151 的多个版本

2.5.6. SSD 的检测特点

SSD 是 Single Shot MultiBox Detector 的缩写，属于典型的一阶段检测器。它的核心特点是：不需要像两阶段检测器那样先生成候选区域，再做分类与回归，而是直接在不同尺度的特征图上预测目标位置和类别。

SSD 的关键思想包括：

- 在多个尺度特征图上进行检测，兼顾大目标和小目标。
- 在每个位置预先定义一组 default boxes，也叫 prior boxes。
- 网络直接回归 default box 的位置偏移量，同时预测各类别分数。

- 经过解码和 NMS 后，输出最终的检测框。

SSD 的优点主要有：

- 推理速度快，适合实时场景。
- 结构相对直接，便于理解和部署。
- 检测头设计清晰，容易解释“分类”和“回归”两个分支的作用。

SSD 的局限也需要明确说明：

- 对小目标和密集目标场景通常不如更新的检测器稳定。
- 检测效果较依赖先验框设计与输入尺寸。
- 在复杂遮挡场景下，单帧检测质量会波动，从而影响后续跟踪。

2.5.7. MobileNet-SSD 用于目标跟踪的优缺点

在目标跟踪任务中，检测器不是孤立存在的，它会直接影响轨迹质量。MobileNet-SSD 作为跟踪前端，有以下优点：

- 轻量高效，适合边缘侧实时处理。
- 输出结构标准，便于接入后续跟踪器。
- 检测速度较快，可以为多目标跟踪提供稳定的帧级观测。
- 工程复杂度低，便于构建完整案例。

但它也存在明显不足：

- 对小目标和远距离目标的检测精度有限。
- 遇到密集遮挡时，漏检和框抖动会增多。
- 当检测框位置不稳定时，跟踪器更容易发生 ID switch。
- 仅靠检测框几何信息，难以应对长时间遮挡或重识别问题。

因此，MobileNet-SSD 的价值不在于“它是最强的检测前端”，而在于“它能以较低复杂度，把检测到跟踪这条主线讲清楚”。

2.6. DeepSORT 作为跟踪后端的选择依据

2.6.1. 从检测到跟踪，还缺少什么

检测器每一帧都输出一组目标框，但这些框之间没有时间关联。要把检测结果变成稳定轨迹，还需要以下机制：

- 轨迹状态表示。
- 帧间运动预测。
- 检测与轨迹之间的匹配。

- 轨迹的创建、保留和删除。

DeepSORT 恰好提供了这样一套结构完整的解决框架。

2.6.2. DeepSORT 为什么适合作为入门算法

完整的 DeepSORT 在 SORT 基础上增加了外观特征建模，因此在目标遮挡、交叉和重识别问题上通常具有更好的稳定性。尽管本案例并未实现完整的工业级 DeepSORT，但保留了其最核心、最适合入门阶段把握的基本思路：

- 用卡尔曼滤波做运动预测。
- 用匈牙利算法做全局匹配。
- 用 IOU 作为主要几何相似度。
- 用轨迹生命周期参数管理目标的出现与消失。

这种简化版方案有三个突出优点：

- 思路完整，已经覆盖多目标跟踪中的核心环节。
- 数学和工程难度适中，不需要先掌握 ReID 网络训练。
- 便于直接观察检测质量、IOU 阈值、轨迹寿命等因素对最终效果的影响。

因此，将 MobileNet-SSD 与 DeepSORT 组合使用，是一条结构清晰、实现成本适中且便于展开分析的入门路径。

2.7. 本案例的代码结构与总体流程

本案例的工程代码主要分为三层：

2.7.1. 入口层

- demo/detection_app.py：只做实时检测。
- demo/tracking_app.py：先做检测，再做跟踪。

2.7.2. 检测层

- ssdlite/backend_base.py：统一检测后端基类，负责预处理、推理调度与输出解码。
- ssdlite/cpu_backend.py：ONNXRuntime CPU 推理实现。
- ssdlite/npu_backend.py：Ascend ACL NPU 推理实现。
- ssdlite/decoder.py：SSD prior boxes、位置解码、类别概率与 NMS。

2.7.3. 跟踪层

- tracking/deepsort.py: 轨迹对象、匹配逻辑、轨迹生命周期管理。
- tracking/kalman_filter.py: 卡尔曼滤波状态预测与观测更新。
- utils/postprocessing.py: 检测结果转换为跟踪器输入，并将轨迹画回图像。

整条链路的执行顺序可以概括为：

1. 读取摄像头或视频流。
2. 对输入帧执行 MobileNet-SSD 检测。
3. 解码输出张量并得到边界框、类别和分数。
4. 将检测结果整理成跟踪器统一输入格式。
5. 对已有轨迹进行预测。
6. 将当前检测结果与历史轨迹做关联。
7. 更新匹配轨迹、创建新轨迹、删除失效轨迹。
8. 在图像上绘制检测结果或轨迹结果。

2.8. 检测部分代码解析

2.8.1. detection_app.py 如何组织检测主流程

在 demo/detection_app.py 中，主程序首先解析参数，然后根据 device 选择后端，再进入逐帧推理循环。这一设计体现了很清晰的工程结构：入口脚本只负责流程编排，把模型相关细节下沉到 ssdlite 模块。

主流程的示意代码如下：

```

1 labels = load_labels(args.labels)
2 model_path = resolve_model_path(args.model, args.backbone, model_dir,
  ↳ args.device)
3 backend = CpuBackend(model_path) or NpuBackend(model_path)
4
5 while True:
6     ok, frame = cap.read()
7     detections, profile_ms = backend.infer_with_profile(
8         frame,
9         args.score_threshold,
10        args.nms_threshold,
11        args.max_detections,
12    )
13    annotated = draw_detections(...)

```

这里需要关注的重点并非某一行语法细节，而是模块职责的划分方式：

- 入口脚本负责参数、视频流、显示和保存。

- backend 负责模型推理。
- decoder 负责把原始张量变成检测框。
- postprocessing 负责可视化。

这是一种很清晰的工程拆分方式。

2.8.2. backend_base.py 如何统一检测后端

ssdlite/backend_base.py 是检测部分最核心的代码之一。它把 CPU 与 NPU 两种后端统一到同一个抽象接口下。这样设计后，入口脚本就不用关心底层到底是 ONNXRuntime 还是 ACL。

这个基类做了三件关键事情：

- 根据模型名推断输入尺寸。
- 统一图像预处理。
- 统一输出解码流程。

其中，infer_with_profile 的核心逻辑如下：

```
1 input_tensor = preprocess_frame(frame, self.input_hw)
2 outputs = self._run_model(input_tensor)
3 detections = decode_detections(
4     outputs,
5     frame.shape,
6     score_threshold,
7     nms_threshold,
8     max_detections,
9     self.ssd_decoder,
10    self.strict_ssd,
11 )
```

这段代码清晰地对应了检测系统中的三个阶段：

1. 预处理：把原始图像缩放、归一化，并变成模型要求的 NCHW 张量。
2. 推理：调用不同后端执行前向计算。
3. 解码：把输出张量还原成真实的目标框与类别分数。

这里需要明确的一点是，模型推理本身并不等于最终检测结果。真正可用于后续跟踪的目标框，需要由模型输出经过解码与 NMS 后处理后才能得到。

2.8.3. preprocess_frame 体现了部署输入规范

在 backend_base.py 中，preprocess_frame 完成了以下操作：

- BGR 转 RGB。
- resize 到固定输入尺寸。

- 像素值归一化到 0 到 1。
- 按 ImageNet 风格均值与方差做标准化。
- 从 HWC 转为 CHW。
- 增加 batch 维度。

这说明了一个重要的部署原则：模型输入的预处理流程必须与训练阶段的约定保持一致，否则推理结果将发生明显偏移。

2.8.4. decoder.py 如何体现 SSD 的核心思想

ssdlite/decoder.py 直接对应了 SSD 的理论结构，是整条检测链路中的关键部分。

default boxes

在 DefaultBoxes 类中，代码根据不同特征图尺度和长宽比生成先验框。这正是 SSD 的基础：网络并不是直接从零开始生成框，而是在预定义框基础上学习偏移量。

位置回归的反变换

在 SSDDecoder.scale_back_batch 中，网络输出的并不是最终框坐标，而是相对于默认框的偏移量。代码中先利用 scale_xy 和 scale_wh 对偏移量进行还原，再把中心点形式转换为左上角和右下角坐标。

更直接地说：

- 网络先预测相对于先验框的中心偏移和宽高缩放。
- 解码阶段再把这些相对量恢复成图像上的实际边界框。

分类分数与 NMS

在 decode_single 中，代码对每个类别分别进行处理：

```
1 for class_id in range(1, scores_in.shape[1]):  
2     class_scores = scores_in[:, class_id]  
3     keep_score = class_scores > 0.05  
4     keep_nms = _nms(class_boxes, class_scores, criteria)
```

这段代码体现了 SSD 后处理的两个关键步骤：

- 先按类别分数进行阈值筛选。
- 再对同一类别的框做 NMS，去掉大量重叠冗余框。

如果没有这一步，检测器往往会输出大量相互重叠的候选框，无法直接作为跟踪器输入。

NMS 的作用是什么

NMS 是 Non-Maximum Suppression，即非极大值抑制。它的核心作用可以概括为一句话：当多个候选框同时指向同一个目标时，只保留其中最可信的候选框，并删除其余高度重叠的框。

这是因为 SSD 会在每个特征图位置、每种先验框形状上产生候选框。对于一个真实目标，模型往往不会只输出一个框，而是会输出一组位置接近、分数相近的候选框。如果不执行 NMS，画面中就会出现大量相互覆盖的检测框，例如同行人周围同时叠加 5 个到 10 个候选框。

NMS 的作用主要有四点：

- 去除重复检测，让一个目标尽量只保留一个主框。
- 降低后续显示和统计的混乱程度。
- 为跟踪器提供更稳定、更稀疏的观测输入。
- 避免同一目标在跟踪阶段被误认为多个目标。

对本案例尤其要强调最后一点。如果检测器把同一目标输出成多个高重叠框，那么 tracking 模块在创建轨迹时，可能会把这些框错误地当成多个独立目标，从而造成重复轨迹、ID 混乱和后续匹配不稳定。

NMS 的处理逻辑可以概括成下面 4 步：

1. 按置信度从高到低排序所有候选框。
2. 取当前分数最高的框作为保留框。
3. 计算其余框与该保留框的 IOU。
4. 删除所有 IOU 超过阈值的框，然后继续处理剩余框。

因此，NMS 本质上是在做“去重”。它不是提高分类能力，而是在后处理阶段把重复候选框压缩成更干净的检测结果。

ssdlite 中 _nms 的实现解析

ssdlite/decoder.py 中实现的 NMS 函数如下：

```
1 def _nms(bboxes, scores, iou_threshold):
2     if len(bboxes) == 0:
3         return np.empty((0, ), dtype=np.int64)
4
5     order = np.argsort(scores)[::-1]
6     keep = []
7     while order.size > 0:
8         index = order[0]
9         keep.append(index)
10        if order.size == 1:
11            break
```

```

12     iou = calc_iou(boxes[order[1:]], boxes[index].reshape(1, 4)).
    ↪ reshape(-1)
13     order = order[np.where(iou <= iou_threshold)[0] + 1]
14     return np.array(keep, dtype=np.int64)

```

这段实现虽然短，但几乎完整体现了经典 NMS 的核心流程。

第一步，处理空输入：

```

1 if len(boxes) == 0:
2     return np.empty((0,), dtype=np.int64)

```

这一步是工程上必不可少的防御式处理。如果当前类别一个候选框都没有，就直接返回空索引，避免后续排序和 IOU 计算报错。

第二步，按照分数降序排列：

```

1 order = np.argsort(scores)[::-1]

```

`np.argsort(scores)` 会返回分数从小到大排列后的索引，`[::-1]` 再把顺序翻转成从大到小。这样一来，`order[0]` 就总是当前剩余候选框中分数最高的那个。

第三步，循环保留最高分框：

```

1 keep = []
2 while order.size > 0:
3     index = order[0]
4     keep.append(index)

```

这里的含义至关重要：每一轮循环都将当前最高分框作为“代表框”保留下来。NMS 的基本原则是，在一组高度重叠的候选框中，分数最高的框通常最应优先保留。

第四步，只剩一个框时直接结束：

```

1 if order.size == 1:
2     break

```

如果当前只剩一个候选框，那么它已经被保留，没有必要再做 IOU 比较。

第五步，计算其余候选框与当前最高分框的 IOU：

```

1 iou = calc_iou(boxes[order[1:]], boxes[index].reshape(1, 4)).reshape(-1)

```

这一步可以这样理解：

- `boxes[index]` 是当前保留框。
- `boxes[order[1:]]` 是除了它以外剩余的候选框。
- `calc_iou(...)` 计算这些候选框与当前保留框之间的重叠程度。

如果某个候选框与保留框的 IOU 很高，就说明这两个框大概率描述的是同一个目标，此时就没有必要同时保留。

第六步，删除 IOU 过高的重复框：

```
1 order = order[np.where(iou <= iou_threshold)[0] + 1]
```

这一行代码是整个 NMS 实现中最关键的语句之一。

- `iou <= iou_threshold` 表示只保留那些与当前主框重叠不太严重的候选框。
- `np.where(...)` 得到满足条件的索引位置。
- 之所以要 `+ 1`，是因为前面计算 IOU 时使用的是 `order[1:]`，也就是跳过了当前最高分框，所以这里需要把索引映射回原来的 `order` 序列。

执行完这一行后，所有与当前主框高度重叠的框都会被移除，剩下的框继续进入下一轮循环。

第七步，返回保留框索引：

```
1 return np.array(keep, dtype=np.int64)
```

最终返回的不是框本身，而是被保留框在原数组中的索引。后续代码再用这些索引去提取真正的框、分数和类别。

calc_iou 为什么是 NMS 的基础

NMS 能否正确工作，取决于 IOU 计算是否准确。本案例在 `decoder.py` 中通过 `calc_iou` 计算两个框集合之间的交并比。其核心思想是：

$$IOU = \frac{\text{交集面积}}{\text{并集面积}}$$

在代码里，这一过程分为三步：

- 先求两个框相交区域的左上角和右下角。
- 再计算交集面积。
- 最后用交集面积除以并集面积。

如果 IOU 接近 1，说明两个框几乎重合；如果 IOU 接近 0，说明两个框几乎没有重叠。NMS 正是利用这一数值来判断“两个候选框是不是在描述同一个目标”。

NMS 阈值对结果的影响

NMS 中最重要的超参数是 `iou_threshold`。它决定了“多大程度的重叠应被视为重复”。

如果阈值较小，例如 0.3，那么算法会更严格：

- 只要两个框稍微重叠得多一点，就可能删除其中一个。
- 输出框更少，更干净。
- 但也可能误删本来相邻但不是同一目标的框。

如果阈值较大，例如 0.6 或 0.7，那么算法会更宽松：

- 允许更多重叠框同时保留。
- 对密集目标场景可能更友好。
- 但重复检测也更容易残留。

因此，NMS 阈值本质上是在“去重强度”和“保留相邻目标”之间做权衡。

NMS 对跟踪效果的直接影响

NMS 容易被视为检测模块内部的技术细节，但在目标跟踪系统中，它实际上会直接影响跟踪质量。

原因是跟踪器把检测器输出当作观测输入。如果检测结果中存在大量重复框，就会产生以下问题：

- 同一目标可能在同一帧中生成多条新轨迹。
- 轨迹关联时会出现多对一竞争。
- 轨迹数目虚高，画面看起来混乱。
- 目标 ID 容易抖动，甚至频繁切换。

从这个角度看，NMS 不是孤立的后处理技巧，而是整个“检测到跟踪”流水线稳定运行的重要保障。

decode_single 中 NMS 的完整上下文

在本案例中，NMS 不是单独执行的，而是被放在 `decode_single` 的类别循环中。其完整语义是：

1. 针对每一个类别单独处理。
2. 先按分数过滤掉置信度过低的框。
3. 再保留该类别中得分最高的一部分候选框。
4. 最后在这个类别内部做 NMS。

这意味着本案例采用的是“按类别分别做 NMS”的策略。这样设计的优点是：不同类别之间不会互相抑制。例如一个行人框和一个自行车框即使重叠较大，也不应该因为彼此重叠就被删除。

2.8.5. CPU 与 NPU 后端如何被统一接入

本案例中的 `ssdlite/cpu_backend.py` 与 `ssdlite/npu_backend.py` 都继承自 `Detection-Backend`，这意味着它们对外暴露的是同一套接口。这种设计有两层价值：

- 从算法角度看，检测逻辑不依赖具体硬件后端。
- 从工程角度看，同一套上层代码可以在 CPU 和 Ascend NPU 之间切换。

这也体现了一个重要的工程原则：算法主线尽量稳定，硬件适配尽量下沉到后端实现层。

2.9. 从检测到跟踪：tracking_app.py 的桥梁作用

如果说 detection_app.py 用于建立检测流程，那么 tracking_app.py 的作用就在于将“检测结果如何转化为轨迹”这一过程完整串联起来。

在 demo/tracking_app.py 中，检测模块与跟踪模块之间的桥接代码如下：

```
1 detections, profile_ms = backend.infer_with_profile(...)
2 tracker_inputs = detections_to_tracker_inputs(detections)
3 tracks = tracker.update(tracker_inputs)
4 annotated = draw_tracks(...)
```

这一过程可以分为四个连续步骤：

1. 先由检测器输出本帧的目标集合。
2. 再把字典形式的检测结果转成跟踪器统一使用的数组格式。
3. 调用跟踪器更新轨迹状态。
4. 把最新轨迹结果绘制到图像上。

其中，utils/postprocessing.py 中的 detections_to_tracker_inputs 函数具有关键作用，因为它完成了检测模块与跟踪模块之间的数据接口统一。转换后的输入格式为：

$$[x_1, y_1, x_2, y_2, score, class_id]$$

这正是后续轨迹匹配与分类约束所依赖的输入格式。

2.10. 跟踪部分代码解析

2.10.1. Track 类如何表示一条轨迹

tracking/deepsort.py 中的 Track 类代表单个目标的历史状态。每条轨迹至少维护了以下信息：

- track_id：目标编号。
- bbox：当前边界框。
- score：当前检测置信度。
- class_id：目标类别。
- trail：历史中心点序列，用于绘制轨迹拖尾。
- kalman_filter：用于状态预测与更新。

- `time_since_update`: 距离上次匹配成功已经过去多少帧。
- `hits`: 这条轨迹累计被成功匹配了多少次。

这表明轨迹并不是静态边界框的简单记录，而是一个具有时间属性的状态对象。因此，“检测框”和“轨迹”必须明确区分。

2.10.2. predict 和 update 对应卡尔曼滤波两阶段

Track 类中最核心的两个方法是 `predict` 和 `update`。

`predict` 的作用是：即使当前帧还没有拿到新的检测结果，也先根据上一时刻状态预测目标现在大概会出现在哪里。

`update` 的作用是：一旦当前帧有新的检测框匹配到这条轨迹，就用新的观测值修正状态估计。

这正对应卡尔曼滤波的两步：

1. **Predict**: 根据上一时刻状态外推当前状态。
2. **Update**: 用当前观测修正预测误差。

本案例里，状态量采用了一个比较直观的简化形式：

$$x = [x, y, v_x, v_y]^T$$

其中 x, y 表示目标中心位置， v_x, v_y 表示速度。这样做的好处是：数学含义直观，代码实现也容易读懂。

2.10.3. kalman_filter.py 如何实现线性状态估计

`tracking/kalman_filter.py` 中定义了最基础的卡尔曼滤波器。其状态转移矩阵为：

$$F = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

这表示系统假设目标在相邻帧之间近似满足匀速运动。观测矩阵为：

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

这表示我们能够直接观测到的是目标中心位置，而不是速度。

这一实现虽然经过简化，但其价值十分明确，因为它将“运动预测”与“观测修正”这两个概念直接落实到了矩阵运算之中。

2.10.4. 卡尔曼滤波器的发展历史与作者信息

卡尔曼滤波器的提出者是鲁道夫·埃米尔·卡尔曼，英文名 Rudolf Emil Kalman。他是匈牙利裔美国数学家和控制理论学者，1960 年发表了著名论文 A New Approach to Linear Filtering and Prediction Problems，系统建立了线性动态系统状态估计的现代形式。

从历史背景看，卡尔曼滤波器并不是突然出现的。它的形成与 20 世纪中叶的控制理论、随机过程、航空航天导航和军事工程需求密切相关。第二次世界大战后，工程界越来越需要解决这样一类问题：

- 系统状态不能被直接完整观测。
- 传感器观测存在噪声。
- 系统本身还在不断运动和变化。

例如，雷达跟踪飞机、导弹制导、航天器轨道估计，本质上都属于这一类问题。

卡尔曼滤波器之所以具有里程碑意义，是因为它把“预测”和“校正”统一进了一个递推框架中：

- 先根据上一时刻状态和系统模型预测当前状态。
- 再根据当前观测对预测结果进行修正。

这一思想后来在 20 世纪 60 年代迅速应用到阿波罗计划的导航系统中，也因此被广泛传播到航空航天、自动控制、机器人、计算机视觉和金融工程等领域。

在发展过程中，卡尔曼滤波形成了多个重要分支：

- 标准卡尔曼滤波：用于线性高斯系统。
- 扩展卡尔曼滤波 EKF：用于非线性系统的一阶近似。
- 无迹卡尔曼滤波 UKF：通过采样点传播非线性分布。
- 粒子滤波：进一步放宽线性和高斯假设。

本案例采用最基础的标准卡尔曼滤波器。采用这一形式，并非因为它代表当前最复杂的状态估计方案，而是因为它在入门阶段具有最清晰的解释路径：

- 数学形式清晰。
- 与目标跟踪中的位置预测问题高度匹配。
- 代码实现短小，便于直接对应矩阵运算理解。

2.10.5. 卡尔曼滤波器到底在解决什么问题

在目标跟踪里，卡尔曼滤波器要解决的问题可以表述为：当检测器每一帧给出的目标框存在抖动、漏检和噪声时，如何根据历史状态更平滑地估计目标当前位置，并在短时没有观测的情况下继续维持轨迹。

这意味着卡尔曼滤波并不是在做“目标分类”，也不是在做“目标匹配”，它做的是状态估计。

在本案例里，这个状态估计问题被简化成了二维平面上的匀速运动模型：

$$x_k = Fx_{k-1} + w_k$$

$$z_k = Hx_k + v_k$$

其中：

- x_k 表示第 k 帧的隐藏状态，也就是目标真实但不可完全直接观测的状态。
- z_k 表示第 k 帧的观测，也就是检测器给出的目标中心位置。
- F 是状态转移矩阵。
- H 是观测矩阵。
- w_k 是过程噪声。
- v_k 是观测噪声。

从工程角度看，这个模型表达的是一个朴素但有效的假设：目标会以相对平滑的方式运动，而检测器给出的观测值只是对真实运动状态的带噪声测量。

2.10.6. 本案例中的 predict 与 update 如何对应算法公式

tracking/kalman_filter.py 中的两个核心函数正好对应卡尔曼滤波的两个阶段。

第一阶段是预测：

```
1 def predict(self):
2     self.x = np.dot(self.F, self.x)
3     self.P = np.dot(np.dot(self.F, self.P), self.F.T) + self.Q
4     return self.x
```

该阶段完成以下两项操作：

- 用状态转移矩阵 F 预测新的状态向量 x 。
- 用协方差传播公式预测新的不确定性 P 。

第二阶段是更新：

```
1 def update(self, z):
2     y = z - np.dot(self.H, self.x)
3     S = np.dot(self.H, np.dot(self.P, self.H.T)) + self.R
4     K = np.dot(np.dot(self.P, self.H.T), np.linalg.inv(S))
5     self.x = self.x + np.dot(K, y)
6     self.P = self.P - np.dot(np.dot(K, self.H), self.P)
```

这段代码中几个量的意义需要讲清楚：

- y 是 innovation，也叫残差，表示“观测值与预测值之间的差”。
- S 是残差协方差，表示当前误差的不确定性。
- K 是卡尔曼增益，决定“应该更相信模型预测还是更相信当前观测”。

这里最重要的量是卡尔曼增益 K 。它并非经验性设定的固定权重，而是由当前状态不确定性与观测噪声共同决定的自适应权重。

可以直观地理解为：

- 如果观测噪声很大，滤波器会更信任模型预测。
- 如果观测噪声较小，滤波器会更信任当前检测结果。

这也是卡尔曼滤波器的重要优势之一。它并不是简单的滑动平均，而是在概率意义上进行最优线性估计。

2.10.7. 卡尔曼滤波在本案例中的适用性

对本案例而言，卡尔曼滤波器之所以合适，并不在于目标运动一定严格符合线性高斯模型，而在于它在工程上实现了较好的综合平衡：

- 算法复杂度低。
- 推理和更新速度快。
- 能明显改善检测框抖动。
- 在短时遮挡和漏检场景下能维持轨迹连续。

如果在入门阶段直接引入 EKF、UKF 或粒子滤波，虽然理论上能够处理更复杂的运动模型，但理解成本会显著上升，主线结构反而更不易把握。因此，本案例采用标准卡尔曼滤波具有充分的合理性。

2.10.8. DeepSORT.update 如何体现多目标跟踪主线

tracking/deepsort.py 中的 DeepSORT.update 是整条跟踪主线最核心的函数，其执行顺序如下：

```

1 for track in self.tracks:
2     track.predict()
3
4 matched, unmatched_detections, _ = self._associate(detections)
5
6 for track_idx, detection_idx in matched:
7     self.tracks[track_idx].update(detections[detection_idx])
8
9 for detection_idx in unmatched_detections:
10    self._create_track(detections[detection_idx])
11
12 self.tracks = [track for track in self.tracks if track.time_since_update
    ↪ <= self.max_age]
```

这一段代码几乎完整体现了多目标跟踪的标准流程：

1. 对所有旧轨迹做预测。
2. 把当前检测结果与旧轨迹做关联。
3. 用匹配到的检测结果更新旧轨迹。
4. 对未匹配检测创建新轨迹。
5. 删除长期未更新的轨迹。

换言之，跟踪并不是对检测框进行简单编号，而是在连续帧之间持续执行预测、匹配、更新与清理。

2.10.9. 数据关联为什么使用 IOU + 匈牙利算法

在 `_associate` 中，本案例先构造轨迹与检测之间的 IOU 矩阵，再调用匈牙利算法获得全局最优分配。对应代码思路如下：

```
1 iou_matrix = self._calculate_iou_matrix(detections)
2 matched_indices = self._linear_assignment(iou_matrix)
```

之所以这样设计，是因为当场景中有多条轨迹和多个检测框时，不能只做局部贪心匹配。匈牙利算法的作用在于：

- 从全局角度寻找一一对应的最优匹配。
- 避免一个检测框同时分配给多条轨迹。
- 避免一条轨迹同时匹配多个检测框。

本案例还加入了类别兼容性判断：

```
1 if not self._is_class_compatible(track.class_id, int(detection[5])):
2     continue
```

这说明系统不仅看几何位置，还利用类别信息避免“行人轨迹匹配到车辆框”这类明显错误。

2.10.10. 匈牙利算法的发展历史与作者信息

匈牙利算法是解决二分图最优匹配和指派问题的经典算法。它最早由美国数学家哈罗德·W·库恩，英文名 Harold W. Kuhn，在 1955 年系统提出。之所以命名为“匈牙利算法”，是因为库恩的工作建立在两位匈牙利数学家的理论成果之上：

- 德奈什·克尼格，英文名 Dénes Kőnig。
- 耶诺·埃格瓦里，英文名 Jenő Egerváry。

他们在图论和指派问题上的研究，为后来算法化求解最优匹配提供了重要理论基础。之后，美国数学家詹姆斯·芒克雷，英文名 James Munkres，在 1957 年进一步

改进了该算法，使其在计算复杂度和工程实现上更加完善。因此，这一算法也常被称为 Kuhn-Munkres 算法。

从历史背景看，匈牙利算法主要服务于一类经典的组合优化问题：

- 一组任务要分配给一组工人。
- 每一种分配都有代价或收益。
- 目标是在一一对应约束下，使总代价最小或总收益最大。

这个问题后来被称为 **assignment problem**，即指派问题。它在生产调度、资源分配、路径规划、视觉跟踪等场景中都较为常见。

在多目标跟踪中，匈牙利算法的角色就是把“轨迹”和“当前检测框”看成两侧节点，把相似度或代价矩阵看成边权，然后求出全局最优的一一匹配关系。

2.10.11. 匈牙利算法到底解决什么问题

在目标跟踪里，如果只有一条轨迹和一个检测框，那么匹配很简单。但一旦场景中存在多条轨迹和多个检测框，问题就会迅速复杂起来。

例如，假设当前有 3 条旧轨迹和 3 个新检测框。此时系统不能只看“哪两个最像就先配对”，因为局部最优未必能得到全局最优。匈牙利算法解决的就是这个问题：

- 它在满足一一对应约束的前提下，寻找整体代价最小或整体相似度最大的匹配方案。

这正是多目标跟踪里数据关联的核心。

2.10.12. 本案例中匈牙利算法是如何被调用的

在 `tracking/deepsort.py` 中，匈牙利算法的调用被封装在 `_linear_assignment` 中：

```
1 def _linear_assignment(self, cost_matrix):
2     if cost_matrix.size == 0:
3         return np.empty((0, 2), dtype=np.int64)
4
5     row_ind, col_ind = linear_sum_assignment(-cost_matrix)
6     return np.array(list(zip(row_ind, col_ind)), dtype=np.int64)
```

这里调用的是 `scipy.optimize.linear_sum_assignment`，它是匈牙利算法在科学计算库中的标准实现。

该函数默认求解最小化问题，而本案例构造的是 IOU 相似度矩阵，IOU 越大表示匹配越优。因此，代码中采用了一个典型的等价变换：

```
1 linear_sum_assignment(-cost_matrix)
```

即将“最大化相似度”转化为“最小化负相似度”。

这里需要强调的一点是，同一个优化算法既可以用于最小代价问题，也可以通过简单变换用于最大收益问题。

2.10.13. 匈牙利算法在本案例中的完整上下文

在 `_associate` 中，数据关联的整体过程是：

1. 先计算每一条轨迹与每一个检测框之间的 IOU。
2. 用这些 IOU 构造一个矩阵。
3. 调用匈牙利算法得到全局最优分配。
4. 再结合 IOU 阈值和类别兼容条件过滤不合理匹配。

对应代码主线是：

```
1 iou_matrix = self._calculate_iou_matrix(detections)
2 matched_indices = self._linear_assignment(iou_matrix)
3
4 for track_idx, detection_idx in matched_indices:
5     if iou_matrix[track_idx, detection_idx] < self.iou_threshold:
6         continue
7     if not self._is_class_compatible(...):
8         continue
```

这段代码说明一个重要事实：匈牙利算法本身只负责“给出最优一一分配”，但它并不直接理解哪些匹配在语义上是合理的。因此，本案例又叠加了两层约束：

- IOU 必须高于阈值。
- 类别必须兼容。

由此可见，匈牙利算法提供的是全局分配框架，而不是完整的业务规则集合。

2.10.14. 匈牙利算法相对于贪心匹配的优势

一个常见问题在于，为什么不直接选择当前最大 IOU 的一对进行匹配。该贪心策略看似简单，但存在明显局限：

- 先做出的局部选择可能破坏后续整体最优。
- 当多个轨迹都与同一个检测框接近时，容易造成冲突。
- 在目标密集场景下，局部贪心更容易导致 ID 交换。

匈牙利算法的价值就在于它能从全局角度处理这类一一匹配问题。虽然本案例的匹配度量比较简单，只用了 IOU，但只要场景中存在多目标竞争，使用全局分配通常都比局部贪心更稳健。

2.10.15. 匈牙利算法在多目标跟踪中的局限

需要说明的是，匈牙利算法并不是万能的。它解决的是“在给定代价矩阵时，如何求最优匹配”，但如果代价矩阵本身构造得不好，算法仍然可能得到错误匹配。

在本案例中，代价矩阵主要由 IOU 构成，因此它会受到以下问题影响：

- 快速运动目标与旧轨迹重叠变小，IOU 不足。
- 相邻目标位置太近，几何信息不够区分。
- 遮挡后重新出现时，仅靠 IOU 难以恢复原身份。

这也是为什么完整 DeepSORT 往往还会引入外观特征和更复杂的关联策略。匈牙利算法负责的是“最优分配”，但“什么样的代价矩阵才合理”仍然是跟踪算法设计的关键。

2.10.16. 轨迹生命周期参数如何影响最终效果

本案例中的三组参数具有较强的实验分析价值：

- `max_age`：轨迹最长允许失配多少帧。
- `min_hits`：轨迹至少命中多少次后才输出。
- `iou_threshold`：检测与轨迹关联所需的最小 IOU。

这些参数分别控制三类问题：

- 轨迹是否容易过早消失。
- 新轨迹是否容易过快出现。
- 匹配是否保守还是激进。

通过调参可以直接观察以下现象：

- `max_age` 过小，目标短时遮挡后容易断轨。
- `min_hits` 过小，误检容易变成短暂轨迹。
- `iou_threshold` 过高，快速移动目标更容易失配。
- `iou_threshold` 过低，邻近目标更容易错配。

2.11. 可视化代码如何帮助理解算法结果

在 `utils/postprocessing.py` 中，`draw_detections` 与 `draw_tracks` 分别负责绘制检测结果和跟踪结果。尽管这部分并不直接构成算法主体，但其作用十分重要，因为可视化能够将抽象状态转化为可直接观察的现象。

其中，`draw_tracks` 中有两点尤其值得关注：

- 每条轨迹使用固定颜色，帮助观察 ID 是否稳定。

- trail 轨迹拖尾可以直观看出运动路径与预测连续性。

因此，在调节参数时，不仅能够观察检测框是否正常输出，还能够观察 ID 是否发生跳变，以及轨迹是否保持连续。

2.12. 如何评价这个案例方案

2.12.1. 这个方案的优点

作为昇腾 310B 开发教程中的案例，本方案有很强的适配性：

- 模型轻量，适合边缘侧实时实验。
- 检测和跟踪两个阶段边界清晰，便于分层讲解。
- 代码结构整洁，便于把入口、后端、解码、跟踪拆开讲。
- DeepSORT 采用简化实现，避免课程一开始就陷入 ReID 细节。
- 既能跑 CPU，也能跑 NPU，便于体现部署层面的思维。

2.12.2. 这个方案的不足

需要明确的是，本案例并不是为了追求最强跟踪精度：

- 检测前端仍是经典 SSD，精度不是当前最先进水平。
- 跟踪器没有使用外观特征，因此遮挡恢复能力有限。
- 数据关联主要依赖 IOU，对交叉目标和密集目标不够鲁棒。
- 卡尔曼状态建模较简化，更偏向实现可解释性而不是工业最优效果。

2.12.3. 作为入门案例的合理性

正因为它没有引入过多复杂部件，目标跟踪的基本问题反而更容易被清晰呈现：

- 检测结果是如何来的。
- 检测结果如何进入跟踪器。
- 为什么需要预测。
- 为什么需要全局匹配。
- 为什么轨迹管理会决定最终 ID 稳定性。

当这些基础概念建立起来后，再引入完整 DeepSORT、ByteTrack、OC-SORT 或更复杂的 ReID 模块，学习曲线会平缓很多。

2.13. 实验设计

为了把本案例组织成一章完整教程，可以安排以下实验任务：

2.13.1. 只运行检测链路

目标：理解 MobileNet-SSD 的输入输出与后处理。

可观察：

- 不同模型输入尺寸对速度和效果的影响。
- `score_threshold` 对检测框数量的影响。
- NMS 阈值对重复框抑制效果的影响。

2.13.2. 在检测基础上启用跟踪

目标：观察轨迹是如何从检测结果中产生的。

可观察：

- 同一目标在连续帧中是否保持稳定 ID。
- 当目标短时遮挡时，轨迹是否断裂。
- 当多个目标接近时，是否发生 ID 交换。

2.13.3. 调节 DeepSORT 参数

目标：理解生命周期管理与关联阈值。

可调节：

- `track_max_age`
- `track_min_hits`
- `track_iou_threshold`

实验中应重点解释参数变化背后的原因，而不是只记录结果。

2.14. 章节总结

本案例展示了一条较为典型的目标跟踪实现路线：

- 用 MobileNet 作为轻量骨干网络，降低边缘侧计算成本。
- 用 SSD 作为一阶段检测器，直接输出类别与边界框。
- 用统一检测后端把 CPU 与 Ascend NPU 推理过程封装起来。

- 用简化版 *DeepSORT* 完成轨迹预测、数据关联与轨迹维护。

从知识结构上看，本章真正要让读者掌握的，不只是某个脚本的使用方法，而是下面这条主线：

理解这条主线之后，读者就能明白：目标跟踪不是一个独立黑盒，而是一套由检测、预测、匹配和管理共同组成的时序识别系统。

对于昇腾 310B 教程而言，这个案例的价值正在于此。它既保留了真实部署场景中需要考虑的实时性与工程组织问题，又把目标跟踪最核心的算法逻辑清楚地呈现出来，适合作为后续学习更复杂视觉任务的基础章节。

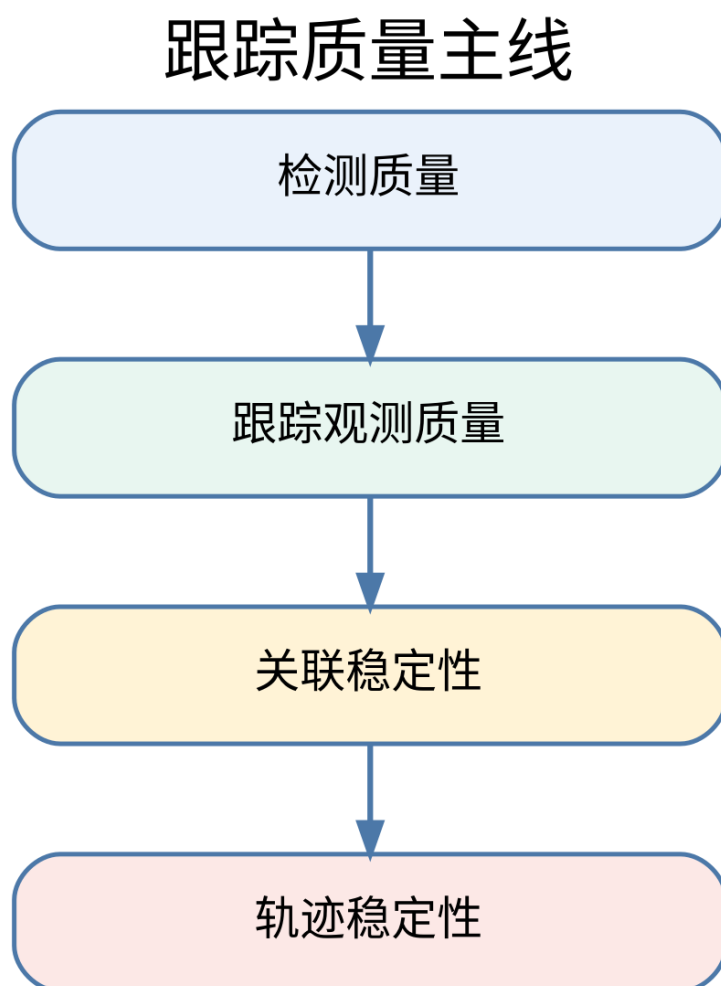


Figure 2.2.: 主线图

3. 案例 3：智能电子琴

3.1. 1. 项目简介

本项目通过结合计算机视觉和音频处理技术，创造了一种全新的交互式音乐体验。系统利用昇腾 310B 的 AI 算力，实时识别用户的手势或特定图像标记，并将其转换为相应的音符和音效，从而实现“隔空弹琴”的神奇效果。该项目不仅展示了 AI 在艺术创作领域的应用潜力，也为音乐教育、娱乐互动和无障碍音乐演奏提供了创新的解决方案。无论是音乐爱好者还是技术探索者，都能从这个项目获得乐趣和启发。项目的源代码可以从[这里](#)下载。

3.2. 2. 内容大纲

3.2.1. 2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 图像采集: 高分辨率 USB 摄像头 (推荐 1080p 30fps)
- 音频输出:
 - USB 音响或蓝牙音箱
 - 3.5mm 耳机接口
 - HDMI 音频输出
- 显示设备: 触摸屏显示器 (可选, 用于虚拟键盘显示)
- LED 指示灯: RGB LED 灯带, 用于视觉反馈
- 电源: 稳定的 12V 电源适配器

硬件系统架构图





3.2.2. 2.2. 软件环境

- 操作系统: Ubuntu 20.04 LTS
- CANN 版本: 7.0.RC1
- Python 版本: 3.8.10
- 音频处理库:
 - pygame: 音频播放和 MIDI 处理
 - pyaudio: 实时音频处理
 - librosa: 音频分析
 - mido: MIDI 文件操作
- 计算机视觉库:
 - opencv-python: 图像处理
 - mediapipe: 手势识别
 - numpy: 数值计算
- 界面开发:
 - tkinter: GUI 界面
 - matplotlib: 波形可视化

快速安装脚本 (install_piano.sh)

```
1 #!/bin/bash
2 # 更新包管理器
3 sudo apt update
4
5 # 安装音频依赖
6 sudo apt install -y python3-pyaudio portaudio19-dev
7
8 # 安装Python包
9 pip3 install pygame librosa mido opencv-python mediapipe numpy tkinter
10 ↪ matplotlib
11
12 # 安装MIDI支持
13 sudo apt install -y timidity timidity-interfaces-extra
14
15 echo "智能电子琴环境安装完成!"
```

3.2.3. 2.3. 手势识别与音符映射

- 手势识别方案:
 - 静态手势: 识别手指数量对应不同音符
 - 动态手势: 手部移动轨迹控制音调变化
 - 双手协作: 左手控制和弦, 右手控制旋律
 - 手势速度: 识别手势速度控制音符强度
- 音符映射系统: python #手势到音符的映射示例 `gesture_to_note = { 'one_finger' ↪ ': 'C4', # 1个手指 = C调 'two_fingers': 'D4', # 2个手指 = D调 'three_fingers' ↪ ': 'E4', # 3个手指 = E调 'four_fingers': 'F4', # 4个手指 = F调 'five_fingers' ↪ ': 'G4', # 5个手指 = G调 'fist': 'REST', # 握拳 = 休止符 'palm_open': ↪ 'CHORD' # 张开手掌 = 和弦 }`
- 高级识别功能:
 - 音量控制: 通过手与摄像头的距离控制音量
 - 音效切换: 特定手势切换乐器音色
 - 节拍控制: 双手拍击节奏控制节拍器

3.2.4. 2.4. 模型训练与优化

- 手势识别模型:
 - 基础方案: 使用 MediaPipe 的预训练手部识别模型
 - 自定义方案: 训练专门的手势分类模型
 - 数据集构建: 收集多角度、多光照条件下的手势数据
- 模型训练流程:
 1. 数据收集: 使用摄像头收集各种手势样本
 2. 数据标注: 为每个手势分配对应的音符标签
 3. 模型训练: 使用 CNN 或 Transformer 架构训练分类器
 4. 模型评估: 在测试集上验证识别准确率
 5. 模型优化: 针对昇腾 310B 进行量化和加速
- 实时优化策略:
 - 帧率控制: 平衡识别精度和实时性
 - 噪声过滤: 减少误识别和抖动
 - 延迟补偿: 最小化从手势到发声的延迟

3.2.5. 2.5. 音频合成与处理

- 音频合成方案:
 - **MIDI 合成**: 使用 MIDI 格式生成标准乐器音色
 - **波形合成**: 直接生成音频波形, 支持自定义音色
 - **采样回放**: 预录制真实乐器采样进行回放
- 音效处理: python #音效处理管道示例 `audio_pipeline = [ 'reverb', # 混响效`
`↪ 果 'equalizer', # 均衡器 'compressor', # 压缩器 'delay', # 延迟效果 '`
`↪ chorus' # 合唱效果 ]`
- 实时音频流处理:
 - **低延迟设计**: 优化音频缓冲区大小
 - **多线程处理**: 分离音频生成和播放线程
 - **动态加载**: 按需加载音色库

3.2.6. 2.6. 3D 打印结构件

- 摄像头支架 (camera_stand.stl):
 - 高度可调节设计 (50-80cm)
 - 角度微调机构 ($\pm 30^\circ$)
 - 防震减振设计
- 设备一体化外壳 (piano_case.stl):
 - 集成散热风扇位
 - LED 灯带安装槽
 - 音响设备安装位
- 用户交互面板 (control_panel.stl):
 - 物理按键布局
 - 旋钮和滑块安装位
 - 显示屏保护框

3D 打印建议: - **材料选择**: ABS (强度好, 适合结构件) - **层高设置**: 0.2mm - **填充密度**: 25% - **支撑设置**: 针对悬垂结构添加支撑

3.2.7. 2.7. 用户手册

2.7.1 快速上手

1. **环境配置**: 运行安装脚本配置软件环境

- 2. 硬件连接: 按照连接图组装所有硬件
- 3. 校准摄像头: 调整摄像头角度和高度
- 4. 启动程序: python3 smart_piano.py
- 5. 手势训练: 跟随屏幕提示学习基本手势

2.7.2 演奏模式

- 自由演奏模式: 随意手势即兴演奏
- 跟随演奏模式: 根据屏幕提示演奏指定曲目
- 录制模式: 录制演奏过程并回放
- 教学模式: 逐步学习手势和音符对应关系

2.7.3 高级功能

- 乐器切换: 手势控制切换钢琴、小提琴、吉他等音色
- 和弦演奏: 双手配合演奏复杂和弦
- 节拍器: 内置节拍器辅助练习
- 录音回放: 保存演奏为音频文件

2.7.4 故障排除

- 手势识别不准确: 检查光照条件和摄像头角度
- 音频延迟: 调整音频缓冲区设置
- 程序崩溃: 查看日志文件排查错误

3.3. 3. 源代码结构

```
1 smart_piano/
2 |— src/
3 |   |— gesture_recognition/    # 手势识别模块
4 |   |— audio_synthesis/       # 音频合成模块
5 |   |— ui/                     # 用户界面
6 |   |— utils/                  # 工具函数
7 |— models/
8 |   |— gesture_classifier.onnx # 手势识别模型
9 |   |— pretrained/            # 预训练模型
10 |— audio/
11 |   |— samples/               # 音频采样
12 |   |— midi/                  # MIDI文件
```

```
13 |— configs/
14 |   |— piano_config.yaml    # 配置文件
15 |— 3d_models/               # 3D打印文件
```

3.4. 4. 效果演示

- **基础演奏:** 展示单手手势演奏简单旋律
- **和弦演奏:** 双手配合演奏和弦进行
- **音色切换:** 实时切换不同乐器音色
- **教学演示:** 跟随模式学习演奏《小星星》等简单曲目

4. 案例 4：智能掌纹识别机

4.1. 1. 项目简介

本项目基于昇腾 310B 平台，构建一个高精度的掌纹识别系统。掌纹识别作为一种新兴的生物识别技术，具有纹理丰富、难以伪造、非接触式采集等优势，在门禁控制、金融支付、身份验证等领域有着广阔的应用前景。

相比传统的指纹识别，掌纹包含更多的特征信息，包括主线、皱纹、纹理方向等，能够提供更高的识别精度和更强的防伪能力。本项目将从掌纹图像采集、特征提取、模式匹配到系统部署的全流程进行详细介绍。

4.2. 2. 内容大纲

4.2.1. 2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 图像采集设备:
 - 高分辨率 USB 摄像头 (≥5MP, 推荐 8MP)
 - 近红外 LED 照明阵列 (增强纹理对比度)
 - 偏振滤光片 (减少皮肤反光)
- 用户交互设备:
 - 7 寸电容触摸屏
 - 蜂鸣器 (音频反馈)
 - 指示 LED 灯 (状态指示)
- 辅助设备:
 - 手掌定位导板
 - 防抖支架
 - UPS 不间断电源

掌纹采集系统架构



4.2.2. 2.2. 软件环境

- 操作系统: Ubuntu 20.04 LTS
- CANN 版本: 7.0.RC1
- Python 版本: 3.8.10
- 图像处理库:
 - opencv-python: 图像预处理和增强
 - scikit-image: 高级图像处理算法
 - PIL: 图像基础操作
- 机器学习库:
 - pytorch: 深度学习框架
 - torchvision: 计算机视觉工具
 - sklearn: 传统机器学习算法
- 数据库系统:
 - sqlite3: 本地掌纹特征数据库
 - redis: 高速缓存系统
- 界面开发:
 - tkinter: GUI 开发
 - pygame: 音效处理

环境部署脚本 (setup_palmprint.sh)

```

1 #!/bin/bash
2 # 系统依赖安装
3 sudo apt update
4 sudo apt install -y python3-dev python3-pip sqlite3 redis-server

```

```

5
6 # Python依赖安装
7 pip3 install opencv-python scikit-image Pillow torch torchvision sklearn
8   ↪ redis pygame
9
10 # 创建数据库目录
11 mkdir -p ./database/palmprints
12 mkdir -p ./models/pretrained
13 echo "掌纹识别系统环境配置完成!"

```

4.2.3. 2.3. 掌纹图像预处理

- 图像采集标准化:
 - 分辨率要求: 300 DPI, 确保纹理清晰度
 - 光照控制: 均匀 LED 阵列, 避免阴影
 - 手掌定位: 引导用户正确放置手掌
 - 质量检测: 自动评估图像质量, 不合格则重新采集
- 预处理管道: “python # 掌纹预处理流程 def preprocess_palmprint(image): #
 1. 手掌区域分割 palm_region = segment_palm(image)

```

1  # 2. ROI提取 (感兴趣区域)
2  roi = extract_roi(palm_region)
3
4  # 3. 图像增强
5  enhanced = enhance_contrast(roi)
6  enhanced = reduce_noise(enhanced)
7
8  # 4. 几何校正
9  normalized = geometric_correction(enhanced)
10
11 # 5. 尺寸标准化
12 resized = resize_to_standard(normalized)
13
14 return resized

```

“

- 图像质量评估:
 - 清晰度检测: 基于梯度的清晰度评估
 - 完整性检查: 确保手掌完整采集
 - 光照均匀性: 检测过度曝光或阴影区域

4.2.4. 2.4. 特征提取与匹配

- 传统特征提取方法：
 - 方向场计算: 提取掌纹主要纹线方向
 - **Gabor 滤波**: 多尺度、多方向纹理特征
 - **LBP (局部二值模式)**: 局部纹理描述子
 - **SIFT 关键点**: 尺度不变特征点
- 深度学习特征提取：
 - **ResNet-50**: 作为特征提取 backbone
 - **注意力机制**: 关注重要纹理区域
 - **对比学习**: 学习区分性特征表示
 - **多尺度融合**: 结合不同尺度的特征信息
- 特征匹配算法: “python # 掌纹匹配流程 def match_palmprint(query_features, database_features): # 1. 特征归一化 query_norm = normalize_features(query_features) db_norm = normalize_features(database_features)

```

1  # 2. 相似度计算
2  similarity = cosine_similarity(query_norm, db_norm)
3
4  # 3. 阈值判断
5  if similarity > MATCH_THRESHOLD:
6      return True, similarity
7  else:
8      return False, similarity

```

“

4.2.5. 2.5. 模型训练与优化

- 数据集构建：
 - 公开数据集: PolyU 掌纹数据库、IITD 掌纹数据库
 - 自建数据集: 多场景、多光照条件的掌纹采集
 - 数据增强: 旋转、缩放、亮度调整、噪声添加
- 模型训练策略：
 - 预训练: 在大规模掌纹数据集上预训练
 - **Fine-tuning**: 在特定应用场景数据上微调
 - 对抗训练: 提高模型鲁棒性
 - 知识蒸馏: 压缩模型以适应边缘设备
- 性能优化：
 - 模型量化: INT8 量化减少计算开销

- 模型剪枝: 移除冗余参数
- 图融合: 算子融合减少内存访问

4.2.6. 2.6. 系统部署与集成

- 模型部署流程: “bash # 1. 模型转换 python3 convert_model.py -input palmprint_model.pth -output palmprint_model.onnx
2. 昇腾模型转换 atc -model=palmprint_model.onnx -framework=5 -output=palmprint_model -input_format=NCHW -input_shape="input:1,3,224,224"
-soc_version=Ascend310B1 “
- 系统架构设计:
 - 模块化设计: 图像采集、预处理、识别、数据库模块独立
 - 多线程处理: 并发处理图像采集和识别任务
 - 异常处理: 完善的错误恢复机制
 - 日志系统: 详细的操作和错误日志

4.2.7. 2.7. 3D 打印结构件

- 掌纹采集支架 (palmprint_scanner.stl):
 - 符合人体工程学的手掌放置槽
 - 摄像头精确定位机构
 - LED 照明最优角度设计
- 设备主体外壳 (main_enclosure.stl):
 - 防尘防水 IP54 级别
 - 散热风扇安装位
 - 线缆整理空间
- 用户交互面板 (user_interface.stl):
 - 触摸屏保护边框
 - 指示灯透光设计
 - 音响开孔优化

3D 打印规格: - 材料: PETG (化学稳定性好) - 层高: 0.15mm (精细结构) - 填充率: 40% (强度要求)

4.2.8. 2.8. 用户手册

2.8.1 系统部署

1. 硬件组装: 按照接线图连接所有硬件
2. 软件安装: 执行环境配置脚本
3. 系统校准: 校准摄像头和照明系统
4. 数据库初始化: 创建用户数据库表结构

2.8.2 用户注册流程

1. 身份信息录入: 输入姓名、工号等基本信息
2. 掌纹采集: 引导用户正确放置手掌
3. 质量检测: 自动评估采集质量
4. 多次采集: 采集 3-5 张不同角度的掌纹图像
5. 特征提取: 生成用户掌纹特征模板
6. 数据库存储: 保存用户信息和特征模板

2.8.3 身份验证流程

1. 掌纹采集: 用户将手掌放置在采集区域
2. 预处理: 自动进行图像预处理
3. 特征提取: 提取当前掌纹特征
4. 数据库匹配: 与注册用户进行匹配
5. 结果输出: 显示验证结果和置信度

2.8.4 系统维护

- 定期清洁: 清洁摄像头镜头和 LED 灯
- 数据备份: 定期备份用户数据库
- 性能监控: 监控识别准确率和响应时间
- 系统更新: 定期更新模型和软件

4.3. 3. 源代码结构

```
1 palmprint_system/  
2 |— src/
```

```
3 |   |— capture/           # 图像采集模块
4 |   |— preprocessing/    # 图像预处理
5 |   |— feature_extraction/ # 特征提取
6 |   |— matching/         # 匹配算法
7 |   |— database/         # 数据库操作
8 |   |— ui/               # 用户界面
9 | — models/
10 |   |— pretrained/       # 预训练模型
11 |   |— custom/          # 自定义模型
12 | — data/
13 |   |— datasets/        # 训练数据集
14 |   |— database/        # 用户数据库
15 | — configs/
16 |   |— system_config.yaml # 系统配置
17 | — hardware/
18 |   |— 3d_models/        # 3D打印文件
19 |   |— circuit_diagrams/ # 电路连接图
```

4.4. 4. 效果演示

- **注册演示:** 展示用户注册的完整流程
- **识别演示:** 展示快速准确的身份验证
- **防伪测试:** 验证系统对伪造掌纹的识别能力
- **性能指标:** 识别准确率、FAR/FRR 指标、响应时间统计

5. 案例 5：智能数据采集仪

5.1. 1. 项目简介

本项目基于昇腾 310B 平台，构建一个多功能的智能数据采集与分析系统。该系统能够同时连接多种传感器，实时采集环境数据，并利用 AI 算法进行智能分析、异常检测和趋势预测，为环境监测、智慧农业、工业 4.0 等应用场景提供强有力的数据支撑。

与传统数据采集设备相比，本系统具备边缘 AI 处理能力，能够在本地进行数据预处理、特征提取和初步分析，大大减少了数据传输量和云端处理负担，实现了真正的边缘智能化数据采集。

5.2. 2. 内容大纲

5.2.1. 2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 数据采集模块:
 - 环境传感器:
 - * DHT22 (温湿度传感器)
 - * BMP280 (气压传感器)
 - * TSL2561 (光照强度传感器)
 - * MQ-2 (烟雾气体传感器)
 - * DS18B20 (防水温度传感器)
 - 运动传感器:
 - * MPU6050 (六轴陀螺仪加速度计)
 - * HMC5883L (电子罗盘)
 - 电气参数:
 - * ACS712 (电流传感器)
 - * 电压分压电路
- 通信接口:

- I2C 总线扩展板
- SPI 接口模块
- RS485 通信模块
- LoRa 无线模块 (长距离通信)
- 显示与交互:
 - 3.5 寸 TFT 彩屏
 - 旋转编码器
 - 多功能按键
- 存储设备:
 - 高速 SD 卡 (32GB+)
 - 实时时钟模块 (DS3231)

系统架构图



5.2.2. 2.2. 软件环境

- 操作系统: Ubuntu 20.04 LTS
- CANN 版本: 7.0.RC1
- Python 版本: 3.8.10
- 硬件接口库:
 - RPi.GPIO: GPIO 控制 (兼容库)
 - smbus: I2C 通信
 - spidev: SPI 通信
 - pyserial: 串口通信
- 数据处理库:

- numpy: 数值计算
- pandas: 数据分析
- scipy: 科学计算
- matplotlib: 数据可视化
- 机器学习库:
 - scikit-learn: 传统机器学习
 - tensorflow-lite: 轻量级深度学习
- 数据库:
 - sqlite3: 本地数据存储
 - influxdb: 时序数据库 (可选)
- 通信协议:
 - mqtt: 物联网通信
 - modbus: 工业通信协议

环境安装脚本 (setup_daq.sh)

```

1 #!/bin/bash
2 # 更新系统
3 sudo apt update && sudo apt upgrade -y
4
5 # 安装系统依赖
6 sudo apt install -y python3-dev python3-pip i2c-tools
7
8 # 启用I2C和SPI接口
9 sudo raspi-config nonint do_i2c 0
10 sudo raspi-config nonint do_spi 0
11
12 # 安装Python依赖
13 pip3 install numpy pandas scipy matplotlib scikit-learn
14 pip3 install RPi.GPIO smbus spidev pyserial
15 pip3 install paho-mqtt pymodbus
16
17 echo "数据采集系统环境配置完成!"

```

5.2.3. 2.3. 传感器集成与数据采集

- 传感器驱动开发: “python # 传感器基类设计 class SensorBase: def **init**(self, address, bus_type='i2c'): self.address = address self.bus_type = bus_type self.init_sensor()

```

1 def init_sensor(self):
2     """传感器初始化"""
3     pass
4
5 def read_data(self):

```

```

6     """读取传感器数据"""
7     pass
8
9     def calibrate(self):
10         """传感器校准"""
11         pass

```

具体传感器实现 class DHT22Sensor(SensorBase): def read_data(self): return { 'temperature': self.read_temperature(), 'humidity': self.read_humidity(), 'timestamp': time.time() } “

- 数据采集调度:
 - 多线程采集: 不同传感器独立线程
 - 采样频率控制: 根据传感器特性设置不同采样率
 - 数据同步: 时间戳同步和数据对齐
 - 异常处理: 传感器故障检测和恢复
- 数据质量控制:
 - 异常值检测: 基于统计方法的异常值识别
 - 数据校准: 定期校准传感器偏差
 - 缺失值处理: 插值和补齐策略
 - 噪声过滤: 数字滤波和平滑处理

5.2.4. 2.4. 智能数据分析

- 实时数据处理: “python # 数据处理管道 class DataProcessor: def init(self): self.filters = [] self.analyzers = []

```

1     def add_filter(self, filter_func):
2         self.filters.append(filter_func)
3
4     def add_analyzer(self, analyzer):
5         self.analyzers.append(analyzer)
6
7     def process(self, raw_data):
8         # 1. 数据过滤
9         filtered_data = raw_data
10        for filter_func in self.filters:
11            filtered_data = filter_func(filtered_data)
12
13        # 2. 特征提取
14        features = self.extract_features(filtered_data)
15
16        # 3. 智能分析
17        results = {}
18        for analyzer in self.analyzers:
19            results.update(analyzer.analyze(features))

```

```
20
21         return results
```

“

- 异常检测算法：
 - 统计方法: 3σ 准则、箱线图方法
 - 机器学习方法: Isolation Forest、One-Class SVM
 - 深度学习方法: AutoEncoder 异常检测
 - 时序分析: ARIMA 模型、LSTM 网络
- 趋势预测：
 - 短期预测: 基于滑动窗口的线性回归
 - 中期预测: 季节性分解和 ARIMA 模型
 - 长期预测: LSTM 神经网络
 - 置信区间: 预测结果的不确定性量化

5.2.5. 2.5. 边缘 AI 模型部署

- 模型选择与训练：
 - 异常检测模型: 基于历史数据训练异常检测器
 - 预测模型: 时间序列预测模型训练
 - 分类模型: 环境状态分类器
 - 回归模型: 传感器数值预测
- 模型优化: “bash # 模型转换和优化 # 1. TensorFlow 模型转换 python3 convert_tf_to_tflite.py -input model.pb -output model.tflite
2. 模型量化 python3 quantize_model.py -input model.tflite -output model_quantized.tflite
3. 昇腾模型转换 atc -model=model.onnx -framework=5 -output=daq_model
-input_format=NCHW -input_shape="input:1,10,1"
-soc_version=Ascend310B1 “
- 实时推理：
 - 流式处理: 实时数据流分析
 - 批处理: 定时批量数据分析
 - 混合模式: 关键数据实时处理，历史数据批处理

5.2.6. 2.6. 数据存储与管理

- 本地存储策略: “python # 数据存储管理 class DataStorage: def **init**(self, db_path): self.db_path = db_path self.init_database()

```

1  def init_database(self):
2      # 创建数据表
3      self.create_sensor_data_table()
4      self.create_analysis_results_table()
5      self.create_system_logs_table()
6
7  def store_sensor_data(self, sensor_id, data, timestamp):
8      # 存储传感器原始数据
9      pass
10
11 def store_analysis_result(self, analysis_type, result, timestamp)
12     ↪ :
13     # 存储分析结果
14     pass

```

“

- 数据压缩与归档:
 - 实时数据: 高频采样, 短期保存
 - 历史数据: 降采样压缩, 长期归档
 - 分析结果: 关键指标永久保存
 - 日志数据: 定期清理和归档

5.2.7. 2.7. 用户界面与可视化

- 实时监控界面:
 - 仪表盘显示: 关键传感器数值
 - 趋势图表: 历史数据趋势
 - 告警提示: 异常情况及时提醒
 - 系统状态: 设备运行状态监控
- 数据可视化: “python # 数据可视化模块 class DataVisualizer: def **init**(self): self.fig, self.axes = plt.subplots(2, 2, figsize=(12, 8))

```

1  def update_real_time_plot(self, data):
2      # 更新实时数据图表
3      pass
4
5  def generate_daily_report(self, date):
6      # 生成日报图表
7      pass
8

```

```
9 def create_trend_analysis(self, sensor_type, time_range):
10     # 创建趋势分析图
11     pass
```

““

5.2.8. 2.8. 3D 打印结构件

- 传感器安装支架 (**sensor_mount.stl**):
 - 模块化传感器安装座
 - 防护罩设计
 - 线缆管理槽
- 主机保护外壳 (**main_enclosure.stl**):
 - IP65 防护等级
 - 散热孔设计
 - 标准 DIN 导轨安装
- 显示屏支架 (**display_mount.stl**):
 - 角度可调设计
 - 防眩光遮光罩
 - 按键操作区域

3D 打印建议: - 材料: ABS 或 PETG (耐候性好) - 层高: 0.2mm - 填充率: 30% - 后处理: 打磨和喷涂保护层

5.2.9. 2.9. 用户手册

2.9.1 系统部署

1. 硬件组装: 按照接线图连接传感器和模块
2. 软件安装: 运行环境配置脚本
3. 传感器校准: 校准各个传感器的基准值
4. 系统测试: 验证数据采集和通信功能

2.9.2 日常操作

1. 启动系统: 开机自检和传感器状态检查
2. 实时监控: 查看当前环境参数
3. 历史查询: 检索历史数据和分析结果
4. 参数配置: 调整采样频率和告警阈值

2.9.3 维护保养

1. 定期校准: 每月校准传感器精度
2. 数据备份: 定期备份重要数据
3. 清洁维护: 清洁传感器和外壳
4. 软件更新: 更新系统软件和 AI 模型

2.9.4 故障排除

- 传感器故障: 检查连接和供电
- 通信异常: 检查网络和协议配置
- 数据异常: 验证传感器校准和环境因素

5.3. 3. 源代码结构

```
1 smart_daq/
2 |— src/
3 |   |— sensors/           # 传感器驱动
4 |   |— data_processing/   # 数据处理
5 |   |— ai_analysis/       # AI分析模块
6 |   |— storage/           # 数据存储
7 |   |— communication/     # 通信模块
8 |   |— ui/                # 用户界面
9 |— models/
10 |   |— anomaly_detection/ # 异常检测模型
11 |   |— prediction/        # 预测模型
12 |   |— classification/    # 分类模型
13 |— data/
14 |   |— raw/               # 原始数据
15 |   |— processed/         # 处理后数据
16 |   |— analysis/          # 分析结果
17 |— configs/
18 |   |— sensors.yaml       # 传感器配置
19 |   |— ai_models.yaml     # AI模型配置
20 |   |— system.yaml        # 系统配置
21 |— hardware/
22 |   |— 3d_models/          # 3D打印文件
23 |   |— pcb_design/         # PCB设计文件
24 |   |— assembly_guide/     # 组装指南
```

5.4. 4. 效果演示

- **多传感器同步采集:** 展示多种传感器数据的实时采集
- **异常检测演示:** 模拟异常情况的自动检测和告警
- **趋势预测展示:** 基于历史数据的未来趋势预测
- **智能分析报告:** 自动生成的数据分析和建议报告

6. 案例 6：智能小车

6.1. 1. 项目简介

本项目基于昇腾 310B 平台，构建一个具备自主导航、障碍物避让、自动循迹等功能的智能小车。该小车集成了计算机视觉、路径规划、运动控制等多项 AI 技术，能够在复杂环境中自主行驶，为无人驾驶、服务机器人、智能巡检等应用领域提供技术验证平台。

相比传统的遥控小车，智能小车具备环境感知、决策规划和自主执行的能力，代表了移动机器人技术的发展方向。本项目将详细介绍从硬件搭建、软件开发到 AI 算法部署的完整实现过程。

6.2. 2. 内容大纲

6.2.1. 2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 底盘系统:
 - 驱动方式: 四轮差速驱动
 - 电机: 直流减速电机 × 4 (12V, 100RPM)
 - 编码器: 增量式光电编码器 × 4
 - 轮胎: 全向轮或麦克纳姆轮
- 传感器系统:
 - 视觉传感器: USB 摄像头 (1080p 30fps)
 - 激光雷达: RPLiDAR A1 或 A2 (可选)
 - 超声波传感器: HC-SR04 × 6 (前后左右)
 - IMU: MPU6050 (姿态检测)
 - GPS 模块: NEO-8M (户外定位)
- 控制系统:
 - 主控板: 树莓派 4B 或专用控制板
 - 电机驱动: L298N 驱动模块 × 2

- 舵机: SG90 (摄像头云台)
- 电源系统:
 - 主电池: 12V 锂电池组 (5000mAh)
 - 辅助电源: 5V/3.3V 稳压模块
 - 电源管理: 充电保护电路

智能小车系统架构



6.2.2. 2.2. 软件环境

- 操作系统: Ubuntu 20.04 LTS
- CANN 版本: 7.0.RC1
- Python 版本: 3.8.10
- 机器人框架:
 - ROS2 Foxy: 机器人操作系统
 - rospy: Python ROS 接口
 - tf2: 坐标变换库
- 计算机视觉:
 - opencv-python: 图像处理
 - ultralytics: YOLOv8 目标检测
 - apriltag: AprilTag 标签识别
- 路径规划:
 - scipy: 科学计算
 - numpy: 数值计算
 - matplotlib: 路径可视化
- 硬件控制:

- RPi.GPIO: GPIO 控制
- pyserial: 串口通信
- smbus: I2C 通信

环境配置脚本 (setup_smartcar.sh)

```

1 #!/bin/bash
2 # ROS2安装
3 sudo apt update
4 sudo apt install curl gnupg2 lsb-release
5 curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc |
   ↪ sudo apt-key add -
6 sudo sh -c 'echo "deb [arch=$(dpkg --print-architecture)] http://packages
   ↪ .ros.org/ros2/ubuntu $(lsb_release -cs) main" > /etc/apt/sources.
   ↪ list.d/ros2-latest.list'
7 sudo apt update
8 sudo apt install ros-foxy-desktop
9
10 # Python依赖安装
11 pip3 install opencv-python ultralytics scipy numpy matplotlib
12 pip3 install RPi.GPIO pyserial smbus rospkg
13
14 echo "智能小车环境配置完成!"

```

6.2.3. 2.3. 计算机视觉系统

- 目标检测与识别: “python # YOLO 目标检测类 class ObjectDetector: def **init**(self, model_path): self.model = YOLO(model_path) self.target_classes = ['person', 'car', 'traffic_light', 'stop_sign']

```

1 def detect_objects(self, image):
2     results = self.model(image)
3     detections = []
4
5     for result in results:
6         for box in result.bboxes:
7             if box.cls in self.target_classes:
8                 detections.append({
9                     'class': self.target_classes[box.cls],
10                    'confidence': box.conf,
11                    'bbox': box.xyxy,
12                    'distance': self.estimate_distance(box)
13                })
14
15     return detections

```

“

- 车道线检测:
 - 图像预处理: 灰度化、高斯滤波、边缘检测

- **ROI 提取**: 感兴趣区域设定
- **Hough 变换**: 直线检测
- **拟合优化**: 车道线拟合和平滑
- **深度估计**:
 - **单目深度估计**: 基于物体大小的距离估算
 - **双目视觉 (可选)**: 立体视觉深度计算
 - **传感器融合**: 结合超声波和视觉信息

6.2.4. 2.4. 路径规划与导航

- **全局路径规划**: “python # A* 路径规划算法 class AStarPlanner: def **init**(self, grid_map): self.grid_map = grid_map self.open_set = [] self.closed_set = set()

```

1  def plan_path(self, start, goal):
2      # A*算法实现
3      current = start
4      path = []
5
6      while current != goal:
7          # 搜索最优路径
8          current = self.find_best_node()
9          path.append(current)
10
11         if self.is_goal_reached(current, goal):
12             break
13
14         return self.smooth_path(path)
15
16     def smooth_path(self, path):
17         # 路径平滑处理
18         return smoothed_path

```

““

- **局部路径规划**:
 - **DWA 算法**: 动态窗口法避障
 - **人工势场法**: 基于势场的路径规划
 - **RRT 算法**: 快速随机树路径搜索
- **SLAM 建图 (可选)**:
 - **激光 SLAM**: 基于激光雷达的地图构建
 - **视觉 SLAM**: 基于摄像头的视觉 SLAM
 - **多传感器融合**: 激光雷达 + 视觉 + IMU

6.2.5. 2.5. 运动控制系统

- 底层运动控制: “python # 差速驱动控制器 class DifferentialDriveController:
def init(self, wheel_base, wheel_radius): self.wheel_base = wheel_base self.wheel_radius = wheel_radius self.max_speed = 1.0 # m/s

```

1  def velocity_to_wheel_speeds(self, linear_vel, angular_vel):
2      # 将线速度和角速度转换为左右轮速度
3      left_speed = linear_vel - angular_vel * self.wheel_base / 2
4      right_speed = linear_vel + angular_vel * self.wheel_base / 2
5
6      # 速度限制
7      left_speed = self.limit_speed(left_speed)
8      right_speed = self.limit_speed(right_speed)
9
10     return left_speed, right_speed
11
12     def execute_motion(self, left_speed, right_speed):
13         # 发送速度指令到电机驱动器
14         self.set_motor_speeds(left_speed, right_speed)

```

““

- PID 控制器:
 - 位置控制: PID 位置控制器
 - 速度控制: PID 速度控制器
 - 姿态控制: PID 姿态稳定控制
- 轨迹跟踪:
 - Pure Pursuit: 纯跟踪算法
 - Stanley 控制器: Stanley 横向控制
 - MPC 控制: 模型预测控制

6.2.6. 2.6. AI 决策系统

- 行为决策树: “python # 智能行为决策系统 class BehaviorPlanner: def init(self):
self.behaviors = { 'follow_lane': self.follow_lane_behavior, 'avoid_obstacle':
self.avoid_obstacle_behavior, 'stop_for_traffic': self.stop_for_traffic_behavior,
'explore': self.exploration_behavior }

```

1  def make_decision(self, sensor_data, current_state):
2      # 根据传感器数据和当前状态做出决策
3      if self.detect_obstacle(sensor_data):
4          return 'avoid_obstacle'
5      elif self.detect_traffic_sign(sensor_data):
6          return 'stop_for_traffic'
7      elif self.lane_detected(sensor_data):

```

```

8         return 'follow_lane'
9     else:
10        return 'explore'

```

““

- 强化学习 (高级功能):
 - **Q-Learning**: 基于值函数的学习
 - **Deep Q-Network**: 深度 Q 网络
 - **Policy Gradient**: 策略梯度方法

6.2.7. 2.7. 模型部署与优化

- 模型转换流程: “bash # YOLOv8 模型转换 # 1. PyTorch 转 ONNX yolo export model=yolov8n.pt format=onnx
2. ONNX 转昇腾模型 atc -model=yolov8n.onnx -framework=5 -output=yolov8n_car -input_format=NCHW -input_shape="images:1,3,640,640"
-soc_version=Ascend310B1 -out_nodes="output0:0" ““
- 实时推理优化:
 - 模型量化: INT8 量化减少计算量
 - 图优化: 算子融合和内存优化
 - 并行处理: 多线程并行推理

6.2.8. 2.8. 安全系统

- 紧急制动系统:
 - 超声波触发: 近距离障碍物检测
 - 视觉确认: 摄像头二次确认
 - 硬件保护: 硬件级别的紧急停止
- 故障检测:
 - 传感器健康监测: 实时检测传感器状态
 - 通信故障检测: 网络和串口通信监控
 - 电源监控: 电池电量和电压监测

6.2.9. 2.9. 3D 打印结构件

- 底盘框架 (chassis_frame.stl):
 - 轻量化设计

- 模块化安装孔
- 线缆走线槽
- **传感器安装座 (sensor_mounts.stl):**
 - 摄像头云台支架
 - 超声波传感器固定座
 - 激光雷达安装底座
- **保护外壳 (protective_covers.stl):**
 - 控制板保护罩
 - 电池仓外壳
 - 防撞缓冲器

3D 打印规格: - 材料: PLA (原型) 或 ABS (实用) - 层高: 0.2mm - 填充率: 25% (结构件) / 15% (外壳)

6.2.10. 2.10. 用户手册

2.10.1 硬件组装

1. **底盘组装:** 安装电机、轮子和编码器
2. **传感器安装:** 固定摄像头、超声波等传感器
3. **电路连接:** 按照接线图连接所有电子模块
4. **系统测试:** 验证各个子系统功能

2.10.2 软件配置

1. **环境安装:** 运行环境配置脚本
2. **ROS 配置:** 配置 ROS 节点和话题
3. **参数标定:** 标定传感器和运动参数
4. **地图构建:** 构建环境地图 (如需要)

2.10.3 操作指南

1. **手动模式:** 遥控器控制小车运动
2. **半自动模式:** 人工指定目标点自动导航
3. **全自动模式:** 完全自主探索和导航
4. **调试模式:** 实时查看传感器数据和算法状态

2.10.4 维护保养

1. 定期检查: 检查轮子、电机和传感器
2. 软件更新: 更新算法和模型
3. 电池保养: 正确充放电保护电池
4. 故障排除: 常见问题解决方案

6.3. 3. 源代码结构

```

1 smart_car/
2 |— src/
3 |   |— perception/           # 感知模块
4 |   |   |— camera/          # 摄像头处理
5 |   |   |— lidar/           # 激光雷达
6 |   |   |— ultrasonic/       # 超声波传感器
7 |   |— planning/            # 规划模块
8 |   |   |— global_planner/   # 全局路径规划
9 |   |   |— local_planner/    # 局部路径规划
10 |   |   |— behavior/         # 行为规划
11 |   |— control/             # 控制模块
12 |   |   |— motion_control/   # 运动控制
13 |   |   |— safety/           # 安全控制
14 |   |— utils/               # 工具模块
15 |— models/
16 |   |— detection/           # 目标检测模型
17 |   |— segmentation/       # 图像分割模型
18 |   |— rl_models/           # 强化学习模型
19 |— configs/
20 |   |— sensors.yaml         # 传感器配置
21 |   |— motion.yaml          # 运动参数配置
22 |   |— behavior.yaml        # 行为配置
23 |— launch/
24 |   |— smartcar.launch.py   # ROS启动文件
25 |— hardware/
26 |   |— 3d_models/           # 3D打印文件
27 |   |— pcb_design/          # 电路设计
28 |   |— assembly/            # 组装指南

```

6.4. 4. 效果演示

- 自动循迹: 沿着地面标线自动行驶

- **障碍物避让:** 检测并绕过静态和动态障碍物
- **目标跟踪:** 跟随指定目标人员或物体
- **自主导航:** 在已知地图中自主导航到目标点
- **智能巡检:** 按照预设路线进行巡逻检查

7. 案例 7：智能相册

7.1. 1. 项目简介

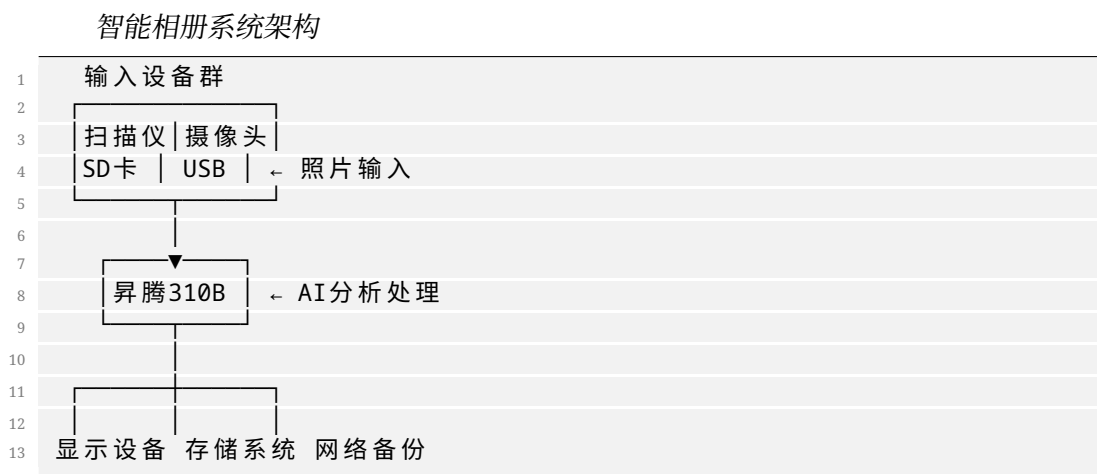
本项目基于昇腾 310B 平台，构建一个智能化的照片管理和检索系统。系统利用先进的计算机视觉和深度学习技术，能够自动识别照片中的人脸、物体、场景等信息，并根据这些特征对照片进行智能分类、标签化和索引，为用户提供便捷高效的照片管理体验。

与传统的相册管理软件相比，智能相册具备自动化程度高、检索精准、个性化推荐等特点，能够帮助用户快速找到目标照片，发现遗忘的美好回忆，甚至自动生成精彩的照片集锦和回忆录。

7.2. 2. 内容大纲

7.2.1. 2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 存储系统:
 - 高速 SSD: 1TB NVMe SSD (照片存储)
 - 机械硬盘: 4TB SATA 硬盘 (备份存储)
 - SD 卡读卡器: 多格式读卡器
 - USB3.0 Hub: 多设备连接
- 显示设备:
 - 主显示器: 4K 高分辨率显示器
 - 触摸屏: 10 寸电容触摸屏 (操作界面)
- 输入设备:
 - 扫描仪: 高精度平板扫描仪 (老照片数字化)
 - 摄像头: 高清网络摄像头 (实时拍照)
- 网络设备:
 - WiFi 模块: 支持 2.4G/5G 双频
 - 网络存储: NAS 设备 (可选)



7.2.2. 2.2. 软件环境

- 操作系统: Ubuntu 20.04 LTS
- CANN 版本: 7.0.RC1
- Python 版本: 3.8.10
- 深度学习框架:
 - torch: PyTorch 深度学习框架
 - torchvision: 计算机视觉库
 - transformers: Transformer 模型库
 - ultralytics: YOLO 系列模型
- 图像处理:
 - opencv-python: OpenCV 图像处理
 - Pillow: Python 图像库
 - scikit-image: 高级图像处理
 - imageio: 图像读写
- 数据库系统:
 - sqlite3: 轻量级关系数据库
 - faiss: 向量相似度搜索
 - elasticsearch: 全文搜索引擎 (可选)
- Web 框架:
 - flask: 轻量级 Web 框架
 - fastapi: 高性能 API 框架
- 前端技术:

- streamlit: 快速 Web 应用开发
- gradio: AI 模型 Web 界面

环境安装脚本 (setup_album.sh)

```

1 #!/bin/bash
2 # 更新系统
3 sudo apt update && sudo apt upgrade -y
4
5 # 安装系统依赖
6 sudo apt install -y python3-dev python3-pip sqlite3 elasticsearch
7
8 # 安装Python依赖
9 pip3 install torch torchvision transformers ultralytics
10 pip3 install opencv-python Pillow scikit-image imageio
11 pip3 install faiss-cpu flask fastapi streamlit gradio
12
13 # 安装图像格式支持
14 sudo apt install -y libraw-dev libexiv2-dev
15
16 echo "智能相册环境配置完成!"

```

7.2.3. 2.3. 智能识别与分析

- 人脸识别系统: “python # 人脸识别和聚类 class FaceRecognitionSystem: def init(self): self.detector = MTCNN() # 人脸检测 self.recognizer = FaceNet() # 人脸特征提取 self.clusterer = DBSCAN() # 人脸聚类

```

1 def detect_faces(self, image):
2     # 检测图像中所有人脸
3     faces = self.detector.detect(image)
4     face_embeddings = []
5
6     for face in faces:
7         # 提取人脸特征向量
8         embedding = self.recognizer.extract_features(face)
9         face_embeddings.append(embedding)
10
11     return faces, face_embeddings
12
13 def cluster_faces(self, all_embeddings):
14     # 对所有人脸进行聚类, 识别同一个人
15     clusters = self.clusterer.fit_predict(all_embeddings)
16     return clusters

```

““

- 物体识别系统:
 - 通用物体检测: YOLO 系列模型检测常见物体
 - 细粒度分类: 针对特定类别的精细分类

- **OCR 文字识别**: 识别照片中的文字信息
- **品牌 logo 识别**: 识别商标和品牌标识
- **场景理解**: “python # 场景分类和描述生成 class SceneAnalyzer: def init(self): self.scene_classifier = ResNet50(pretrained=True) self.caption_model = BLIP() # 图像描述生成

```

1  def analyze_scene(self, image):
2      # 场景分类
3      scene_type = self.classify_scene(image)
4
5      # 生成自然语言描述
6      caption = self.caption_model.generate_caption(image)
7
8      # 提取关键信息
9      metadata = self.extract_metadata(image)
10
11     return {
12         'scene_type': scene_type,
13         'caption': caption,
14         'metadata': metadata
15     }

```

“

- **情感分析**:
 - **人脸表情识别**: 识别喜怒哀乐等基本情感
 - **场景情感分析**: 分析照片整体氛围
 - **色彩情感**: 基于色彩心理学的情感分析

7.2.4. 2.4. 智能分类与标签

- **自动分类系统**: “python # 智能照片分类器 class PhotoClassifier: def init(self): self.categories = { 'people': ['family', 'friends', 'portrait', 'group'], 'events': ['wedding', 'birthday', 'graduation', 'travel'], 'places': ['home', 'office', 'restaurant', 'nature'], 'activities': ['sports', 'cooking', 'reading', 'shopping'] }

```

1  def classify_photo(self, image, metadata):
2      results = {}
3
4      # 基于人脸数量分类
5      face_count = len(metadata['faces'])
6      if face_count == 1:
7          results['type'] = 'portrait'
8      elif face_count > 1:
9          results['type'] = 'group'
10
11     # 基于场景分类

```

```

12     scene = metadata['scene_type']
13     results['scene'] = scene
14
15     # 基于时间分类
16     timestamp = metadata['timestamp']
17     results['time_period'] = self.get_time_period(timestamp)
18
19     return results

```

“

- 智能标签生成:
 - 视觉标签: 基于图像内容的标签
 - 时空标签: 基于拍摄时间和地点的标签
 - 人物标签: 基于人脸识别的人物标签
 - 情境标签: 基于事件和活动的标签
- 个性化分类:
 - 用户偏好学习: 学习用户的分类习惯
 - 自定义类别: 支持用户自定义分类体系
 - 关联分析: 发现照片之间的关联关系

7.2.5. 2.5. 智能检索系统

- 多模态检索: “python # 多模态照片检索引擎 class PhotoSearchEngine: def
init(self): self.text_encoder = CLIP.load_text_encoder() self.image_encoder
 = CLIP.load_image_encoder() self.vector_db = FaissIndex()

```

1  def search_by_text(self, query_text):
2      # 文本转向量
3      text_embedding = self.text_encoder.encode(query_text)
4
5      # 向量检索
6      similar_indices = self.vector_db.search(text_embedding, k=50)
7
8      return self.get_photos_by_indices(similar_indices)
9
10 def search_by_image(self, query_image):
11     # 图像转向量
12     image_embedding = self.image_encoder.encode(query_image)
13
14     # 相似图像检索
15     similar_indices = self.vector_db.search(image_embedding, k
16     ↪ =50)
17
18     return self.get_photos_by_indices(similar_indices)
19
20 def search_by_face(self, face_image):

```

```

20     # 人脸检索
21     face_embedding = self.face_recognizer.extract_features(
22         ↪ face_image)
23     similar_faces = self.face_db.search(face_embedding)
24     return self.get_photos_with_faces(similar_faces)

```

““

- 智能筛选器:
 - 时间范围筛选: 按年月日时间段筛选
 - 地理位置筛选: 按拍摄地点筛选
 - 人物筛选: 按出现人物筛选
 - 质量筛选: 按照照片质量和美感筛选

7.2.6. 2.6. 自动整理与推荐

- 重复照片检测: “python # 重复和相似照片检测 class DuplicateDetector: def init(self): self.hasher = ImageHash() self.similarity_threshold = 0.85

```

1  def find_duplicates(self, photo_list):
2      duplicates = []
3
4      for i, photo1 in enumerate(photo_list):
5          for j, photo2 in enumerate(photo_list[i+1:], i+1):
6              similarity = self.calculate_similarity(photo1, photo2
7              ↪ )
8
9              if similarity > self.similarity_threshold:
10                 duplicates.append((i, j, similarity))
11
12     return duplicates
13
14 def suggest_best_photo(self, duplicate_group):
15     # 从重复照片中推荐最佳的一张
16     best_photo = max(duplicate_group, key=self.quality_score)
17     return best_photo

```

““

- 智能相册生成:
 - 主题相册: 按主题自动生成相册
 - 时光相册: 按时间线组织的回忆相册
 - 人物相册: 为每个人生成专属相册
 - 精选集: AI 挑选的高质量照片集
- 个性化推荐:
 - 回忆推送: 根据历史同期推送旧照片

- **相关推荐:** 基于当前浏览推荐相关照片
- **收藏建议:** 推荐值得收藏的精彩照片

7.2.7. 2.7. 用户界面设计

- **Web 界面开发:** “python # Flask Web 应用 from flask import Flask, render_template, request, jsonify
app = Flask(name)
@app.route('/') def index(): return render_template('gallery.html')
@app.route('/search', methods=['POST']) def search_photos(): query = request.json.get('query') results = photo_search_engine.search_by_text(query)
return jsonify(results)
@app.route('/upload', methods=['POST']) def upload_photos(): files = request.files.getlist('photos') for file in files: # 处理上传的照片 process_uploaded_photo(file)
return jsonify({'status': 'success'})”
- **移动端适配:**
 - **响应式设计:** 适配不同屏幕尺寸
 - **触摸手势:** 支持滑动、缩放等手势操作
 - **离线缓存:** 关键功能的离线支持

7.2.8. 2.8. 模型部署与优化

- **模型转换与部署:** “bash # 模型转换流程 # 1. 人脸识别模型转换 atc -model=facenet.onnx -framework=5 -output=facenet_ascend -input_format=NCHW -input_shape="input:1,3,160,160" -soc_version=Ascend310B1
2. 物体检测模型转换 atc -model=yolov8.onnx -framework=5 -output=yolov8_ascend -input_format=NCHW -input_shape="images:1,3,640,640" -soc_version=Ascend310B1
3. 场景分类模型转换 atc -model=resnet50.onnx -framework=5 -output=resnet50_ascend -input_format=NCHW -input_shape="input:1,3,224,224" -soc_version=Ascend310B1”
- **性能优化策略:**
 - **批处理推理:** 批量处理多张照片
 - **异步处理:** 后台异步分析新上传照片

- **缓存机制:** 缓存分析结果避免重复计算
- **增量更新:** 仅处理新增和修改的照片

7.2.9. 2.9. 数据管理与安全

- **数据库设计:** “sql – 照片信息表 CREATE TABLE photos (id INTEGER PRIMARY KEY, filename TEXT NOT NULL, filepath TEXT NOT NULL, timestamp DATETIME, gps_latitude REAL, gps_longitude REAL, image_hash TEXT, analysis_status INTEGER DEFAULT 0);
– 人脸信息表 CREATE TABLE faces (id INTEGER PRIMARY KEY, photo_id INTEGER, person_id INTEGER, bbox TEXT, embedding BLOB, confidence REAL, FOREIGN KEY (photo_id) REFERENCES photos (id));
– 标签信息表 CREATE TABLE tags (id INTEGER PRIMARY KEY, photo_id INTEGER, tag_name TEXT, tag_type TEXT, confidence REAL, FOREIGN KEY (photo_id) REFERENCES photos (id)); “
- **隐私保护:**
 - **本地处理:** 照片不上传到云端, 保护隐私
 - **访问控制:** 多用户权限管理
 - **数据加密:** 敏感信息加密存储
 - **安全备份:** 定期安全备份重要数据

7.2.10. 2.10. 用户手册

2.10.1 系统安装

1. **硬件连接:** 连接存储设备和显示器
2. **软件安装:** 运行环境配置脚本
3. **初始化:** 创建数据库和目录结构
4. **模型下载:** 下载预训练 AI 模型

2.10.2 照片导入

1. **批量导入:** 从 SD 卡或硬盘批量导入
2. **实时拍摄:** 使用摄像头实时拍照
3. **扫描导入:** 扫描纸质照片数字化
4. **网络同步:** 从云存储同步照片

2.10.3 智能分析

1. 自动分析: 后台自动分析新照片
2. 手动标注: 用户手动补充标签信息
3. 人脸训练: 训练个人专属人脸模型
4. 质量评估: 评估和筛选高质量照片

2.10.4 检索使用

1. 文本搜索: 使用自然语言描述搜索
2. 图像搜索: 上传图片搜索相似照片
3. 人脸搜索: 搜索包含特定人物的照片
4. 高级筛选: 使用多种条件组合筛选

7.3. 3. 源代码结构

```
1 smart_album/
2 |— src/
3 |   |— analysis/           # 图像分析模块
4 |   |   |— face_recognition/
5 |   |   |— object_detection/
6 |   |   |— scene_analysis/
7 |   |   |— emotion_analysis/
8 |   |— search/            # 检索模块
9 |   |   |— text_search/
10 |   |   |— image_search/
11 |   |   |— vector_db/
12 |   |— classification/   # 分类模块
13 |   |   |— auto_classify/
14 |   |   |— tag_generation/
15 |   |— web/              # Web界面
16 |   |   |— templates/
17 |   |   |— static/
18 |   |   |— api/
19 |   |— utils/            # 工具模块
20 |— models/
21 |   |— face_models/      # 人脸相关模型
22 |   |— object_models/    # 物体检测模型
23 |   |— scene_models/     # 场景分析模型
24 |   |— text_models/      # 文本处理模型
25 |— database/
26 |   |— schema.sql        # 数据库结构
```

```
27 |   └─ migrations/           # 数据库迁移
28 | └─ configs/
29 |   └─ models.yaml          # 模型配置
30 |   └─ database.yaml         # 数据库配置
31 |   └─ app.yaml              # 应用配置
32 | └─ tests/
33 |   └─ unit_tests/           # 单元测试
34 |   └─ integration_tests/    # 集成测试
```

7.4. 4. 效果演示

- **智能分类展示**: 自动将照片按人物、场景、事件分类
- **人脸识别演示**: 识别和聚类照片中的不同人物
- **智能搜索体验**: 使用自然语言搜索特定照片
- **自动相册生成**: 根据主题自动生成精美相册
- **重复照片清理**: 检测并建议删除重复和低质量照片

8. 案例 8：手势识别

8.1. 1. 项目简介

本项目基于昇腾 310B 平台，构建一个高精度的手势识别系统，能够实时识别多种静态和动态手势，并将识别结果转换为相应的控制指令。该系统在人机交互、智能家居控制、虚拟现实、辅助医疗等领域具有广泛的应用前景。

与传统的接触式控制方式相比，手势识别技术提供了更自然、更直观的交互体验，特别适合在需要保持距离、无法直接接触设备的场景中使用。本项目将详细介绍从手势数据采集、模型训练到实时识别部署的完整实现过程。

8.2. 2. 内容大纲

8.2.1. 2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 图像采集系统:
 - RGB 摄像头: 高清 USB 摄像头 (1080p 60fps)
 - 深度摄像头: Intel RealSense D435i (可选)
 - 红外摄像头: 夜视环境下的手势识别
 - 多视角摄像头: 2-3 个摄像头实现多角度捕捉
- 照明系统:
 - LED 补光灯: 可调节亮度的环形补光灯
 - 红外补光: 红外 LED 阵列
- 显示与反馈:
 - 显示屏: 7 寸触摸屏显示识别结果
 - 指示灯: RGB LED 指示系统状态
 - 扬声器: 语音反馈系统
- 控制设备:
 - 智能插座: 控制家电开关

- 舵机: 机械臂控制演示
- 无线模块: WiFi/蓝牙控制模块

手势识别系统架构



8.2.2. 2.2. 软件环境

- 操作系统: Ubuntu 20.04 LTS
- CANN 版本: 7.0.RC1
- Python 版本: 3.8.10
- 深度学习框架:
 - pytorch: 深度学习框架
 - torchvision: 计算机视觉库
 - opencv-python: 图像处理库
 - mediapipe: Google 手势识别库
- 图像处理:
 - scikit-image: 高级图像处理
 - albumentations: 数据增强库
 - imgaug: 图像增强
- 数据处理:
 - numpy: 数值计算
 - pandas: 数据处理
 - matplotlib: 数据可视化
 - seaborn: 统计可视化
- 硬件控制:
 - pyserial: 串口通信
 - RPi.GPIO: GPIO 控制

- paho-mqtt: MQTT 通信协议
- 实时处理:
 - threading: 多线程处理
 - queue: 队列管理
 - asyncio: 异步编程

环境配置脚本 (setup_gesture.sh)

```

1 #!/bin/bash
2 # 更新系统
3 sudo apt update && sudo apt upgrade -y
4
5 # 安装系统依赖
6 sudo apt install -y python3-dev python3-pip cmake pkg-config
7 sudo apt install -y libgtk-3-dev libavcodec-dev libavformat-dev
8   ↪ libswscale-dev
9 sudo apt install -y libv4l-dev libxvidcore-dev libx264-dev libjpeg-dev
10   ↪ libpng-dev libtiff-dev
11
12 # 安装Python依赖
13 pip3 install torch torchvision opencv-python mediapipe
14 pip3 install scikit-image albumentations imgaug
15 pip3 install numpy pandas matplotlib seaborn
16 pip3 install pyserial RPi.GPIO paho-mqtt
17
18 # 安装深度摄像头支持 (Intel RealSense)
19 sudo apt install -y librealsense2-dev librealsense2-utils
20 pip3 install pyrealsense2
21
22 echo "手势识别环境配置完成!"

```

8.2.3. 2.3. 手势数据采集与预处理

- 手势数据采集: “python # 多模态手势数据采集器 class GestureDataCollector:


```
def init(self): self.rgb_camera = cv2.VideoCapture(0) self.depth_camera =
rs.pipeline() # RealSense 深度摄像头 self.hand_detector = mp.solutions.hands.Hands()
```

```

1 def collect_gesture_sequence(self, gesture_name, duration=3.0):
2     frames = []
3     landmarks_sequence = []
4
5     start_time = time.time()
6     while time.time() - start_time < duration:
7         # 获取RGB图像
8         ret, rgb_frame = self.rgb_camera.read()
9
10        # 获取深度图像
11        depth_frame = self.get_depth_frame()
12

```

```

13         # 提取手部关键点
14         landmarks = self.extract_hand_landmarks(rgb_frame)
15
16         frames.append({
17             'rgb': rgb_frame,
18             'depth': depth_frame,
19             'timestamp': time.time(),
20             'landmarks': landmarks
21         })
22
23     return frames

```

““

- 手势预处理管道: “python # 手势数据预处理 class GesturePreprocessor: def
init(self): self.target_size = (224, 224) self.sequence_length = 30

```

1     def preprocess_gesture_sequence(self, frames):
2         processed_frames = []
3
4         for frame in frames:
5             # 手部区域提取
6             hand_roi = self.extract_hand_region(frame['rgb'])
7
8             # 图像标准化
9             normalized = self.normalize_image(hand_roi)
10
11            # 尺寸调整
12            resized = cv2.resize(normalized, self.target_size)
13
14            processed_frames.append(resized)
15
16            # 序列长度标准化
17            standardized_sequence = self.standardize_sequence_length(
18                processed_frames, self.sequence_length
19            )
20
21            return standardized_sequence

```

““

- 数据增强策略:
 - 几何变换: 旋转、平移、缩放、翻转
 - 光照变化: 亮度、对比度、饱和度调整
 - 噪声添加: 高斯噪声、椒盐噪声
 - 时序增强: 时间拉伸、压缩、抖动

8.2.4. 2.4. 手势识别模型设计

- 静态手势识别: “python # CNN 静态手势分类器 class StaticGestureClassifier(nn.Module): def **init**(self, num_classes=10): super().__init__() self.backbone = torchvision.models.mobilenet_v3_large(pretrained=True) self.backbone.classifier = nn.Sequential(nn.Dropout(0.2), nn.Linear(960, 512), nn.ReLU(), nn.Dropout(0.2), nn.Linear(512, num_classes))

```
1  def forward(self, x):
2      return self.backbone(x)
```

““

- 动态手势识别: “python # LSTM 动态手势识别器 class DynamicGestureRecognizer(nn.Module): def **init**(self, input_size=42, hidden_size=128, num_classes=15): super().__init__() self.lstm = nn.LSTM(input_size, hidden_size, batch_first=True, num_layers=2) self.classifier = nn.Sequential(nn.Linear(hidden_size, 64), nn.ReLU(), nn.Dropout(0.3), nn.Linear(64, num_classes))

```
1  def forward(self, x):
2      # x shape: (batch_size, sequence_length, input_size)
3      lstm_out, _ = self.lstm(x)
4      # 使用最后一个时间步的输出
5      last_output = lstm_out[:, -1, :]
6      return self.classifier(last_output)
```

““

- 多模态融合模型: “python # RGB + 深度 + 关键点多模态融合 class MultiModalGestureModel(nn.Module): def **init**(self, num_classes=20): super().__init__() # RGB 分支 self.rgb_branch = mobilenet_v3_large(pretrained=True) self.rgb_branch.classifier = nn.Linear(960, 256)

```
1      # 深度分支
2      self.depth_branch = mobilenet_v3_small(pretrained=True)
3      self.depth_branch.classifier = nn.Linear(576, 128)
4
5      # 关键点分支
6      self.landmark_branch = nn.Sequential(
7          nn.Linear(42, 128),
8          nn.ReLU(),
9          nn.Linear(128, 64)
10     )
11
12     # 融合层
13     self.fusion = nn.Sequential(
14         nn.Linear(256 + 128 + 64, 256),
15         nn.ReLU(),
16         nn.Dropout(0.3),
```



```

17         nn.Linear(256, num_classes)
18     )
19
20     def forward(self, rgb, depth, landmarks):
21         rgb_feat = self.rgb_branch(rgb)
22         depth_feat = self.depth_branch(depth)
23         landmark_feat = self.landmark_branch(landmarks)
24
25         combined = torch.cat([rgb_feat, depth_feat, landmark_feat],
26                               ↪ dim=1)
27         return self.fusion(combined)

```

““

8.2.5. 2.5. 模型训练与优化

- 训练策略: “python # 手势识别模型训练器 class GestureModelTrainer: def
init(self, model, train_loader, val_loader): self.model = model self.train_loader
 = train_loader self.val_loader = val_loader self.optimizer = torch.optim.AdamW(model.parameters(),
 lr=1e-3) self.criterion = nn.CrossEntropyLoss() self.scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
 self.optimizer, T_max=100)

```

1     def train_epoch(self):
2         self.model.train()
3         total_loss = 0
4         correct = 0
5         total = 0
6
7         for batch_idx, (data, target) in enumerate(self.train_loader):
8             ↪ :
9             self.optimizer.zero_grad()
10            output = self.model(data)
11            loss = self.criterion(output, target)
12            loss.backward()
13            self.optimizer.step()
14
15            total_loss += loss.item()
16            pred = output.argmax(dim=1)
17            correct += pred.eq(target).sum().item()
18            total += target.size(0)
19
20            accuracy = correct / total
21            avg_loss = total_loss / len(self.train_loader)
22
23            return avg_loss, accuracy

```

““

- 数据增强与正则化:
 - 标签平滑: 减少过拟合

- 混合精度训练: 加速训练过程
- 梯度累积: 模拟大批量训练
- 早停策略: 防止过拟合

8.2.6. 2.6. 实时识别系统

- 实时识别引擎: “python # 实时手势识别系统 class RealTimeGestureRecognizer: def init(self, model_path): self.model = self.load_model(model_path) self.gesture_buffer = deque(maxlen=30) # 30 帧缓冲 self.confidence_threshold = 0.8 self.gesture_labels = self.load_gesture_labels()

```

1  def process_frame(self, frame):
2      # 预处理当前帧
3      processed_frame = self.preprocess_frame(frame)
4
5      # 添加到缓冲区
6      self.gesture_buffer.append(processed_frame)
7
8      # 当缓冲区满时进行识别
9      if len(self.gesture_buffer) == 30:
10         prediction = self.predict_gesture()
11         return prediction
12
13     return None
14
15 def predict_gesture(self):
16     # 准备输入数据
17     input_sequence = np.array(list(self.gesture_buffer))
18     input_tensor = torch.FloatTensor(input_sequence).unsqueeze(0)
19
20     # 模型推理
21     with torch.no_grad():
22         output = self.model(input_tensor)
23         probabilities = torch.softmax(output, dim=1)
24         confidence, predicted = torch.max(probabilities, 1)
25
26     # 置信度检查
27     if confidence.item() > self.confidence_threshold:
28         gesture_name = self.gesture_labels[predicted.item()]
29         return {
30             'gesture': gesture_name,
31             'confidence': confidence.item(),
32             'timestamp': time.time()
33         }
34
35     return None

```

““

- 手势指令映射: “python # 手势到指令的映射系统 class GestureCommandMapper: def init(self): self.command_mapping = { 'thumbs_up': self.turn_on_light, 'thumbs_down': self.turn_off_light, 'peace': self.play_music, 'fist': self.stop_music, 'open_hand': self.increase_volume, 'pointing': self.next_track, 'wave': self.previous_track, 'ok_sign': self.confirm_action }

```

1  def execute_gesture_command(self, gesture_result):
2      gesture_name = gesture_result['gesture']
3
4      if gesture_name in self.command_mapping:
5          command_func = self.command_mapping[gesture_name]
6          return command_func()
7      else:
8          return {'status': 'unknown_gesture', 'gesture':
9                  ↪ gesture_name}
10
11 def turn_on_light(self):
12     # 控制智能灯泡
13     mqtt_client.publish("home/light/living_room", "ON")
14     return {'status': 'success', 'action': 'light_on'}

```

““

8.2.7. 2.7. 模型部署与优化

- 模型转换流程: “bash # 模型转换到昇腾格式 # 1. PyTorch 模型转 ONNX
python3 convert_gesture_model.py
-model_path gesture_model.pth
-output_path gesture_model.onnx
-input_shape 1,3,224,224
2. ONNX 转昇腾离线模型 atc -model=gesture_model.onnx -framework=5
-output=gesture_model_ascend
-input_format=NCHW
-input_shape="input:1,3,224,224"
-soc_version=Ascend310B1
-precision_mode=allow_fp32_to_fp16 “
- 推理性能优化:
 - 模型量化: INT8 量化减少计算量
 - 算子融合: 减少内存访问次数
 - 批处理: 批量处理多帧图像
 - 异步推理: 推理和预处理并行执行

8.2.8. 2.8. 应用场景集成

- 智能家居控制: “python # 智能家居手势控制系统 class SmartHomeGesture-Controller: def **init**(self): self.mqtt_client = mqtt.Client() self.device_mapping = { 'living_room_light': 'home/light/living_room', 'air_conditioner': 'home-/ac/living_room', 'tv': 'home/tv/living_room', 'music_player': 'home/music/living_room' }

```
1 def control_device(self, device, action, value=None):
2     topic = self.device_mapping.get(device)
3     if topic:
4         message = {'action': action, 'value': value}
5         self.mqtt_client.publish(topic, json.dumps(message))
```

“

- 虚拟现实交互:
 - 3D 手势映射: 手势控制 3D 场景
 - 虚拟按钮: 空中虚拟按钮交互
 - 手势绘图: 空中绘制和操作
- 辅助医疗应用:
 - 康复训练: 手势动作康复评估
 - 无接触控制: 医疗设备无接触操作
 - 手语翻译: 手语到文字的转换

8.2.9. 2.9. 用户界面与反馈

- 可视化界面: “python # 手势识别可视化界面 class GestureRecognitionUI: def **init**(self): self.window_size = (800, 600) self.gesture_history = deque(maxlen=10)

```
1 def draw_gesture_overlay(self, frame, gesture_result):
2     if gesture_result:
3         # 绘制识别结果
4         gesture_name = gesture_result['gesture']
5         confidence = gesture_result['confidence']
6
7         # 在图像上绘制文字
8         text = f"{gesture_name}: {confidence:.2f}"
9         cv2.putText(frame, text, (10, 30),
10                     cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
11
12         # 绘制手部关键点
13         self.draw_hand_landmarks(frame, gesture_result.get('
14             ↪ landmarks'))
15
16     return frame
```

```

16
17 def update_gesture_history(self, gesture_result):
18     self.gesture_history.append({
19         'gesture': gesture_result['gesture'],
20         'timestamp': gesture_result['timestamp'],
21         'confidence': gesture_result['confidence']
22     })

```

“

- 多模态反馈:
 - 视觉反馈: 屏幕显示识别结果
 - 声音反馈: 语音提示和音效
 - 触觉反馈: 震动反馈 (如有设备)

8.2.10. 2.10. 用户手册

2.10.1 系统部署

1. 硬件连接: 连接摄像头和控制设备
2. 软件安装: 运行环境配置脚本
3. 模型部署: 部署训练好的手势识别模型
4. 系统校准: 校准摄像头和光照环境

2.10.2 手势训练

1. 手势录制: 录制个人专属手势数据
2. 数据标注: 为手势数据添加标签
3. 模型微调: 基于个人数据微调模型
4. 准确率测试: 测试个性化模型效果

2.10.3 日常使用

1. 启动系统: 开启手势识别程序
2. 手势操作: 在摄像头前做出标准手势
3. 指令执行: 系统执行对应的控制指令
4. 状态查看: 查看识别历史和系统状态

2.10.4 故障排除

1. 识别不准确: 检查光照和手势标准性

2. 延迟问题: 优化模型和硬件配置
3. 误识别: 调整置信度阈值和手势规范

8.3. 3. 源代码结构

```
1 gesture_recognition/
2   └─ src/
3       └─ data_collection/      # 数据采集模块
4       └─ preprocessing/      # 数据预处理
5       └─ models/              # 模型定义
6       └─ training/            # 模型训练
7       └─ inference/           # 实时推理
8       └─ commands/            # 指令映射
9       └─ ui/                  # 用户界面
10  └─ models/
11      └─ static_gesture/      # 静态手势模型
12      └─ dynamic_gesture/     # 动态手势模型
13      └─ multimodal/          # 多模态融合模型
14  └─ data/
15      └─ raw/                  # 原始手势数据
16      └─ processed/            # 处理后数据
17      └─ annotations/          # 标注文件
18  └─ configs/
19      └─ model_config.yaml     # 模型配置
20      └─ camera_config.yaml    # 摄像头配置
21      └─ command_mapping.yaml  # 指令映射配置
22  └─ applications/
23      └─ smart_home/           # 智能家居应用
24      └─ vr_control/           # VR控制应用
25      └─ medical_assist/       # 医疗辅助应用
```

8.4. 4. 效果演示

- 静态手势识别: 识别竖拇指、OK 手势、数字手势等
- 动态手势识别: 识别挥手、指向、画圈等动作
- 智能家居控制: 手势控制灯光、空调、音响等设备
- 实时性能展示: 展示识别速度和准确率指标
- 多人手势识别: 同时识别多个人的手势动作

9. 案例 9：智能聊天机器人

9.1. 1. 项目简介

本项目基于昇腾 310B 平台，构建一个能够在边缘设备上运行的智能聊天机器人。该机器人集成了自然语言处理、语音识别、语音合成等多项 AI 技术，能够与用户进行自然流畅的对话交互，为智能客服、教育辅助、老人陪护、智能助手等应用场景提供解决方案。

相比云端聊天机器人，边缘端部署具有响应速度快、隐私保护好、离线可用等优势。本项目将展示如何在资源受限的边缘设备上部署和优化大语言模型，实现高效的对话服务。

9.2. 2. 内容大纲

9.2.1. 2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 音频输入输出:
 - 麦克风阵列: 4 麦克风阵列 (支持远场拾音)
 - 扬声器: 高保真音响 (支持立体声输出)
 - 音频处理板: USB 音频接口卡
 - 降噪设备: 硬件降噪模块
- 人机交互界面:
 - 触摸显示屏: 10 寸电容触摸屏
 - LED 指示灯: RGB LED 状态指示
 - 物理按键: 唤醒按键、音量调节键
- 网络通信:
 - WiFi 模块: 2.4G/5G 双频 WiFi
 - 蓝牙模块: 支持音频传输
 - 4G 模块: 移动网络支持 (可选)

- 存储扩展:
 - 高速存储: 512GB NVMe SSD
 - 内存扩展: 16GB DDR4 内存
- 电源管理:
 - 锂电池: 大容量锂电池组
 - 电源管理: 智能电源管理模块

智能聊天机器人系统架构



9.2.2. 2.2. 软件环境

- 操作系统: Ubuntu 20.04 LTS
- CANN 版本: 7.0.RC1
- Python 版本: 3.8.10
- 深度学习框架:
 - torch: PyTorch 深度学习框架
 - transformers: Hugging Face Transformers 库
 - accelerate: 模型加速库
 - peft: 参数高效微调库
- 自然语言处理:
 - tokenizers: 高效分词器
 - datasets: 数据集处理
 - nltk: 自然语言处理工具包
 - jieba: 中文分词库
- 语音处理:
 - librosa: 音频分析库
 - soundfile: 音频文件处理

- pyaudio: 实时音频处理
- speech_recognition: 语音识别库
- pyttsx3: 文本转语音

- **Web 服务:**

- fastapi: 高性能 Web 框架
- websockets: WebSocket 支持
- uvicorn: ASGI 服务器

- **数据库:**

- sqlite3: 轻量级数据库
- redis: 内存数据库

环境配置脚本 (setup_chatbot.sh)

```

1 #!/bin/bash
2 # 更新系统
3 sudo apt update && sudo apt upgrade -y
4
5 # 安装系统依赖
6 sudo apt install -y python3-dev python3-pip build-essential
7 sudo apt install -y portaudio19-dev redis-server sqlite3
8 sudo apt install -y espeak espeak-data libespeak1 libespeak-dev
9
10 # 安装Python依赖
11 pip3 install torch transformers accelerate peft
12 pip3 install tokenizers datasets nltk jieba
13 pip3 install librosa soundfile pyaudio SpeechRecognition pyttsx3
14 pip3 install fastapi websockets uvicorn
15 pip3 install redis sqlite3
16
17 # 下载NLTK数据
18 python3 -c "import nltk; nltk.download('punkt'); nltk.download('stopwords'
    ↪ ' ')"
19
20 echo "智能聊天机器人环境配置完成!"

```

9.2.3. 2.3. 语言模型选择与优化

- **模型选择策略:** python #适合边缘设备的语言模型 model_options = { "chinese_models
 ↪ ": ["baichuan2-7b-chat", # 百川2-7B对话模型 "chatglm2-6b", # ChatGLM2
 ↪ -6B "qwen-7b-chat", # 通义千问7B "internlm-chat-7b" # InternLM-7B],
 ↪ "multilingual_models": ["llama2-7b-chat", # LLaMA2-7B对话版 "mistral
 ↪ -7b-instruct", # Mistral-7B指令版 "vicuna-7b-v1.5" # Vicuna-7B], "
 ↪ lightweight_models": ["phi-2", # Microsoft Phi-2 "stablelm-3b-4e1t
 ↪ ", # StableLM-3B "opt-2.7b" # OPT-2.7B] }

- **模型量化与压缩**: “python # 模型量化配置 from transformers import AutoModelForCausalLM, AutoTokenizer from transformers import BitsAndBytesConfig
4-bit 量化配置 quantization_config = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_compute_dtype=torch.float16, bnb_4bit_use_double_quant=True, bnb_4bit_quant_type="nf4")
加载量化模型 model = AutoModelForCausalLM.from_pretrained("baichuan2-7b-chat", quantization_config=quantization_config, device_map="auto", trust_remote_code=True)
) “
- **LoRA 微调**: “python # LoRA 参数高效微调 from peft import get_peft_model, LoraConfig, TaskType
LoRA 配置 lora_config = LoraConfig(task_type=TaskType.CAUSAL_LM, inference_mode=False, r=16, # LoRA 秩 lora_alpha=32, # LoRA 缩放参数 lora_dropout=0.1, # Dropout 率 target_modules=["q_proj", "v_proj", "k_proj", "o_proj"])
应用 LoRA model = get_peft_model(base_model, lora_config) “

9.2.4. 2.4. 对话管理系统

- **对话状态跟踪**: “python # 对话状态管理器 class DialogueStateManager: def
init(self): self.conversation_history = [] self.user_context = {} self.current_topic = None self.dialogue_state = “greeting”

```

1  def update_state(self, user_input, bot_response):
2      # 更新对话历史
3      self.conversation_history.append({
4          "user": user_input,
5          "bot": bot_response,
6          "timestamp": time.time(),
7          "state": self.dialogue_state
8      })
9
10     # 更新对话状态
11     self.dialogue_state = self.predict_next_state(user_input)
12
13     # 提取用户信息
14     user_info = self.extract_user_context(user_input)
15     self.user_context.update(user_info)
16
17     def get_context_prompt(self):
18         # 生成包含上下文的提示词
19         context = ""
20         if self.conversation_history:

```

```

21         recent_turns = self.conversation_history[-3:] # 最近3轮
           ↪ 对话
22         for turn in recent_turns:
23             context += f"用户: {turn['user']}\n助手: {turn['bot']}\n"
           ↪ '']\n"
24
25         return context

```

““

- 意图识别与槽填充: “python # 意图识别系统 class IntentRecognizer: def init(self): self.intent_classifier = self.load_intent_model() self.slot_extractor = self.load_slot_model()

```

1  def recognize_intent(self, user_input):
2      # 预处理用户输入
3      processed_input = self.preprocess_text(user_input)
4
5      # 意图分类
6      intent_probs = self.intent_classifier.predict(processed_input)
           ↪ )
7      intent = max(intent_probs, key=intent_probs.get)
8
9      # 槽位提取
10     slots = self.slot_extractor.extract(processed_input)
11
12     return {
13         "intent": intent,
14         "confidence": intent_probs[intent],
15         "slots": slots
16     }

```

““

- 多轮对话管理: “python # 多轮对话控制器 class MultiTurnDialogueController: def init(self, language_model): self.language_model = language_model self.state_manager = DialogueStateManager() self.intent_recognizer = IntentRecognizer()

```

1  def generate_response(self, user_input):
2      # 意图识别
3      intent_result = self.intent_recognizer.recognize_intent(
           ↪ user_input)
4
5      # 获取对话上下文
6      context = self.state_manager.get_context_prompt()
7
8      # 构建完整提示词
9      full_prompt = self.build_prompt(context, user_input,
           ↪ intent_result)
10
11     # 生成回复
12     response = self.language_model.generate(
13         full_prompt,

```

```

14         max_length=512,
15         temperature=0.7,
16         do_sample=True
17     )
18
19     # 更新对话状态
20     self.state_manager.update_state(user_input, response)
21
22     return response

```

““

9.2.5. 2.5. 语音交互系统

- 语音识别 (ASR): “python # 实时语音识别系统 class SpeechRecognitionSystem: def init(self): self.recognizer = sr.Recognizer() self.microphone = sr.Microphone() self.is_listening = False

```

1  def continuous_recognition(self, callback):
2      """持续语音识别"""
3      with self.microphone as source:
4          self.recognizer.adjust_for_ambient_noise(source)
5
6      def listen_continuously():
7          while self.is_listening:
8              try:
9                  with self.microphone as source:
10                     # 监听音频
11                     audio = self.recognizer.listen(source,
12                                                         ↪ timeout=1, phrase_time_limit=5)
13
14                     # 识别语音
15                     text = self.recognizer.recognize_google(audio,
16                                                         ↪ language='zh-CN')
17                     callback(text)
18
19             except sr.WaitTimeoutError:
20                 pass
21             except sr.UnknownValueError:
22                 pass
23             except sr.RequestError as e:
24                 print(f"语音识别服务错误: {e}")
25
26      # 启动监听线程
27      listen_thread = threading.Thread(target=listen_continuously)
28      listen_thread.daemon = True
29      listen_thread.start()

```

““

- 语音合成 (TTS): “python # 语音合成系统 class TextToSpeechSystem: def init(self): self.tts_engine = pyttsx3.init() self.configure_voice()

```

1  def configure_voice(self):
2      # 配置语音参数
3      voices = self.tts_engine.getProperty('voices')
4
5      # 选择中文语音
6      for voice in voices:
7          if 'chinese' in voice.name.lower() or 'zh' in voice.id.
           ↳ lower():
8              self.tts_engine.setProperty('voice', voice.id)
9              break
10
11     # 设置语速和音量
12     self.tts_engine.setProperty('rate', 150)    # 语速
13     self.tts_engine.setProperty('volume', 0.8)  # 音量
14
15     def speak(self, text):
16         """异步语音播放"""
17         def speak_async():
18             self.tts_engine.say(text)
19             self.tts_engine.runAndWait()
20
21         speak_thread = threading.Thread(target=speak_async)
22         speak_thread.daemon = True
23         speak_thread.start()

```

““

- 语音增强与降噪: “python # 音频预处理和增强 class AudioPreprocessor: def init(self): self.sample_rate = 16000

```

1  def noise_reduction(self, audio_data):
2      # 谱减法降噪
3      import scipy.signal
4
5      # 计算功率谱
6      f, t, Sxx = scipy.signal.spectrogram(audio_data, self.
           ↳ sample_rate)
7
8      # 噪声估计和抑制
9      noise_power = np.mean(Sxx[:, :10], axis=1, keepdims=True) #
           ↳ 假设前10帧为噪声
10     enhanced_Sxx = Sxx - 0.5 * noise_power
11     enhanced_Sxx = np.maximum(enhanced_Sxx, 0.1 * Sxx) # 保留10%
           ↳ 原信号
12
13     # 重构音频
14     enhanced_audio = scipy.signal.istft(enhanced_Sxx, self.
           ↳ sample_rate)[1]
15
16     return enhanced_audio

```

““

9.2.6. 2.6. 知识库与检索增强

- 本地知识库构建: “python # 知识库管理系统 class KnowledgeBase: def **init**(self, db_path): self.db_path = db_path self.embedding_model = SentenceTransformer(‘all-MiniLM-L6-v2’) self.vector_index = faiss.IndexFlatIP(384) # 向量维度 self.knowledge_store = []

```

1  def add_knowledge(self, text, metadata=None):
2      # 生成文本嵌入
3      embedding = self.embedding_model.encode([text])
4
5      # 添加到向量索引
6      self.vector_index.add(embedding)
7
8      # 存储原始文本和元数据
9      self.knowledge_store.append({
10         'text': text,
11         'metadata': metadata or {},
12         'id': len(self.knowledge_store)
13     })
14
15  def search_similar(self, query, k=5):
16      # 查询向量化
17      query_embedding = self.embedding_model.encode([query])
18
19      # 向量检索
20      scores, indices = self.vector_index.search(query_embedding, k
21         ↪ )
22
23      # 返回相关知识
24      results = []
25      for score, idx in zip(scores[0], indices[0]):
26          if idx < len(self.knowledge_store):
27              knowledge_item = self.knowledge_store[idx]
28              knowledge_item['score'] = float(score)
29              results.append(knowledge_item)
30
31      return results

```

““

- 检索增强生成 (RAG): “python # RAG 对话系统 class RAGChatBot: def **init**(self, language_model, knowledge_base): self.language_model = language_model self.knowledge_base = knowledge_base

```

1  def generate_with_knowledge(self, user_query):
2      # 检索相关知识

```

```

3     relevant_knowledge = self.knowledge_base.search_similar(
4         ↪ user_query, k=3)
5
6     # 构建包含知识的提示词
7     knowledge_context = ""
8     for item in relevant_knowledge:
9         knowledge_context += f"知识: {item['text']}\n"
10
11     prompt = f"""基于以下知识回答用户问题:

```

```

{knowledge_context}

```

```

用户问题: {user_query} 助手回答:“ ”

```

```

1     # 生成回复
2     response = self.language_model.generate(prompt)
3
4     return response, relevant_knowledge
5

```

9.2.7. 2.7. 模型部署与推理优化

- **昇腾模型转换:** “bash # 语言模型转换流程 # 1. PyTorch 模型转 ONNX (需要特殊处理 Transformer 架构) python3 convert_llm_to_onnx.py
 -model_path ./chatbot_model
 -output_path ./chatbot_model.onnx
 -seq_length 512
 # 2. ONNX 转昇腾格式 (可能需要分块处理) atc -model=chatbot_encoder.onnx
 -framework=5
 -output=chatbot_encoder_ascend
 -input_format=ND
 -input_shape="input_ids:1,512;attention_mask:1,512"
 -soc_version=Ascend310B1 “
- **推理加速策略:** “python # 推理优化管理器 class InferenceOptimizer: def
init(self, model): self.model = model self.kv_cache = {} # 键值缓存 self.generation_config
 = { “max_new_tokens”: 512, “temperature”: 0.7, “top_p”: 0.9, “do_sample”:
 True, “pad_token_id”: 0 }

```

1     def generate_with_cache(self, input_ids, attention_mask):
2         # 使用KV缓存加速生成
3         with torch.no_grad():
4             outputs = self.model.generate(
5                 input_ids=input_ids,
6                 attention_mask=attention_mask,

```

```

7         **self.generation_config,
8         use_cache=True,
9         past_key_values=self.kv_cache.get("past_key_values")
10    )
11
12    # 更新缓存
13    self.kv_cache["past_key_values"] = outputs.past_key_values
14
15    return outputs

```

“

9.2.8. 2.8. Web 界面与 API 服务

- **WebSocket 实时通信:** “python # WebSocket 聊天服务 from fastapi import FastAPI, WebSocket from fastapi.staticfiles import StaticFiles
app = FastAPI()
class ChatWebSocketManager: def **init**(self): self.active_connections = []
self.chatbot = ChatBot() # 聊天机器人实例

```

1  async def connect(self, websocket: WebSocket):
2      await websocket.accept()
3      self.active_connections.append(websocket)
4
5  def disconnect(self, websocket: WebSocket):
6      self.active_connections.remove(websocket)
7
8  async def handle_message(self, websocket: WebSocket, message: str
9      ↪ ):
10     # 生成回复
11     response = await self.chatbot.generate_response(message)
12
13     # 发送回复
14     await websocket.send_text(response)

```

manager = ChatWebSocketManager()

@app.websocket("/ws/chat") async def websocket_endpoint(websocket: WebSocket): await manager.connect(websocket) try: while True: message = await websocket.receive_text() await manager.handle_message(websocket, message) except Exception as e: print(f"WebSocket 错误: {e}") finally: manager.disconnect(websocket) “

- **RESTful API 接口:** “python # REST API 服务 from fastapi import FastAPI, HTTPException from pydantic import BaseModel
class ChatRequest(BaseModel): message: str session_id: str = None context: dict = None


```
class ChatResponse(BaseModel): response: str session_id: str confidence:
float timestamp: float
@app.post("/api/chat", response_model=ChatResponse) async def chat_endpoint(request:
ChatRequest): try: # 处理聊天请求 response = await chatbot_service.process_message(
message=request.message, session_id=request.session_id, context=request.context
)
```

```
1     return ChatResponse(
2         response=response["text"],
3         session_id=response["session_id"],
4         confidence=response["confidence"],
5         timestamp=time.time()
6     )
7
8     except Exception as e:
9         raise HTTPException(status_code=500, detail=str(e))
```

““

9.2.9. 2.9. 用户手册

2.9.1 系统部署

1. 硬件连接: 连接音频设备和显示器
2. 软件安装: 运行环境配置脚本
3. 模型部署: 下载和部署语言模型
4. 服务启动: 启动聊天机器人服务

2.9.2 功能配置

1. 语音设置: 配置语音识别和合成参数
2. 知识库管理: 添加和管理自定义知识
3. 对话策略: 配置对话风格和策略
4. 安全设置: 配置内容过滤和安全策略

2.9.3 使用指南

1. 语音交互: 语音唤醒和对话流程
2. 文本交互: 通过界面进行文字对话
3. 多模态交互: 结合语音、文字、图像的交互
4. 个性化设置: 个人偏好和习惯配置

2.9.4 维护管理

1. **性能监控:** 监控响应时间和资源使用
2. **日志分析:** 分析对话日志和错误信息
3. **模型更新:** 更新和优化对话模型
4. **数据备份:** 备份对话历史和用户数据

9.3. 3. 源代码结构

```

1 intelligent_chatbot/
2 |— src/
3 |   |— models/           # 模型管理
4 |   |   |— language_model/
5 |   |   |— intent_recognition/
6 |   |   |— voice_models/
7 |   |— dialogue/        # 对话管理
8 |   |   |— state_manager/
9 |   |   |— intent_recognition/
10 |   |   |— response_generation/
11 |   |— speech/          # 语音处理
12 |   |   |— asr/          # 语音识别
13 |   |   |— tts/          # 语音合成
14 |   |   |— audio_processing/
15 |   |— knowledge/       # 知识管理
16 |   |   |— knowledge_base/
17 |   |   |— retrieval/
18 |   |   |— rag/
19 |   |— api/             # API服务
20 |   |   |— rest_api/
21 |   |   |— websocket/
22 |   |   |— voice_api/
23 |   |— ui/              # 用户界面
24 |   |   |— web_ui/
25 |   |   |— voice_ui/
26 |— models/
27 |   |— language_models/  # 语言模型文件
28 |   |— voice_models/    # 语音模型文件
29 |   |— intent_models/   # 意图识别模型
30 |— data/
31 |   |— knowledge_base/   # 知识库数据
32 |   |— dialogue_history/ # 对话历史
33 |   |— user_profiles/   # 用户画像
34 |— configs/
35 |   |— model_config.yaml # 模型配置

```

```
36 |   ├── dialogue_config.yaml # 对话配置
37 |   └── speech_config.yaml # 语音配置
38 | └── deployment/
39 |     ├── docker/           # Docker部署
40 |     ├── scripts/          # 部署脚本
41 |     └── monitoring/        # 监控配置
```

9.4. 4. 效果演示

- 自然对话展示: 流畅的多轮对话演示
- 语音交互体验: 语音识别和合成的完整流程
- 知识问答: 基于知识库的专业问答
- 个性化对话: 根据用户偏好的个性化回复
- 多语言支持: 中英文混合对话能力展示